

Беру v9.62 именно как integration pass: сначала читаю существующие системы и grep'аю уже имеющиеся loop/reward/service hooks. Если разрывов нет, добавлю smoke; если есть маленький разрыв, сделаю минимальный bridge без новых profile fields.

Ran 4 commands

Ran command

\$ git status -sb

feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

Success

Ran command

\$ git branch --show-current

feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

feature/start-bunker-bt72-pippad-v10

Success

Ran command

\$ git fetch origin --prune

feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

Success

Ran command

\$ git log --oneline --decorate -5

feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

70dce93 (HEAD -> feature/start-bunker-bt72-pippad-v10, origin/feature/start-bunker-bt72-pippad-v10) Update v 9.61 - Bind active Pip device customization hotkeys

45e7ea4 Update v 9.60 - Add active Pip device access wrapper

bc2e85b Update v 9.59 - Gate Pip customization commands behind active device

7b9c20c Update v 9.58 - Gate Pip device customization by active model

30fbc65 Update v 9.57 - Add active Pip device customization commands

Success

Preflight чистый. Сейчас ищу уже существующие scavenging/recovery/service/Lanline hooks и места, где они вызываются из runtime/events, чтобы не изобретать второй слой.

Ran 4 commands

Ran command

```
$ git grep -n "scavenger\\|scavenge\\|scouting\\|field  
route\\|caravan\\|drone\\|trade\\|rail\\|fatigue\\|stress\\|medicine\\|medical\\|field  
service\\|service\\|returnedToBase\\|exitedBunker\\|surfaceArrivalReached\\|Lanline\\|Recovery" -- include src  
tests  
  
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10  
  
include/AppPaths.hpp:27:inline fs::path LanlineSessionsDirectory() {  
  
include/AppPaths.hpp:35: fs::create_directories(LanlineSessionsDirectory());  
  
include/AppPaths.hpp:152:inline fs::path LanlineSessionPath() {  
  
include/AppPaths.hpp:156:inline fs::path LanlineSessionSnapshotPath(std::string_view sessionId) {  
  
include/AppPaths.hpp:157: return LanlineSessionsDirectory() / (SanitizeWorldKey(sessionId) + ".state");  
  
include/GameRuntime.hpp:76: bool stressThresholdTriggered = false;  
  
include/GameRuntime.hpp:82: float scavengerTimer = 180.0f;  
  
include/GameRuntime.hpp:83: float caravanTimer = 260.0f;  
  
include/GameRuntime.hpp:84: float droneTimer = 210.0f;  
  
include/GameRuntime.hpp:85: float tradeTimer = 240.0f;  
  
include/GameRuntime.hpp:86: float railTimer = 320.0f;  
  
include/GameRuntime.hpp:88: float railFortressTimer = 520.0f;  
  
include/GameRuntime.hpp:97: float serviceBayTimer = 300.0f;  
  
include/GameRuntime.hpp:110: bool medicalSupportNearby = false;  
  
include/GameRuntime.hpp:119: "SYSTEM: 'Cryo wing unstable. Sweep the shared cryo tier, recover an access  
card, and follow the paper trail to the Pip-Pad.'",  
  
include/GameRuntime.hpp:120: "ARCHIVE: 'One reactor core is missing. One body is missing. Garage service  
traces remain incomplete.'",  
  
include/GameRuntime.hpp:284:void UpdateRecoveryFabricator(SessionProfile& profile, GameState& gameState,  
float dt);  
  
include/GameRuntime.hpp:285:void UpdateRecoveryMilestones(SessionProfile& profile, GameState&  
gameState);  
  
include/HangarSystem.hpp:35: hangar.lastAction = "Partner tank serviced, rearmed, and fully repaired."  
  
include/HangarSystem.hpp:118: *eventText = "Turret service kit applied. Servo ring and stabilizer alignment  
restored."  
  
include/HangarSystem.hpp:128: *eventText = "Engine service kit applied. Core seals replaced and reserve charge  
restored.";
```

```

include/HangarSystem.hpp:207: *eventText = "Tank service kit applied to the selected subsystem.";

include/HangarSystem.hpp:213: *eventText = "Required tank service kit not found.";

include/HangarSystem.hpp:256: *eventText = "Compatible tank service kit not found.";

include/HangarSystem.hpp:264: ? "BT-72 service anchor reports no damaged subsystem that matches the
available kits."

include/HangarSystem.hpp:265: : "Compatible tank service kit not found.";

include/LanlineLobbyLogic.hpp:5:#include "LanlineSession.hpp"

include/LanlineLobbyLogic.hpp:9:inline bool IsLanlineAwaitingSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:13:inline bool IsLanlinePendingSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:17:inline bool IsLanlineReservedSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:21:inline bool IsLanlineAcceptedSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:25:inline bool IsLanlineReadyEligibleSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:29:inline const char* LanlineSlotStateLabel(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:30: if (IsLanlineAwaitingSlot(entry)) {

include/LanlineLobbyLogic.hpp:33: if (IsLanlinePendingSlot(entry)) {

include/LanlineLobbyLogic.hpp:36: if (IsLanlineReservedSlot(entry)) {

include/LanlineLobbyLogic.hpp:39: if (IsLanlineAcceptedSlot(entry)) {

include/LanlineLobbyLogic.hpp:45:inline const char* LanlineReadyLabel(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:46: if (!IsLanlineReadyEligibleSlot(entry)) {

include/LanlineLobbyLogic.hpp:52:inline int FindFirstAwaitingSlotIndex(const LanlineSessionState& state) {

include/LanlineLobbyLogic.hpp:54: if (IsLanlineAwaitingSlot(state.players[static_cast<std::size_t>(index)])) {

include/LanlineLobbyLogic.hpp:61:inline int MaxLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:68:inline int OccupiedLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:71: if (session.mode == "LAN Host" && IsLanlineAwaitingSlot(player)) {

include/LanlineLobbyLogic.hpp:81:inline int AvailableLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:82: return std::max(0, MaxLanlineSessionSlots(session) -
OccupiedLanlineSessionSlots(session));

include/LanlineLobbyLogic.hpp:85:inline int PendingLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:88: if (IsLanlinePendingSlot(player)) {

include/LanlineLobbyLogic.hpp:95:inline int ReservedLanlineSessionSlots(const LanlineSessionState& session) {

```

```
include/LanlineLobbyLogic.hpp:98: if (IsLanlineReservedSlot(player)) {  
  
include/LanlineLobbyLogic.hpp:105:inline int AcceptedLanlineSessionSlots(const LanlineSessionState& session) {  
  
include/LanlineLobbyLogic.hpp:108: if (IsLanlineAcceptedSlot(player)) {  
  
include/LanlineLobbyLogic.hpp:115:inline int ReadyLanlineSessionSlots(const LanlineSessionState& session) {  
  
include/LanlineLobbyLogic.hpp:118: if (IsLanlineReadyEligibleSlot(player) && player.ready) {  
  
include/LanlineLobbyLogic.hpp:125:inline bool IsJoinableLanlineSession(const LanlineSessionState& session) {  
  
include/LanlineLobbyLogic.hpp:137:inline bool IsLanlineMatchStartReady(const LanlineSessionState& session) {  
  
include/LanlineLobbyLogic.hpp:141: if (ReservedLanlineSessionSlots(session) > 0 ||  
PendingLanlineSessionSlots(session) > 0) {  
  
include/LanlineLobbyLogic.hpp:147: if (!IsLanlineReadyEligibleSlot(player)) {  
  
include/LanlineServices.hpp:9:#include "LanlineSession.hpp"  
  
include/LanlineServices.hpp:16: LanlineLocal,  
  
include/LanlineServices.hpp:33: bool serviceBayActive = false;  
  
include/LanlineServices.hpp:50:inline bool IsLanlineServicesUnlocked(const ServicesUnlockState& state) {  
  
include/LanlineServices.hpp:55: return state.serviceBayActive || state.tier >=  
ServicesUnlockTier::BackboneStable;  
  
include/LanlineServices.hpp:183:struct LanlineServicesState {  
  
include/LanlineServices.hpp:195:struct LanlineServicesSave {  
  
include/LanlineServices.hpp:206:LanlineServicesState MakeDefaultLanlineServicesState(std::int64_t nowUnix);  
  
include/LanlineServices.hpp:207:LanlineServicesState MakeLanlineServicesStateFromSave(const  
LanlineServicesSave& save, std::int64_t nowUnix);  
  
include/LanlineServices.hpp:208:LanlineServicesSave BuildLanlineServicesSave(const LanlineServicesState&  
state);  
  
include/LanlineServices.hpp:209:void ApplyLanlineServicesProfileSnapshot(LanlineServicesState& state, const  
LanlineServicesProfile& profile);  
  
include/LanlineServices.hpp:210:void SyncLanlineServicesProfileSnapshot(LanlineServicesProfile& profile, const  
LanlineServicesState& state);  
  
include/LanlineServices.hpp:211:void SyncLanlineServicesSessionProfile(SessionProfile& profile, const  
LanlineServicesState& state);  
  
include/LanlineServices.hpp:212:void AdvanceLanlineSupportOrders(LanlineServicesState& state, std::int64_t  
nowUnix);  
  
include/LanlineServices.hpp:213:int CountSupportOrdersInState(const LanlineServicesState& state,  
SupportOrderState orderState);
```

```
include/LanlineServices.hpp:214:int ClaimDeliveredSupportOrders(LanlineServicesState& state, SessionProfile&
profile, std::string* summary = nullptr);

include/LanlineServices.hpp:215:ServiceHubMode ResolveLanlineServicesMode(const ServicesUnlockState&
unlockState, const LanlineSessionState* sessionState);

include/LanlineServices.hpp:216:void SyncLanlineServicesPresence(LanlineServicesState& state,

include/LanlineServices.hpp:217: const LanlineSessionState* sessionState,

include/LanlineServices.hpp:221:std::filesystem::path DefaultLanlineServicesSavePath());

include/LanlineServices.hpp:222:bool SaveLanlineServicesSave(const LanlineServicesSave& save, const
std::filesystem::path& path);

include/LanlineServices.hpp:223:bool LoadLanlineServicesSave(const std::filesystem::path& path,
LanlineServicesSave& outSave);

include/LanlineServices.hpp:224:void DrawLanlineServicesPanel(LanlineServicesState& state,

include/LanlineSession.hpp:10:struct LanlinePlayerEntry {

include/LanlineSession.hpp:18:struct LanlineRelayMessage {

include/LanlineSession.hpp:25:struct LanlineVoicePresence {

include/LanlineSession.hpp:34:struct LanlineSessionState {

include/LanlineSession.hpp:48: std::vector<LanlinePlayerEntry> players{};

include/LanlineSession.hpp:49: std::vector<LanlineRelayMessage> relayMessages{};

include/LanlineSession.hpp:50: std::vector<LanlineVoicePresence> voicePresence{};

include/LanlineSession.hpp:54:struct LanlineDiagnostics {

include/LanlineSession.hpp:65:bool SaveLanlineSessionState(const LanlineSessionState& state);

include/LanlineSession.hpp:66:bool LoadLanlineSessionState(LanlineSessionState& state);

include/LanlineSession.hpp:67:bool SaveLanlineSessionState(const LanlineSessionState& state, const
std::filesystem::path& path);

include/LanlineSession.hpp:68:bool LoadLanlineSessionState(const std::filesystem::path& path,
LanlineSessionState& state);

include/LanlineSession.hpp:69:std::vector<LanlineSessionState> DiscoverLanlineSessionSnapshots();

include/LanlineSession.hpp:70:LanlineDiagnostics ProbeLanlineHost(const LanlineSessionState& session,
std::string_view runtimeWorldName);

include/SessionProfiles.hpp:70:struct LanlineServicesProfile {

include/SessionProfiles.hpp:74: bool serviceHubKnown = false;

include/SessionProfiles.hpp:91: bool caravanRouteActive = false;
```

```
include/SessionProfiles.hpp:92: bool droneStationsActive = false;

include/SessionProfiles.hpp:93: bool tradeNetworkActive = false;

include/SessionProfiles.hpp:94: bool railFreightActive = false;

include/SessionProfiles.hpp:96: bool railFortressActive = false;

include/SessionProfiles.hpp:101: int caravanRunsCompleted = 0;

include/SessionProfiles.hpp:102: int droneRunsCompleted = 0;

include/SessionProfiles.hpp:103: int tradeCyclesCompleted = 0;

include/SessionProfiles.hpp:105: int railRunsCompleted = 0;

include/SessionProfiles.hpp:107: int railFortressDeployments = 0;

include/SessionProfiles.hpp:125: bool serviceBayActive = false;

include/SessionProfiles.hpp:126: int serviceCyclesCompleted = 0;

include/SessionProfiles.hpp:151: int stressSurvivals = 0;

include/SessionProfiles.hpp:169: bool surfaceArrivalReached = false;

include/SessionProfiles.hpp:172: bool firstRecoveryNodeActivated = false;

include/SessionProfiles.hpp:182: bool exitedBunker = false;

include/SessionProfiles.hpp:185: bool returnedToBase = false;

include/SessionProfiles.hpp:417: LanlineServicesProfile lanlineServices{};

include/SessionProfiles.hpp:427: int scavengerRunsCompleted = 0;

include/SessionProfiles.hpp:521: target.caravanRouteActive = target.caravanRouteActive ||
source.caravanRouteActive;

include/SessionProfiles.hpp:522: target.droneStationsActive = target.droneStationsActive ||
source.droneStationsActive;

include/SessionProfiles.hpp:523: target.tradeNetworkActive = target.tradeNetworkActive ||
source.tradeNetworkActive;

include/SessionProfiles.hpp:524: target.railFreightActive = target.railFreightActive || source.railFreightActive;

include/SessionProfiles.hpp:526: target.railFortressActive = target.railFortressActive || source.railFortressActive;

include/SessionProfiles.hpp:531: target.caravanRunsCompleted = std::max(target.caravanRunsCompleted,
source.caravanRunsCompleted);

include/SessionProfiles.hpp:532: target.droneRunsCompleted = std::max(target.droneRunsCompleted,
source.droneRunsCompleted);

include/SessionProfiles.hpp:533: target.tradeCyclesCompleted = std::max(target.tradeCyclesCompleted,
source.tradeCyclesCompleted);
```

```
include/SessionProfiles.hpp:535: target.railRunsCompleted = std::max(target.railRunsCompleted,
source.railRunsCompleted);

include/SessionProfiles.hpp:537: target.railFortressDeployments = std::max(target.railFortressDeployments,
source.railFortressDeployments);

include/SessionProfiles.hpp:555: target.serviceBayActive = target.serviceBayActive || source.serviceBayActive;

include/SessionProfiles.hpp:556: target.serviceCyclesCompleted = std::max(target.serviceCyclesCompleted,
source.serviceCyclesCompleted);

include/SessionProfiles.hpp:634: return worldState.caravanRouteActive &&

include/SessionProfiles.hpp:641: return worldState.droneStationsActive && HasRegionalGridOnline(profile);

include/SessionProfiles.hpp:645: return worldState.tradeNetworkActive &&

include/SessionProfiles.hpp:648: (worldState.caravanRouteActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:652: return worldState.railFreightActive &&

include/SessionProfiles.hpp:657: (worldState.caravanRouteActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:663: worldState.railFreightActive &&

include/SessionProfiles.hpp:665: (worldState.tradeNetworkActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:669: return worldState.railFortressActive &&

include/SessionProfiles.hpp:670: worldState.railFreightActive &&

include/SessionProfiles.hpp:676:inline bool IsRecoveryFabricatorOperational(const SessionProfile& profile, const
WorldFieldState& worldState) {

include/SessionProfiles.hpp:679: (worldState.railFreightActive || worldState.droneStationsActive) &&

include/SessionProfiles.hpp:680: (worldState.tradeNetworkActive || worldState.orbitalUplinkActive);

include/SessionProfiles.hpp:687: worldState.tradeNetworkActive;

include/SessionProfiles.hpp:694: worldState.tradeNetworkActive;

include/SessionProfiles.hpp:708: worldState.railFortressActive;

include/SessionProfiles.hpp:733: return worldState.serviceBayActive &&

include/SessionProfiles.hpp:741: worldState.serviceBayActive &&

include/SessionProfiles.hpp:746:inline bool IsStableRecoveryBackbone(const SessionProfile& profile, const
WorldFieldState& worldState) {

include/SessionProfiles.hpp:750: IsRecoveryFabricatorOperational(profile, worldState);

include/StoryRoute.hpp:33:struct RecoveryBackboneStatus {

include/StoryRoute.hpp:43:std::string CurrentRecoveryHandoffSummary(const SessionProfile& profile);
```

```
include/StoryRoute.hpp:44:std::string CurrentRecoveryHandoffSummary(const SessionProfile& profile,
std::string_view worldReference);

include/StoryRoute.hpp:45:RecoveryBackboneStatus CurrentRecoveryBackboneStatus(const SessionProfile&
profile);

include/StoryRoute.hpp:46:RecoveryBackboneStatus CurrentRecoveryBackboneStatus(const SessionProfile&
profile, std::string_view worldReference);

include/StoryRoute.hpp:56:bool HasLanlineServicesObjective(const SessionProfile& profile);

include/WorldExport.hpp:202: std::filesystem::path auditTrailPath{};

include/WorldExport.hpp:223:std::filesystem::path ExportAuditTrailPathForWorld(const std::filesystem::path&
worldPath);

src/GameRuntime.cpp:5:#include "../include/LanlineLobbyLogic.hpp"

src/GameRuntime.cpp:6:#include "../include/LanlineServices.hpp"

src/GameRuntime.cpp:7:#include "../include/LanlineSession.hpp"

src/GameRuntime.cpp:83:void AppendRecoveryBackboneReadabilityHint(std::string& message, const
SessionProfile& profile) {

src/GameRuntime.cpp:84: const auto backbone = CurrentRecoveryBackboneStatus(profile);

src/GameRuntime.cpp:128: registryIdLower.find("drone") != std::string::npos ||

src/GameRuntime.cpp:131: displayNameLower.find("drone") != std::string::npos ||

src/GameRuntime.cpp:1386: eventText += " Pilot Sync trimmed service losses and tightened the link.";

src/GameRuntime.cpp:1411: eventText += " Medical doctrine stabilized the operator during service.";

src/GameRuntime.cpp:1430:float DoctrineCampRecoveryBoost(const SessionProfile& profile) {

src/GameRuntime.cpp:1441:float WaterRecoveryBoost(const SessionProfile& profile) {

src/GameRuntime.cpp:1449:const char* RecoveryStatusLabel(const SessionProfile& profile, const
WorldFieldState* worldState) {

src/GameRuntime.cpp:1450: if (worldState != nullptr && IsStableRecoveryBackbone(profile, *worldState)) {

src/GameRuntime.cpp:1453: return profile.story.returnedToBase ? "Recovery buildout active" : "Starter route";

src/GameRuntime.cpp:1460: {"drone_station", "Drone station ledger mirrored. Sweep routes registered to Pip-
Pad."},

src/GameRuntime.cpp:1461: {"rail_depot", "Rail freight depot records mirrored. Heavy spur logistics
registered."},

src/GameRuntime.cpp:1463: {"rail_fortress_hub", "Rail Fortress patrol package mirrored. Spur security doctrine
updated."},
```



```
src/GameRuntime.cpp:1464: {"recovery_fabricator", "Recovery fabricator recipes mirrored. Shelter supply pipeline updated."},

src/GameRuntime.cpp:1473: {"service_bay", "Service bay notes mirrored. BT-72 support route updated."},

src/GameRuntime.cpp:1475: {"lanline_service_hub", "Lanline service hub mirrored. Shelter 17 service catalog now resolves through authored relay infrastructure."},

src/GameRuntime.cpp:1477: {"medical_support", "Medical support node mirrored. Field treatment requests now route through authored relay anchors."},

src/GameRuntime.cpp:1478: {"tank_service", "Tank service anchor mirrored. BT-72 maintenance route updated."},

src/GameRuntime.cpp:1481: {"workshop_service", "Workshop terminal mirrored. BT-72 service route and repair notes updated."},

src/GameRuntime.cpp:1502: const LanlineDiagnostics& CachedLanlineDiagnostics(const LanlineSessionState& session, std::string_view runtimeWorldName) {

src/GameRuntime.cpp:1506: static LanlineDiagnostics cachedDiagnostics;

src/GameRuntime.cpp:1512: cachedDiagnostics = ProbeLanlineHost(session, normalizedWorldName);

src/GameRuntime.cpp:1520: const std::vector<std::string>& UpdateLanlineRuntimeNotifications(const LanlineSessionState* session,

src/GameRuntime.cpp:1524: static LanlineSessionState previousSession;

src/GameRuntime.cpp:1540: pushNotification("Lanline session link lost. Waiting for refreshed launcher/session state.");

src/GameRuntime.cpp:1552: pushNotification("Lanline session linked: " + session->sessionId + " on " + normalizedSessionWorld + ".");

src/GameRuntime.cpp:1554: pushNotification("Lanline world mismatch: runtime is on " + normalizedRuntimeWorld +

src/GameRuntime.cpp:1564: pushNotification("Lanline session switched to " + session->sessionId + " (" + session->mode + ").");

src/GameRuntime.cpp:1567: pushNotification("Lanline mode changed to " + session->mode + ".");

src/GameRuntime.cpp:1570: pushNotification("Lanline lifecycle advanced to " + session->lifecycleStage + ".");

src/GameRuntime.cpp:1574: ? "Lanline pending peer queue cleared."

src/GameRuntime.cpp:1575: : "Lanline pending peer updated: " + session->pendingPeer + ".");

src/GameRuntime.cpp:1579: ? "Lanline connected peer link cleared."

src/GameRuntime.cpp:1580: : "Lanline peer link confirmed with " + session->connectedPeer + ".");

src/GameRuntime.cpp:1583: pushNotification("Lanline session world updated: " + previousNormalizedWorld + " -> " + normalizedSessionWorld + ".");
```

```
src/GameRuntime.cpp:1589: ? "Lanline session world now matches the active runtime world."

src/GameRuntime.cpp:1590: : "Lanline session world drifted away from the active runtime world.");

src/GameRuntime.cpp:1595: [&](const LanlinePlayerEntry& previousEntry) {

src/GameRuntime.cpp:1599: pushNotification(playerEntry.displayName + " joined Lanline roster as " +
playerEntry.role + ".");

src/GameRuntime.cpp:1603: pushNotification(playerEntry.displayName + " moved from " + previousIt->role + " to
" + playerEntry.role + " in Lanline lobby.");

src/GameRuntime.cpp:1607: (playerEntry.online ? " came online in Lanline." : " went offline in Lanline."));

src/GameRuntime.cpp:1609: if (bunker::IsLanlineReadyEligibleSlot(playerEntry) && playerEntry.ready !=
previousIt->ready) {

src/GameRuntime.cpp:1611: (playerEntry.ready ? " is now ready for Lanline deployment." : " is no longer ready
for Lanline deployment."));

src/GameRuntime.cpp:1617: [&](const LanlinePlayerEntry& currentEntry) {

src/GameRuntime.cpp:1621: pushNotification(previousEntry.displayName + " left the Lanline roster.");

src/GameRuntime.cpp:1637: [&](const LanlineVoicePresence& previousVoice) {

src/GameRuntime.cpp:1660: [&](const LanlineVoicePresence& currentVoice) {

src/GameRuntime.cpp:1697: const char* JoinabilityLabel(const LanlineSessionState& session) {

src/GameRuntime.cpp:1713: bool RecoveryFabricatorReady(const SessionProfile& profile, const WorldFieldState&
worldState) {

src/GameRuntime.cpp:1714: return IsRecoveryFabricatorOperational(profile, worldState);

src/GameRuntime.cpp:1881: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";

src/GameRuntime.cpp:1885: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the gunner
station.";

src/GameRuntime.cpp:1889: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";

src/GameRuntime.cpp:1894: gameState.lastEvent = "BT-72 is moving too fast for field service. Stop the hull first.";

src/GameRuntime.cpp:1898: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable
condition.";

src/GameRuntime.cpp:1902: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a repair
patch.";

src/GameRuntime.cpp:1906: const bool hasEngineer = HasAssignedSpecialistRole(profile, "engineer",
"scavenger_support");

src/GameRuntime.cpp:1923: ? "Field service rack cycled. Scavenger-side engineer tuning improved the repair
pass."
```

```
src/GameRuntime.cpp:1924: : "Field service rack cycled. BT-72 patched and partially resupplied in the field.";
src/GameRuntime.cpp:1928: gameState.lastEvent += " First service halt logged for the route.";
src/GameRuntime.cpp:2004: missing.push_back("service notes");
src/GameRuntime.cpp:2074: : "First surface contact resolved. Run a BT-72 service halt before relay sync.";
src/GameRuntime.cpp:2167: float ShelterRecoveryIndex(const SessionProfile& profile) {
src/GameRuntime.cpp:2174: const bool caravanOperational = IsCaravanOperational(profile, *worldState);
src/GameRuntime.cpp:2175: const bool droneOperational = IsDroneStationOperational(profile, *worldState);
src/GameRuntime.cpp:2176: const bool tradeOperational = IsTradeNetworkOperational(profile, *worldState);
src/GameRuntime.cpp:2177: const bool railOperational = IsRailFreightOperational(profile, *worldState);
src/GameRuntime.cpp:2180: const bool fabricatorOperational = IsRecoveryFabricatorOperational(profile,
*worldState);
src/GameRuntime.cpp:2188: const bool serviceOperational = IsServiceBayOperational(*worldState);
src/GameRuntime.cpp:2190: if (profile.story.returnedToBase) score += 8.0f;
src/GameRuntime.cpp:2194: if (caravanOperational) score += 8.0f;
src/GameRuntime.cpp:2195: if (droneOperational) score += 7.0f;
src/GameRuntime.cpp:2196: if (tradeOperational) score += 8.0f;
src/GameRuntime.cpp:2197: if (railOperational) score += 10.0f;
src/GameRuntime.cpp:2209: if (serviceOperational) score += 8.0f;
src/GameRuntime.cpp:2219: score += std::min(6.0f, static_cast<float>(worldState->serviceCyclesCompleted) *
1.0f);
src/GameRuntime.cpp:2222: score += std::min(8.0f, static_cast<float>(worldState->tradeCyclesCompleted) *
0.8f);
src/GameRuntime.cpp:2223: score += std::min(8.0f, static_cast<float>(worldState->railRunsCompleted) * 0.8f);
src/GameRuntime.cpp:2233: const char* ShelterRecoveryBand(float recoveryIndex) {
src/GameRuntime.cpp:2249: int ShelterRecoveryMilestoneTier(float recoveryIndex) {
src/GameRuntime.cpp:2262: const char* ShelterRecoveryMilestoneLabel(int tier) {
src/GameRuntime.cpp:2347: if (scriptTag == "drone_station") {
src/GameRuntime.cpp:2353: if (!worldState.droneStationsActive) {
src/GameRuntime.cpp:2360: worldState.droneStationsActive = true;
src/GameRuntime.cpp:2361: gameState.droneTimer = 210.0f;
```

```
src/GameRuntime.cpp:2362: gameState.lastEvent = "Drone station brought online. Automated scavenger drones
launched to local sweep routes.";

src/GameRuntime.cpp:2364: worldState.droneStationsActive = false;

src/GameRuntime.cpp:2370: if (scriptTag == "rail_depot") {

src/GameRuntime.cpp:2372: if (!worldState.railFreightActive) {

src/GameRuntime.cpp:2383: worldState.railFreightActive = true;

src/GameRuntime.cpp:2384: gameState.railTimer = 320.0f;

src/GameRuntime.cpp:2385: gameState.lastEvent = "Rail freight spur restored. Heavy salvage trains can now
service the bunker route.";

src/GameRuntime.cpp:2387: worldState.railFreightActive = false;

src/GameRuntime.cpp:2396: if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {

src/GameRuntime.cpp:2397: gameState.lastEvent = "Orbital uplink requires a live grid and active rail freight
backbone.";

src/GameRuntime.cpp:2416: if (scriptTag == "rail_fortress_hub") {

src/GameRuntime.cpp:2418: if (!worldState.railFortressActive) {

src/GameRuntime.cpp:2419: if (!worldState.railFreightActive || !worldState.orbitalUplinkActive) {

src/GameRuntime.cpp:2420: gameState.lastEvent = "Rail fortress requires active rail freight and orbital uplink
support.";

src/GameRuntime.cpp:2429: worldState.railFortressActive = true;

src/GameRuntime.cpp:2430: gameState.railFortressTimer = 520.0f;

src/GameRuntime.cpp:2433: worldState.railFortressActive = false;

src/GameRuntime.cpp:2442: if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {

src/GameRuntime.cpp:2443: gameState.lastEvent = "Recovery fabricator requires a live grid and rail freight
support.";

src/GameRuntime.cpp:2447: gameState.lastEvent = "Recovery fabricator boot needs steel_scrap and
ether_shard.";

src/GameRuntime.cpp:2454: gameState.lastEvent = "Recovery fabricator primed. Shelter industry can now refine
salvage into field supplies.";

src/GameRuntime.cpp:2457: gameState.lastEvent = "Recovery fabricator returned to standby.";

src/GameRuntime.cpp:2470: if (!worldState.recoveryFabricatorActive || !worldState.railFortressActive ||
!worldState.orbitalUplinkActive) {

src/GameRuntime.cpp:2499: if (!worldState.orbitalUplinkActive || !worldState.tradeNetworkActive) {
```

```
src/GameRuntime.cpp:2500: gameState.lastEvent = "Industrial survey needs orbital uplink and trade network support.";

src/GameRuntime.cpp:2569: if (!worldState.assemblyCellActive || !worldState.industrialOutpostActive || !worldState.railFortressActive) {

src/GameRuntime.cpp:2665: if (scriptTag == "service_bay") {

src/GameRuntime.cpp:2671: if (!worldState.serviceBayActive) {

src/GameRuntime.cpp:2679: worldState.serviceBayActive = true;

src/GameRuntime.cpp:2680: gameState.serviceBayTimer = 300.0f;

src/GameRuntime.cpp:2681: gameState.lastEvent = "Inner spur service bay brought online. BT-72 can now be serviced deeper into the factory belt.";

src/GameRuntime.cpp:2683: worldState.serviceBayActive = false;

src/GameRuntime.cpp:2684: gameState.lastEvent = "Inner spur service bay returned to standby.";

src/GameRuntime.cpp:2691: if (!worldState.serviceBayActive || !worldState.relaySubstationActive || !worldState.recoveryFabricatorActive) {

src/GameRuntime.cpp:2692: gameState.lastEvent = "Water reclaimer needs a live service bay, relay substation, and recovery fabricator first.";

src/GameRuntime.cpp:2713: if (scriptTag == "lanline_service_hub") {

src/GameRuntime.cpp:2716: gameState.lastEvent = "Lanline Services stay locked until the first tower is synchronized.";

src/GameRuntime.cpp:2720: profile.lanlineServices.serviceHubKnown = true;

src/GameRuntime.cpp:2723: ? "Lanline service hub handshake complete."

src/GameRuntime.cpp:2724: : "Lanline service hub handshake complete -> " + nearest->linkTarget + ".";

src/GameRuntime.cpp:2729: if (scriptTag == "tank_service") {

src/GameRuntime.cpp:2731: if (!worldState.serviceBayActive) {

src/GameRuntime.cpp:2732: gameState.lastEvent = "Tank service anchor is authored, but the service bay backbone is still offline.";

src/GameRuntime.cpp:2737: gameState.lastSupportAction = "BT-72 serviced through authored tank service anchor.";

src/GameRuntime.cpp:2741: ? "Tank service anchor ready. Bring a compatible service kit from Lanline support or field salvage."

src/GameRuntime.cpp:2742: : "Tank service anchor ready. " + repairEvent;

src/GameRuntime.cpp:2747: if (scriptTag == "medical_support") {
```

```
src/GameRuntime.cpp:2763: gameState.lastEvent = "Fey Ring routing remains sealed until Lanline Services come
online through tower sync.";

src/GameRuntime.cpp:2816: AddCollectedTapelfMissing(profile, "bt72_service_reel_001", "BT-72 Service Reel");

src/GameRuntime.cpp:2817: gameState.lastEvent = "Maintenance echo resolved. BT-72 service notes and holo-
records mirrored to Pip-Pad.";

src/GameRuntime.cpp:2848: AddCollectedTapelfMissing(profile, nearest->registryId + "_personnel", nearest-
>displayName + " // Personnel Recovery");

src/GameRuntime.cpp:2859: if (scriptTag == "workshop_service") {

src/GameRuntime.cpp:3283: return profile.story.exitedBunker ||

src/GameRuntime.cpp:3284: profile.firstPlayableRoute.surfaceArrivalReached ||

src/GameRuntime.cpp:3287: profile.story.returnedToBase;

src/GameRuntime.cpp:3741: profile.firstPlayableRoute.surfaceArrivalReached =

src/GameRuntime.cpp:3742: profile.firstPlayableRoute.surfaceArrivalReached || profile.story.outerRoadCleared;

src/GameRuntime.cpp:3745: profile.firstPlayableRoute.firstRecoveryNodeActivated =

src/GameRuntime.cpp:3746: profile.firstPlayableRoute.firstRecoveryNodeActivated ||
profile.story.relayRecovered;

src/GameRuntime.cpp:3748: profile.firstPlayableRoute.debriefSummaryViewed || profile.story.returnedToBase;

src/GameRuntime.cpp:3760: profile.story.relayRecovered ||
profile.firstPlayableRoute.firstRecoveryNodeActivated || HasInventoryItem(profile, "relay_reconstruction_data");

src/GameRuntime.cpp:3786: RecoveryStatusLabel(sessionProfile, worldState),

src/GameRuntime.cpp:3861: const bool isWorkshop = object.interaction == InteractionType::Workshop ||
object.scriptTag == "workshop_service";

src/GameRuntime.cpp:3902: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:3966: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:4012: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:4107: const bool scavengerReady =

src/GameRuntime.cpp:4112: if (!scavengerReady) {

src/GameRuntime.cpp:4113: gameState.scavengerTimer = 180.0f;

src/GameRuntime.cpp:4117: const bool engineerAssignedToScavengers = HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support");
```

```
src/GameRuntime.cpp:4118: gameState.scavengerTimer -= dt;

src/GameRuntime.cpp:4119: if (gameState.scavengerTimer > 0.0f) {

src/GameRuntime.cpp:4137: profile.scavengerRunsCompleted += 1;

src/GameRuntime.cpp:4138: gameState.scavengerTimer = (engineerAssignedToScavengers ? 150.0f : 180.0f) /
doctrineScavengerBoost;

src/GameRuntime.cpp:4148: if (worldState == nullptr || !worldState->caravanRouteActive) {

src/GameRuntime.cpp:4149: gameState.caravanTimer = 260.0f;

src/GameRuntime.cpp:4153: const bool caravanReady = CaravanRouteReady(profile);

src/GameRuntime.cpp:4154: if (!caravanReady) {

src/GameRuntime.cpp:4155: gameState.caravanTimer = 260.0f;

src/GameRuntime.cpp:4165: const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer",
"scavenger_support");

src/GameRuntime.cpp:4167: gameState.caravanTimer -= dt;

src/GameRuntime.cpp:4168: if (gameState.caravanTimer > 0.0f) {

src/GameRuntime.cpp:4183: worldState->caravanRunsCompleted += 1;

src/GameRuntime.cpp:4184: gameState.caravanTimer = 260.0f / doctrineBoost;

src/GameRuntime.cpp:4186: ? "Autopilot caravan returned under escort support with reinforced cargo intake."

src/GameRuntime.cpp:4187: : "Autopilot caravan returned from the bunker route with bulk salvage.";

src/GameRuntime.cpp:4192: if (worldState == nullptr || !worldState->droneStationsActive) {

src/GameRuntime.cpp:4193: gameState.droneTimer = 210.0f;

src/GameRuntime.cpp:4198: gameState.droneTimer = 210.0f;

src/GameRuntime.cpp:4208: gameState.droneTimer -= dt;

src/GameRuntime.cpp:4209: if (gameState.droneTimer > 0.0f) {

src/GameRuntime.cpp:4218: worldState->droneRunsCompleted += 1;

src/GameRuntime.cpp:4220: gameState.droneTimer = 210.0f / (doctrineBoost * std::max(1.0f, pylonBoost *
0.95f));

src/GameRuntime.cpp:4221: gameState.lastEvent = "Automated drone sweep completed. Salvage and ether
traces transferred to shelter stores.";

src/GameRuntime.cpp:4226: if (worldState == nullptr || !worldState->tradeNetworkActive) {

src/GameRuntime.cpp:4227: gameState.tradeTimer = 240.0f;

src/GameRuntime.cpp:4231: const bool tradeReady = TradeNetworkReady(profile, *worldState);
```

```
src/GameRuntime.cpp:4232: if (!tradeReady) {  
src/GameRuntime.cpp:4233: gameState.tradeTimer = 240.0f;  
src/GameRuntime.cpp:4244: gameState.tradeTimer -= dt;  
src/GameRuntime.cpp:4245: if (gameState.tradeTimer > 0.0f) {  
src/GameRuntime.cpp:4250: AddInventoryItem(profile, "trade_voucher", vouchers, 0.0f);  
src/GameRuntime.cpp:4251: worldState->tradeCyclesCompleted += 1;  
src/GameRuntime.cpp:4252: gameState.tradeTimer = 240.0f / doctrineBoost;  
src/GameRuntime.cpp:4258: if (worldState == nullptr || !worldState->railFreightActive) {  
src/GameRuntime.cpp:4259: gameState.railTimer = 320.0f;  
src/GameRuntime.cpp:4263: const bool railReady = RailFreightReady(profile, *worldState);  
src/GameRuntime.cpp:4264: if (!railReady) {  
src/GameRuntime.cpp:4265: gameState.railTimer = 320.0f;  
src/GameRuntime.cpp:4275: const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer",  
"scavenger_support");  
src/GameRuntime.cpp:4277: gameState.railTimer -= dt;  
src/GameRuntime.cpp:4278: if (gameState.railTimer > 0.0f) {  
src/GameRuntime.cpp:4287: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);  
src/GameRuntime.cpp:4291: worldState->railRunsCompleted += 1;  
src/GameRuntime.cpp:4293: gameState.railTimer = 320.0f / doctrineBoost;  
src/GameRuntime.cpp:4325: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);  
src/GameRuntime.cpp:4339: if (worldState == nullptr || !worldState->railFortressActive) {  
src/GameRuntime.cpp:4340: gameState.railFortressTimer = 520.0f;  
src/GameRuntime.cpp:4346: gameState.railFortressTimer = 520.0f;  
src/GameRuntime.cpp:4356: gameState.railFortressTimer -= dt;  
src/GameRuntime.cpp:4357: if (gameState.railFortressTimer > 0.0f) {  
src/GameRuntime.cpp:4364: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);  
src/GameRuntime.cpp:4365: worldState->railFortressDeployments += 1;  
src/GameRuntime.cpp:4368: gameState.railFortressTimer = 520.0f / (doctrineBoost * infrastructurePenalty);  
src/GameRuntime.cpp:4374: void UpdateRecoveryFabricator(SessionProfile& profile, GameState& gameState,  
float dt) {
```



```
src/GameRuntime.cpp:4381: const bool fabricatorReady = RecoveryFabricatorReady(profile, *worldState);

src/GameRuntime.cpp:4408: gameState.lastEvent = "Recovery fabricator completed a refinement cycle and
issued fresh field supplies.";

src/GameRuntime.cpp:4411: void UpdateRecoveryMilestones(SessionProfile& profile, GameState& gameState) {

src/GameRuntime.cpp:4417: const int currentTier =
ShelterRecoveryMilestoneTier(ShelterRecoveryIndex(profile));

src/GameRuntime.cpp:4429: gameState.lastEvent = "Shelter Recovery Checkpoint I secured. Emergency stores
unlocked for the crew. " + xpEvent;

src/GameRuntime.cpp:4434: gameState.lastEvent = "Shelter Recovery Checkpoint II secured. Grid and field
service reserves expanded. " + xpEvent;

src/GameRuntime.cpp:4436: AddInventoryItem(profile, "trade_voucher", 2, 0.0f);

src/GameRuntime.cpp:4441: gameState.lastEvent = "Shelter Recovery Checkpoint III secured. Shelter 17 now has
a stable recovery backbone and sustained water reserves. " + xpEvent;

src/GameRuntime.cpp:4472: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4509: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4689: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4702: if (worldState == nullptr || !worldState->serviceBayActive) {

src/GameRuntime.cpp:4703: gameState.serviceBayTimer = 300.0f;

src/GameRuntime.cpp:4707: const bool serviceReady = ServiceBayReady(*worldState);

src/GameRuntime.cpp:4708: if (!serviceReady) {

src/GameRuntime.cpp:4709: gameState.serviceBayTimer = 300.0f;

src/GameRuntime.cpp:4719: gameState.serviceBayTimer -= dt;

src/GameRuntime.cpp:4720: if (gameState.serviceBayTimer > 0.0f) {

src/GameRuntime.cpp:4731: worldState->serviceCyclesCompleted += 1;

src/GameRuntime.cpp:4733: gameState.serviceBayTimer = 300.0f / (doctrineBoost * infrastructurePenalty);

src/GameRuntime.cpp:4734: gameState.lastEvent = "Inner spur service bay completed a repair cycle for BT-72
and the backbone fleet.";

src/GameRuntime.cpp:4751: DoctrineCampRecoveryBoost(profile) *

src/GameRuntime.cpp:4752: WaterRecoveryBoost(profile) *

src/GameRuntime.cpp:4784: profile.story.returnedToBase ||

src/GameRuntime.cpp:4793: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4815: if (routeEventType == "service_call") {
```

```
src/GameRuntime.cpp:4858: if (routeEventType == "service_call") {  
  
src/GameRuntime.cpp:4859: return "ROUTE EVENT OFFERED: Service call tagged near the recovery fringe.  
Stabilize BT-72 and complete one honest repair/service pass before the window closes.";  
  
src/GameRuntime.cpp:4881: if (routeEventType == "service_call") {  
  
src/GameRuntime.cpp:4909: if (routeEventType == "service_call") {  
  
src/GameRuntime.cpp:4932: if (routeEventType == "service_call") {  
  
src/GameRuntime.cpp:4964: if (worldState.activeRouteEventType == "service_call") {  
  
src/GameRuntime.cpp:4967: worldState.serviceCyclesCompleted > 0 ||  
  
src/GameRuntime.cpp:4968: worldState.serviceBayActive) {  
  
src/GameRuntime.cpp:4984: if (worldState.caravanRouteActive ||  
  
src/GameRuntime.cpp:4985: worldState.droneStationsActive ||  
  
src/GameRuntime.cpp:4986: worldState.tradeNetworkActive ||  
  
src/GameRuntime.cpp:4987: worldState.railFreightActive) {  
  
src/GameRuntime.cpp:5012: worldState.railFortressActive ||  
  
src/GameRuntime.cpp:5013: worldState.caravanRouteActive ||  
  
src/GameRuntime.cpp:5014: worldState.droneStationsActive) {  
  
src/GameRuntime.cpp:5021: if (worldState.caravanRouteActive ||  
  
src/GameRuntime.cpp:5022: worldState.droneStationsActive ||  
  
src/GameRuntime.cpp:5023: worldState.tradeNetworkActive ||  
  
src/GameRuntime.cpp:5024: worldState.railFreightActive) {  
  
src/GameRuntime.cpp:5027: if (worldState.serviceBayActive ||  
  
src/GameRuntime.cpp:5035: return worldState.tradeCyclesCompleted > 0 ? 1 : 0;  
  
src/GameRuntime.cpp:5041: if (worldState.activeRouteEventType == "service_call") {  
  
src/GameRuntime.cpp:5055: !worldState.caravanRouteActive &&  
  
src/GameRuntime.cpp:5056: !worldState.droneStationsActive &&  
  
src/GameRuntime.cpp:5057: !worldState.tradeNetworkActive &&  
  
src/GameRuntime.cpp:5058: !worldState.railFreightActive;  
  
src/GameRuntime.cpp:5068: if (routeEventType == "service_call") {  
  
src/GameRuntime.cpp:5071: worldState.serviceCyclesCompleted = std::max(worldState.serviceCyclesCompleted,  
1);
```

```
src/GameRuntime.cpp:5078: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:5097: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:5100: worldState.tradeCyclesCompleted = std::max(worldState.tradeCyclesCompleted, 1);

src/GameRuntime.cpp:5103: worldState.tradeCyclesCompleted = std::max(worldState.tradeCyclesCompleted, 1);

src/GameRuntime.cpp:5111: gameState.lastEvent += " " + CurrentRecoveryHandoffSummary(profile);

src/GameRuntime.cpp:5112: AppendRecoveryBackboneReadabilityHint(gameState.lastEvent, profile);

src/GameRuntime.cpp:5117: if (routeEventType == "service_call") {

src/GameRuntime.cpp:5121: ? "ROUTE EVENT EXPIRED: The service call window closed. BT-72 reserve pressure
climbed and field upkeep drifted upward."

src/GameRuntime.cpp:5122: : "ROUTE EVENT FAILED: The service call slipped. BT-72 reserve pressure climbed
and field upkeep drifted upward.";

src/GameRuntime.cpp:5164: AppendRecoveryBackboneReadabilityHint(gameState.lastEvent, profile);

src/GameRuntime.cpp:5175: !worldState.serviceBayActive &&

src/GameRuntime.cpp:5176: (TankNeedsRepair(profile) || worldState.serviceCyclesCompleted == 0)) {

src/GameRuntime.cpp:5177: candidates.push_back({"service_call", 5 +
RouteEventWeatherWeightBonus("service_call", gameState)});

src/GameRuntime.cpp:5185: if (profile.firstPlayableRoute.firstRecoveryNodeActivated &&

src/GameRuntime.cpp:5198: if ((worldState.caravanRouteActive ||

src/GameRuntime.cpp:5199: worldState.droneStationsActive ||

src/GameRuntime.cpp:5200: worldState.tradeNetworkActive ||

src/GameRuntime.cpp:5201: worldState.railFreightActive) &&

src/GameRuntime.cpp:5202: worldState.serviceCyclesCompleted == 0) {

src/GameRuntime.cpp:5208: (worldState.tradeNetworkActive ||

src/GameRuntime.cpp:5209: worldState.caravanRouteActive ||

src/GameRuntime.cpp:5210: worldState.droneStationsActive ||

src/GameRuntime.cpp:5211: worldState.railFreightActive ||

src/GameRuntime.cpp:5212: worldState.serviceBayActive) &&

src/GameRuntime.cpp:5214: InventoryCount(profile, "trade_voucher") > 0) &&

src/GameRuntime.cpp:5219: candidates.push_back({"service_call", 1 +
RouteEventWeatherWeightBonus("service_call", gameState)});

src/GameRuntime.cpp:5355: bool paidWithVoucher = ConsumeInventoryItem(profile, "trade_voucher", 1);
```

```
src/GameRuntime.cpp:5359: gameState.lastEvent = "Merchant window needs 1 trade voucher or 90 relay credits before the broker will commit.";

src/GameRuntime.cpp:5381: worldState->tradeCyclesCompleted += 1;

src/GameRuntime.cpp:5946: NormalizeGameplayDescriptorTag(nearest->scriptTag) != "workshop_service") {

src/GameRuntime.cpp:5969: gameState.lastEvent = "Cryostasis terminated. Adjacent pods are split open, several berths stand empty, and an emergency baton plus stained service sketch were recovered from the capsule tray.";

src/GameRuntime.cpp:5997: AddCollectedTapelfMissing(profile, "bt72_core_service_001", "BT-72 Starter Core Ledger");

src/GameRuntime.cpp:5998: gameState.lastEvent = "Starter core package recovered from the rack. BT-72 can be powered once the hull and service notes are ready.";

src/GameRuntime.cpp:6011: : "Garage lift survey already mirrored. BT-72 still needs its core and service notes.";

src/GameRuntime.cpp:6017: gameState.lastEvent = "Garage lift survey mirrored. One serviceable hull plate recovered for BT-72 restoration.";

src/GameRuntime.cpp:6082: gameState.lastEvent = "BT-72 hull attached to the hangar crane hook. Move it to the service lift cradle.";

src/GameRuntime.cpp:6086: if (nearest->registryId == "[%bt72_service_lift_0001]") {

src/GameRuntime.cpp:6088: gameState.lastEvent = "BT-72 service lift is waiting for a crane-attached hull.";

src/GameRuntime.cpp:6092: gameState.lastEvent = "BT-72 service lift could not receive the hull. Check crane power, control, and path state.";

src/GameRuntime.cpp:6095: gameState.lastEvent = "BT-72 hull moved to the service lift and locked in the restoration cradle.";

src/GameRuntime.cpp:6110: gameState.lastEvent = "The Pip-Boy/Pip-Pad selection station is still under mechanical card-lock. Pull a bunker access card from the core service racks first.";

src/GameRuntime.cpp:6117: ? "Pip-Boy/Pip-Pad station selected the default Pip-Pad 3500. Press TAB to open the Pip-Pad; the bunker paper trail now makes sense."

src/GameRuntime.cpp:6118: : "Pip-Boy/Pip-Pad station selected the default Pip-Pad 3500. Press TAB to open the Pip-Pad; the bunker paper trail is still incomplete.";

src/GameRuntime.cpp:6179: gameState.lastEvent = "Archive sync complete. One reactor core and one body are still missing, and BT-72 service traces terminate near the garage.";

src/GameRuntime.cpp:6186: if (nearest->registryId == "[%bt72_service_notes_0001]") {

src/GameRuntime.cpp:6188: gameState.lastEvent = "BT-72 service notes require Pip-Pad recovery before holo-records can be mirrored.";

src/GameRuntime.cpp:6192: gameState.lastEvent = "BT-72 service notes remain encrypted until archive personnel records are mirrored.";
```

```
src/GameRuntime.cpp:6196: gameState.lastEvent = "BT-72 service notes already mirrored to Pip-Pad.";

src/GameRuntime.cpp:6200: AddCollectedTapelfMissing(profile, "bt72_service_reel_001", "BT-72 Service Reel");

src/GameRuntime.cpp:6201: gameState.lastEvent = "BT-72 service notes and holo-records mirrored to Pip-Pad.";

src/GameRuntime.cpp:6241: gameState.lastEvent = "BT-72 hull is disabled. Run workshop service before attempting another link.";

src/GameRuntime.cpp:6339: profile.story.exitedBunker = true;

src/GameRuntime.cpp:6396: gameState.lastEvent = "BT-72 is still hot off first contact. Run one service/rest cycle before syncing the node.";

src/GameRuntime.cpp:6402: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

src/GameRuntime.cpp:6416: gameState.lastEvent = "Recovery node synchronized. Shelter 17 now reads a live route toward the city fringe. Return for debrief. " + progressionEvent;

src/GameRuntime.cpp:6422: AppendRecoveryBackboneReadabilityHint(gameState.lastEvent, profile);

src/GameRuntime.cpp:6434: if (!profile.story.returnedToBase) {

src/GameRuntime.cpp:6436: profile.story.returnedToBase = true;

src/GameRuntime.cpp:6452: AppendRecoveryBackboneReadabilityHint(gameState.lastEvent, profile);

src/GameRuntime.cpp:6460: const bool firstSurfaceArrival = profile.story.exitedBunker && !profile.firstPlayableRoute.surfaceArrivalReached;

src/GameRuntime.cpp:6461: if (profile.story.exitedBunker) {

src/GameRuntime.cpp:6462: profile.firstPlayableRoute.surfaceArrivalReached = true;

src/GameRuntime.cpp:6468: const float doctrineCampBoost = DoctrineCampRecoveryBoost(profile) * WaterRecoveryBoost(profile) * PylonGridBoost(profile);

src/GameRuntime.cpp:6478: const char* recoveryStatus = RecoveryStatusLabel(profile, &worldState);

src/GameRuntime.cpp:6521: gameState.lastEvent = "Workshop grid is offline. Use field service or return after first combat for a manual recovery pass.";

src/GameRuntime.cpp:6525: gameState.lastEvent = "Workshop cannot locate BT-72 in the local service grid.";

src/GameRuntime.cpp:6529: gameState.lastEvent = "Bring BT-72 into the workshop service bay first.";

src/GameRuntime.cpp:6581: gameState.lastEvent += " First service halt logged for the route.";

src/GameRuntime.cpp:6696: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service cycles logged", std::min(fieldServiceUses, 3));

src/GameRuntime.cpp:6698: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the recipe.", cyclesRemaining, cyclesRemaining == 1 ? "" : "s");

src/GameRuntime.cpp:6702: if (worldFieldState != nullptr && worldFieldState->tradeNetworkActive) {
```

```
src/GameRuntime.cpp:6705: ImGui::TextDisabled("Trade vouchers: %s", HasInventoryItem(profile,
"trade_voucher") ? "available" : "none");

src/GameRuntime.cpp:6707: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6711: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6716: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6720: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6724: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6728: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6758: ImGui::BulletText("Field Engineer: %s", HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support") ? "assigned" : "not assigned");

src/GameRuntime.cpp:6870: LanlineSessionState sessionState;

src/GameRuntime.cpp:6871: const bool hasSessionState = LoadLanlineSessionState(sessionState);

src/GameRuntime.cpp:6872: const auto& runtimeNotifications = UpdateLanlineRuntimeNotifications(

src/GameRuntime.cpp:6876: ImGui::Text("Lanline - optime");

src/GameRuntime.cpp:6878: ImGui::TextDisabled("No active Lanline session state found. Launch through
BunkerLauncher to seed roster and snapshot data.");

src/GameRuntime.cpp:6891: const auto& diagnostics = CachedLanlineDiagnostics(sessionState,
profile.selectedWorld);

src/GameRuntime.cpp:6898: ImGui::Text("Reserved Slots: %d",
bunker::ReservedLanlineSessionSlots(sessionState));

src/GameRuntime.cpp:6899: ImGui::Text("Pending Slots: %d", bunker::PendingLanlineSessionSlots(sessionState));

src/GameRuntime.cpp:6900: ImGui::Text("Accepted Client Slots: %d",
bunker::AcceptedLanlineSessionSlots(sessionState));

src/GameRuntime.cpp:6901: ImGui::Text("Ready Seats: %d", bunker::ReadyLanlineSessionSlots(sessionState));

src/GameRuntime.cpp:6921: bunker::LanlineSlotStateLabel(playerEntry),

src/GameRuntime.cpp:6922: bunker::LanlineReadyLabel(playerEntry));

src/GameRuntime.cpp:6927: ImGui::TextDisabled("No Lanline events recorded yet.");

src/GameRuntime.cpp:6953: ImGui::TextDisabled("No Lanline voice presence mirrored into this session yet.");

src/GameRuntime.cpp:6967: ImGui::TextDisabled("No runtime Lanline notifications yet.");

src/GameRuntime.cpp:6974: const auto knownSessions = DiscoverLanlineSessionSnapshots();

src/GameRuntime.cpp:6981: const auto& knownDiagnostics = CachedLanlineDiagnostics(knownSession,
profile.selectedWorld);
```

```
src/GameRuntime.cpp:6990: bunker::OccupiedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:6991: bunker::MaxLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:6998: if (bunker::IsJoinableLanlineSession(knownSession)) {
src/GameRuntime.cpp:7000: bunker::AvailableLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7001: bunker::PendingLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7002: bunker::ReservedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7003: bunker::AcceptedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7004: bunker::ReadyLanlineSessionSlots(knownSession));

src/GameRuntime.cpp:7009: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and snapshot
trail even without Steam/Xbox auth.");

src/GameRuntime.cpp:7015: static LanlineServicesState lanlineServices =
MakeDefaultLanlineServicesState(std::time(nullptr));

src/GameRuntime.cpp:7017: LanlineServicesSave lanlineSave{};

src/GameRuntime.cpp:7018: if (LoadLanlineServicesSave(DefaultLanlineServicesSavePath(), lanlineSave)) {

src/GameRuntime.cpp:7019: lanlineServices = MakeLanlineServicesStateFromSave(lanlineSave,
std::time(nullptr));

src/GameRuntime.cpp:7021: ApplyLanlineServicesProfileSnapshot(lanlineServices, profile.lanlineServices);

src/GameRuntime.cpp:7025: LanlineSessionState sessionState;

src/GameRuntime.cpp:7026: const bool hasSessionState = LoadLanlineSessionState(sessionState);

src/GameRuntime.cpp:7027: const auto servicesUnlock = BuildServicesUnlockState(profile,
currentWorldFieldState);

src/GameRuntime.cpp:7028: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState :
nullptr, servicesUnlock);

src/GameRuntime.cpp:7029: gameState.supportTerminalNearby = IsNearTaggedObject(world, player.x, player.y,
"lanline_service_hub", 4.0f);

src/GameRuntime.cpp:7030: gameState.tankServiceNearby = IsNearTaggedObject(world, player.x, player.y,
"tank_service", 4.0f);

src/GameRuntime.cpp:7031: gameState.medicalSupportNearby = IsNearTaggedObject(world, player.x, player.y,
"medical_support", 4.0f);

src/GameRuntime.cpp:7034: ImGui::Text("Lanline Services");

src/GameRuntime.cpp:7036: ImGui::BulletText("Tank service nearby: %s", gameState.tankServiceNearby ? "yes" :
"no");
```

```
src/GameRuntime.cpp:7037: ImGui::BulletText("Medical support nearby: %s", gameState.medicalSupportNearby ? "yes" : "no");

src/GameRuntime.cpp:7045: DrawLanlineServicesPanel(lanlineServices, servicesUnlock, static_cast<std::int64_t>(std::time(nullptr)));

src/GameRuntime.cpp:7046: SyncLanlineServicesProfileSnapshot(profile.lanlineServices, lanlineServices);

src/GameRuntime.cpp:7047: SaveLanlineServicesSave(BuildLanlineServicesSave(lanlineServices), DefaultLanlineServicesSavePath());

src/GameRuntime.cpp:7070: ImGui::Begin("PIP-PAD // Recovery Shell", nullptr, ImGuiWindowFlags_NoCollapse);

src/GameRuntime.cpp:7085: const float recoveryIndex = ShelterRecoveryIndex(profile);

src/GameRuntime.cpp:7087: const bool stableRecoveryBackbone = worldFieldState != nullptr &&

src/GameRuntime.cpp:7088: IsStableRecoveryBackbone(profile, *worldFieldState);

src/GameRuntime.cpp:7090: const auto backboneStatus = CurrentRecoveryBackboneStatus(profile);

src/GameRuntime.cpp:7097: ImGui::TextWrapped("Recovery Handoff: %s", CurrentRecoveryHandoffSummary(profile).c_str());

src/GameRuntime.cpp:7111: ImGui::TextWrapped("One discreet broker exchange is open on the recovery fringe. Spend 1 trade voucher or 90 relay credits before the trace folds.");

src/GameRuntime.cpp:7130: ImGui::Text("Recovery Support Stock");

src/GameRuntime.cpp:7131: ImGui::BulletText("Trade Vouchers: %d", InventoryCount(profile, "trade_voucher"));

src/GameRuntime.cpp:7136: ImGui::Text("Shelter Recovery Index");

src/GameRuntime.cpp:7137: ImGui::BulletText("State: %s", ShelterRecoveryBand(recoveryIndex));

src/GameRuntime.cpp:7140: const int recoveryTier = ShelterRecoveryMilestoneTier(recoveryIndex);

src/GameRuntime.cpp:7141: ImGui::BulletText("Milestone: %s", ShelterRecoveryMilestoneLabel(recoveryTier));

src/GameRuntime.cpp:7143: ImGui::BulletText("Recovery Backbone: %s", stableRecoveryBackbone ? "stable" : "still assembling");

src/GameRuntime.cpp:7183: const char* buttonLabel = specialist.assignment == "scavenger_support"

src/GameRuntime.cpp:7187: specialist.assignment = specialist.assignment == "scavenger_support" ? "workshop" : "scavenger_support";

src/GameRuntime.cpp:7189: ? specialist.displayName + " reassigned to workshop support. Heavy service cycles now get the engineer boost."

src/GameRuntime.cpp:7190: : specialist.displayName + " reassigned to scavenger support. Field service and salvage teams now get the engineer boost.";

src/GameRuntime.cpp:7232: ? "Relay and tower sync are suppressing bloom growth. Camp and scavengers can gradually reclaim the route."
```



```
src/GameRuntime.cpp:7238: ImGui::TextWrapped("Offline grids slowly wear down workshop, camps, and route  
service. Restored power stabilizes structures over time.");  
  
src/GameRuntime.cpp:7245: const bool scavengerReady =  
  
src/GameRuntime.cpp:7251: if (scavengerReady) {  
  
src/GameRuntime.cpp:7253: ImGui::BulletText("Next return: %.0fs", gameState.scavengerTimer);  
  
src/GameRuntime.cpp:7254: ImGui::BulletText("Completed runs: %d", profile.scavengerRunsCompleted);  
  
src/GameRuntime.cpp:7255: ImGui::BulletText("Engineer support: %s", HasAssignedSpecialistRole(profile,  
"engineer", "scavenger_support") ? "scavenger_support" : "workshop");  
  
src/GameRuntime.cpp:7263: const bool caravanReady = CaravanRouteReady(profile);  
  
src/GameRuntime.cpp:7265: worldFieldState->caravanRouteActive  
  
src/GameRuntime.cpp:7266: ? (caravanReady ? "active" : "blocked")  
  
src/GameRuntime.cpp:7268: ImGui::BulletText("Completed runs: %d", worldFieldState->caravanRunsCompleted);  
  
src/GameRuntime.cpp:7269: if (worldFieldState->caravanRouteActive) {  
  
src/GameRuntime.cpp:7270: ImGui::BulletText("Next return: %.0fs", gameState.caravanTimer);  
  
src/GameRuntime.cpp:7272: if (ImGui::SmallButton(worldFieldState->caravanRouteActive ? "Disable Caravan  
Route" : "Enable Caravan Route")) {  
  
src/GameRuntime.cpp:7273: if (!worldFieldState->caravanRouteActive && !caravanReady) {  
  
src/GameRuntime.cpp:7274: gameState.lastEvent = "Autopilot caravan needs a field checkpoint, a cleared outer  
route, and a live regional grid.";  
  
src/GameRuntime.cpp:7276: worldFieldState->caravanRouteActive = !worldFieldState->caravanRouteActive;  
  
src/GameRuntime.cpp:7277: gameState.caravanTimer = 260.0f;  
  
src/GameRuntime.cpp:7278: gameState.lastEvent = worldFieldState->caravanRouteActive  
  
src/GameRuntime.cpp:7279: ? "Autopilot caravan route activated between shelter and forward camp."  
  
src/GameRuntime.cpp:7280: : "Autopilot caravan route suspended.";  
  
src/GameRuntime.cpp:7284: if (worldFieldState->caravanRouteActive && !caravanReady) {  
  
src/GameRuntime.cpp:7293: const bool tradeReady = TradeNetworkReady(profile, *worldFieldState);  
  
src/GameRuntime.cpp:7295: worldFieldState->tradeNetworkActive  
  
src/GameRuntime.cpp:7296: ? (tradeReady ? "active" : "blocked")  
  
src/GameRuntime.cpp:7298: ImGui::BulletText("Completed cycles: %d", worldFieldState->  
>tradeCyclesCompleted);  
  
src/GameRuntime.cpp:7299: ImGui::BulletText("Trade vouchers on hand: %d", InventoryCount(profile,  
"trade_voucher"));
```

```
src/GameRuntime.cpp:7300: if (worldFieldState->tradeNetworkActive) {  
src/GameRuntime.cpp:7301: ImGui::BulletText("Next convoy sync: %.0fs", gameState.tradeTimer);  
src/GameRuntime.cpp:7303: if (ImGui::SmallButton(worldFieldState->tradeNetworkActive ? "Disable Trade  
Network" : "Enable Trade Network")) {  
src/GameRuntime.cpp:7304: if (!worldFieldState->tradeNetworkActive && !tradeReady) {  
src/GameRuntime.cpp:7305: gameState.lastEvent = "Trade network needs a field checkpoint, a live grid, and  
either caravan or drone logistics.";  
src/GameRuntime.cpp:7307: worldFieldState->tradeNetworkActive = !worldFieldState->tradeNetworkActive;  
src/GameRuntime.cpp:7308: gameState.tradeTimer = 240.0f;  
src/GameRuntime.cpp:7309: gameState.lastEvent = worldFieldState->tradeNetworkActive  
src/GameRuntime.cpp:7314: ImGui::TextWrapped("Requires a field checkpoint, a live grid, and either caravan or  
drone logistics. Generates trade vouchers for supply exchange.");  
src/GameRuntime.cpp:7315: if (worldFieldState->tradeNetworkActive && !tradeReady) {  
src/GameRuntime.cpp:7322: const bool railReady = RailFreightReady(profile, *worldFieldState);  
src/GameRuntime.cpp:7324: worldFieldState->railFreightActive  
src/GameRuntime.cpp:7325: ? (railReady ? "active" : "blocked")  
src/GameRuntime.cpp:7327: ImGui::BulletText("Completed runs: %d", worldFieldState->railRunsCompleted);  
src/GameRuntime.cpp:7328: if (worldFieldState->railFreightActive) {  
src/GameRuntime.cpp:7329: ImGui::BulletText("Next freight return: %.0fs", gameState.railTimer);  
src/GameRuntime.cpp:7331: if (ImGui::SmallButton(worldFieldState->railFreightActive ? "Disable Rail Freight" :  
"Enable Rail Freight")) {  
src/GameRuntime.cpp:7332: if (!worldFieldState->railFreightActive && !railReady) {  
src/GameRuntime.cpp:7335: worldFieldState->railFreightActive = !worldFieldState->railFreightActive;  
src/GameRuntime.cpp:7336: gameState.railTimer = 320.0f;  
src/GameRuntime.cpp:7337: gameState.lastEvent = worldFieldState->railFreightActive  
src/GameRuntime.cpp:7343: if (worldFieldState->railFreightActive && !railReady) {  
src/GameRuntime.cpp:7361: gameState.lastEvent = "Orbital uplink needs a live grid, active rail freight, two  
restored pylons, and trade or drone support.";  
src/GameRuntime.cpp:7370: ImGui::TextWrapped("Requires live grid, restored pylons, active rail freight, and a  
functioning logistics backbone. Grants orbital scans and route intelligence.");  
src/GameRuntime.cpp:7372: ImGui::TextDisabled("Orbital uplink is enabled but waiting on rail freight, pylon  
coverage, grid stability, or logistics support.");
```

```
src/GameRuntime.cpp:7380: worldFieldState->railFortressActive

src/GameRuntime.cpp:7383: ImGui::BulletText("Patrol cycles: %d", worldFieldState->railFortressDeployments);

src/GameRuntime.cpp:7384: if (worldFieldState->railFortressActive) {

src/GameRuntime.cpp:7385: ImGui::BulletText("Next return: %.0fs", gameState.railFortressTimer);

src/GameRuntime.cpp:7387: if (ImGui::SmallButton(worldFieldState->railFortressActive ? "Recall Rail Fortress" :
"Deploy Rail Fortress")) {

src/GameRuntime.cpp:7388: if (!worldFieldState->railFortressActive && !fortressReady) {

src/GameRuntime.cpp:7389: gameState.lastEvent = "Rail fortress needs active rail freight, an orbital uplink, a
healthy grid, and two restored pylons.";

src/GameRuntime.cpp:7391: worldFieldState->railFortressActive = !worldFieldState->railFortressActive;

src/GameRuntime.cpp:7392: gameState.railFortressTimer = 520.0f;

src/GameRuntime.cpp:7393: gameState.lastEvent = worldFieldState->railFortressActive

src/GameRuntime.cpp:7398: ImGui::TextWrapped("Requires active rail freight, orbital uplink and a healthy grid.
Defense doctrine improves armored patrol tempo.");

src/GameRuntime.cpp:7399: if (worldFieldState->railFortressActive && !fortressReady) {

src/GameRuntime.cpp:7404: ImGui::Text("Recovery Fabricator");

src/GameRuntime.cpp:7406: const bool fabricatorReady = RecoveryFabricatorReady(profile, *worldFieldState);

src/GameRuntime.cpp:7417: gameState.lastEvent = "Recovery fabricator needs a live grid, logistics input, and
either trade or orbital support.";

src/GameRuntime.cpp:7422: ? "Recovery fabricator activated for Shelter 17."

src/GameRuntime.cpp:7423: : "Recovery fabricator returned to standby.";

src/GameRuntime.cpp:7426: ImGui::TextWrapped("Requires live grid, logistics input, and either trade or orbital
support. Converts salvage into field supplies and recovery stock.");

src/GameRuntime.cpp:7428: ImGui::TextDisabled("Recovery fabricator is enabled but waiting on grid stability,
inbound logistics, or convoy/uplink support.");

src/GameRuntime.cpp:7449: ImGui::TextWrapped("Requires the unlocked industrial gate, orbital uplink, and
trade support. Survey sweeps feed inner spur intel back to Shelter 17.");

src/GameRuntime.cpp:7451: ImGui::TextDisabled("Survey beacon is enabled but waiting on gate access, orbital
uplink, or trade support.");

src/GameRuntime.cpp:7466: ImGui::TextWrapped("Requires gate access, survey coverage, and trade support.
Outpost supply runs reinforce the inner spur foothold.");

src/GameRuntime.cpp:7468: ImGui::TextDisabled("Inner spur outpost is enabled but waiting on gate access,
survey coverage, or trade support.");
```

```
src/GameRuntime.cpp:7500: ImGui::TextWrapped("Requires assembly, outpost support, and rail security. This is  
the first heavy fabrication line beyond the gate.");  
  
src/GameRuntime.cpp:7559: const bool serviceReady = ServiceBayReady(*worldFieldState);  
  
src/GameRuntime.cpp:7561: worldFieldState->serviceBayActive  
  
src/GameRuntime.cpp:7562: ? (serviceReady ? "active" : "blocked")  
  
src/GameRuntime.cpp:7564: ImGui::BulletText("Service cycles: %d", worldFieldState->serviceCyclesCompleted);  
  
src/GameRuntime.cpp:7565: if (worldFieldState->serviceBayActive) {  
  
src/GameRuntime.cpp:7566: ImGui::BulletText("Next service: %.0fs", gameState.serviceBayTimer);  
  
src/GameRuntime.cpp:7569: if (worldFieldState->serviceBayActive && !serviceReady) {  
  
src/GameRuntime.cpp:7585: ImGui::TextWrapped("Requires service bay, relay support, and the recovery  
fabricator. Stabilizes camp recovery and produces clean water.");  
  
src/GameRuntime.cpp:7587: ImGui::TextDisabled("Water reclaimer is enabled but waiting on service bay  
support, relay return flow, or fabricator output.");  
  
src/GameRuntime.cpp:7593: const bool droneReady = IsRegionalGridOnline(profile);  
  
src/GameRuntime.cpp:7595: worldFieldState->droneStationsActive  
  
src/GameRuntime.cpp:7596: ? (droneReady ? "active" : "blocked")  
  
src/GameRuntime.cpp:7598: ImGui::BulletText("Completed runs: %d", worldFieldState->droneRunsCompleted);  
  
src/GameRuntime.cpp:7599: if (worldFieldState->droneStationsActive) {  
  
src/GameRuntime.cpp:7600: ImGui::BulletText("Next sweep: %.0fs", gameState.droneTimer);  
  
src/GameRuntime.cpp:7602: ImGui::TextWrapped("Requires a powered drone station in the world. Industry  
doctrine and restored pylons improve drone efficiency.");  
  
src/GameRuntime.cpp:7603: if (worldFieldState->droneStationsActive && !droneReady) {  
  
src/GameRuntime.cpp:7608: ImGui::Text("Recovery Route");  
  
src/GameRuntime.cpp:7618: stableRecoveryBackbone  
  
src/GameRuntimePipPad.cpp:9:#include "../include/LanlineLobbyLogic.hpp"  
  
src/GameRuntimePipPad.cpp:10:#include "../include/LanlineServices.hpp"  
  
src/GameRuntimePipPad.cpp:17:const LanlineDiagnostics& CachedLanlineDiagnostics(const  
LanlineSessionState& session, std::string_view runtimeWorldName) {  
  
src/GameRuntimePipPad.cpp:21: static LanlineDiagnostics cachedDiagnostics;  
  
src/GameRuntimePipPad.cpp:27: cachedDiagnostics = ProbeLanlineHost(session, normalizedWorldName);  
  
src/GameRuntimePipPad.cpp:35:const std::vector<std::string>& UpdateLanlineRuntimeNotifications(const  
LanlineSessionState* session,
```

```
src/GameRuntimePipPad.cpp:39: static LanlineSessionState previousSession;

src/GameRuntimePipPad.cpp:55: pushNotification("Lanline session link lost. Waiting for refreshed
launcher/session state.");

src/GameRuntimePipPad.cpp:67: pushNotification("Lanline session linked: " + session->sessionId + " on " +
normalizedSessionWorld + ".");

src/GameRuntimePipPad.cpp:69: pushNotification("Lanline world mismatch: runtime is on " +
normalizedRuntimeWorld +

src/GameRuntimePipPad.cpp:79: pushNotification("Lanline session switched to " + session->sessionId + " (" +
session->mode + ").");

src/GameRuntimePipPad.cpp:82: pushNotification("Lanline mode changed to " + session->mode + ".");

src/GameRuntimePipPad.cpp:85: pushNotification("Lanline lifecycle advanced to " + session->lifecycleStage + ".");

src/GameRuntimePipPad.cpp:89: ? "Lanline pending peer queue cleared."

src/GameRuntimePipPad.cpp:90: : "Lanline pending peer updated: " + session->pendingPeer + ".");

src/GameRuntimePipPad.cpp:94: ? "Lanline connected peer link cleared."

src/GameRuntimePipPad.cpp:95: : "Lanline peer link confirmed with " + session->connectedPeer + ".");

src/GameRuntimePipPad.cpp:98: pushNotification("Lanline session world updated: " + previousNormalizedWorld
+ " -> " + normalizedSessionWorld + ".");

src/GameRuntimePipPad.cpp:104: ? "Lanline session world now matches the active runtime world."

src/GameRuntimePipPad.cpp:105: : "Lanline session world drifted away from the active runtime world.");

src/GameRuntimePipPad.cpp:110: [&](const LanlinePlayerEntry& previousEntry) {

src/GameRuntimePipPad.cpp:114: pushNotification(playerEntry.displayName + " joined Lanline roster as " +
playerEntry.role + ".");

src/GameRuntimePipPad.cpp:118: pushNotification(playerEntry.displayName + " moved from " + previousIt->role
+ " to " + playerEntry.role + " in Lanline lobby.");

src/GameRuntimePipPad.cpp:122: (playerEntry.online ? " came online in Lanline." : " went offline in Lanline.");

src/GameRuntimePipPad.cpp:124: if (IsLanlineReadyEligibleSlot(playerEntry) && playerEntry.ready != previousIt-
>ready) {

src/GameRuntimePipPad.cpp:126: (playerEntry.ready ? " is now ready for Lanline deployment." : " is no longer
ready for Lanline deployment.");

src/GameRuntimePipPad.cpp:132: [&](const LanlinePlayerEntry& currentEntry) {

src/GameRuntimePipPad.cpp:136: pushNotification(previousEntry.displayName + " left the Lanline roster.");

src/GameRuntimePipPad.cpp:150: [&](const LanlineVoicePresence& previousVoice) {

src/GameRuntimePipPad.cpp:170: [&](const LanlineVoicePresence& currentVoice) {
```

```
src/GameRuntimePipPad.cpp:183:const char* JoinabilityLabel(const LanlineSessionState& session) {  
  
src/GameRuntimePipPad.cpp:303:void TrustLanlinePeerAsBt72Gunner(SessionProfile& profile, const  
LanlineSessionState* sessionState, GameState& gameState) {  
  
src/GameRuntimePipPad.cpp:309: gameState.lastEvent = "No Lanline session state found. Launch through  
BunkerLauncher to seed a trusted gunner.";  
  
src/GameRuntimePipPad.cpp:327: gameState.lastEvent = "No Lanline peer is currently available to trust as BT-72  
gunner.";  
  
src/GameRuntimePipPad.cpp:385: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";  
  
src/GameRuntimePipPad.cpp:389: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the  
gunner station.";  
  
src/GameRuntimePipPad.cpp:393: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";  
  
src/GameRuntimePipPad.cpp:398: gameState.lastEvent = "BT-72 is moving too fast for field service. Stop the hull  
first.";  
  
src/GameRuntimePipPad.cpp:402: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable  
condition.";  
  
src/GameRuntimePipPad.cpp:406: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a  
repair patch.";  
  
src/GameRuntimePipPad.cpp:410: const bool hasEngineer = HasAssignedSpecialistRole(profile, "engineer",  
"scavenger_support");  
  
src/GameRuntimePipPad.cpp:425: ? "Field service rack cycled. Scavenger-side engineer tuning improved the  
repair pass."  
  
src/GameRuntimePipPad.cpp:426: : "Field service rack cycled. BT-72 patched and partially resupplied in the  
field.";  
  
src/GameRuntimePipPad.cpp:429: gameState.lastEvent += " First service halt logged for the route.";  
  
src/GameRuntimePipPad.cpp:534: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service  
cycles logged", std::min(fieldServiceUses, 3));  
  
src/GameRuntimePipPad.cpp:536: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the  
recipe.", cyclesRemaining, cyclesRemaining == 1 ? "" : "s");  
  
src/GameRuntimePipPad.cpp:540: if (worldFieldState != nullptr && worldFieldState->tradeNetworkActive) {  
  
src/GameRuntimePipPad.cpp:543: ImGui::TextDisabled("Trade vouchers: %s", HasInventoryItem(profile,  
"trade_voucher") ? "available" : "none");  
  
src/GameRuntimePipPad.cpp:545: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {  
  
src/GameRuntimePipPad.cpp:549: gameState.lastEvent = "No trade voucher available.";  
  
src/GameRuntimePipPad.cpp:554: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {
```

```
src/GameRuntimePipPad.cpp:558: gameState.lastEvent = "No trade voucher available.";
src/GameRuntimePipPad.cpp:562: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {
src/GameRuntimePipPad.cpp:566: gameState.lastEvent = "No trade voucher available.";
src/GameRuntimePipPad.cpp:574: LanlineSessionState sessionState;
src/GameRuntimePipPad.cpp:575: const bool hasSessionState = LoadLanlineSessionState(sessionState);
src/GameRuntimePipPad.cpp:613: ImGui::BulletText("Field Engineer: %s", HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support") ? "assigned" : "not assigned");
src/GameRuntimePipPad.cpp:625: if (ImGui::Button("Trust Lanline Peer")) {
src/GameRuntimePipPad.cpp:626: TrustLanlinePeerAsBt72Gunner(profile, hasSessionState ? &sessionState :
nullptr, gameState);
src/GameRuntimePipPad.cpp:757: LanlineSessionState sessionState;
src/GameRuntimePipPad.cpp:758: const bool hasSessionState = LoadLanlineSessionState(sessionState);
src/GameRuntimePipPad.cpp:759: const auto& runtimeNotifications = UpdateLanlineRuntimeNotifications(
src/GameRuntimePipPad.cpp:763: ImGui::Text("Lanline - optime");
src/GameRuntimePipPad.cpp:765: ImGui::TextDisabled("No active Lanline session state found. Launch through
BunkerLauncher to seed roster and snapshot data.");
src/GameRuntimePipPad.cpp:778: const auto& diagnostics = CachedLanlineDiagnostics(sessionState,
profile.selectedWorld);
src/GameRuntimePipPad.cpp:785: ImGui::Text("Reserved Slots: %d", ReservedLanlineSessionSlots(sessionState));
src/GameRuntimePipPad.cpp:786: ImGui::Text("Pending Slots: %d", PendingLanlineSessionSlots(sessionState));
src/GameRuntimePipPad.cpp:787: ImGui::Text("Accepted Client Slots: %d",
AcceptedLanlineSessionSlots(sessionState));
src/GameRuntimePipPad.cpp:788: ImGui::Text("Ready Seats: %d", ReadyLanlineSessionSlots(sessionState));
src/GameRuntimePipPad.cpp:817: LanlineSlotStateLabel(playerEntry),
src/GameRuntimePipPad.cpp:818: LanlineReadyLabel(playerEntry));
src/GameRuntimePipPad.cpp:823: ImGui::TextDisabled("No Lanline events recorded yet.");
src/GameRuntimePipPad.cpp:849: ImGui::TextDisabled("No Lanline voice presence mirrored into this session
yet.");
src/GameRuntimePipPad.cpp:863: ImGui::TextDisabled("No runtime Lanline notifications yet.");
src/GameRuntimePipPad.cpp:870: const auto knownSessions = DiscoverLanlineSessionSnapshots();
src/GameRuntimePipPad.cpp:877: const auto& knownDiagnostics = CachedLanlineDiagnostics(knownSession,
profile.selectedWorld);
```

```
src/GameRuntimePipPad.cpp:886: OccupiedLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:887: MaxLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:894: if (IsJoinableLanlineSession(knownSession)) {
src/GameRuntimePipPad.cpp:896: AvailableLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:897: PendingLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:898: ReservedLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:899: AcceptedLanlineSessionSlots(knownSession),
src/GameRuntimePipPad.cpp:900: ReadyLanlineSessionSlots(knownSession));

src/GameRuntimePipPad.cpp:905: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and
snapshot trail even without Steam/Xbox auth.");

src/GameRuntimePipPad.cpp:911: static LanlineServicesState lanlineServices =
MakeDefaultLanlineServicesState(std::time(nullptr));

src/GameRuntimePipPad.cpp:913: LanlineServicesSave lanlineSave{};

src/GameRuntimePipPad.cpp:914: if (LoadLanlineServicesSave(DefaultLanlineServicesSavePath(), lanlineSave)) {
src/GameRuntimePipPad.cpp:915: lanlineServices = MakeLanlineServicesStateFromSave(lanlineSave,
std::time(nullptr));

src/GameRuntimePipPad.cpp:920: ApplyLanlineServicesProfileSnapshot(lanlineServices, profile.lanlineServices);

src/GameRuntimePipPad.cpp:921: LanlineSessionState sessionState;

src/GameRuntimePipPad.cpp:922: const bool hasSessionState = LoadLanlineSessionState(sessionState);

src/GameRuntimePipPad.cpp:924: const auto servicesUnlock = BuildServicesUnlockState(profile,
currentWorldFieldState);

src/GameRuntimePipPad.cpp:925: const auto backboneStatus = CurrentRecoveryBackboneStatus(profile);

src/GameRuntimePipPad.cpp:926: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState
: nullptr, servicesUnlock);

src/GameRuntimePipPad.cpp:927: AdvanceLanlineSupportOrders(lanlineServices, nowUnix);

src/GameRuntimePipPad.cpp:928: gameState.supportTerminalNearby = IsNearTaggedObject(world, player.x,
player.y, "lanline_service_hub", 4.0f);

src/GameRuntimePipPad.cpp:929: gameState.tankServiceNearby = IsNearTaggedObject(world, player.x, player.y,
"tank_service", 4.0f);

src/GameRuntimePipPad.cpp:930: gameState.medicalSupportNearby = IsNearTaggedObject(world, player.x,
player.y, "medical_support", 4.0f);

src/GameRuntimePipPad.cpp:933: ImGui::Text("Lanline Services");
```



```
src/GameRuntimePipPad.cpp:935: ImGui::BulletText("Tank service nearby: %s", gameState.tankServiceNearby ?
"yes" : "no");

src/GameRuntimePipPad.cpp:936: ImGui::BulletText("Medical support nearby: %s",
gameState.medicalSupportNearby ? "yes" : "no");

src/GameRuntimePipPad.cpp:945: servicesUnlock.localRelayAvailable &&

src/GameRuntimePipPad.cpp:946: (gameState.supportTerminalNearby ||
profile.lanlineServices.serviceHubKnown);

src/GameRuntimePipPad.cpp:958: ImGui::TextDisabled("Delivered orders can be claimed after a Lanline service
hub handshake.");

src/GameRuntimePipPad.cpp:972: ImGui::TextWrapped("A rare merchant trace is open. Broker one exchange
from the services layer before the window closes.");

src/GameRuntimePipPad.cpp:977: DrawLanlineServicesPanel(lanlineServices, servicesUnlock, nowUnix);

src/GameRuntimePipPad.cpp:978: SyncLanlineServicesSessionProfile(profile, lanlineServices);

src/GameRuntimePipPad.cpp:979: SaveLanlineServicesSave(BuildLanlineServicesSave(lanlineServices),
DefaultLanlineServicesSavePath());

src/GameplayDescriptorRegistry.cpp:13: {"workshop_service", "Workshop Service", InteractionType::Workshop,
ObjectCategory::Terminal, {}, false},

src/GameplayDescriptorRegistry.cpp:16: {"drone_station", "Drone Station", InteractionType::Terminal,
ObjectCategory::Terminal, "salvage_sweep", false},

src/GameplayDescriptorRegistry.cpp:17: {"rail_depot", "Rail Depot", InteractionType::Terminal,
ObjectCategory::Terminal, "industrial_spur_alpha", false},

src/GameplayDescriptorRegistry.cpp:19: {"rail_fortress_hub", "Rail Fortress Hub", InteractionType::Terminal,
ObjectCategory::Terminal, "magistral_anchor_alpha", false},

src/GameplayDescriptorRegistry.cpp:20: {"recovery_fabricator", "Recovery Fabricator", InteractionType::Terminal,
ObjectCategory::Terminal, "shelter17_refinery", false},

src/GameplayDescriptorRegistry.cpp:29: {"service_bay", "Service Bay", InteractionType::Terminal,
ObjectCategory::Terminal, "inner_spur_service", false},

src/GameplayDescriptorRegistry.cpp:31: {"lanline_service_hub", "Lanline Service Hub",
InteractionType::Terminal, ObjectCategory::Terminal, "shelter17_services", false},

src/GameplayDescriptorRegistry.cpp:32: {"tank_service", "Tank Service", InteractionType::Workshop,
ObjectCategory::Hangar, "bt72_service", false},

src/GameplayDescriptorRegistry.cpp:33: {"medical_support", "Medical Support", InteractionType::Terminal,
ObjectCategory::Terminal, "field_medical", false},

src/GameplayDescriptorRegistry.cpp:41: {"workshop_field_service", "workshop_service"},
```

```
src/GameplayDescriptorRegistry.cpp:42: {"rail_freight", "rail_depot"},
src/GameplayDescriptorRegistry.cpp:43: {"rail_fortress", "rail_fortress_hub"},
src/LanlineServices.cpp:1:#include "../include/LanlineServices.hpp"
src/LanlineServices.cpp:32: case StoreCurrency::InGame: return "Recovery Scrip";
src/LanlineServices.cpp:98:std::string CurrentLanlineTimestamp() {
src/LanlineServices.cpp:176:const SupportCatalogItem* FindSupportCatalogItem(const LanlineServicesState&
state, std::string_view itemId) {
src/LanlineServices.cpp:248:ChatChannel* FindChatChannel(LanlineServicesState& state, const std::string&
channelId) {
src/LanlineServices.cpp:259: {"session", "Session", {"Lanline", "Session relay ready. Presence log restored.",
"boot"}},
src/LanlineServices.cpp:260: {"relay", "Relay", {"Shelter 17 Relay", "Tower sync can reopen broader relay
services.", "boot"}},
src/LanlineServices.cpp:265:FriendEntry* FindFriendEntry(LanlineServicesState& state, const std::string& handle)
{
src/LanlineServices.cpp:275: LanlineSessionState sessionState;
src/LanlineServices.cpp:276: if (!LoadLanlineSessionState(sessionState)) {
src/LanlineServices.cpp:278: *errorText = "No active Lanline session is available for relay transport.";
src/LanlineServices.cpp:283: sessionState.updatedAt = CurrentLanlineTimestamp();
src/LanlineServices.cpp:288: sessionState.relayMessages.begin() +
static_cast<std::vector<LanlineRelayMessage>::difference_type>(sessionState.relayMessages.size() - 32));
src/LanlineServices.cpp:291: if (!SaveLanlineSessionState(sessionState) ||
src/LanlineServices.cpp:292: !SaveLanlineSessionState(sessionState,
LanlineSessionSnapshotPath(sessionState.sessionId))) {
src/LanlineServices.cpp:294: *errorText = "Relay transport failed to write Lanline session state.";
src/LanlineServices.cpp:307: LanlineSessionState sessionState;
src/LanlineServices.cpp:308: if (!LoadLanlineSessionState(sessionState)) {
src/LanlineServices.cpp:310: *errorText = "No active Lanline session is available for voice relay.";
src/LanlineServices.cpp:315: sessionState.updatedAt = CurrentLanlineTimestamp();
src/LanlineServices.cpp:319: [&](const LanlineVoicePresence& entry) { return entry.handle == handle; });
src/LanlineServices.cpp:333: sessionState.voicePresence.begin() +
static_cast<std::vector<LanlineVoicePresence>::difference_type>(sessionState.voicePresence.size() - 16));
```

```
src/LanlineServices.cpp:336: if (!SaveLanlineSessionState(sessionState) ||  
src/LanlineServices.cpp:337: !SaveLanlineSessionState(sessionState,  
LanlineSessionSnapshotPath(sessionState.sessionId))) {  
src/LanlineServices.cpp:339: *errorText = "Voice relay failed to write Lanline session state.";  
src/LanlineServices.cpp:347: void DrawLanlineServicesLockedScreen(const ServicesUnlockState& unlockState) {  
src/LanlineServices.cpp:348: ImGui::Text("Lanline Services: offline");  
src/LanlineServices.cpp:351: ImGui::TextWrapped("Relay access unavailable. Restore and synchronize the first  
tower to open Lanline Services.");  
src/LanlineServices.cpp:355: ImGui::TextWrapped("Relay node detected, but services are still restricted while  
Shelter 17 stabilizes.");  
src/LanlineServices.cpp:359: if (!unlockState.serviceBayActive) {  
src/LanlineServices.cpp:360: ImGui::BulletText("Requirement: service bay operational");  
src/LanlineServices.cpp:375: void DrawFriendsTab(LanlineServicesState& state, char* searchBuffer, std::size_t  
searchBufferSize) {  
src/LanlineServices.cpp:403: void DrawChatTab(LanlineServicesState& state, int& selectedChannelIndex, char*  
messageBuffer, std::size_t messageBufferSize) {  
src/LanlineServices.cpp:421: ImGui::BeginChild("LanlineChatHistory", ImVec2(0.0f, 180.0f), true);  
src/LanlineServices.cpp:443: ImGui::TextDisabled("Relay chat rides on the active Lanline session state and  
snapshot mirror.");  
src/LanlineServices.cpp:447: void DrawVoiceTab(LanlineServicesState& state) {  
src/LanlineServices.cpp:470: ImGui::TextDisabled("Voice activity now relays through the active Lanline session  
state; raw audio transport remains future work.");  
src/LanlineServices.cpp:490: void DrawOperationalSupportCategory(LanlineServicesState& state,  
src/LanlineServices.cpp:539: ImGui::TextDisabled("Insufficient Recovery Scrip.");  
src/LanlineServices.cpp:547: ImGui::TextDisabled("Tank service remains locked until backbone stability is  
restored.");  
src/LanlineServices.cpp:556: void DrawCosmeticSupportCategory(LanlineServicesState& state, SupportCategory  
category, const char* title) {  
src/LanlineServices.cpp:602: void DrawSupportOrdersSummary(const LanlineServicesState& state, std::int64_t  
nowUnix) {  
src/LanlineServices.cpp:624: void DrawSupportOverview(LanlineServicesState& state, std::int64_t nowUnix) {  
src/LanlineServices.cpp:626: ImGui::BulletText("Recovery Scrip: %d", state.relayCredits);
```

```
src/LanlineServices.cpp:632:void DrawFeyGateTab(LanlineServicesState& state, const ServicesUnlockState&
unlockState, std::int64_t nowUnix) {

src/LanlineServices.cpp:709: state.serviceBayActive = resolvedWorldState != nullptr &&

src/LanlineServices.cpp:714: IsStableRecoveryBackbone(scopedProfile, *resolvedWorldState);

src/LanlineServices.cpp:725: const auto backboneStatus = CurrentRecoveryBackboneStatus(scopedProfile);

src/LanlineServices.cpp:773: {"tank_sensor_kit", "BT-72 Sensor Recovery Kit", "Optics, relay and calibration tools
for damaged sensor arrays.", SupportCategory::TankService, StoreCurrency::InGame, 185, "", false, false, false,
TankSubsystem::Sensors, {"lens_pack", "sensor_relay", "calibration_spool"}},

src/LanlineServices.cpp:786:LanlineServicesState MakeDefaultLanlineServicesState(std::int64_t nowUnix) {

src/LanlineServices.cpp:787: LanlineServicesState state;

src/LanlineServices.cpp:790: {"spur.mechanic.ira", "Inner Spur Depot", "Waiting for stable backbone before
sending heavier service loads.", false, false, false, 1.0f},

src/LanlineServices.cpp:791: {"fortress.dispatch", "Rail Fortress Net", "Patrol routing and support manifests
mirrored through Lanline.", true, false, false, 1.0f},

src/LanlineServices.cpp:799:LanlineServicesState MakeLanlineServicesStateFromSave(const LanlineServicesSave&
save, std::int64_t nowUnix) {

src/LanlineServices.cpp:800: LanlineServicesState state = MakeDefaultLanlineServicesState(nowUnix);

src/LanlineServices.cpp:809:LanlineServicesSave BuildLanlineServicesSave(const LanlineServicesState& state) {

src/LanlineServices.cpp:810: LanlineServicesSave save;

src/LanlineServices.cpp:819:void ApplyLanlineServicesProfileSnapshot(LanlineServicesState& state, const
LanlineServicesProfile& profile) {

src/LanlineServices.cpp:827:void SyncLanlineServicesProfileSnapshot(LanlineServicesProfile& profile, const
LanlineServicesState& state) {

src/LanlineServices.cpp:835:void SyncLanlineServicesSessionProfile(SessionProfile& profile, const
LanlineServicesState& state) {

src/LanlineServices.cpp:837: SyncLanlineServicesProfileSnapshot(profile.lanlineServices, state);

src/LanlineServices.cpp:855:void AdvanceLanlineSupportOrders(LanlineServicesState& state, std::int64_t
nowUnix) {

src/LanlineServices.cpp:867:int CountSupportOrdersInState(const LanlineServicesState& state,
SupportOrderState orderState) {

src/LanlineServices.cpp:874:int ClaimDeliveredSupportOrders(LanlineServicesState& state, SessionProfile&
profile, std::string* summary) {

src/LanlineServices.cpp:911: *summary = "Lanline delivery received: " + claimedOrderSummaries.front() + ".";

src/LanlineServices.cpp:913: *summary = "Lanline deliveries received: " + std::to_string(claimedCount) +
```

```
src/LanlineServices.cpp:921:ServiceHubMode ResolveLanlineServicesMode(const ServicesUnlockState&
unlockState, const LanlineSessionState* sessionState) {

src/LanlineServices.cpp:922: if (!IsLanlineServicesUnlocked(unlockState)) {

src/LanlineServices.cpp:926: return ServiceHubMode::LanlineLocal;

src/LanlineServices.cpp:928: return unlockState.backboneStable ? ServiceHubMode::RelayOnline :
ServiceHubMode::LanlineLocal;

src/LanlineServices.cpp:931:void SyncLanlineServicesPresence(LanlineServicesState& state,

src/LanlineServices.cpp:932: const LanlineSessionState* sessionState,

src/LanlineServices.cpp:934: state.mode = ResolveLanlineServicesMode(unlockState, sessionState);

src/LanlineServices.cpp:942: AddChatMessage(*sessionChannel, "Lanline", "No active session mirror is currently
available.", "now");

src/LanlineServices.cpp:953: AddChatMessage(*sessionChannel, "Lanline", presenceBuffer, sessionState-
>updatedAt);

src/LanlineServices.cpp:972: state.friends.push_back({handle, "Lanline Session", player.role, player.online, true,
false, 1.0f});

src/LanlineServices.cpp:975: friendEntry->nodeLabel = "Lanline Session";

src/LanlineServices.cpp:989: "Lanline Voice",

src/LanlineServices.cpp:997: friendEntry->nodeLabel = "Lanline Voice";

src/LanlineServices.cpp:1019:std::filesystem::path DefaultLanlineServicesSavePath() {

src/LanlineServices.cpp:1020: return ProfilesDirectory() / "lanline_services.state";

src/LanlineServices.cpp:1023:bool SaveLanlineServicesSave(const LanlineServicesSave& save, const
std::filesystem::path& path) {

src/LanlineServices.cpp:1058:bool LoadLanlineServicesSave(const std::filesystem::path& path,
LanlineServicesSave& outSave) {

src/LanlineServices.cpp:1116:void DrawLanlineServicesPanel(LanlineServicesState& state,

src/LanlineServices.cpp:1119: AdvanceLanlineSupportOrders(state, nowUnix);

src/LanlineServices.cpp:1121: if (!IsLanlineServicesUnlocked(unlockState)) {

src/LanlineServices.cpp:1122: DrawLanlineServicesLockedScreen(unlockState);

src/LanlineServices.cpp:1130: ImGui::Text("Lanline Services Online");

src/LanlineServices.cpp:1135: ImGui::BulletText("Service bay: %s", unlockState.serviceBayActive ? "online" :
"offline");

src/LanlineServices.cpp:1161: if (ImGui::BeginTabBar("LanlineServicesTabs")) {
```

```
src/LanlineServices.cpp:1181: "Recovery materials and basic workshop stock on in-game currency.");

src/LanlineSession.cpp:1:#include "../include/LanlineSession.hpp"

src/LanlineSession.cpp:77:void TrimRelayMessages(LanlineSessionState& state, std::size_t maxMessages) {

src/LanlineSession.cpp:81: state.relayMessages.begin() +
static_cast<std::vector<LanlineRelayMessage>::difference_type>(state.relayMessages.size() - maxMessages));

src/LanlineSession.cpp:85:void TrimVoicePresence(LanlineSessionState& state, std::size_t maxEntries) {

src/LanlineSession.cpp:89: state.voicePresence.begin() +
static_cast<std::vector<LanlineVoicePresence>::difference_type>(state.voicePresence.size() - maxEntries));

src/LanlineSession.cpp:110:bool SaveLanlineSessionState(const LanlineSessionState& state) {

src/LanlineSession.cpp:111: return SaveLanlineSessionState(state, LanlineSessionPath());

src/LanlineSession.cpp:114:bool SaveLanlineSessionState(const LanlineSessionState& state, const
std::filesystem::path& path) {

src/LanlineSession.cpp:164:bool LoadLanlineSessionState(LanlineSessionState& state) {

src/LanlineSession.cpp:165: return LoadLanlineSessionState(LanlineSessionPath(), state);

src/LanlineSession.cpp:168:bool LoadLanlineSessionState(const std::filesystem::path& path,
LanlineSessionState& state) {

src/LanlineSession.cpp:174: state = LanlineSessionState{};

src/LanlineSession.cpp:215: LanlinePlayerEntry player{};

src/LanlineSession.cpp:268:std::vector<LanlineSessionState> DiscoverLanlineSessionSnapshots() {

src/LanlineSession.cpp:269: std::vector<LanlineSessionState> sessions;

src/LanlineSession.cpp:270: const auto sessionsDir = LanlineSessionsDirectory();

src/LanlineSession.cpp:280: LanlineSessionState state;

src/LanlineSession.cpp:281: if (LoadLanlineSessionState(entry.path(), state)) {

src/LanlineSession.cpp:286: std::sort(sessions.begin(), sessions.end(), [](const LanlineSessionState& left, const
LanlineSessionState& right) {

src/LanlineSession.cpp:296:LanlineDiagnostics ProbeLanlineHost(const LanlineSessionState& session,
std::string_view runtimeWorldName) {

src/LanlineSession.cpp:297: LanlineDiagnostics out;

src/LanlineSession.cpp:329: out.lastError = "Unable to resolve Lanline host.";

src/PrefabLibrary.cpp:118:void TrimTrailingEmptyLootEntries(std::vector<LootEntry>& entries) {

src/PrefabLibrary.cpp:212: TrimTrailingEmptyLootEntries(prefab.object.lootEntries);
```

```
src/Progression.cpp:33: *eventText = "Level up achieved. Recovery stats refreshed and attribute points awarded.";

src/SessionProfiles.cpp:63: HasCollectedTapeId(profile, "bt72_service_reel_001");

src/SessionProfiles.cpp:70: route.surfaceArrivalReached = route.surfaceArrivalReached ||

src/SessionProfiles.cpp:73: profile.story.returnedToBase;

src/SessionProfiles.cpp:74: route.firstTankCombatResolved = route.firstTankCombatResolved ||
profile.story.relayRecovered || profile.story.returnedToBase;

src/SessionProfiles.cpp:77: profile.story.returnedToBase ||

src/SessionProfiles.cpp:79: route.firstRecoveryNodeActivated = route.firstRecoveryNodeActivated ||
profile.story.relayRecovered;

src/SessionProfiles.cpp:81: profile.story.returnedToBase ||

src/SessionProfiles.cpp:93: profile.story.relayRecovered = profile.story.relayRecovered ||
route.firstRecoveryNodeActivated;

src/SessionProfiles.cpp:94: profile.story.returnedToBase = profile.story.returnedToBase ||
route.debriefSummaryViewed;

src/SessionProfiles.cpp:117: route.surfaceArrivalReached = profile.story.outerRoadCleared ||
profile.story.relayRecovered || profile.story.returnedToBase;

src/SessionProfiles.cpp:118: route.firstTankCombatResolved = profile.story.relayRecovered ||
profile.story.returnedToBase;

src/SessionProfiles.cpp:120: profile.story.returnedToBase ||

src/SessionProfiles.cpp:122: route.firstRecoveryNodeActivated = profile.story.relayRecovered;

src/SessionProfiles.cpp:123: route.debriefSummaryViewed = profile.story.returnedToBase;

src/SessionProfiles.cpp:136: profile.firstPlayableRoute.surfaceArrivalReached =

src/SessionProfiles.cpp:139: profile.story.returnedToBase;

src/SessionProfiles.cpp:171: profile.character.collectedTapes.push_back({"music_recovery_001", "Recovery
Station Mixtape", false, false, false});

src/SessionProfiles.cpp:282: profile.lanlineServices.serviceHubKnown = false;

src/SessionProfiles.cpp:301: profile.character.collectedTapes.push_back({"music_recovery_001", "Recovery
Station Mixtape", false, false, false});

src/SessionProfiles.cpp:375: out << "lanline_service_hub_known=" << (profile.lanlineServices.serviceHubKnown ?
1 : 0) << '\n';

src/SessionProfiles.cpp:386: out << "scavenger_runs_completed=" << profile.scavengerRunsCompleted << '\n';
```

```
src/SessionProfiles.cpp:401: out << "bt72_hull_moved_to_service_lift=" << (profile.bt72HullMovedToServiceLift ? 1 : 0) << '\n';

src/SessionProfiles.cpp:432: out << "story_exit=" << (profile.story.exitedBunker ? 1 : 0) << '\n';

src/SessionProfiles.cpp:435: out << "story_return=" << (profile.story.returnedToBase ? 1 : 0) << '\n';

src/SessionProfiles.cpp:448: out << "route_surface_arrival=" << (profile.firstPlayableRoute.surfaceArrivalReached ? 1 : 0) << '\n';

src/SessionProfiles.cpp:450: out << "route_first_service=" << (profile.firstPlayableRoute.firstServicePerformed ? 1 : 0) << '\n';

src/SessionProfiles.cpp:451: out << "route_first_recovery=" <<
(profile.firstPlayableRoute.firstRecoveryNodeActivated ? 1 : 0) << '\n';

src/SessionProfiles.cpp:456: out << "awakening_stress=" << profile.character.awakening.stressSurvivals << '\n';

src/SessionProfiles.cpp:458: out << "awakening_field_service=" << profile.character.awakening.fieldServiceUses << '\n';

src/SessionProfiles.cpp:485: << (worldState.caravanRouteActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:486: << (worldState.droneStationsActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:487: << (worldState.tradeNetworkActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:488: << (worldState.railFreightActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:490: << (worldState.railFortressActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:495: << worldState.caravanRunsCompleted << ',';

src/SessionProfiles.cpp:496: << worldState.droneRunsCompleted << ',';

src/SessionProfiles.cpp:497: << worldState.tradeCyclesCompleted << ',';

src/SessionProfiles.cpp:499: << worldState.railRunsCompleted << ',';

src/SessionProfiles.cpp:501: << worldState.railFortressDeployments << ',';

src/SessionProfiles.cpp:519: << (worldState.serviceBayActive ? 1 : 0) << ',';

src/SessionProfiles.cpp:520: << worldState.serviceCyclesCompleted << ',';

src/SessionProfiles.cpp:614: else if (key == "lanline_service_hub_known")
outProfile.lanlineServices.serviceHubKnown = (std::stoi(value) != 0);

src/SessionProfiles.cpp:625: else if (key == "scavenger_runs_completed") outProfile.scavengerRunsCompleted =
std::stoi(value);

src/SessionProfiles.cpp:635: else if (key == "bt72_hull_moved_to_service_lift")
outProfile.bt72HullMovedToServiceLift = (std::stoi(value) != 0);

src/SessionProfiles.cpp:666: else if (key == "story_exit") outProfile.story.exitedBunker = (std::stoi(value) != 0);
```



```
src/SessionProfiles.cpp:669: else if (key == "story_return") outProfile.story.returnedToBase = (std::stoi(value) != 0);

src/SessionProfiles.cpp:682: else if (key == "route_surface_arrival")
outProfile.firstPlayableRoute.surfaceArrivalReached = (std::stoi(value) != 0);

src/SessionProfiles.cpp:684: else if (key == "route_first_service")
outProfile.firstPlayableRoute.firstServicePerformed = (std::stoi(value) != 0);

src/SessionProfiles.cpp:685: else if (key == "route_first_recovery")
outProfile.firstPlayableRoute.firstRecoveryNodeActivated = (std::stoi(value) != 0);

src/SessionProfiles.cpp:690: else if (key == "awakening_stress") outProfile.character.awakening.stressSurvivals =
std::stoi(value);

src/SessionProfiles.cpp:692: else if (key == "awakening_field_service")
outProfile.character.awakening.fieldServiceUses = std::stoi(value);

src/SessionProfiles.cpp:787: state.caravanRouteActive = std::stoi(value.substr(third + 1, fourth - third - 1)) != 0;

src/SessionProfiles.cpp:788: state.caravanRunsCompleted = std::stoi(value.substr(fourth + 1, fifth - fourth - 1));

src/SessionProfiles.cpp:789: state.droneRunsCompleted = std::stoi(value.substr(fifth + 1, sixth - fifth - 1));

src/SessionProfiles.cpp:802: const bool hasLanlineFields = parts.size() >= 37;

src/SessionProfiles.cpp:803: const std::size_t baseOffset = hasLanlineFields ? 6 : 3;

src/SessionProfiles.cpp:804: state.towerSyncRecovered = hasLanlineFields ? (std::stoi(parts[3]) != 0) : false;

src/SessionProfiles.cpp:805: state.localRelayAvailable = hasLanlineFields ? (std::stoi(parts[4]) != 0) : false;

src/SessionProfiles.cpp:806: state.regionalGridOnline = hasLanlineFields ? (std::stoi(parts[5]) != 0) : false;

src/SessionProfiles.cpp:807: state.caravanRouteActive = std::stoi(parts[baseOffset]) != 0;

src/SessionProfiles.cpp:808: state.droneStationsActive = std::stoi(parts[baseOffset + 1]) != 0;

src/SessionProfiles.cpp:809: state.tradeNetworkActive = std::stoi(parts[baseOffset + 2]) != 0;

src/SessionProfiles.cpp:812: state.railFreightActive = std::stoi(parts[baseOffset + 3]) != 0;

src/SessionProfiles.cpp:814: state.railFortressActive = std::stoi(parts[baseOffset + 5]) != 0;

src/SessionProfiles.cpp:821: state.caravanRunsCompleted = std::stoi(parts[baseIndex + 2]);

src/SessionProfiles.cpp:822: state.droneRunsCompleted = std::stoi(parts[baseIndex + 3]);

src/SessionProfiles.cpp:823: state.tradeCyclesCompleted = std::stoi(parts[baseIndex + 4]);

src/SessionProfiles.cpp:825: state.railRunsCompleted = std::stoi(parts[baseIndex + 6]);

src/SessionProfiles.cpp:827: state.railFortressDeployments = std::stoi(parts[baseIndex + 8]);

src/SessionProfiles.cpp:845: state.serviceBayActive = parts.size() >= baseIndex + 27 ? (std::stoi(parts[baseIndex +
26]) != 0) : false;
```

```
src/SessionProfiles.cpp:846: state.serviceCyclesCompleted = parts.size() >= baseIndex + 28 ?
std::stoi(parts[baseIndex + 27]) : 0;

src/SessionProfiles.cpp:874: state.railFreightActive = std::stoi(parts[baseOffset + 3]) != 0;

src/SessionProfiles.cpp:876: state.railFortressActive = std::stoi(parts[baseOffset + 5]) != 0;

src/SessionProfiles.cpp:880: state.caravanRunsCompleted = std::stoi(parts[baseOffset + 8]);

src/SessionProfiles.cpp:881: state.droneRunsCompleted = std::stoi(parts[baseOffset + 9]);

src/SessionProfiles.cpp:882: state.tradeCyclesCompleted = std::stoi(parts[baseOffset + 10]);

src/SessionProfiles.cpp:884: state.railRunsCompleted = std::stoi(parts[baseOffset + 12]);

src/SessionProfiles.cpp:886: state.railFortressDeployments = std::stoi(parts[baseOffset + 14]);

src/SessionProfiles.cpp:904: state.serviceBayActive = false;

src/SessionProfiles.cpp:905: state.serviceCyclesCompleted = 0;

src/SessionProfiles.cpp:909: state.railFreightActive = std::stoi(parts[6]) != 0;

src/SessionProfiles.cpp:913: state.caravanRunsCompleted = std::stoi(parts[9]);

src/SessionProfiles.cpp:914: state.droneRunsCompleted = std::stoi(parts[10]);

src/SessionProfiles.cpp:915: state.tradeCyclesCompleted = std::stoi(parts[11]);

src/SessionProfiles.cpp:917: state.railRunsCompleted = std::stoi(parts[13]);

src/SessionProfiles.cpp:919: state.railFortressDeployments = 0;

src/SessionProfiles.cpp:937: state.serviceBayActive = false;

src/SessionProfiles.cpp:938: state.serviceCyclesCompleted = 0;

src/SessionProfiles.cpp:946: state.caravanRunsCompleted = std::stoi(parts[8]);

src/SessionProfiles.cpp:947: state.droneRunsCompleted = std::stoi(parts[9]);

src/SessionProfiles.cpp:948: state.tradeCyclesCompleted = std::stoi(parts[10]);

src/SessionProfiles.cpp:953: state.railRunsCompleted = 0;

src/SessionProfiles.cpp:955: state.railFortressDeployments = 0;

src/SessionProfiles.cpp:973: state.serviceBayActive = false;

src/SessionProfiles.cpp:974: state.serviceCyclesCompleted = 0;

src/SkillSystem.cpp:76: profile.character.awakening.stressSurvivals += 1;

src/SkillSystem.cpp:77: if (profile.character.awakening.stressSurvivals >= 1) {

src/StoryRoute.cpp:45: HasCollectedTapelId(profile, "bt72_service_reel_001");

src/StoryRoute.cpp:65: profile.story.exitedBunker;
```

```
src/StoryRoute.cpp:71: profile.story.exitedBunker;

src/StoryRoute.cpp:77: profile.story.exitedBunker;

src/StoryRoute.cpp:81: return profile.firstPlayableRoute.surfaceArrivalReached ||

src/StoryRoute.cpp:84: profile.story.returnedToBase;

src/StoryRoute.cpp:90: profile.story.returnedToBase;

src/StoryRoute.cpp:100: profile.story.returnedToBase ||

src/StoryRoute.cpp:104: bool FirstRecoveryNodeActivated(const SessionProfile& profile) {

src/StoryRoute.cpp:105: return profile.firstPlayableRoute.firstRecoveryNodeActivated ||
profile.story.relayRecovered;

src/StoryRoute.cpp:110: profile.story.returnedToBase ||

src/StoryRoute.cpp:126: if (routeEventType == "service_call") {

src/StoryRoute.cpp:148: if (worldState.activeRouteEventType == "service_call" ||

src/StoryRoute.cpp:175: return "Restore the industrial rail depot and bring heavy freight back to Shelter 17.";

src/StoryRoute.cpp:183: if (!IsRecoveryFabricatorOperational(profile, worldState)) {

src/StoryRoute.cpp:184: return "Prime the Recovery Fabricator to turn salvage into operational supplies.";

src/StoryRoute.cpp:211: return "Bring the inner spur service bay online to push BT-72 repairs deeper into the
factory belt.";

src/StoryRoute.cpp:243: return "rail freight";

src/StoryRoute.cpp:251: if (!IsRecoveryFabricatorOperational(profile, worldState)) {

src/StoryRoute.cpp:252: return "Recovery Fabricator";

src/StoryRoute.cpp:279: return "service bay";

src/StoryRoute.cpp:297: if (!IsRecoveryFabricatorOperational(profile, worldState)) {

src/StoryRoute.cpp:298: return "The rail spur is secured, but salvage is not yet turning into field stock.";

src/StoryRoute.cpp:328: return "Deep BT-72 service is online, but frontier recovery still lacks long-range water
support.";

src/StoryRoute.cpp:330: return "Relay, service, and water loops are all online; Shelter 17 can now sustain a stable
recovery backbone.";

src/StoryRoute.cpp:333: RecoveryBackboneStatus CurrentRecoveryBackboneStatusForProfile(const
SessionProfile& profile) {

src/StoryRoute.cpp:334: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:344: "Recovery payoff is captured, but Shelter 17 has not archived the debrief yet.",
```

```
src/StoryRoute.cpp:345: "Upload the debrief to turn the route payoff into honest rail, service, and industrial work."

src/StoryRoute.cpp:358: const bool railOperational = IsRailFreightOperational(profile, *worldState);

src/StoryRoute.cpp:361: const bool fabricatorOperational = IsRecoveryFabricatorOperational(profile, *worldState);

src/StoryRoute.cpp:369: const bool serviceOperational = IsServiceBayOperational(*worldState);

src/StoryRoute.cpp:373: AppendBackboneNodeLabel(activeNodes, railOperational, "rail freight");

src/StoryRoute.cpp:376: AppendBackboneNodeLabel(activeNodes, fabricatorOperational, "Recovery Fabricator");

src/StoryRoute.cpp:385: AppendBackboneNodeLabel(activeNodes, serviceOperational, "service bay");

src/StoryRoute.cpp:389: (railOperational ? 1 : 0) +

src/StoryRoute.cpp:402: (serviceOperational ? 1 : 0) +

src/StoryRoute.cpp:405: RecoveryBackboneStatus status{};

src/StoryRoute.cpp:406: if (IsStableRecoveryBackbone(profile, *worldState)) {

src/StoryRoute.cpp:408: status.status = "Recovery backbone " + std::to_string(starterOnline + innerOnline) +

src/StoryRoute.cpp:428: bool HasLanlineServicesObjective(const SessionProfile& profile) {

src/StoryRoute.cpp:452: return route.prePipPadClueCount >= 2 ? "Pip-Pad Recovery" : "Bunker Passage";

src/StoryRoute.cpp:466: if (!profile.story.exitedBunker) {

src/StoryRoute.cpp:481: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:482: return "Recovery Node";

src/StoryRoute.cpp:499: return "Recover a bunker access card and trace the paper trail to the missing Pip-Pad.";

src/StoryRoute.cpp:502: return "Sweep the bunker passage, gather the paper trail, and recover the missing Pip-Pad.";

src/StoryRoute.cpp:508: ? "Sync the archive and pull the missing personnel trail into the Pip-Pad."

src/StoryRoute.cpp:513: return "Survey the BT-72 hull, recover the starter core, and decode the service notes.";

src/StoryRoute.cpp:516: return "Restore hangar power, clear the crane path, and move the BT-72 hull into the service lift cradle.";

src/StoryRoute.cpp:535: if (!profile.story.exitedBunker) {

src/StoryRoute.cpp:548: return "Take a first service/rest stop before pushing the recovery node.";

src/StoryRoute.cpp:550: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:558: return "Recovery buildout active. Runtime world state not loaded yet.";
```

```
src/StoryRoute.cpp:583: return "Sweep the bunker passage, gather the paper trail, and recover the missing Pip-Pad.";

src/StoryRoute.cpp:595: return "Survey the BT-72 hull, recover the starter core, and decode the service notes.";

src/StoryRoute.cpp:598: return "Power the hangar crane, clear the crane path, and lock BT-72 into the service lift cradle.";

src/StoryRoute.cpp:617: if (!profile.story.exitedBunker) {

src/StoryRoute.cpp:630: return "Run a first service cycle before pushing the relay node.";

src/StoryRoute.cpp:632: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:640: return "Recovery buildout active. Runtime world state not loaded yet.";

src/StoryRoute.cpp:647: std::string CurrentRecoveryHandoffSummaryForProfile(const SessionProfile& profile) {

src/StoryRoute.cpp:648: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:649: return "Recovery handoff locked until the first relay node comes online.";

src/StoryRoute.cpp:657: return "Recovery handoff waiting for selected-world state.";

src/StoryRoute.cpp:661: return "Handoff: restore rail freight and move heavy salvage off the starter route.";

src/StoryRoute.cpp:669: if (!IsRecoveryFabricatorOperational(profile, *worldState)) {

src/StoryRoute.cpp:670: return "Handoff: prime the Recovery Fabricator and turn salvage into field stock.";

src/StoryRoute.cpp:697: return "Handoff: bring the service bay online for deep BT-72 support.";

src/StoryRoute.cpp:722: if (!(DebriefSummaryViewed(profile) || profile.story.returnedToBase || HasCollectedTapId(profile, "debrief_shelter17"))) {

src/StoryRoute.cpp:753: if (!profile.story.exitedBunker) {

src/StoryRoute.cpp:766: return "contact resolved, service halt pending";

src/StoryRoute.cpp:768: if (!FirstRecoveryNodeActivated(profile)) {

src/StoryRoute.cpp:769: return "service complete, recovery node pending";

src/StoryRoute.cpp:791: : CurrentRecoveryHandoffSummary(profile);

src/StoryRoute.cpp:807: "Recover the access card, stabilize the paper trail, and pull the missing Pip-Pad into the route.",

src/StoryRoute.cpp:808: "The archive trail should point cleanly toward the hangar and BT-72."

src/StoryRoute.cpp:814: "Clear the vermin gate and sync the missing personnel trail into the Pip-Pad.",

src/StoryRoute.cpp:820: "Hangar Recovery",

src/StoryRoute.cpp:821: "Read the BT-72 hull, core, and service notes, move the hull through the crane and service lift cradle, then restore the chassis from bunker salvage.",
```

```
src/StoryRoute.cpp:839: if (!profile.story.exitedBunker) {  
  
src/StoryRoute.cpp:864: "A short service halt should feel earned instead of menu-like."  
  
src/StoryRoute.cpp:870: "Run one field or workshop service cycle to cool BT-72 and reset the push.",  
  
src/StoryRoute.cpp:874: if (!FirstRecoveryNodeActivated(profile)) {  
  
src/StoryRoute.cpp:876: "Recovery Sync",  
  
src/StoryRoute.cpp:890: CurrentRecoveryHandoffSummaryForProfile(profile),  
  
src/StoryRoute.cpp:891: "Route events, recovery state, and BT-72 service now carry progression beyond the  
starter lane."  
  
src/StoryRoute.cpp:897: std::string CurrentRecoveryHandoffSummary(const SessionProfile& profile) {  
  
src/StoryRoute.cpp:898: return CurrentRecoveryHandoffSummaryForProfile(profile);  
  
src/StoryRoute.cpp:901: std::string CurrentRecoveryHandoffSummary(const SessionProfile& profile,  
std::string_view worldReference) {  
  
src/StoryRoute.cpp:902: return CurrentRecoveryHandoffSummaryForProfile(BuildWorldScopedProfile(profile,  
worldReference));  
  
src/StoryRoute.cpp:905: RecoveryBackboneStatus CurrentRecoveryBackboneStatus(const SessionProfile& profile)  
{  
  
src/StoryRoute.cpp:906: return CurrentRecoveryBackboneStatusForProfile(profile);  
  
src/StoryRoute.cpp:909: RecoveryBackboneStatus CurrentRecoveryBackboneStatus(const SessionProfile& profile,  
std::string_view worldReference) {  
  
src/StoryRoute.cpp:910: return CurrentRecoveryBackboneStatusForProfile(BuildWorldScopedProfile(profile,  
worldReference));  
  
src/StoryRoute.cpp:941: {"Decode BT-72 service notes and holo-records.", Bt72ServiceNotesRecovered(profile)},  
  
src/StoryRoute.cpp:942: {"Lock the BT-72 hull in the service lift cradle.",  
Bt72HullLockedInRestorationCradle(profile)},  
  
src/StoryRoute.cpp:955: {"Recover the bunker access card and paper trail.", HasAccessCardRecovered(profile) &&  
  
src/StoryRoute.cpp:963: {"Open the outer bulkhead.", profile.story.exitedBunker},  
  
src/StoryRoute.cpp:967: {"Take the first service/rest halt.", FirstServicePerformed(profile)},  
  
src/StoryRoute.cpp:968: {"Sync the first recovery node.", FirstRecoveryNodeActivated(profile)},  
  
src/StoryRoute.cpp:975: const bool railOperational = worldState != nullptr && IsRailFreightOperational(profile,  
*worldState);  
  
src/StoryRoute.cpp:978: const bool fabricatorOperational = worldState != nullptr &&  
IsRecoveryFabricatorOperational(profile, *worldState);
```

```
src/StoryRoute.cpp:986: const bool serviceOperational = worldState != nullptr &&
IsServiceBayOperational(*worldState);

src/StoryRoute.cpp:999: {"Decode BT-72 service notes.", profile.firstPlayableRoute.bt72ServiceNotesRecovered},

src/StoryRoute.cpp:1005: {"Open the outer bulkhead.", profile.story.exitedBunker},

src/StoryRoute.cpp:1006: {"Reach the first surface arrival foothold.",
profile.firstPlayableRoute.surfaceArrivalReached},

src/StoryRoute.cpp:1009: {"Take a first service/rest cycle.", profile.firstPlayableRoute.firstServicePerformed},

src/StoryRoute.cpp:1010: {"Sync the first recovery node.", profile.firstPlayableRoute.firstRecoveryNodeActivated},

src/StoryRoute.cpp:1012: {"Restore the rail freight depot.", railOperational},

src/StoryRoute.cpp:1015: {"Prime the Recovery Fabricator.", fabricatorOperational},

src/StoryRoute.cpp:1024: {"Bring the inner spur service bay online.", serviceOperational},

src/World.cpp:137: "[%bt72_service_lift_0001]",

src/World.cpp:154: "[%bt72_service_notes_0001]",

src/World.cpp:259: return normalizedTag == "rail_depot" || normalizedTag == "rail_fortress_hub";

src/World.cpp:274: normalizedTag == "workshop_service" ||

src/World.cpp:276: normalizedTag == "drone_station" ||

src/World.cpp:279: normalizedTag == "service_bay" ||

src/World.cpp:281: normalizedTag == "lanline_service_hub" ||

src/World.cpp:282: normalizedTag == "medical_support" ||

src/World.cpp:283: normalizedTag == "tank_service" ||

src/World.cpp:324: void TrimTrailingEmptyLootEntries(std::vector<LootEntry>& entries) {

src/World.cpp:354: TrimTrailingEmptyLootEntries(object.lootEntries);

src/World.cpp:625: TrimTrailingEmptyLootEntries(persistedLootEntries);

src/World.cpp:677: "Pip-Pad Recovery Locker",

src/World.cpp:1010: "[%drone_0001]",

src/World.cpp:1025: "drone_station",

src/World.cpp:1030: "[%rail_0001]",

src/World.cpp:1045: "rail_depot",

src/World.cpp:1085: "rail_fortress_hub",

src/World.cpp:1091: "Recovery Fabricator Node",
```

src/World.cpp:1270: "[%servicebay_0001]",
src/World.cpp:1285: "service_bay",
src/World.cpp:1286: "inner_spur_service"
src/World.cpp:1310: "[%services_0001]",
src/World.cpp:1311: "Shelter 17 Lanline Service Hub",
src/World.cpp:1325: "lanline_service_hub",
src/World.cpp:1326: "shelter17_services"
src/World.cpp:1365: "medical_support",
src/World.cpp:1366: "field_medical"
src/World.cpp:1370: "[%tankservice_0001]",
src/World.cpp:1385: "tank_service",
src/World.cpp:1386: "bt72_service"
src/World.cpp:1556: if (IsStarterScenarioWorld() && !HasObject("[%drone_0001]")) {
src/World.cpp:1558: "[%drone_0001]",
src/World.cpp:1573: "drone_station",
src/World.cpp:1577: if (IsStarterScenarioWorld() && !HasObject("[%rail_0001]")) {
src/World.cpp:1579: "[%rail_0001]",
src/World.cpp:1594: "rail_depot",
src/World.cpp:1636: "rail_fortress_hub",
src/World.cpp:1643: "Recovery Fabricator Node",
src/World.cpp:1829: if (IsStarterScenarioWorld() && !HasObject("[%servicebay_0001]")) {
src/World.cpp:1831: "[%servicebay_0001]",
src/World.cpp:1846: "service_bay",
src/World.cpp:1847: "inner_spur_service"
src/World.cpp:1871: if (IsStarterScenarioWorld() && !HasObject("[%services_0001]")) {
src/World.cpp:1873: "[%services_0001]",
src/World.cpp:1874: "Shelter 17 Lanline Service Hub",
src/World.cpp:1888: "lanline_service_hub",
src/World.cpp:1889: "shelter17_services"
src/World.cpp:1930: "medical_support",


```
src/World.cpp:1931: "field_medical"

src/World.cpp:1934: if (IsStarterScenarioWorld() && !HasObject("[%tankservice_0001]")) {

src/World.cpp:1936: "[%tankservice_0001]",

src/World.cpp:1951: "tank_service",

src/World.cpp:1952: "bt72_service"

src/WorldEvents.cpp:61: : "LOCKER BAY: 'Emergency recovery locker detected. Mechanical interlock wants a
bunker access card from the core service racks.'";

src/WorldEvents.cpp:79: if (profile.story.exitedBunker &&

src/WorldEvents.cpp:80: !profile.firstPlayableRoute.surfaceArrivalReached &&

src/WorldEvents.cpp:82: profile.firstPlayableRoute.surfaceArrivalReached = true;

src/WorldEvents.cpp:96: ? "CONTACT: 'Outer Ghoul charging the cleared lane. Hold BT-72 through the rush, then
take one service halt before syncing the relay.'"

src/WorldEvents.cpp:102: if (profile.story.relayRecovered && !profile.story.returnedToBase &&
!gameState.zoneEventReturn &&

src/WorldEvents.cpp:105: gameState.lastEvent = "DEBRIEF: 'Recovery node recognized. Upload the route
summary and authorize Shelter 17 planning: rail, orbital, fabrication, then the inner spur.'";

src/WorldEvents.cpp:110: if (profile.story.returnedToBase && !gameState.zoneEventReturn &&

src/WorldEvents.cpp:113: gameState.lastEvent = "DEBRIEF: 'Recovery planning remains active. Keep rebuilding
rail freight, orbital support, fabrication, and the inner spur backbone.'";

src/WorldExport.cpp:270: if (result.auditTrailPath.empty()) {

src/WorldExport.cpp:274: std::ofstream auditFile(result.auditTrailPath, std::ios::app);

src/WorldExport.cpp:786: std::filesystem::path ExportAuditTrailPathForWorld(const std::filesystem::path&
worldPath) {

src/WorldExport.cpp:828: std::ifstream file(ExportAuditTrailPathForWorld(worldPath));

src/WorldExport.cpp:1346: report << "Audit trail: " << result.auditTrailPath.string() << '\n';

src/WorldExport.cpp:1425: result.auditTrailPath = ExportAuditTrailPathForWorld(path);

src/WorldExport.cpp:1439: " Audit: " + result.auditTrailPath.string();

src/WorldExport.cpp:1446: " Audit: " + result.auditTrailPath.string();

src/WorldExport.cpp:1452: " Audit: " + result.auditTrailPath.string();

src/WorldExport.cpp:1459: " Audit: " + result.auditTrailPath.string();

src/WorldExport.cpp:1462: " Audit: " + result.auditTrailPath.string();

src/WorldSemanticAuthoring.cpp:64: } else if (spec.scriptTag == "service_bay") {
```

```
src/WorldSemanticAuthoring.cpp:139: if (scriptTag == "lanline_service_hub" || scriptTag == "medical_support") {
src/WorldSemanticAuthoring.cpp:142: if (scriptTag == "tank_service") {
src/WorldSemanticAuthoring.cpp:143: return {"service_bay"};
src/WorldSemanticAuthoring.cpp:148: if (scriptTag == "service_bay") {
src/WorldSemanticAuthoring.cpp:152: return {"service_bay", "relay_substation", "recovery_fabricator"};
src/main.cpp:15:#include "../include/LanlineLobbyLogic.hpp"
src/main.cpp:16:#include "../include/LanlineSession.hpp"
src/main.cpp:72:void TrimLanlineEventLog(bunker::LanlineSessionState& state, std::size_t maxEntries) {
src/main.cpp:78:void UpsertLanlinePlayer(bunker::LanlineSessionState& state,
src/main.cpp:85: [&](const bunker::LanlinePlayerEntry& entry) {
src/main.cpp:98:void AcceptLanlineLobbySlot(bunker::LanlineSessionState& state, const std::string& peerName) {
src/main.cpp:118: UpsertLanlinePlayer(state, peerName, "Client", true, false);
src/main.cpp:128:void SyncLanlineRuntimeLaunchState(const bunker::LaunchTicketInfo& launchTicket, const
bunker::SessionProfile& sessionProfile) {
src/main.cpp:129: bunker::LanlineSessionState state;
src/main.cpp:130: if (!bunker::LoadLanlineSessionState(state)) {
src/main.cpp:131: state = bunker::LanlineSessionState{};
src/main.cpp:174: AcceptLanlineLobbySlot(state, state.connectedPeer);
src/main.cpp:175: state.eventLog.push_back("Host accepted Lanline peer " + state.connectedPeer + ".");
src/main.cpp:187: if (player.displayName == actorName && bunker::IsLanlineReservedSlot(player)) {
src/main.cpp:211: UpsertLanlinePlayer(state, actorName, actorRole, true, currentReady, seatAssignment);
src/main.cpp:214: state.eventLog.push_back("Lanline lifecycle advanced to " + state.lifecycleStage + ".");
src/main.cpp:218: TrimLanlineEventLog(state, 12);
src/main.cpp:219: bunker::SaveLanlineSessionState(state);
src/main.cpp:220: bunker::SaveLanlineSessionState(state, bunker::LanlineSessionSnapshotPath(state.sessionId));
src/main.cpp:223:void SyncLanlineRuntimePresence(const bunker::PlayerState& player, const
bunker::SessionProfile& sessionProfile) {
src/main.cpp:246: bunker::LanlineSessionState state;
src/main.cpp:247: if (!bunker::LoadLanlineSessionState(state)) {
src/main.cpp:269: UpsertLanlinePlayer(state, actorName, actorRole, true, currentReady, seatAssignment);
```

```
src/main.cpp:275: TrimLanlineEventLog(state, 12);

src/main.cpp:276: bunker::SaveLanlineSessionState(state);

src/main.cpp:277: bunker::SaveLanlineSessionState(state, bunker::LanlineSessionSnapshotPath(state.sessionId));

src/main.cpp:365: SyncLanlineRuntimeLaunchState(launchTicket, sessionProfile);

src/main.cpp:453: SyncLanlineRuntimePresence(player, sessionProfile);

src/main.cpp:469: bunker::UpdateRecoveryFabricator(sessionProfile, gameState, dt);

src/main.cpp:470: bunker::UpdateRecoveryMilestones(sessionProfile, gameState);

src/main.cpp:490: gameState.lastEvent = "Operator fatigue rising. Recover MP before pushing further.";

src/main.cpp:494: if (atCriticalHp && !gameState.stressThresholdTriggered) {

src/main.cpp:495: gameState.stressThresholdTriggered = true;

src/main.cpp:502: gameState.stressThresholdTriggered = false;

src/main.cpp:514: gameState.lastEvent = "Second Wind engaged. Operator stabilized under stress.";

src/main.cpp:699: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
sessionProfile.story.exitedBunker) : true;

src/main.cpp:864: gameState.stressThresholdTriggered = false;

src/main.cpp:964: ImGui::TextColored(ImVec4(0.95f, 0.7f, 0.22f, 1.0f), "Ammo reserves low. Workshop or field
service recommended.");

tests/SmokeChecks.cpp:8:#include "../include/LanlineServices.hpp"

tests/SmokeChecks.cpp:201: Check(loaderWorld.HasLinkTarget("inner_spur_service"), "service bay link target
missing after roundtrip") &&

tests/SmokeChecks.cpp:203: Check(loaderWorld.CountObjectsInEditorLayer("Service") >= 1, "world roundtrip
should preserve inferred service layers") &&

tests/SmokeChecks.cpp:231: "world roundtrip should trim trailing empty rows after a filled row 10 entry") &&

tests/SmokeChecks.cpp:237: "world roundtrip should trim all-empty trailing loot rows to zero persisted entries");

tests/SmokeChecks.cpp:334: savedProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:344: savedWorldState->activeRouteEventType = "service_call";

tests/SmokeChecks.cpp:382: Check(loaderProfile.firstPlayableRoute.surfaceArrivalReached, "profile first route
surface arrival flag mismatch") &&

tests/SmokeChecks.cpp:388: Check(bunker::FindWorldFieldState(loaderProfile, loaderProfile.selectedWorld)-
>activeRouteEventType == "service_call",

tests/SmokeChecks.cpp:602: ContainsText(savedText, "bt72_hull_moved_to_service_lift=") &&

tests/SmokeChecks.cpp:699: if (!Check(bunker::CurrentStoryCheckpointLabel(profile) == "Pip-Pad Recovery",
```

```
tests/SmokeChecks.cpp:743: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:757: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:761: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:772: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:779: Check(bunker::CurrentStoryObjectivePreview(profile).find("rail depot") !=
std::string::npos,

tests/SmokeChecks.cpp:821: missingPreview.find("service notes") != std::string::npos,

tests/SmokeChecks.cpp:822: "bt72_objective_requires_knowledge_before_cradle expected hull/core/service
notes objective")) {

tests/SmokeChecks.cpp:832: if (!Check(hasAny(cradlePreview, {"crane", "service lift", "cradle"}) &&

tests/SmokeChecks.cpp:833: hasAny(cradleObjective, {"crane", "service lift", "cradle"}) &&

tests/SmokeChecks.cpp:834: cradlePreview.find("Survey the BT-72 hull, recover the starter core, and decode the
service notes") == std::string::npos &&

tests/SmokeChecks.cpp:835: cradleObjective.find("Survey the BT-72 hull, recover the starter core, and decode the
service notes") == std::string::npos &&

tests/SmokeChecks.cpp:879: return entry.text.find("cradle") != std::string::npos || entry.text.find("service lift") !=
std::string::npos;

tests/SmokeChecks.cpp:885: return entry.text.find("cradle") != std::string::npos || entry.text.find("service lift") !=
std::string::npos;

tests/SmokeChecks.cpp:930: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:941: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:955: "route beat smoke expected service-halt label after first combat")) {

tests/SmokeChecks.cpp:960: if (!Check(bunker::CurrentFirstPlayableRouteBeat(profile).label == "Recovery Sync",

tests/SmokeChecks.cpp:961: "route beat smoke expected recovery-sync label after first service")) {

tests/SmokeChecks.cpp:966: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:972: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:978: Check(industrialBeat.cue.find("rail freight") != std::string::npos,

tests/SmokeChecks.cpp:979: "route beat smoke expected industrial handoff cue to point at rail freight");

tests/SmokeChecks.cpp:1014: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1033: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1036: const auto serviceReadout = bunker::BuildFirstPlayableRouteReadout(profile);
```

tests/SmokeChecks.cpp:1037: if (!Check(serviceReadout.surfaceStatus == "contact resolved, service halt pending",

tests/SmokeChecks.cpp:1038: "route readout smoke expected post-contact service-halt surface status")) {

tests/SmokeChecks.cpp:1041: if (!Check(serviceReadout.nextPayoff == "Take the first service/rest halt.",

tests/SmokeChecks.cpp:1042: "route readout smoke expected first service as the next payoff")) {

tests/SmokeChecks.cpp:1048: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1059: Check(industrialReadout.nextPayoff.find("rail freight") != std::string::npos,

tests/SmokeChecks.cpp:1060: "route readout smoke expected industrial handoff to point at rail freight");

tests/SmokeChecks.cpp:1106: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1115: return Check(profile.firstPlayableRoute.surfaceArrivalReached, "surface arrival smoke expected route flag to flip at camp anchor") &&

tests/SmokeChecks.cpp:1140: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1141: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1159: Check(gameState.lastEvent.find("service halt") != std::string::npos,

tests/SmokeChecks.cpp:1160: "first combat smoke expected service-halt cue in contact event") &&

tests/SmokeChecks.cpp:1187: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1188: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1203: "first combat handoff smoke expected service-halt resolution cue") &&

tests/SmokeChecks.cpp:1204: Check(gameState.lastEvent.find("Run a first service cycle before pushing the relay node.") != std::string::npos,

tests/SmokeChecks.cpp:1205: "first combat handoff smoke expected first-service runtime objective hint") &&

tests/SmokeChecks.cpp:1207: "first combat handoff smoke expected service-halt route beat");

tests/SmokeChecks.cpp:1219: profile.selectedWorld = "service_handoff_smoke.bwld";

tests/SmokeChecks.cpp:1228: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1229: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1247: return Check(profile.firstPlayableRoute.firstServicePerformed, "workshop handoff smoke expected first service flag") &&

tests/SmokeChecks.cpp:1248: Check(gameState.lastEvent.find("First service halt logged") != std::string::npos,

tests/SmokeChecks.cpp:1249: "workshop handoff smoke expected first service log copy") &&

tests/SmokeChecks.cpp:1252: Check(gameState.lastEvent.find("Route beat: Recovery Sync") != std::string::npos,

tests/SmokeChecks.cpp:1412: profile.story.exitedBunker = true;

```
tests/SmokeChecks.cpp:1413: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1418: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1429: return Check(profile.story.returnedToBase, "debrief handoff smoke expected
returned-to-base flag") &&

tests/SmokeChecks.cpp:1433: Check(gameState.lastEvent.find("rail depot") != std::string::npos,

tests/SmokeChecks.cpp:1434: "debrief handoff smoke expected industrial rail-depot handoff") &&

tests/SmokeChecks.cpp:1441: bool RunRecoveryHandoffSummarySmoke() {

tests/SmokeChecks.cpp:1452: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1453: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1458: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1459: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1468: if (!Check(bunker::CurrentRecoveryHandoffSummary(profile).find("rail freight") !=
std::string::npos,

tests/SmokeChecks.cpp:1469: "recovery handoff smoke expected rail-freight first step")) {

tests/SmokeChecks.cpp:1476: worldState->caravanRouteActive = true;

tests/SmokeChecks.cpp:1477: worldState->railFreightActive = true;

tests/SmokeChecks.cpp:1479: if (!Check(bunker::CurrentRecoveryHandoffSummary(profile).find("orbital uplink")
!= std::string::npos,

tests/SmokeChecks.cpp:1484: worldState->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1487: if (!Check(bunker::CurrentRecoveryHandoffSummary(profile).find("Rail Fortress")
!= std::string::npos,

tests/SmokeChecks.cpp:1492: worldState->railFortressActive = true;

tests/SmokeChecks.cpp:1493: return Check(bunker::CurrentRecoveryHandoffSummary(profile).find("Recovery
Fabricator") != std::string::npos,

tests/SmokeChecks.cpp:1497: bool RunRecoveryBackboneStatusSmoke() {

tests/SmokeChecks.cpp:1501: const auto lockedStatus = bunker::CurrentRecoveryBackboneStatus(profile);

tests/SmokeChecks.cpp:1515: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1516: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1521: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1523: const auto pendingStatus = bunker::CurrentRecoveryBackboneStatus(profile);

tests/SmokeChecks.cpp:1529: profile.story.returnedToBase = true;
```

```
tests/SmokeChecks.cpp:1538: const auto starterStatus = bunker::CurrentRecoveryBackboneStatus(profile);
tests/SmokeChecks.cpp:1544: starterStatus.status.find("rail freight") != std::string::npos,
tests/SmokeChecks.cpp:1545: "recovery backbone smoke expected starter status to point at rail freight: " +
starterStatus.status)) {
tests/SmokeChecks.cpp:1552: worldState->caravanRouteActive = true;
tests/SmokeChecks.cpp:1553: worldState->railFreightActive = true;
tests/SmokeChecks.cpp:1555: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:1559: const auto freightStatus = bunker::CurrentRecoveryBackboneStatus(profile);
tests/SmokeChecks.cpp:1574: worldState->railFortressActive = true;
tests/SmokeChecks.cpp:1581: const auto innerSpurStatus = bunker::CurrentRecoveryBackboneStatus(profile);
tests/SmokeChecks.cpp:1600: worldState->serviceBayActive = true;
tests/SmokeChecks.cpp:1603: const auto stableStatus = bunker::CurrentRecoveryBackboneStatus(profile);
tests/SmokeChecks.cpp:1640: profile.story.exitedBunker = true;
tests/SmokeChecks.cpp:1641: profile.firstPlayableRoute.surfaceArrivalReached = true;
tests/SmokeChecks.cpp:1646: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;
tests/SmokeChecks.cpp:1647: profile.story.returnedToBase = true;
tests/SmokeChecks.cpp:1659: if (!Check(worldState->activeRouteEventType == "service_call",
tests/SmokeChecks.cpp:1660: "route event smoke expected deterministic service-call spawn")) {
tests/SmokeChecks.cpp:1688: return Check(!bunker::HasActiveRouteEvent(*worldState), "route event smoke
expected service-call resolution") &&
tests/SmokeChecks.cpp:1719: profile.story.exitedBunker = true;
tests/SmokeChecks.cpp:1720: profile.firstPlayableRoute.surfaceArrivalReached = true;
tests/SmokeChecks.cpp:1725: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;
tests/SmokeChecks.cpp:1726: profile.story.returnedToBase = true;
tests/SmokeChecks.cpp:1729: profile.character.inventory.push_back({"trade_voucher", 1, 0.0f});
tests/SmokeChecks.cpp:1739: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:1740: worldState->serviceBayActive = true;
tests/SmokeChecks.cpp:1741: worldState->serviceCyclesCompleted = 1;
tests/SmokeChecks.cpp:1751: const int vouchersBefore = countInventory(profile, "trade_voucher");
tests/SmokeChecks.cpp:1767: Check(countInventory(profile, "trade_voucher") == vouchersBefore - 1,
```

```
tests/SmokeChecks.cpp:1787: acidProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1788: acidProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1793: acidProfile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1794: acidProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1797: acidProfile.character.inventory.push_back({"trade_voucher", 1, 0.0f});

tests/SmokeChecks.cpp:1807: acidWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1808: acidWorld->serviceCyclesCompleted = 0;

tests/SmokeChecks.cpp:1818: if (!Check(acidWorld->activeRouteEventType == "service_call",

tests/SmokeChecks.cpp:1819: "weather-weighted route event smoke expected acid rain to bias into service_call
instead of merchant")) {

tests/SmokeChecks.cpp:1837: fogProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1838: fogProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1843: fogProfile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1844: fogProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1847: fogProfile.character.inventory.push_back({"trade_voucher", 1, 0.0f});

tests/SmokeChecks.cpp:1857: fogWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1858: fogWorld->serviceBayActive = true;

tests/SmokeChecks.cpp:1859: fogWorld->serviceCyclesCompleted = 1;

tests/SmokeChecks.cpp:1889: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1890: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1895: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

tests/SmokeChecks.cpp:1896: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1915: alphaWorld->caravanRouteActive = true;

tests/SmokeChecks.cpp:1916: alphaWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1917: alphaWorld->railFreightActive = true;

tests/SmokeChecks.cpp:1919: alphaWorld->railFortressActive = true;

tests/SmokeChecks.cpp:1929: alphaWorld->serviceBayActive = true;

tests/SmokeChecks.cpp:1939: const std::string alphaHandoff =
bunker::CurrentRecoveryHandoffSummary(profile);

tests/SmokeChecks.cpp:1940: const std::string betaHandoff = bunker::CurrentRecoveryHandoffSummary(profile,
"route_summary_beta.bwld");
```



```
tests/SmokeChecks.cpp:1941: const auto alphaBackbone = bunker::CurrentRecoveryBackboneStatus(profile);
tests/SmokeChecks.cpp:1942: const auto betaBackbone = bunker::CurrentRecoveryBackboneStatus(profile,
"route_summary_beta.bwld");
tests/SmokeChecks.cpp:1948: Check(betaObjective.find("rail depot") != std::string::npos,
tests/SmokeChecks.cpp:1949: "world-scoped route summary smoke expected beta preview objective to stay at
rail depot: " + betaObjective) &&
tests/SmokeChecks.cpp:1952: Check(betaHandoff.find("rail freight") != std::string::npos,
tests/SmokeChecks.cpp:1953: "world-scoped route summary smoke expected beta handoff to point at rail
freight: " + betaHandoff) &&
tests/SmokeChecks.cpp:1958: betaBackbone.status.find("rail freight") != std::string::npos,
tests/SmokeChecks.cpp:1959: "world-scoped route summary smoke expected beta backbone status to stay on
rail freight: " + betaBackbone.status) &&
tests/SmokeChecks.cpp:2626: if (!Check(workshop != nullptr, "service choice smoke expected workshop anchor"))
{
tests/SmokeChecks.cpp:2632: profile.selectedWorld = "service_choice_smoke.bwld";
tests/SmokeChecks.cpp:2681: "service choice smoke expected doctrine-weighted service to recover more
operator HP") &&
tests/SmokeChecks.cpp:2683: "service choice smoke expected doctrine-weighted service to recover more
operator MP") &&
tests/SmokeChecks.cpp:2685: "service choice smoke expected Pilot Sync service pass to recover more BT-72
energy") &&
tests/SmokeChecks.cpp:2687: "service choice smoke expected readable doctrine-weight feedback copy");
tests/SmokeChecks.cpp:2719: bool RunLanlineServicesRoundtripSmoke() {
tests/SmokeChecks.cpp:2732: profile.story.exitedBunker = true;
tests/SmokeChecks.cpp:2733: profile.firstPlayableRoute.surfaceArrivalReached = true;
tests/SmokeChecks.cpp:2738: profile.firstPlayableRoute.firstRecoveryNodeActivated = true;
tests/SmokeChecks.cpp:2739: profile.story.returnedToBase = true;
tests/SmokeChecks.cpp:2744: if (!Check(worldState != nullptr, "lanline services smoke failed to create world field
state")) {
tests/SmokeChecks.cpp:2751: worldState->caravanRouteActive = true;
tests/SmokeChecks.cpp:2755: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:2756: worldState->railFreightActive = true;
tests/SmokeChecks.cpp:2757: worldState->railFortressActive = true;
```

```
tests/SmokeChecks.cpp:2764: worldState->serviceBayActive = true;

tests/SmokeChecks.cpp:2771: if (!Check(unlockState.towerSyncRecovered, "lanline services smoke expected
tower sync recovery flag")) {

tests/SmokeChecks.cpp:2774: if (!Check(unlockState.relaySubstationActive, "lanline services smoke expected
relay substation unlock flag")) {

tests/SmokeChecks.cpp:2777: if (!Check(unlockState.serviceBayActive, "lanline services smoke expected service
bay unlock flag")) {

tests/SmokeChecks.cpp:2780: if (!Check(unlockState.waterReclaimerActive, "lanline services smoke expected
water reclaimer unlock flag")) {

tests/SmokeChecks.cpp:2783: if (!Check(unlockState.backboneStable, "lanline services smoke expected stable
backbone")) {

tests/SmokeChecks.cpp:2786: if (!Check(unlockState.feyRingIntercityUnlocked, "lanline services smoke expected
inter-city Fey unlock")) {

tests/SmokeChecks.cpp:2789: if (!Check(unlockState.feyRingInterserverUnlocked, "lanline services smoke
expected inter-server Fey unlock")) {

tests/SmokeChecks.cpp:2792: if (!Check(bunker::IsTankServiceUnlocked(unlockState), "lanline services smoke
expected tank service unlock")) {

tests/SmokeChecks.cpp:2795: if (!Check(bunker::IsMedicalSupportUnlocked(unlockState), "lanline services smoke
expected medical support unlock")) {

tests/SmokeChecks.cpp:2799: "lanline services smoke expected mirrored stable backbone stage")) {

tests/SmokeChecks.cpp:2803: "lanline services smoke expected mirrored full backbone status")) {

tests/SmokeChecks.cpp:2807: "lanline services smoke expected mirrored backbone payoff")) {

tests/SmokeChecks.cpp:2811: "lanline services smoke expected mirrored idle route-event summary")) {

tests/SmokeChecks.cpp:2814: if (!Check(!unlockState.merchantWindowActive, "lanline services smoke expected
merchant window to start inactive")) {

tests/SmokeChecks.cpp:2818: bunker::LanlineServicesState state =
bunker::MakeDefaultLanlineServicesState(1000);

tests/SmokeChecks.cpp:2822: order.orderId = "order_service_01";

tests/SmokeChecks.cpp:2833: bunker::LanlineServicesProfile profileSnapshot{};

tests/SmokeChecks.cpp:2834: bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, state);

tests/SmokeChecks.cpp:2835: if (!Check(profileSnapshot.relayCredits == 777, "lanline services smoke expected
profile relay credits sync")) {

tests/SmokeChecks.cpp:2838: if (!Check(profileSnapshot.ownedCosmetics.size() == 2, "lanline services smoke
expected cosmetic snapshot sync")) {
```

```
tests/SmokeChecks.cpp:2842: profileSnapshot.pendingSupportOrders[0] == "order_service_01",
tests/SmokeChecks.cpp:2843: "lanline services smoke expected pending support order sync")) {
tests/SmokeChecks.cpp:2847: const bunker::LanlineServicesSave save = bunker::BuildLanlineServicesSave(state);
tests/SmokeChecks.cpp:2848: if (!Check(bunker::SaveLanlineServicesSave(save,
bunker::DefaultLanlineServicesSavePath()),
tests/SmokeChecks.cpp:2849: "lanline services smoke failed to save services state")) {
tests/SmokeChecks.cpp:2853: bunker::LanlineServicesSave loadedSave{};
tests/SmokeChecks.cpp:2854: if
(!Check(bunker::LoadLanlineServicesSave(bunker::DefaultLanlineServicesSavePath(), loadedSave),
tests/SmokeChecks.cpp:2855: "lanline services smoke failed to load services state")) {
tests/SmokeChecks.cpp:2859: bunker::LanlineServicesState loadedState =
bunker::MakeLanlineServicesStateFromSave(loadedSave, 1200);
tests/SmokeChecks.cpp:2860: bunker::ApplyLanlineServicesProfileSnapshot(loadedState, profileSnapshot);
tests/SmokeChecks.cpp:2861: bunker::AdvanceLanlineSupportOrders(loadedState, 1500);
tests/SmokeChecks.cpp:2863: "lanline services smoke expected one delivered support order after time
advance")) {
tests/SmokeChecks.cpp:2871: if (!Check(merchantUnlockState.merchantWindowActive, "lanline services smoke
expected merchant window mirror when active")) {
tests/SmokeChecks.cpp:2875: "lanline services smoke expected merchant route-event summary mirror")) {
tests/SmokeChecks.cpp:2881: "lanline services smoke expected delivered support order claim")) {
tests/SmokeChecks.cpp:2886: bunker::SyncLanlineServicesSessionProfile(profile, loadedState);
tests/SmokeChecks.cpp:2887: bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, loadedState);
tests/SmokeChecks.cpp:2888: return Check(loadedState.relayCredits == 777, "lanline services smoke expected
relay credits after roundtrip") &&
tests/SmokeChecks.cpp:2889: Check(loadedState.ownedCosmetics.size() == 2, "lanline services smoke expected
owned cosmetics after roundtrip") &&
tests/SmokeChecks.cpp:2890: Check(loadedState.supportOrders.size() == 1, "lanline services smoke expected one
support order after roundtrip") &&
tests/SmokeChecks.cpp:2892: "lanline services smoke expected destination node after roundtrip") &&
tests/SmokeChecks.cpp:2894: "lanline services smoke expected claimed support order after runtime claim") &&
tests/SmokeChecks.cpp:2896: "lanline services smoke expected pending support orders to clear after claim") &&
tests/SmokeChecks.cpp:2898: "lanline services smoke expected world relay credit spend mirror after session
sync") &&
```

```
tests/SmokeChecks.cpp:2900: "inline services smoke expected claim summary to mention delivered order") &&
tests/SmokeChecks.cpp:2907: "inline services smoke expected engine service kit delivery to grant engine_seal
inventory");

tests/SmokeChecks.cpp:2935: "tank service smoke expected engine kit to apply first")) {

tests/SmokeChecks.cpp:2938: if (!Check(countItem("engine_seal") == 0, "tank service smoke expected engine
seal to be consumed")) {

tests/SmokeChecks.cpp:2942: "tank service smoke expected engine kit to restore power core and reserve
charge")) {

tests/SmokeChecks.cpp:2945: if (!Check(eventText.find("Engine service kit applied") != std::string::npos,
tests/SmokeChecks.cpp:2946: "tank service smoke expected engine kit event text")) {

tests/SmokeChecks.cpp:2952: "tank service smoke expected suspension kit to apply second")) {

tests/SmokeChecks.cpp:2955: if (!Check(countItem("track_patch") == 0, "tank service smoke expected track
patch to be consumed")) {

tests/SmokeChecks.cpp:2959: "tank service smoke expected suspension kit to restore hull carriage and bucket
rig")) {

tests/SmokeChecks.cpp:2963: "tank service smoke expected suspension kit event text")) {

tests/SmokeChecks.cpp:2969: "tank service smoke expected sensor kit to apply third")) {

tests/SmokeChecks.cpp:2972: if (!Check(countItem("lens_pack") == 0, "tank service smoke expected lens pack to
be consumed")) {

tests/SmokeChecks.cpp:2976: "tank service smoke expected sensor kit to restore optics and cockpit feed")) {

tests/SmokeChecks.cpp:2980: "tank service smoke expected sensor kit event text")) {

tests/SmokeChecks.cpp:2986: "tank service smoke expected turret kit to apply fourth")) {

tests/SmokeChecks.cpp:2989: if (!Check(countItem("servo_patch") == 0, "tank service smoke expected servo
patch to be consumed")) {

tests/SmokeChecks.cpp:2993: "tank service smoke expected turret kit to restore turret integrity")) {

tests/SmokeChecks.cpp:2996: if (!Check(eventText.find("Turret service kit applied") != std::string::npos,
tests/SmokeChecks.cpp:2997: "tank service smoke expected turret kit event text")) {

tests/SmokeChecks.cpp:3001: "tank service smoke expected successful service to clear in-repair flag")) {

tests/SmokeChecks.cpp:3007: "tank service smoke expected missing kit fallback after all kits are consumed")) {

tests/SmokeChecks.cpp:3010: if (!Check(eventText.find("Compatible tank service kit not found.") !=
std::string::npos,
tests/SmokeChecks.cpp:3011: "tank service smoke expected missing kit message")) {
```

```
tests/SmokeChecks.cpp:3019: "tank service smoke expected idle tank not to consume a valid kit") &&
tests/SmokeChecks.cpp:3021: "tank service smoke expected idle tank to keep the suspension kit") &&
tests/SmokeChecks.cpp:3023: "tank service smoke expected explicit no-damage guidance");
tests/SmokeChecks.cpp:3026: bool RunLanlineSeatRoundtripSmoke() {
tests/SmokeChecks.cpp:3027: bunker::LanlineSessionState savedState;
tests/SmokeChecks.cpp:3041: if (!Check(bunker::SaveLanlineSessionState(savedState), "lanline seat smoke failed
to save session state")) {
tests/SmokeChecks.cpp:3045: bunker::LanlineSessionState loadedState;
tests/SmokeChecks.cpp:3046: if (!Check(bunker::LoadLanlineSessionState(loadedState), "lanline seat smoke
failed to load session state")) {
tests/SmokeChecks.cpp:3096: if (!Check(bunker::NormalizeGameplayDescriptorTag("workshop_field_service") ==
"workshop_service", "workshop alias normalization failed")) {
tests/SmokeChecks.cpp:3150: (issue.relatedValue == "service_bay" ||
tests/SmokeChecks.cpp:3172: if (!Check(dependencyTags[0] == "service_bay" &&
tests/SmokeChecks.cpp:3210: if (edge.sourceScriptTag == "water_reclaimer" && edge.dependencyScriptTag ==
"service_bay")) {
tests/SmokeChecks.cpp:3219: if (edge.sourceScriptTag == "service_bay" && edge.dependencyScriptTag ==
"foundry_line")) {
tests/SmokeChecks.cpp:3230: Check(sawWaterToService, "semantic dependency graph missing water_reclaimer -
> service_bay edge") &&
tests/SmokeChecks.cpp:3233: Check(sawServiceToFoundry, "semantic dependency graph missing service_bay ->
foundry_line edge") &&
tests/SmokeChecks.cpp:3263: const auto* serviceBay = world.FindObjectByScriptTag("service_bay");
tests/SmokeChecks.cpp:3271: if (!Check(serviceBay != nullptr &&
tests/SmokeChecks.cpp:3284: const float firstLayerX = serviceBay->x;
tests/SmokeChecks.cpp:3299: Check(serviceBay->y < relaySubstation->y && relaySubstation->y <
recoveryFabricator->y,
tests/SmokeChecks.cpp:3320: bunker::MapObject serviceBay;
tests/SmokeChecks.cpp:3321: serviceBay.registryId = "[%service_manual_0001]";
tests/SmokeChecks.cpp:3322: serviceBay.displayName = "Service Bay";
tests/SmokeChecks.cpp:3323: serviceBay.interaction = bunker::InteractionType::Terminal;
tests/SmokeChecks.cpp:3324: serviceBay.category = bunker::ObjectCategory::Terminal;
```

```
tests/SmokeChecks.cpp:3325: serviceBay.scriptTag = "service_bay";

tests/SmokeChecks.cpp:3326: serviceBay.linkTarget = "inner_spur_service";

tests/SmokeChecks.cpp:3327: serviceBay.x = 27.0f;

tests/SmokeChecks.cpp:3328: serviceBay.y = -17.0f;

tests/SmokeChecks.cpp:3329: world.objects.push_back(serviceBay);

tests/SmokeChecks.cpp:3352: auto* pinnedRecoveryFabricator =
world.FindObjectByScriptTag("recovery_fabricator");

tests/SmokeChecks.cpp:3353: if (!Check(pinnedRecoveryFabricator != nullptr, "preserve-manual layout smoke
expected recovery_fabricator after cascade")) {

tests/SmokeChecks.cpp:3356: pinnedRecoveryFabricator->semanticLayoutPinned = true;

tests/SmokeChecks.cpp:3357: const float pinnedRecoveryX = pinnedRecoveryFabricator->x;

tests/SmokeChecks.cpp:3358: const float pinnedRecoveryY = pinnedRecoveryFabricator->y;

tests/SmokeChecks.cpp:3367: const auto* authoredServiceBay = world.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:3390: "preserve-manual layout should not move authored service bay")) {

tests/SmokeChecks.cpp:3397: if (!Check(almostEqual(recoveryFabricator->x, pinnedRecoveryX) &&
almostEqual(recoveryFabricator->y, pinnedRecoveryY),

tests/SmokeChecks.cpp:3430: "full semantic reflow should align authored service bay into the first-layer column")
&&

tests/SmokeChecks.cpp:3471: "service_bay",

tests/SmokeChecks.cpp:3618: Check(loaderPrefabs[0].object.lootEntries.size() == 5, "prefab library should
preserve scalable loot entries without trailing empty UI rows") &&

tests/SmokeChecks.cpp:3688: relayPrefab.id = "prefab_relay_service";

tests/SmokeChecks.cpp:3731: brokenObject.prefabSourceId = "prefab_missing_service";

tests/SmokeChecks.cpp:4092: if (!Check(std::filesystem::exists(prototypeExport.auditTrailPath), "prototype
export should append export audit trail")) {

tests/SmokeChecks.cpp:4169: bool RunWorldExportAuditTrailSmoke() {

tests/SmokeChecks.cpp:4171: world.metadata.name = "World Export Audit Trail Smoke";

tests/SmokeChecks.cpp:4184: if (!Check(batch.createdCount == 7, "export audit trail smoke expected seven auto-
created dependency anchors")) {

tests/SmokeChecks.cpp:4193: if (!Check(prototypeExport.ok, "export audit trail smoke expected prototype export
to pass")) {

tests/SmokeChecks.cpp:4202: "export audit trail smoke expected strict export to block by policy")) {
```

```
tests/SmokeChecks.cpp:4207: if (!Check(adoptedAnchors == 7, "export audit trail smoke expected adopt-all to
convert seven anchors")) {

tests/SmokeChecks.cpp:4215: if (!Check(shippingExport.ok, "export audit trail smoke expected shipping export to
pass after adopt-all")) {

tests/SmokeChecks.cpp:4218: if (!Check(std::filesystem::exists(shippingExport.auditTrailPath), "export audit trail
smoke expected audit trail file")) {

tests/SmokeChecks.cpp:4222: const std::string auditTrail =
bunker::LoadTextArtifactPreview(shippingExport.auditTrailPath, 20000);

tests/SmokeChecks.cpp:4223: return Check(auditTrail.find("Policy: prototype / allow warnings") !=
std::string::npos,

tests/SmokeChecks.cpp:4224: "export audit trail should include prototype export entry") &&

tests/SmokeChecks.cpp:4225: Check(auditTrail.find("Policy: shipping-safe") != std::string::npos,

tests/SmokeChecks.cpp:4226: "export audit trail should include shipping export entries") &&

tests/SmokeChecks.cpp:4227: Check(auditTrail.find("Decision: blocked by policy") != std::string::npos,

tests/SmokeChecks.cpp:4228: "export audit trail should record blocked shipping export") &&

tests/SmokeChecks.cpp:4229: Check(auditTrail.find("Auto-created semantic anchors: 0") != std::string::npos,

tests/SmokeChecks.cpp:4230: "export audit trail should record zero semantic debt after adopt-all") &&

tests/SmokeChecks.cpp:4231: Check(auditTrail.find("Decision: exported") != std::string::npos,

tests/SmokeChecks.cpp:4232: "export audit trail should record successful exports");

tests/SmokeChecks.cpp:4353: auto* serviceBay = world.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:4355: if (!Check(serviceBay != nullptr, "shipping baseline object-aware smoke expected
service_bay")) {

tests/SmokeChecks.cpp:4362: const std::string serviceBayRegistryId = serviceBay->registryId;

tests/SmokeChecks.cpp:4363: serviceBay->linkTarget = "service_bay_drift";

tests/SmokeChecks.cpp:4374: serviceBay->linkTarget = "inner_spur_service";

tests/SmokeChecks.cpp:4396: if (!Check(objectAwareDelta.issueImprovements[0].objectId ==
serviceBayRegistryId,

tests/SmokeChecks.cpp:4397: "shipping baseline object-aware smoke expected service_bay baseline drift to be
marked as improved")) {

tests/SmokeChecks.cpp:4404: Check(objectAwareReport.find(serviceBayRegistryId) != std::string::npos,

tests/SmokeChecks.cpp:4405: "shipping baseline object-aware report should mention improved service_bay
object");

tests/SmokeChecks.cpp:4432: auto* serviceBay = world.FindObjectByScriptTag("service_bay");
```

```
tests/SmokeChecks.cpp:4434: if (!Check(serviceBay != nullptr, "export history selection smoke expected
service_bay")) {

tests/SmokeChecks.cpp:4441: const std::string serviceBayRegistryId = serviceBay->registryId;

tests/SmokeChecks.cpp:4446: std::filesystem::remove(bunker::ExportAuditTrailPathForWorld(exportPath),
cleanupEc);

tests/SmokeChecks.cpp:4475: serviceBay->linkTarget = "service_bay_history_drift";

tests/SmokeChecks.cpp:4488: serviceBay->linkTarget = "inner_spur_service";

tests/SmokeChecks.cpp:4670: Check(olderDelta.issueImprovements[0].objectId == serviceBayRegistryId,

tests/SmokeChecks.cpp:4671: "export history selection smoke expected older checkpoint compare to point at
service_bay improvement") &&

tests/SmokeChecks.cpp:4676: Check(olderReport.find(serviceBayRegistryId) != std::string::npos,

tests/SmokeChecks.cpp:4677: "export history selection smoke expected report to mention improved service_bay
object") &&

tests/SmokeChecks.cpp:4714: WriteRawString(file, "[%service_bay_auto_0001]");

tests/SmokeChecks.cpp:4716: WriteRawString(file, "service_bay");

tests/SmokeChecks.cpp:4717: WriteRawString(file, "inner_spur_service");

tests/SmokeChecks.cpp:4755: const auto* serviceBay = loadedWorld.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:4756: return Check(serviceBay != nullptr, "legacy semantic inference should load
service_bay") &&

tests/SmokeChecks.cpp:4757: Check(bunker::IsAutoGeneratedSemanticAnchor(*serviceBay), "legacy BWL2 auto
anchor should infer semanticAutoCreated from registryId") &&

tests/SmokeChecks.cpp:4758: Check(!bunker::IsPinnedSemanticAnchor(*serviceBay), "legacy BWL2 auto anchor
should not infer pinned semantic placement");

tests/SmokeChecks.cpp:4848: WriteRawString(file, "[%services_0001]");

tests/SmokeChecks.cpp:4849: WriteRawString(file, "Lanline Services Relay");

tests/SmokeChecks.cpp:4850: WriteRawString(file, "lanline_service_hub");

tests/SmokeChecks.cpp:4851: WriteRawString(file, "shelter17_services");

tests/SmokeChecks.cpp:4893: const auto* servicesHub =
loadedWorld.FindObjectByScriptTag("lanline_service_hub");

tests/SmokeChecks.cpp:4895: return Check(servicesHub != nullptr, "legacy editor-layer inference should load
service hub") &&

tests/SmokeChecks.cpp:4896: Check(servicesHub->editorLayer == "Service", "legacy BWL3 object should infer
Service editor layer from scriptTag") &&
```



```
tests/SmokeChecks.cpp:4921: addedObject.registryId = "[%service_0001]";

tests/SmokeChecks.cpp:4925: addedObject.scriptTag = "lanline_service_hub";

tests/SmokeChecks.cpp:4926: addedObject.linkTarget = "shelter17_services";

tests/SmokeChecks.cpp:4928: undoStack.PushObjectAdded("Add service relay", world.objects.back(), 1,
addedObject.registryId);

tests/SmokeChecks.cpp:4950: undoStack.PushObjectUpdated("Edit service relay", beforeFirstUpdate,
world.objects[1], 1);

tests/SmokeChecks.cpp:4960: undoStack.PushObjectUpdated("Edit service relay", beforeSecondUpdate,
world.objects[1], 1);

tests/SmokeChecks.cpp:5022: undoStack.PushObjectRemoved("Delete service relay", removedObject, 1,
removedObject.registryId, rootObject.registryId);

tests/SmokeChecks.cpp:5176: "service_bay",

tests/SmokeChecks.cpp:5247: "rail_depot",

tests/SmokeChecks.cpp:5249: "rail_fortress_hub",

tests/SmokeChecks.cpp:5259: "service_bay",

tests/SmokeChecks.cpp:5261: "lanline_service_hub",

tests/SmokeChecks.cpp:5263: "medical_support",

tests/SmokeChecks.cpp:5264: "tank_service",

tests/SmokeChecks.cpp:5615: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:5715: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:5744: "bt72_hull_moved_to_service_lift_before_core_install expected crane transfer to
lock cradle")) {

tests/SmokeChecks.cpp:5762: "bt72_staged_restoration_requires_service_notes expected service notes to gate
final restoration")) {

tests/SmokeChecks.cpp:5767: "bt72_staged_restoration_requires_service_notes expected staged restoration
after notes")) {

tests/SmokeChecks.cpp:5815: addTerminal("[%bt72_service_lift_0001]", "BT-72 Service Lift");

tests/SmokeChecks.cpp:5828: const auto* lift = world.FindObjectByRegistryId("[%bt72_service_lift_0001]");

tests/SmokeChecks.cpp:5870: "bt72_service_lift_requires_attached_hull expected lift to reject unattached hull"))
{

tests/SmokeChecks.cpp:5922: addObject("[%bt72_service_notes_0001]", "BT-72 Service Notes",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);
```

tests/SmokeChecks.cpp:5934: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, lockedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5936: !bunker::HasCollectedTapeld(lockedProfile, "bt72_service_reel_001"),

tests/SmokeChecks.cpp:5937: "bt72_service_notes_require_pippad expected service notes to stay locked before Pip-Pad")) {

tests/SmokeChecks.cpp:5944: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, notesProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5946: bunker::HasCollectedTapeld(notesProfile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:5949: "bt72_service_notes_runtime_recovered expected runtime service notes without tank unlock")) {

tests/SmokeChecks.cpp:5963: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, notesProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5966: bunker::HasCollectedTapeld(notesProfile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:5970: "bt72_service_material_pickups_are_idempotent expected stable duplicate interactions");

tests/SmokeChecks.cpp:5987: "[%bt72_service_lift_0001]",

tests/SmokeChecks.cpp:5988: "[%bt72_service_notes_0001]",

tests/SmokeChecks.cpp:6002: "[%bt72_service_lift_0001]",

tests/SmokeChecks.cpp:6003: "[%bt72_service_notes_0001]",

tests/SmokeChecks.cpp:6041: hasTypedObject("[%bt72_service_lift_0001]", bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal) &&

tests/SmokeChecks.cpp:6042: hasTypedObject("[%bt72_service_notes_0001]", bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal) &&

tests/SmokeChecks.cpp:6078: hasObject("[%bt72_service_lift_0001]"),

tests/SmokeChecks.cpp:6082: return Check(hasObject("[%bt72_service_notes_0001]") &&

tests/SmokeChecks.cpp:6084: "starter_world_contains_bt72_service_material_objects expected service/material runtime objects");

tests/SmokeChecks.cpp:6119: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:6129: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:6137: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6144: bunker::HasCollectedTapeld(profile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:6156: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6420: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6440: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6563: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), loadedWorld, player, loadedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:6583: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld, player, loadedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:6757: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6763: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6804: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6824: surfaceProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:6830: returnedProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:6879: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6912: pipBoy10Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6925: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6937: noDeviceProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6982: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6990: pipBoy10Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6998: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7005: noDeviceProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7055: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7086: invalidProfile.pipDeviceCarryMode = "drone";

tests/SmokeChecks.cpp:7146: loadedProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7238: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7433: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7496: pippadProfile.story.exitedBunker = true;

```
tests/SmokeChecks.cpp:8132: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"),
loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8149: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld,
player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8201: addObject("[%bt72_service_lift_0001]", "BT-72 Service Lift",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);

tests/SmokeChecks.cpp:8202: addObject("[%bt72_service_notes_0001]", "BT-72 Service Notes",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);

tests/SmokeChecks.cpp:8228: "first_route_e2e_pippad_recovered_before_ui expected access card and pre-Pip-
Pad trail")) {

tests/SmokeChecks.cpp:8256: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:8272: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world,
player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8288: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player,
profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8472: RunRecoveryHandoffSummarySmoke() &&

tests/SmokeChecks.cpp:8473: RunRecoveryBackboneStatusSmoke() &&

tests/SmokeChecks.cpp:8491: RunLanlineServicesRoundtripSmoke() &&

tests/SmokeChecks.cpp:8493: RunLanlineSeatRoundtripSmoke() &&

tests/SmokeChecks.cpp:8509: RunWorldExportAuditTrailSmoke() &&
```

Success

Ran command

```
$ Get-Content include\SessionProfiles.hpp | Select-Object -Skip 120 -First 130
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
bool capacitorBankActive = false;
```

```
int capacitorDischargeCycles = 0;
```

```
bool relaySubstationActive = false;
```

```
int relaySyncCycles = 0;
```

```
bool serviceBayActive = false;
```

```
int serviceCyclesCompleted = 0;
```

```
bool waterReclaimerActive = false;
```

```
int waterCyclesCompleted = 0;
```

```
bool feyRingIntercityUnlocked = false;

bool feyRingInterserverUnlocked = false;

int relayCreditsEarned = 0;

int relayCreditsSpent = 0;

std::string activeRouteEventType{};

float routeEventTimeRemaining = 0.0f;

float routeEventCooldown = 0.0f;

float routeEventOfferTimeRemaining = 0.0f;

int routeEventProgress = 0;

int routeEventStage = 0;

int routeEventSerial = 0;

int routeEventsResolved = 0;

int routeEventsFailed = 0;

int routeEventsExpired = 0;

std::string lastRouteEventType{};

std::string lastRouteEventOutcome{};

};

struct SkillAwakeningProgress {

int archiveSynchs = 0;

int footKills = 0;

int tankActions = 0;

int stressSurvivals = 0;

int heavyCarryDrills = 0;

int fieldServiceUses = 0;

};

struct FirstPlayableRouteProgress {

bool introSeen = false;

bool emergencyMeleeRecovered = false;

bool earlyVerminEncounterResolved = false;

bool accessCardRecovered = false;
```

```
int prePipPadClueCount = 0;

bool bt72HullInspected = false;

bool bt72CoreRecovered = false;

bool bt72ServiceNotesRecovered = false;

bool bt72Restored = false;

bool clearanceBlueprintRecovered = false;

bool clearanceMaterialsRecovered = false;

bool clearanceModuleInstalled = false;

bool surfaceArrivalReached = false;

bool firstTankCombatResolved = false;

bool firstServicePerformed = false;

bool firstRecoveryNodeActivated = false;

bool debriefSummaryViewed = false;

};

struct StoryProgress {

bool awakenedFromCryo = false;

bool pipPadRecovered = false;

bool archiveRecovered = false;

bool tankLinked = false;

bool bucketRecovered = false;

bool exitedBunker = false;

bool outerRoadCleared = false;

bool relayRecovered = false;

bool returnedToBase = false;

};

enum class VehicleType {

Motorcycle,

Car,

Truck,

Helicopter,
```

```
Boat,  
  
Utility  
  
};  
  
enum class TankClass {  
  
Vanguard,  
  
Destroyer,  
  
Tactician  
  
};  
  
enum class ShelterDoctrine {  
  
Balanced,  
  
Industry,  
  
Defense,  
  
Medical  
  
};  
  
enum class TankModuleSlotType {  
  
Chassis,  
  
Turret,  
  
PrimaryWeapon,  
  
SecondaryWeapon,  
  
Utility,  
  
Sensor,  
  
PowerCore,  
  
Bucket  
  
};  
  
struct TankModuleSlot {  
  
TankModuleSlotType type = TankModuleSlotType::Utility;  
  
std::string moduleId;  
  
std::string displayName;  
  
bool installed = false;  
  
float durability = 100.0f;
```

```

};

struct TankDamageState {

float hull = 100.0f;

float turret = 100.0f;

float bucket = 100.0f;

float sensors = 100.0f;

float cockpit = 100.0f;

float powerCore = 100.0f;

};

struct TankLoadout {

std::string chassisId = "[#tr9300]";

std::string hullName = "BT-Partner Hull";

std::string turretId = "turret_field_mk1";

std::string turretName = "Field Turret Mk.I";

std::vector<std::string> primaryWeapons = {"ptrs_41_mount"};

std::vector<std::string> secondaryWeapons = {"coax_mg"};

std::vector<TankModuleSlot> modules = {

{TankModuleSlotType::Chassis, "chassis_bt_partner", "Partner Chassis", true, 100.0f},

{TankModuleSlotType::Turret, "turret_field_mk1", "Field Turret Mk.I", true, 100.0f},

{TankModuleSlotType::PrimaryWeapon, "ptrs_41_mount", "PTRS-41 Mount", true, 100.0f},

{TankModuleSlotType::SecondaryWeapon, "coax_mg", "Coax MG", true, 100.0f},

{TankModuleSlotType::Sensor, "nerv_sensor_mk1", "Nerv Sensor Mk.I", true, 100.0f},

Success

Ran command

$ Get-Content src\WorldEvents.cpp

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

#include "../include/WorldEvents.hpp"

#include <cmath>

namespace bunker {

namespace {

```



```

bool IsNear(const PlayerState& player, float x, float y, float radius) {
    const float dx = player.x - x;
    const float dy = player.y - y;
    return (dx * dx) + (dy * dy) <= radius * radius;
}

void AppendObjectiveHint(std::string& message, const std::string& objective) {
    if (!objective.empty() && message.find(objective) == std::string::npos) {
        message += " Next: " + objective;
    }
}

void AppendObjectivePreviewHint(std::string& message, const SessionProfile& profile) {
    AppendObjectiveHint(message, CurrentStoryObjectivePreview(profile));
}

void AppendRouteBeatReadabilityHint(std::string& message, const SessionProfile& profile) {
    const auto beat = CurrentFirstPlayableRouteBeat(profile);
    if (!beat.label.empty() && message.find("Route beat: " + beat.label) == std::string::npos) {
        message += " Route beat: " + beat.label + ".";
    }
    if (!beat.payoff.empty() && message.find(beat.payoff) == std::string::npos) {
        message += " Readable payoff: " + beat.payoff;
    }
}

bool IsAtSurfaceArrivalAnchor(const World& world, const PlayerState& player) {
    if (const auto* campMarker = world.FindObjectByRegistryId("[%camp_0001]"); campMarker != nullptr) {
        return IsNear(player, campMarker->x, campMarker->y, 5.0f);
    }
    return player.x >= 18.0f;
}

bool IsAtFirstCombatAnchor(const World& world, const PlayerState& player) {
    if (const auto* firstContact = world.FindObjectByRegistryId("[%enemy_ghoul_0001]"); firstContact != nullptr) {

```

```

return IsNear(player, firstContact->x, firstContact->y, 6.5f);
}

return player.x >= 17.0f;
}

} // namespace

void ProcessScriptedWorldEvents(const World& world, const PlayerState& player, SessionProfile& profile,
GameState& gameState) {
if (!world.IsStarterScenarioWorld()) {
return;
}

if (profile.story.awakenedFromCryo && !profile.story.pipPadRecovered && !gameState.zoneEventCryoLocker &&
IsNear(player, -9.0f, -6.0f, 2.5f)) {
gameState.zoneEventCryoLocker = true;

gameState.lastEvent = profile.firstPlayableRoute.accessCardRecovered
? "LOCKER BAY: 'Emergency recovery locker detected. Card interlock accepts the bunker access card. Pip-Pad
signature nearby.'"
: "LOCKER BAY: 'Emergency recovery locker detected. Mechanical interlock wants a bunker access card from the
core service racks.'";

return;
}

if (profile.story.pipPadRecovered && !profile.story.archiveRecovered && !gameState.zoneEventArchive &&
IsNear(player, -7.0f, -1.0f, 3.0f)) {
gameState.zoneEventArchive = true;

gameState.lastEvent = "ARCHIVE: 'Missing personnel logs available. Expect vermin contamination in the corridor
and garage references in the recovered notes.'";

return;
}

if (profile.story.archiveRecovered && !profile.firstPlayableRoute.bt72Restored && !gameState.zoneEventGarage
&&
IsNear(player, 4.0f, -1.5f, 4.0f)) {
gameState.zoneEventGarage = true;

```

```
gameState.lastEvent = "GARAGE: 'BT-72 hull located. Pairing is still locked: inspect the hull, recover the core, and pull the maintenance echo first.'";

return;

}

if (profile.story.exitedBunker &&

!profile.firstPlayableRoute.surfaceArrivalReached &&

IsAtSurfaceArrivalAnchor(world, player)) {

profile.firstPlayableRoute.surfaceArrivalReached = true;

gameState.zoneEventExterior = true;

gameState.lastEvent = "SURFACE ARRIVAL: 'BT-72 reached the first exterior foothold. Skyline active, debris barrier ahead, and the first hostile contact is forming beyond the lift route.'";

AppendObjectivePreviewHint(gameState.lastEvent, profile);

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

}

if (profile.story.outerRoadCleared &&

!profile.firstPlayableRoute.firstTankCombatResolved &&

!gameState.zoneEventFirstCombat &&

IsAtFirstCombatAnchor(world, player)) {

gameState.zoneEventFirstCombat = true;

gameState.lastEvent = player.insideTank

? "CONTACT: 'Outer Ghoul charging the cleared lane. Hold BT-72 through the rush, then take one service halt before syncing the relay.'"

: "CONTACT: 'Outer Ghoul detected beyond the cleared debris lane. BT-72 is the intended answer for the first surface contact.'";

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

}

if (profile.story.relayRecovered && !profile.story.returnedToBase && !gameState.zoneEventReturn &&

IsNear(player, -2.5f, 3.2f, 3.0f)) {

gameState.zoneEventReturn = true;
```

```

gameState.lastEvent = "DEBRIEF: 'Recovery node recognized. Upload the route summary and authorize Shelter 17
planning: rail, orbital, fabrication, then the inner spur.'";

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

}

if (profile.story.returnedToBase && !gameState.zoneEventReturn &&
IsNear(player, -2.5f, 3.2f, 3.0f)) {

gameState.zoneEventReturn = true;

gameState.lastEvent = "DEBRIEF: 'Recovery planning remains active. Keep rebuilding rail freight, orbital support,
fabrication, and the inner spur backbone.'";

}

}

} // namespace bunker

Success

Ran command

$ Get-Content src\LanlineServices.cpp | Select-Object -First 220

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

#include "../include/LanlineServices.hpp"

#include <algorithm>

#include <cstdio>

#include <cstring>

#include <fstream>

#include <sstream>

#include "imgui.h"

#include "../include/AppPaths.hpp"

#include "../include/StoryRoute.hpp"

namespace bunker {

namespace {

const char* ToLabel(SupportCategory category) {

switch (category) {

```

```
case SupportCategory::Materials: return "Materials";
case SupportCategory::Skins: return "Skins";
case SupportCategory::Cosmetics: return "Cosmetics";
case SupportCategory::Utility: return "Utility";
case SupportCategory::TankService: return "Tank Service";
case SupportCategory::Medical: return "Medical";
}

return "Unknown";

}

const char* ToLabel(StoreCurrency currency) {
switch (currency) {
case StoreCurrency::InGame: return "Recovery Scrip";
case StoreCurrency::SymbolicSupport: return "Symbolic Support";
}

return "Unknown";

}

const char* ToLabel(SupportOrderState state) {
switch (state) {
case SupportOrderState::Draft: return "Draft";
case SupportOrderState::Queued: return "Queued";
case SupportOrderState::Routed: return "Routed";
case SupportOrderState::Delivered: return "Delivered";
case SupportOrderState::Claimed: return "Claimed";
}

return "Unknown";

}

const char* ToLabel(FeyGateState state) {
switch (state) {
case FeyGateState::Closed: return "Closed";
case FeyGateState::Open: return "Open";
```

```

case FeyGateState::Transit: return "Transit";
}

return "Unknown";
}

const char* ToLabel(ServicesUnlockTier tier) {
    switch (tier) {
        case ServicesUnlockTier::Locked: return "Locked";
        case ServicesUnlockTier::TowerLinked: return "Tower Linked";
        case ServicesUnlockTier::BackboneStable: return "Backbone Stable";
        case ServicesUnlockTier::RelayExpanded: return "Relay Expanded";
    }
    return "Unknown";
}

const char* DescribeFeyLoad(int queueSize, int capacity) {
    if (capacity <= 0) {
        return "Unknown";
    }

    const float loadRatio = static_cast<float>(queueSize) / static_cast<float>(capacity);

    if (loadRatio >= 0.85f) {
        return "Heavy";
    }

    if (loadRatio >= 0.5f) {
        return "Moderate";
    }

    return "Light";
}

std::string CurrentChatTimeLabel() {
    const std::time_t now = std::time(nullptr);

    std::tm localTime{};

    #if defined(_WIN32)

```

```

localtime_s(&localTime, &now);

#else

localTime = *std::localtime(&now);

#endif

std::ostringstream out;

if (localTime.tm_hour < 10) out << '0';

out << localTime.tm_hour << ':';

if (localTime.tm_min < 10) out << '0';

out << localTime.tm_min;

return out.str();

}

std::string CurrentLanlineTimestamp() {

const std::time_t now = std::time(nullptr);

std::tm localTime{};

#ifdef _WIN32

localtime_s(&localTime, &now);

#else

localTime = *std::localtime(&now);

#endif

std::ostringstream out;

out << (localTime.tm_year + 1900) << '-';

if (localTime.tm_mon + 1 < 10) out << '0';

out << (localTime.tm_mon + 1) << '-';

if (localTime.tm_mday < 10) out << '0';

out << localTime.tm_mday << ' ';

if (localTime.tm_hour < 10) out << '0';

out << localTime.tm_hour << ':';

if (localTime.tm_min < 10) out << '0';

out << localTime.tm_min << ':';

if (localTime.tm_sec < 10) out << '0';

```

```

out << localTime.tm_sec;

return out.str();

}

void NormalizeStringInventory(std::vector<std::string>& values) {
    values.erase(
        std::remove_if(
            values.begin(),
            values.end(),
            [](const std::string& value) { return value.empty(); }),
            values.end());

    std::sort(values.begin(), values.end());

    values.erase(std::unique(values.begin(), values.end()), values.end());
}

std::vector<std::string> CollectPendingSupportOrderIds(const std::vector<SupportOrder>& orders) {
    std::vector<std::string> ids;

    ids.reserve(orders.size());

    for (const auto& order : orders) {
        if (order.orderId.empty() ||
            order.state == SupportOrderState::Delivered ||
            order.state == SupportOrderState::Claimed) {
            continue;
        }

        ids.push_back(order.orderId);
    }

    NormalizeStringInventory(ids);

    return ids;
}

std::string JoinLabels(const std::vector<std::string>& labels) {
    std::ostringstream out;

    for (std::size_t index = 0; index < labels.size(); ++index) {

```



```

if (index > 0) {
    out << ", ";
}
out << labels[index];
}
return out.str();
}

void AddProfileInventoryItem(SessionProfile& profile, const std::string& itemId, int count, float unitWeight) {
    if (itemId.empty() || count <= 0) {
        return;
    }
    for (auto& entry : profile.character.inventory) {
        if (entry.itemId == itemId) {
            entry.count += count;
            if (entry.unitWeight <= 0.0f && unitWeight > 0.0f) {
                entry.unitWeight = unitWeight;
            }
        }
    }
    return;
}

profile.character.inventory.push_back({itemId, count, unitWeight});
}

const SupportCatalogItem* FindSupportCatalogItem(const LanlineServicesState& state, std::string_view itemId) {
    for (const auto& item : state.supportCatalog) {
        if (item.id == itemId) {
            return &item;
        }
    }
    return nullptr;
}

```

```
void GrantSupportDeliveryItem(SessionProfile& profile,
const std::string& itemId,
int count,
float unitWeight,
std::vector<std::string>& grantedLabels) {
    AddProfileInventoryItem(profile, itemId, count, unitWeight);
    grantedLabels.push_back(itemId + " x" + std::to_string(count));
}

void GrantSupportDeliveryContent(SessionProfile& profile,
std::string_view contentId,
std::vector<std::string>& grantedLabels) {
    if (contentId == "bulk_salvage") {
        GrantSupportDeliveryItem(profile, "steel_scrap", 3, 0.5f, grantedLabels);
        GrantSupportDeliveryItem(profile, "old_plate", 1, 0.5f, grantedLabels);
        return;
    }
    if (contentId == "repair_parts" || contentId == "sealant_roll") {
        GrantSupportDeliveryItem(profile, "repair_patch", 1, 0.2f, grantedLabels);
        return;
    }
    if (contentId == "circuit_scrap") {
        GrantSupportDeliveryItem(profile, "copper_wire", 2, 0.2f, grantedLabels);
        return;
    }
    if (contentId == "filter_media") {
        GrantSupportDeliveryItem(profile, "clean_water", 1, 0.4f, grantedLabels);
        return;
    }
    if (contentId == "field_battery") {
        GrantSupportDeliveryItem(profile, "power_cell", 1, 0.3f, grantedLabels);
```

```
return;
}

if (contentId == "track_patch") {

GrantSupportDeliveryItem(profile, "track_patch", 1, 0.2f, grantedLabels);

return;

Success
```

Context automatically compacted

Принял v9.62 как integration pass, не как новую систему. Preflight уже чистый; сейчас сначала читаю существующие петли и счётчики, чтобы найти уже готовый путь и не изобрести второй слой рядом с первым.

Ran 3 commands

Ran command

```
$ git grep -n "scavenger\|scouting\|caravan\|drone\|trade\|rail\|field route\|route
event\|fatigue\|stress\|medicine\|medkit\|field service\|service" -- include src tests

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

include/GameRuntime.hpp:76: bool stressThresholdTriggered = false;

include/GameRuntime.hpp:82: float scavengerTimer = 180.0f;

include/GameRuntime.hpp:83: float caravanTimer = 260.0f;

include/GameRuntime.hpp:84: float droneTimer = 210.0f;

include/GameRuntime.hpp:85: float tradeTimer = 240.0f;

include/GameRuntime.hpp:86: float railTimer = 320.0f;

include/GameRuntime.hpp:88: float railFortressTimer = 520.0f;

include/GameRuntime.hpp:97: float serviceBayTimer = 300.0f;

include/GameRuntime.hpp:119: "SYSTEM: 'Cryo wing unstable. Sweep the shared cryo tier, recover an access
card, and follow the paper trail to the Pip-Pad.'",

include/GameRuntime.hpp:120: "ARCHIVE: 'One reactor core is missing. One body is missing. Garage service
traces remain incomplete.'",

include/HangarSystem.hpp:35: hangar.lastAction = "Partner tank serviced, rearmed, and fully repaired.";

include/HangarSystem.hpp:118: *eventText = "Turret service kit applied. Servo ring and stabilizer alignment
restored.";

include/HangarSystem.hpp:128: *eventText = "Engine service kit applied. Core seals replaced and reserve charge
restored.";
```

```
include/HangarSystem.hpp:207: *eventText = "Tank service kit applied to the selected subsystem.";

include/HangarSystem.hpp:213: *eventText = "Required tank service kit not found.";

include/HangarSystem.hpp:256: *eventText = "Compatible tank service kit not found.";

include/HangarSystem.hpp:264: ? "BT-72 service anchor reports no damaged subsystem that matches the
available kits."

include/HangarSystem.hpp:265: : "Compatible tank service kit not found.";

include/LanlineServices.hpp:33: bool serviceBayActive = false;

include/LanlineServices.hpp:55: return state.serviceBayActive || state.tier >=
ServicesUnlockTier::BackboneStable;

include/SessionProfiles.hpp:74: bool serviceHubKnown = false;

include/SessionProfiles.hpp:91: bool caravanRouteActive = false;

include/SessionProfiles.hpp:92: bool droneStationsActive = false;

include/SessionProfiles.hpp:93: bool tradeNetworkActive = false;

include/SessionProfiles.hpp:94: bool railFreightActive = false;

include/SessionProfiles.hpp:96: bool railFortressActive = false;

include/SessionProfiles.hpp:101: int caravanRunsCompleted = 0;

include/SessionProfiles.hpp:102: int droneRunsCompleted = 0;

include/SessionProfiles.hpp:103: int tradeCyclesCompleted = 0;

include/SessionProfiles.hpp:105: int railRunsCompleted = 0;

include/SessionProfiles.hpp:107: int railFortressDeployments = 0;

include/SessionProfiles.hpp:125: bool serviceBayActive = false;

include/SessionProfiles.hpp:126: int serviceCyclesCompleted = 0;

include/SessionProfiles.hpp:151: int stressSurvivals = 0;

include/SessionProfiles.hpp:427: int scavengerRunsCompleted = 0;

include/SessionProfiles.hpp:521: target.caravanRouteActive = target.caravanRouteActive ||
source.caravanRouteActive;

include/SessionProfiles.hpp:522: target.droneStationsActive = target.droneStationsActive ||
source.droneStationsActive;

include/SessionProfiles.hpp:523: target.tradeNetworkActive = target.tradeNetworkActive ||
source.tradeNetworkActive;

include/SessionProfiles.hpp:524: target.railFreightActive = target.railFreightActive || source.railFreightActive;
```

```
include/SessionProfiles.hpp:526: target.railFortressActive = target.railFortressActive || source.railFortressActive;

include/SessionProfiles.hpp:531: target.caravanRunsCompleted = std::max(target.caravanRunsCompleted,
source.caravanRunsCompleted);

include/SessionProfiles.hpp:532: target.droneRunsCompleted = std::max(target.droneRunsCompleted,
source.droneRunsCompleted);

include/SessionProfiles.hpp:533: target.tradeCyclesCompleted = std::max(target.tradeCyclesCompleted,
source.tradeCyclesCompleted);

include/SessionProfiles.hpp:535: target.railRunsCompleted = std::max(target.railRunsCompleted,
source.railRunsCompleted);

include/SessionProfiles.hpp:537: target.railFortressDeployments = std::max(target.railFortressDeployments,
source.railFortressDeployments);

include/SessionProfiles.hpp:555: target.serviceBayActive = target.serviceBayActive || source.serviceBayActive;

include/SessionProfiles.hpp:556: target.serviceCyclesCompleted = std::max(target.serviceCyclesCompleted,
source.serviceCyclesCompleted);

include/SessionProfiles.hpp:634: return worldState.caravanRouteActive &&

include/SessionProfiles.hpp:641: return worldState.droneStationsActive && HasRegionalGridOnline(profile);

include/SessionProfiles.hpp:645: return worldState.tradeNetworkActive &&

include/SessionProfiles.hpp:648: (worldState.caravanRouteActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:652: return worldState.railFreightActive &&

include/SessionProfiles.hpp:657: (worldState.caravanRouteActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:663: worldState.railFreightActive &&

include/SessionProfiles.hpp:665: (worldState.tradeNetworkActive || worldState.droneStationsActive);

include/SessionProfiles.hpp:669: return worldState.railFortressActive &&

include/SessionProfiles.hpp:670: worldState.railFreightActive &&

include/SessionProfiles.hpp:679: (worldState.railFreightActive || worldState.droneStationsActive) &&

include/SessionProfiles.hpp:680: (worldState.tradeNetworkActive || worldState.orbitalUplinkActive);

include/SessionProfiles.hpp:687: worldState.tradeNetworkActive;

include/SessionProfiles.hpp:694: worldState.tradeNetworkActive;

include/SessionProfiles.hpp:708: worldState.railFortressActive;

include/SessionProfiles.hpp:733: return worldState.serviceBayActive &&

include/SessionProfiles.hpp:741: worldState.serviceBayActive &&

include/WorldExport.hpp:202: std::filesystem::path auditTrailPath{};
```

```
include/WorldExport.hpp:223:std::filesystem::path ExportAuditTrailPathForWorld(const std::filesystem::path&
worldPath);

src/GameRuntime.cpp:128: registryIdLower.find("drone") != std::string::npos ||

src/GameRuntime.cpp:131: displayNameLower.find("drone") != std::string::npos ||

src/GameRuntime.cpp:1386: eventText += " Pilot Sync trimmed service losses and tightened the link.";

src/GameRuntime.cpp:1411: eventText += " Medical doctrine stabilized the operator during service.";

src/GameRuntime.cpp:1460: {"drone_station", "Drone station ledger mirrored. Sweep routes registered to Pip-
Pad."},

src/GameRuntime.cpp:1461: {"rail_depot", "Rail freight depot records mirrored. Heavy spur logistics
registered."},

src/GameRuntime.cpp:1463: {"rail_fortress_hub", "Rail Fortress patrol package mirrored. Spur security doctrine
updated."},

src/GameRuntime.cpp:1473: {"service_bay", "Service bay notes mirrored. BT-72 support route updated."},

src/GameRuntime.cpp:1475: {"lanline_service_hub", "Lanline service hub mirrored. Shelter 17 service catalog
now resolves through authored relay infrastructure."},

src/GameRuntime.cpp:1478: {"tank_service", "Tank service anchor mirrored. BT-72 maintenance route
updated."},

src/GameRuntime.cpp:1481: {"workshop_service", "Workshop terminal mirrored. BT-72 service route and repair
notes updated."},

src/GameRuntime.cpp:1881: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";

src/GameRuntime.cpp:1885: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the gunner
station.";

src/GameRuntime.cpp:1889: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";

src/GameRuntime.cpp:1894: gameState.lastEvent = "BT-72 is moving too fast for field service. Stop the hull first.";

src/GameRuntime.cpp:1898: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable
condition.";

src/GameRuntime.cpp:1902: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a repair
patch.";

src/GameRuntime.cpp:1906: const bool hasEngineer = HasAssignedSpecialistRole(profile, "engineer",
"scavenger_support");

src/GameRuntime.cpp:1923: ? "Field service rack cycled. Scavenger-side engineer tuning improved the repair
pass."

src/GameRuntime.cpp:1924: : "Field service rack cycled. BT-72 patched and partially resupplied in the field.";

src/GameRuntime.cpp:1928: gameState.lastEvent += " First service halt logged for the route.";
```

```
src/GameRuntime.cpp:2004: missing.push_back("service notes");

src/GameRuntime.cpp:2074: : "First surface contact resolved. Run a BT-72 service halt before relay sync.";

src/GameRuntime.cpp:2174: const bool caravanOperational = IsCaravanOperational(profile, *worldState);

src/GameRuntime.cpp:2175: const bool droneOperational = IsDroneStationOperational(profile, *worldState);

src/GameRuntime.cpp:2176: const bool tradeOperational = IsTradeNetworkOperational(profile, *worldState);

src/GameRuntime.cpp:2177: const bool railOperational = IsRailFreightOperational(profile, *worldState);

src/GameRuntime.cpp:2188: const bool serviceOperational = IsServiceBayOperational(*worldState);

src/GameRuntime.cpp:2194: if (caravanOperational) score += 8.0f;

src/GameRuntime.cpp:2195: if (droneOperational) score += 7.0f;

src/GameRuntime.cpp:2196: if (tradeOperational) score += 8.0f;

src/GameRuntime.cpp:2197: if (railOperational) score += 10.0f;

src/GameRuntime.cpp:2209: if (serviceOperational) score += 8.0f;

src/GameRuntime.cpp:2219: score += std::min(6.0f, static_cast<float>(worldState->serviceCyclesCompleted) *
1.0f);

src/GameRuntime.cpp:2222: score += std::min(8.0f, static_cast<float>(worldState->tradeCyclesCompleted) *
0.8f);

src/GameRuntime.cpp:2223: score += std::min(8.0f, static_cast<float>(worldState->railRunsCompleted) * 0.8f);

src/GameRuntime.cpp:2347: if (scriptTag == "drone_station") {

src/GameRuntime.cpp:2353: if (!worldState.droneStationsActive) {

src/GameRuntime.cpp:2360: worldState.droneStationsActive = true;

src/GameRuntime.cpp:2361: gameState.droneTimer = 210.0f;

src/GameRuntime.cpp:2362: gameState.lastEvent = "Drone station brought online. Automated scavenger drones
launched to local sweep routes.";

src/GameRuntime.cpp:2364: worldState.droneStationsActive = false;

src/GameRuntime.cpp:2370: if (scriptTag == "rail_depot") {

src/GameRuntime.cpp:2372: if (!worldState.railFreightActive) {

src/GameRuntime.cpp:2383: worldState.railFreightActive = true;

src/GameRuntime.cpp:2384: gameState.railTimer = 320.0f;

src/GameRuntime.cpp:2385: gameState.lastEvent = "Rail freight spur restored. Heavy salvage trains can now
service the bunker route.";

src/GameRuntime.cpp:2387: worldState.railFreightActive = false;
```

```
src/GameRuntime.cpp:2396: if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {  
src/GameRuntime.cpp:2397: gameState.lastEvent = "Orbital uplink requires a live grid and active rail freight  
backbone.";  
src/GameRuntime.cpp:2416: if (scriptTag == "rail_fortress_hub") {  
src/GameRuntime.cpp:2418: if (!worldState.railFortressActive) {  
src/GameRuntime.cpp:2419: if (!worldState.railFreightActive || !worldState.orbitalUplinkActive) {  
src/GameRuntime.cpp:2420: gameState.lastEvent = "Rail fortress requires active rail freight and orbital uplink  
support.";  
src/GameRuntime.cpp:2429: worldState.railFortressActive = true;  
src/GameRuntime.cpp:2430: gameState.railFortressTimer = 520.0f;  
src/GameRuntime.cpp:2433: worldState.railFortressActive = false;  
src/GameRuntime.cpp:2442: if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {  
src/GameRuntime.cpp:2443: gameState.lastEvent = "Recovery fabricator requires a live grid and rail freight  
support.";  
src/GameRuntime.cpp:2470: if (!worldState.recoveryFabricatorActive || !worldState.railFortressActive ||  
!worldState.orbitalUplinkActive) {  
src/GameRuntime.cpp:2499: if (!worldState.orbitalUplinkActive || !worldState.tradeNetworkActive) {  
src/GameRuntime.cpp:2500: gameState.lastEvent = "Industrial survey needs orbital uplink and trade network  
support.";  
src/GameRuntime.cpp:2569: if (!worldState.assemblyCellActive || !worldState.industrialOutpostActive ||  
!worldState.railFortressActive) {  
src/GameRuntime.cpp:2665: if (scriptTag == "service_bay") {  
src/GameRuntime.cpp:2671: if (!worldState.serviceBayActive) {  
src/GameRuntime.cpp:2679: worldState.serviceBayActive = true;  
src/GameRuntime.cpp:2680: gameState.serviceBayTimer = 300.0f;  
src/GameRuntime.cpp:2681: gameState.lastEvent = "Inner spur service bay brought online. BT-72 can now be  
serviced deeper into the factory belt.";  
src/GameRuntime.cpp:2683: worldState.serviceBayActive = false;  
src/GameRuntime.cpp:2684: gameState.lastEvent = "Inner spur service bay returned to standby.";  
src/GameRuntime.cpp:2691: if (!worldState.serviceBayActive || !worldState.relaySubstationActive ||  
!worldState.recoveryFabricatorActive) {  
src/GameRuntime.cpp:2692: gameState.lastEvent = "Water reclaimer needs a live service bay, relay substation,  
and recovery fabricator first.";
```



```
src/GameRuntime.cpp:2713: if (scriptTag == "lanline_service_hub") {  
src/GameRuntime.cpp:2720: profile.lanlineServices.serviceHubKnown = true;  
src/GameRuntime.cpp:2723: ? "Lanline service hub handshake complete."  
src/GameRuntime.cpp:2724: : "Lanline service hub handshake complete -> " + nearest->linkTarget + ".";  
src/GameRuntime.cpp:2729: if (scriptTag == "tank_service") {  
src/GameRuntime.cpp:2731: if (!worldState.serviceBayActive) {  
src/GameRuntime.cpp:2732: gameState.lastEvent = "Tank service anchor is authored, but the service bay  
backbone is still offline.";  
src/GameRuntime.cpp:2737: gameState.lastSupportAction = "BT-72 serviced through authored tank service  
anchor.";  
src/GameRuntime.cpp:2741: ? "Tank service anchor ready. Bring a compatible service kit from Lanline support or  
field salvage."  
src/GameRuntime.cpp:2742: : "Tank service anchor ready. " + repairEvent;  
src/GameRuntime.cpp:2816: AddCollectedTapelfMissing(profile, "bt72_service_reel_001", "BT-72 Service Reel");  
src/GameRuntime.cpp:2817: gameState.lastEvent = "Maintenance echo resolved. BT-72 service notes and holo-  
records mirrored to Pip-Pad.";  
src/GameRuntime.cpp:2859: if (scriptTag == "workshop_service") {  
src/GameRuntime.cpp:3861: const bool isWorkshop = object.interaction == InteractionType::Workshop ||  
object.scriptTag == "workshop_service";  
src/GameRuntime.cpp:4107: const bool scavengerReady =  
src/GameRuntime.cpp:4112: if (!scavengerReady) {  
src/GameRuntime.cpp:4113: gameState.scavengerTimer = 180.0f;  
src/GameRuntime.cpp:4117: const bool engineerAssignedToScavengers = HasAssignedSpecialistRole(profile,  
"engineer", "scavenger_support");  
src/GameRuntime.cpp:4118: gameState.scavengerTimer -= dt;  
src/GameRuntime.cpp:4119: if (gameState.scavengerTimer > 0.0f) {  
src/GameRuntime.cpp:4137: profile.scavengerRunsCompleted += 1;  
src/GameRuntime.cpp:4138: gameState.scavengerTimer = (engineerAssignedToScavengers ? 150.0f : 180.0f) /  
doctrineScavengerBoost;  
src/GameRuntime.cpp:4148: if (worldState == nullptr || !worldState->caravanRouteActive) {  
src/GameRuntime.cpp:4149: gameState.caravanTimer = 260.0f;  
src/GameRuntime.cpp:4153: const bool caravanReady = CaravanRouteReady(profile);
```

```
src/GameRuntime.cpp:4154: if (!caravanReady) {  
  
src/GameRuntime.cpp:4155: gameState.caravanTimer = 260.0f;  
  
src/GameRuntime.cpp:4165: const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer",  
"scavenger_support");  
  
src/GameRuntime.cpp:4167: gameState.caravanTimer -= dt;  
  
src/GameRuntime.cpp:4168: if (gameState.caravanTimer > 0.0f) {  
  
src/GameRuntime.cpp:4183: worldState->caravanRunsCompleted += 1;  
  
src/GameRuntime.cpp:4184: gameState.caravanTimer = 260.0f / doctrineBoost;  
  
src/GameRuntime.cpp:4186: ? "Autopilot caravan returned under escort support with reinforced cargo intake."  
src/GameRuntime.cpp:4187: : "Autopilot caravan returned from the bunker route with bulk salvage.";   
  
src/GameRuntime.cpp:4192: if (worldState == nullptr || !worldState->droneStationsActive) {  
  
src/GameRuntime.cpp:4193: gameState.droneTimer = 210.0f;  
  
src/GameRuntime.cpp:4198: gameState.droneTimer = 210.0f;  
  
src/GameRuntime.cpp:4208: gameState.droneTimer -= dt;  
  
src/GameRuntime.cpp:4209: if (gameState.droneTimer > 0.0f) {  
  
src/GameRuntime.cpp:4218: worldState->droneRunsCompleted += 1;  
  
src/GameRuntime.cpp:4220: gameState.droneTimer = 210.0f / (doctrineBoost * std::max(1.0f, pylonBoost *  
0.95f));  
  
src/GameRuntime.cpp:4221: gameState.lastEvent = "Automated drone sweep completed. Salvage and ether  
traces transferred to shelter stores.";   
  
src/GameRuntime.cpp:4226: if (worldState == nullptr || !worldState->tradeNetworkActive) {  
  
src/GameRuntime.cpp:4227: gameState.tradeTimer = 240.0f;  
  
src/GameRuntime.cpp:4231: const bool tradeReady = TradeNetworkReady(profile, *worldState);  
  
src/GameRuntime.cpp:4232: if (!tradeReady) {  
  
src/GameRuntime.cpp:4233: gameState.tradeTimer = 240.0f;  
  
src/GameRuntime.cpp:4244: gameState.tradeTimer -= dt;  
  
src/GameRuntime.cpp:4245: if (gameState.tradeTimer > 0.0f) {  
  
src/GameRuntime.cpp:4250: AddInventoryItem(profile, "trade_voucher", vouchers, 0.0f);  
  
src/GameRuntime.cpp:4251: worldState->tradeCyclesCompleted += 1;  
  
src/GameRuntime.cpp:4252: gameState.tradeTimer = 240.0f / doctrineBoost;  
  
src/GameRuntime.cpp:4258: if (worldState == nullptr || !worldState->railFreightActive) {
```

```
src/GameRuntime.cpp:4259: gameState.railTimer = 320.0f;

src/GameRuntime.cpp:4263: const bool railReady = RailFreightReady(profile, *worldState);

src/GameRuntime.cpp:4264: if (!railReady) {

src/GameRuntime.cpp:4265: gameState.railTimer = 320.0f;

src/GameRuntime.cpp:4275: const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer",
"scavenger_support");

src/GameRuntime.cpp:4277: gameState.railTimer -= dt;

src/GameRuntime.cpp:4278: if (gameState.railTimer > 0.0f) {

src/GameRuntime.cpp:4287: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4291: worldState->railRunsCompleted += 1;

src/GameRuntime.cpp:4293: gameState.railTimer = 320.0f / doctrineBoost;

src/GameRuntime.cpp:4325: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4339: if (worldState == nullptr || !worldState->railFortressActive) {

src/GameRuntime.cpp:4340: gameState.railFortressTimer = 520.0f;

src/GameRuntime.cpp:4346: gameState.railFortressTimer = 520.0f;

src/GameRuntime.cpp:4356: gameState.railFortressTimer -= dt;

src/GameRuntime.cpp:4357: if (gameState.railFortressTimer > 0.0f) {

src/GameRuntime.cpp:4364: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4365: worldState->railFortressDeployments += 1;

src/GameRuntime.cpp:4368: gameState.railFortressTimer = 520.0f / (doctrineBoost * infrastructurePenalty);

src/GameRuntime.cpp:4403: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:4426: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:4434: gameState.lastEvent = "Shelter Recovery Checkpoint II secured. Grid and field
service reserves expanded. " + xpEvent;

src/GameRuntime.cpp:4436: AddInventoryItem(profile, "trade_voucher", 2, 0.0f);

src/GameRuntime.cpp:4472: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4509: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4689: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4702: if (worldState == nullptr || !worldState->serviceBayActive) {

src/GameRuntime.cpp:4703: gameState.serviceBayTimer = 300.0f;
```

```
src/GameRuntime.cpp:4707: const bool serviceReady = ServiceBayReady(*worldState);

src/GameRuntime.cpp:4708: if (!serviceReady) {

src/GameRuntime.cpp:4709: gameState.serviceBayTimer = 300.0f;

src/GameRuntime.cpp:4719: gameState.serviceBayTimer -= dt;

src/GameRuntime.cpp:4720: if (gameState.serviceBayTimer > 0.0f) {

src/GameRuntime.cpp:4731: worldState->serviceCyclesCompleted += 1;

src/GameRuntime.cpp:4733: gameState.serviceBayTimer = 300.0f / (doctrineBoost * infrastructurePenalty);

src/GameRuntime.cpp:4734: gameState.lastEvent = "Inner spur service bay completed a repair cycle for BT-72
and the backbone fleet.";

src/GameRuntime.cpp:4763: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:4793: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4815: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4858: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4859: return "ROUTE EVENT OFFERED: Service call tagged near the recovery fringe.
Stabilize BT-72 and complete one honest repair/service pass before the window closes.";

src/GameRuntime.cpp:4881: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4909: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4932: if (routeEventType == "service_call") {

src/GameRuntime.cpp:4964: if (worldState.activeRouteEventType == "service_call") {

src/GameRuntime.cpp:4967: worldState.serviceCyclesCompleted > 0 ||

src/GameRuntime.cpp:4968: worldState.serviceBayActive) {

src/GameRuntime.cpp:4984: if (worldState.caravanRouteActive ||

src/GameRuntime.cpp:4985: worldState.droneStationsActive ||

src/GameRuntime.cpp:4986: worldState.tradeNetworkActive ||

src/GameRuntime.cpp:4987: worldState.railFreightActive) {

src/GameRuntime.cpp:5012: worldState.railFortressActive ||

src/GameRuntime.cpp:5013: worldState.caravanRouteActive ||

src/GameRuntime.cpp:5014: worldState.droneStationsActive) {

src/GameRuntime.cpp:5021: if (worldState.caravanRouteActive ||

src/GameRuntime.cpp:5022: worldState.droneStationsActive ||
```

```
src/GameRuntime.cpp:5023: worldState.tradeNetworkActive ||
src/GameRuntime.cpp:5024: worldState.railFreightActive) {
src/GameRuntime.cpp:5027: if (worldState.serviceBayActive ||
src/GameRuntime.cpp:5035: return worldState.tradeCyclesCompleted > 0 ? 1 : 0;
src/GameRuntime.cpp:5041: if (worldState.activeRouteEventType == "service_call") {
src/GameRuntime.cpp:5055: !worldState.caravanRouteActive &&
src/GameRuntime.cpp:5056: !worldState.droneStationsActive &&
src/GameRuntime.cpp:5057: !worldState.tradeNetworkActive &&
src/GameRuntime.cpp:5058: !worldState.railFreightActive;
src/GameRuntime.cpp:5068: if (routeEventType == "service_call") {
src/GameRuntime.cpp:5070: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:5071: worldState.serviceCyclesCompleted = std::max(worldState.serviceCyclesCompleted,
1);
src/GameRuntime.cpp:5078: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
src/GameRuntime.cpp:5097: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
src/GameRuntime.cpp:5100: worldState.tradeCyclesCompleted = std::max(worldState.tradeCyclesCompleted, 1);
src/GameRuntime.cpp:5103: worldState.tradeCyclesCompleted = std::max(worldState.tradeCyclesCompleted, 1);
src/GameRuntime.cpp:5117: if (routeEventType == "service_call") {
src/GameRuntime.cpp:5121: ? "ROUTE EVENT EXPIRED: The service call window closed. BT-72 reserve pressure
climbed and field upkeep drifted upward."
src/GameRuntime.cpp:5122: : "ROUTE EVENT FAILED: The service call slipped. BT-72 reserve pressure climbed
and field upkeep drifted upward.";
src/GameRuntime.cpp:5175: !worldState.serviceBayActive &&
src/GameRuntime.cpp:5176: (TankNeedsRepair(profile) || worldState.serviceCyclesCompleted == 0)) {
src/GameRuntime.cpp:5177: candidates.push_back({"service_call", 5 +
RouteEventWeatherWeightBonus("service_call", gameState)});
src/GameRuntime.cpp:5198: if ((worldState.caravanRouteActive ||
src/GameRuntime.cpp:5199: worldState.droneStationsActive ||
src/GameRuntime.cpp:5200: worldState.tradeNetworkActive ||
src/GameRuntime.cpp:5201: worldState.railFreightActive) &&
src/GameRuntime.cpp:5202: worldState.serviceCyclesCompleted == 0) {
```

```
src/GameRuntime.cpp:5208: (worldState.tradeNetworkActive ||
src/GameRuntime.cpp:5209: worldState.caravanRouteActive ||
src/GameRuntime.cpp:5210: worldState.droneStationsActive ||
src/GameRuntime.cpp:5211: worldState.railFreightActive ||
src/GameRuntime.cpp:5212: worldState.serviceBayActive) &&
src/GameRuntime.cpp:5214: InventoryCount(profile, "trade_voucher") > 0) &&
src/GameRuntime.cpp:5219: candidates.push_back({"service_call", 1 +
RouteEventWeatherWeightBonus("service_call", gameState)});
src/GameRuntime.cpp:5348: gameState.lastEvent = "No merchant route event is currently open.";
src/GameRuntime.cpp:5355: bool paidWithVoucher = ConsumeInventoryItem(profile, "trade_voucher", 1);
src/GameRuntime.cpp:5359: gameState.lastEvent = "Merchant window needs 1 trade voucher or 90 relay credits
before the broker will commit.";
src/GameRuntime.cpp:5376: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:5381: worldState->tradeCyclesCompleted += 1;
src/GameRuntime.cpp:5946: NormalizeGameplayDescriptorTag(nearest->scriptTag) != "workshop_service") {
src/GameRuntime.cpp:5969: gameState.lastEvent = "Cryostasis terminated. Adjacent pods are split open, several
berths stand empty, and an emergency baton plus stained service sketch were recovered from the capsule tray.";
src/GameRuntime.cpp:5997: AddCollectedTapelfMissing(profile, "bt72_core_service_001", "BT-72 Starter Core
Ledger");
src/GameRuntime.cpp:5998: gameState.lastEvent = "Starter core package recovered from the rack. BT-72 can be
powered once the hull and service notes are ready.";
src/GameRuntime.cpp:6011: : "Garage lift survey already mirrored. BT-72 still needs its core and service notes.";
src/GameRuntime.cpp:6017: gameState.lastEvent = "Garage lift survey mirrored. One serviceable hull plate
recovered for BT-72 restoration.";
src/GameRuntime.cpp:6082: gameState.lastEvent = "BT-72 hull attached to the hangar crane hook. Move it to the
service lift cradle.";
src/GameRuntime.cpp:6086: if (nearest->registryId == "[%bt72_service_lift_0001]") {
src/GameRuntime.cpp:6088: gameState.lastEvent = "BT-72 service lift is waiting for a crane-attached hull.";
src/GameRuntime.cpp:6092: gameState.lastEvent = "BT-72 service lift could not receive the hull. Check crane
power, control, and path state.";
src/GameRuntime.cpp:6095: gameState.lastEvent = "BT-72 hull moved to the service lift and locked in the
restoration cradle.";
```

```
src/GameRuntime.cpp:6110: gameState.lastEvent = "The Pip-Boy/Pip-Pad selection station is still under mechanical card-lock. Pull a bunker access card from the core service racks first.";

src/GameRuntime.cpp:6115: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:6117: ? "Pip-Boy/Pip-Pad station selected the default Pip-Pad 3500. Press TAB to open the Pip-Pad; the bunker paper trail now makes sense."

src/GameRuntime.cpp:6118: : "Pip-Boy/Pip-Pad station selected the default Pip-Pad 3500. Press TAB to open the Pip-Pad; the bunker paper trail is still incomplete.";

src/GameRuntime.cpp:6179: gameState.lastEvent = "Archive sync complete. One reactor core and one body are still missing, and BT-72 service traces terminate near the garage.";

src/GameRuntime.cpp:6186: if (nearest->registryId == "[%bt72_service_notes_0001]") {

src/GameRuntime.cpp:6188: gameState.lastEvent = "BT-72 service notes require Pip-Pad recovery before holo-records can be mirrored.";

src/GameRuntime.cpp:6192: gameState.lastEvent = "BT-72 service notes remain encrypted until archive personnel records are mirrored.";

src/GameRuntime.cpp:6196: gameState.lastEvent = "BT-72 service notes already mirrored to Pip-Pad.";

src/GameRuntime.cpp:6200: AddCollectedTapelfMissing(profile, "bt72_service_reel_001", "BT-72 Service Reel");

src/GameRuntime.cpp:6201: gameState.lastEvent = "BT-72 service notes and holo-records mirrored to Pip-Pad.";

src/GameRuntime.cpp:6241: gameState.lastEvent = "BT-72 hull is disabled. Run workshop service before attempting another link.";

src/GameRuntime.cpp:6396: gameState.lastEvent = "BT-72 is still hot off first contact. Run one service/rest cycle before syncing the node.";

src/GameRuntime.cpp:6521: gameState.lastEvent = "Workshop grid is offline. Use field service or return after first combat for a manual recovery pass.";

src/GameRuntime.cpp:6525: gameState.lastEvent = "Workshop cannot locate BT-72 in the local service grid.";

src/GameRuntime.cpp:6529: gameState.lastEvent = "Bring BT-72 into the workshop service bay first.";

src/GameRuntime.cpp:6581: gameState.lastEvent += " First service halt logged for the route.";

src/GameRuntime.cpp:6653: ImGui::Text("Field Medkits: %s", HasInventoryItem(profile, "cryo_medkit") ? "Available" : "None");

src/GameRuntime.cpp:6696: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service cycles logged", std::min(fieldServiceUses, 3));

src/GameRuntime.cpp:6698: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the recipe.", cyclesRemaining, cyclesRemaining == 1 ? "" : "s");

src/GameRuntime.cpp:6702: if (worldFieldState != nullptr && worldFieldState->tradeNetworkActive) {
```

```
src/GameRuntime.cpp:6705: ImGui::TextDisabled("Trade vouchers: %s", HasInventoryItem(profile,
"trade_voucher") ? "available" : "none");

src/GameRuntime.cpp:6707: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6708: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:6709: gameState.lastEvent = "Trade voucher exchanged for one cryo_medkit.";

src/GameRuntime.cpp:6711: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6716: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6720: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6724: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntime.cpp:6728: gameState.lastEvent = "No trade voucher available.";

src/GameRuntime.cpp:6732: ImGui::TextWrapped("Field action: press H to use one cryo medkit if available.");

src/GameRuntime.cpp:6758: ImGui::BulletText("Field Engineer: %s", HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support") ? "assigned" : "not assigned");

src/GameRuntime.cpp:7009: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and snapshot
trail even without Steam/Xbox auth.");

src/GameRuntime.cpp:7027: const auto servicesUnlock = BuildServicesUnlockState(profile,
currentWorldFieldState);

src/GameRuntime.cpp:7028: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState :
nullptr, servicesUnlock);

src/GameRuntime.cpp:7029: gameState.supportTerminalNearby = IsNearTaggedObject(world, player.x, player.y,
"lanline_service_hub", 4.0f);

src/GameRuntime.cpp:7030: gameState.tankServiceNearby = IsNearTaggedObject(world, player.x, player.y,
"tank_service", 4.0f);

src/GameRuntime.cpp:7036: ImGui::BulletText("Tank service nearby: %s", gameState.tankServiceNearby ? "yes" :
"no");

src/GameRuntime.cpp:7045: DrawLanlineServicesPanel(lanlineServices, servicesUnlock,
static_cast<std::int64_t>(std::time(nullptr)));

src/GameRuntime.cpp:7111: ImGui::TextWrapped("One discreet broker exchange is open on the recovery fringe.
Spend 1 trade voucher or 90 relay credits before the trace folds.");

src/GameRuntime.cpp:7131: ImGui::BulletText("Trade Vouchers: %d", InventoryCount(profile, "trade_voucher"));

src/GameRuntime.cpp:7183: const char* buttonLabel = specialist.assignment == "scavenger_support"

src/GameRuntime.cpp:7187: specialist.assignment = specialist.assignment == "scavenger_support" ? "workshop"
: "scavenger_support";
```



```
src/GameRuntime.cpp:7189: ? specialist.displayName + " reassigned to workshop support. Heavy service cycles
now get the engineer boost."

src/GameRuntime.cpp:7190: : specialist.displayName + " reassigned to scavenger support. Field service and
salvage teams now get the engineer boost.";

src/GameRuntime.cpp:7232: ? "Relay and tower sync are suppressing bloom growth. Camp and scavengers can
gradually reclaim the route."

src/GameRuntime.cpp:7238: ImGui::TextWrapped("Offline grids slowly wear down workshop, camps, and route
service. Restored power stabilizes structures over time.");

src/GameRuntime.cpp:7245: const bool scavengerReady =

src/GameRuntime.cpp:7251: if (scavengerReady) {

src/GameRuntime.cpp:7253: ImGui::BulletText("Next return: %.0fs", gameState.scavengerTimer);

src/GameRuntime.cpp:7254: ImGui::BulletText("Completed runs: %d", profile.scavengerRunsCompleted);

src/GameRuntime.cpp:7255: ImGui::BulletText("Engineer support: %s", HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support") ? "scavenger_support" : "workshop");

src/GameRuntime.cpp:7263: const bool caravanReady = CaravanRouteReady(profile);

src/GameRuntime.cpp:7265: worldFieldState->caravanRouteActive

src/GameRuntime.cpp:7266: ? (caravanReady ? "active" : "blocked")

src/GameRuntime.cpp:7268: ImGui::BulletText("Completed runs: %d", worldFieldState->caravanRunsCompleted);

src/GameRuntime.cpp:7269: if (worldFieldState->caravanRouteActive) {

src/GameRuntime.cpp:7270: ImGui::BulletText("Next return: %.0fs", gameState.caravanTimer);

src/GameRuntime.cpp:7272: if (ImGui::SmallButton(worldFieldState->caravanRouteActive ? "Disable Caravan
Route" : "Enable Caravan Route")) {

src/GameRuntime.cpp:7273: if (!worldFieldState->caravanRouteActive && !caravanReady) {

src/GameRuntime.cpp:7274: gameState.lastEvent = "Autopilot caravan needs a field checkpoint, a cleared outer
route, and a live regional grid.";

src/GameRuntime.cpp:7276: worldFieldState->caravanRouteActive = !worldFieldState->caravanRouteActive;

src/GameRuntime.cpp:7277: gameState.caravanTimer = 260.0f;

src/GameRuntime.cpp:7278: gameState.lastEvent = worldFieldState->caravanRouteActive

src/GameRuntime.cpp:7279: ? "Autopilot caravan route activated between shelter and forward camp."

src/GameRuntime.cpp:7280: : "Autopilot caravan route suspended.";

src/GameRuntime.cpp:7284: if (worldFieldState->caravanRouteActive && !caravanReady) {

src/GameRuntime.cpp:7293: const bool tradeReady = TradeNetworkReady(profile, *worldFieldState);
```

```
src/GameRuntime.cpp:7295: worldFieldState->tradeNetworkActive

src/GameRuntime.cpp:7296: ? (tradeReady ? "active" : "blocked")

src/GameRuntime.cpp:7298: ImGui::BulletText("Completed cycles: %d", worldFieldState-
>tradeCyclesCompleted);

src/GameRuntime.cpp:7299: ImGui::BulletText("Trade vouchers on hand: %d", InventoryCount(profile,
"trade_voucher"));

src/GameRuntime.cpp:7300: if (worldFieldState->tradeNetworkActive) {

src/GameRuntime.cpp:7301: ImGui::BulletText("Next convoy sync: %.0fs", gameState.tradeTimer);

src/GameRuntime.cpp:7303: if (ImGui::SmallButton(worldFieldState->tradeNetworkActive ? "Disable Trade
Network" : "Enable Trade Network")) {

src/GameRuntime.cpp:7304: if (!worldFieldState->tradeNetworkActive && !tradeReady) {

src/GameRuntime.cpp:7305: gameState.lastEvent = "Trade network needs a field checkpoint, a live grid, and
either caravan or drone logistics.";

src/GameRuntime.cpp:7307: worldFieldState->tradeNetworkActive = !worldFieldState->tradeNetworkActive;

src/GameRuntime.cpp:7308: gameState.tradeTimer = 240.0f;

src/GameRuntime.cpp:7309: gameState.lastEvent = worldFieldState->tradeNetworkActive

src/GameRuntime.cpp:7314: ImGui::TextWrapped("Requires a field checkpoint, a live grid, and either caravan or
drone logistics. Generates trade vouchers for supply exchange.");

src/GameRuntime.cpp:7315: if (worldFieldState->tradeNetworkActive && !tradeReady) {

src/GameRuntime.cpp:7322: const bool railReady = RailFreightReady(profile, *worldFieldState);

src/GameRuntime.cpp:7324: worldFieldState->railFreightActive

src/GameRuntime.cpp:7325: ? (railReady ? "active" : "blocked")

src/GameRuntime.cpp:7327: ImGui::BulletText("Completed runs: %d", worldFieldState->railRunsCompleted);

src/GameRuntime.cpp:7328: if (worldFieldState->railFreightActive) {

src/GameRuntime.cpp:7329: ImGui::BulletText("Next freight return: %.0fs", gameState.railTimer);

src/GameRuntime.cpp:7331: if (ImGui::SmallButton(worldFieldState->railFreightActive ? "Disable Rail Freight" :
"Enable Rail Freight")) {

src/GameRuntime.cpp:7332: if (!worldFieldState->railFreightActive && !railReady) {

src/GameRuntime.cpp:7335: worldFieldState->railFreightActive = !worldFieldState->railFreightActive;

src/GameRuntime.cpp:7336: gameState.railTimer = 320.0f;

src/GameRuntime.cpp:7337: gameState.lastEvent = worldFieldState->railFreightActive

src/GameRuntime.cpp:7343: if (worldFieldState->railFreightActive && !railReady) {
```

```
src/GameRuntime.cpp:7361: gameState.lastEvent = "Orbital uplink needs a live grid, active rail freight, two
restored pylons, and trade or drone support.";

src/GameRuntime.cpp:7370: ImGui::TextWrapped("Requires live grid, restored pylons, active rail freight, and a
functioning logistics backbone. Grants orbital scans and route intelligence.");

src/GameRuntime.cpp:7372: ImGui::TextDisabled("Orbital uplink is enabled but waiting on rail freight, pylon
coverage, grid stability, or logistics support.");

src/GameRuntime.cpp:7380: worldFieldState->railFortressActive

src/GameRuntime.cpp:7383: ImGui::BulletText("Patrol cycles: %d", worldFieldState->railFortressDeployments);

src/GameRuntime.cpp:7384: if (worldFieldState->railFortressActive) {

src/GameRuntime.cpp:7385: ImGui::BulletText("Next return: %.0fs", gameState.railFortressTimer);

src/GameRuntime.cpp:7387: if (ImGui::SmallButton(worldFieldState->railFortressActive ? "Recall Rail Fortress" :
"Deploy Rail Fortress")) {

src/GameRuntime.cpp:7388: if (!worldFieldState->railFortressActive && !fortressReady) {

src/GameRuntime.cpp:7389: gameState.lastEvent = "Rail fortress needs active rail freight, an orbital uplink, a
healthy grid, and two restored pylons.";

src/GameRuntime.cpp:7391: worldFieldState->railFortressActive = !worldFieldState->railFortressActive;

src/GameRuntime.cpp:7392: gameState.railFortressTimer = 520.0f;

src/GameRuntime.cpp:7393: gameState.lastEvent = worldFieldState->railFortressActive

src/GameRuntime.cpp:7398: ImGui::TextWrapped("Requires active rail freight, orbital uplink and a healthy grid.
Defense doctrine improves armored patrol tempo.");

src/GameRuntime.cpp:7399: if (worldFieldState->railFortressActive && !fortressReady) {

src/GameRuntime.cpp:7417: gameState.lastEvent = "Recovery fabricator needs a live grid, logistics input, and
either trade or orbital support.";

src/GameRuntime.cpp:7426: ImGui::TextWrapped("Requires live grid, logistics input, and either trade or orbital
support. Converts salvage into field supplies and recovery stock.");

src/GameRuntime.cpp:7449: ImGui::TextWrapped("Requires the unlocked industrial gate, orbital uplink, and
trade support. Survey sweeps feed inner spur intel back to Shelter 17.");

src/GameRuntime.cpp:7451: ImGui::TextDisabled("Survey beacon is enabled but waiting on gate access, orbital
uplink, or trade support.");

src/GameRuntime.cpp:7466: ImGui::TextWrapped("Requires gate access, survey coverage, and trade support.
Outpost supply runs reinforce the inner spur foothold.");

src/GameRuntime.cpp:7468: ImGui::TextDisabled("Inner spur outpost is enabled but waiting on gate access,
survey coverage, or trade support.");
```

```
src/GameRuntime.cpp:7500: ImGui::TextWrapped("Requires assembly, outpost support, and rail security. This is the first heavy fabrication line beyond the gate.");

src/GameRuntime.cpp:7559: const bool serviceReady = ServiceBayReady(*worldFieldState);

src/GameRuntime.cpp:7561: worldFieldState->serviceBayActive

src/GameRuntime.cpp:7562: ? (serviceReady ? "active" : "blocked")

src/GameRuntime.cpp:7564: ImGui::BulletText("Service cycles: %d", worldFieldState->serviceCyclesCompleted);

src/GameRuntime.cpp:7565: if (worldFieldState->serviceBayActive) {

src/GameRuntime.cpp:7566: ImGui::BulletText("Next service: %.0fs", gameState.serviceBayTimer);

src/GameRuntime.cpp:7569: if (worldFieldState->serviceBayActive && !serviceReady) {

src/GameRuntime.cpp:7585: ImGui::TextWrapped("Requires service bay, relay support, and the recovery fabricator. Stabilizes camp recovery and produces clean water.");

src/GameRuntime.cpp:7587: ImGui::TextDisabled("Water reclaimer is enabled but waiting on service bay support, relay return flow, or fabricator output.");

src/GameRuntime.cpp:7593: const bool droneReady = IsRegionalGridOnline(profile);

src/GameRuntime.cpp:7595: worldFieldState->droneStationsActive

src/GameRuntime.cpp:7596: ? (droneReady ? "active" : "blocked")

src/GameRuntime.cpp:7598: ImGui::BulletText("Completed runs: %d", worldFieldState->droneRunsCompleted);

src/GameRuntime.cpp:7599: if (worldFieldState->droneStationsActive) {

src/GameRuntime.cpp:7600: ImGui::BulletText("Next sweep: %.0fs", gameState.droneTimer);

src/GameRuntime.cpp:7602: ImGui::TextWrapped("Requires a powered drone station in the world. Industry doctrine and restored pylons improve drone efficiency.");

src/GameRuntime.cpp:7603: if (worldFieldState->droneStationsActive && !droneReady) {

src/GameRuntimePipPad.cpp:385: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";

src/GameRuntimePipPad.cpp:389: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the gunner station.";

src/GameRuntimePipPad.cpp:393: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";

src/GameRuntimePipPad.cpp:398: gameState.lastEvent = "BT-72 is moving too fast for field service. Stop the hull first.";

src/GameRuntimePipPad.cpp:402: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable condition.";

src/GameRuntimePipPad.cpp:406: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a repair patch.";
```

```
src/GameRuntimePipPad.cpp:410: const bool hasEngineer = HasAssignedSpecialistRole(profile, "engineer",
"scavenger_support");

src/GameRuntimePipPad.cpp:425: ? "Field service rack cycled. Scavenger-side engineer tuning improved the
repair pass."

src/GameRuntimePipPad.cpp:426: : "Field service rack cycled. BT-72 patched and partially resupplied in the
field.";

src/GameRuntimePipPad.cpp:429: gameState.lastEvent += " First service halt logged for the route.";

src/GameRuntimePipPad.cpp:491: ImGui::Text("Field Medkits: %s", HasInventoryItem(profile, "cryo_medkit") ?
"Available" : "None");

src/GameRuntimePipPad.cpp:534: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service
cycles logged", std::min(fieldServiceUses, 3));

src/GameRuntimePipPad.cpp:536: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the
recipe.", cyclesRemaining, cyclesRemaining == 1 ? "" : "s");

src/GameRuntimePipPad.cpp:540: if (worldFieldState != nullptr && worldFieldState->tradeNetworkActive) {

src/GameRuntimePipPad.cpp:543: ImGui::TextDisabled("Trade vouchers: %s", HasInventoryItem(profile,
"trade_voucher") ? "available" : "none");

src/GameRuntimePipPad.cpp:545: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntimePipPad.cpp:546: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntimePipPad.cpp:547: gameState.lastEvent = "Trade voucher exchanged for one cryo_medkit.";

src/GameRuntimePipPad.cpp:549: gameState.lastEvent = "No trade voucher available.";

src/GameRuntimePipPad.cpp:554: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntimePipPad.cpp:558: gameState.lastEvent = "No trade voucher available.";

src/GameRuntimePipPad.cpp:562: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {

src/GameRuntimePipPad.cpp:566: gameState.lastEvent = "No trade voucher available.";

src/GameRuntimePipPad.cpp:570: ImGui::TextWrapped("Field action: press H to use one cryo medkit if
available.");

src/GameRuntimePipPad.cpp:613: ImGui::BulletText("Field Engineer: %s", HasAssignedSpecialistRole(profile,
"engineer", "scavenger_support") ? "assigned" : "not assigned");

src/GameRuntimePipPad.cpp:905: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and
snapshot trail even without Steam/Xbox auth.");

src/GameRuntimePipPad.cpp:924: const auto servicesUnlock = BuildServicesUnlockState(profile,
currentWorldFieldState);

src/GameRuntimePipPad.cpp:926: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState
: nullptr, servicesUnlock);
```

```
src/GameRuntimePipPad.cpp:928: gameState.supportTerminalNearby = IsNearTaggedObject(world, player.x,
player.y, "lanline_service_hub", 4.0f);

src/GameRuntimePipPad.cpp:929: gameState.tankServiceNearby = IsNearTaggedObject(world, player.x, player.y,
"tank_service", 4.0f);

src/GameRuntimePipPad.cpp:935: ImGui::BulletText("Tank service nearby: %s", gameState.tankServiceNearby ?
"yes" : "no");

src/GameRuntimePipPad.cpp:945: servicesUnlock.localRelayAvailable &&

src/GameRuntimePipPad.cpp:946: (gameState.supportTerminalNearby ||
profile.lanlineServices.serviceHubKnown);

src/GameRuntimePipPad.cpp:958: ImGui::TextDisabled("Delivered orders can be claimed after a Lanline service
hub handshake.");

src/GameRuntimePipPad.cpp:972: ImGui::TextWrapped("A rare merchant trace is open. Broker one exchange
from the services layer before the window closes.");

src/GameRuntimePipPad.cpp:977: DrawLanlineServicesPanel(lanlineServices, servicesUnlock, nowUnix);

src/GameplayDescriptorRegistry.cpp:13: {"workshop_service", "Workshop Service", InteractionType::Workshop,
ObjectCategory::Terminal, {}, false},

src/GameplayDescriptorRegistry.cpp:16: {"drone_station", "Drone Station", InteractionType::Terminal,
ObjectCategory::Terminal, "salvage_sweep", false},

src/GameplayDescriptorRegistry.cpp:17: {"rail_depot", "Rail Depot", InteractionType::Terminal,
ObjectCategory::Terminal, "industrial_spur_alpha", false},

src/GameplayDescriptorRegistry.cpp:19: {"rail_fortress_hub", "Rail Fortress Hub", InteractionType::Terminal,
ObjectCategory::Terminal, "magistral_anchor_alpha", false},

src/GameplayDescriptorRegistry.cpp:29: {"service_bay", "Service Bay", InteractionType::Terminal,
ObjectCategory::Terminal, "inner_spur_service", false},

src/GameplayDescriptorRegistry.cpp:31: {"lanline_service_hub", "Lanline Service Hub",
InteractionType::Terminal, ObjectCategory::Terminal, "shelter17_services", false},

src/GameplayDescriptorRegistry.cpp:32: {"tank_service", "Tank Service", InteractionType::Workshop,
ObjectCategory::Hangar, "bt72_service", false},

src/GameplayDescriptorRegistry.cpp:41: {"workshop_field_service", "workshop_service"},

src/GameplayDescriptorRegistry.cpp:42: {"rail_freight", "rail_depot"},

src/GameplayDescriptorRegistry.cpp:43: {"rail_fortress", "rail_fortress_hub"},

src/LanlineServices.cpp:234: if (contentId == "medkit") {

src/LanlineServices.cpp:235: GrantSupportDeliveryItem(profile, "cryo_medkit", 1, 0.5f, grantedLabels);

src/LanlineServices.cpp:239: GrantSupportDeliveryItem(profile, "cryo_medkit", 2, 0.5f, grantedLabels);
```

```
src/LanlineServices.cpp:260: {"relay", "Relay", {"Shelter 17 Relay", "Tower sync can reopen broader relay
services.", "boot"}}},

src/LanlineServices.cpp:355: ImGui::TextWrapped("Relay node detected, but services are still restricted while
Shelter 17 stabilizes.");

src/LanlineServices.cpp:359: if (!unlockState.serviceBayActive) {

src/LanlineServices.cpp:360: ImGui::BulletText("Requirement: service bay operational");

src/LanlineServices.cpp:547: ImGui::TextDisabled("Tank service remains locked until backbone stability is
restored.");

src/LanlineServices.cpp:709: state.serviceBayActive = resolvedWorldState != nullptr &&

src/LanlineServices.cpp:774: {"medkit_standard", "Field Medkit", "Standard expedition med supply for operator
recovery.", SupportCategory::Medical, StoreCurrency::InGame, 75, "", false, false, false, TankSubsystem::None,
{"medkit", "bandage_roll", "sterile_patch"}},

src/LanlineServices.cpp:775: {"medkit_trauma", "Trauma Response Pack", "Advanced trauma bundle for severe
field damage recovery.", SupportCategory::Medical, StoreCurrency::InGame, 120, "", false, false, false,
TankSubsystem::None, {"trauma_kit", "injector", "coagulant_pack"}},

src/LanlineServices.cpp:790: {"spur.mechanic.ira", "Inner Spur Depot", "Waiting for stable backbone before
sending heavier service loads.", false, false, false, 1.0f},

src/LanlineServices.cpp:1020: return ProfilesDirectory() / "lanline_services.state";

src/LanlineServices.cpp:1135: ImGui::BulletText("Service bay: %s", unlockState.serviceBayActive ? "online" :
"offline");

src/PrefabLibrary.cpp:118: void TrimTrailingEmptyLootEntries(std::vector<LootEntry>& entries) {

src/PrefabLibrary.cpp:212: TrimTrailingEmptyLootEntries(prefab.object.lootEntries);

src/SessionProfiles.cpp:63: HasCollectedTapId(profile, "bt72_service_reel_001");

src/SessionProfiles.cpp:282: profile.lanlineServices.serviceHubKnown = false;

src/SessionProfiles.cpp:375: out << "lanline_service_hub_known=" << (profile.lanlineServices.serviceHubKnown ?
1 : 0) << '\n';

src/SessionProfiles.cpp:386: out << "scavenger_runs_completed=" << profile.scavengerRunsCompleted << '\n';

src/SessionProfiles.cpp:401: out << "bt72_hull_moved_to_service_lift=" << (profile.bt72HullMovedToServiceLift ?
1 : 0) << '\n';

src/SessionProfiles.cpp:450: out << "route_first_service=" << (profile.firstPlayableRoute.firstServicePerformed ? 1
: 0) << '\n';

src/SessionProfiles.cpp:456: out << "awakening_stress=" << profile.character.awakening.stressSurvivals << '\n';

src/SessionProfiles.cpp:458: out << "awakening_field_service=" << profile.character.awakening.fieldServiceUses
<< '\n';
```

```
src/SessionProfiles.cpp:485: << (worldState.caravanRouteActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:486: << (worldState.droneStationsActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:487: << (worldState.tradeNetworkActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:488: << (worldState.railFreightActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:490: << (worldState.railFortressActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:495: << worldState.caravanRunsCompleted << ','  
src/SessionProfiles.cpp:496: << worldState.droneRunsCompleted << ','  
src/SessionProfiles.cpp:497: << worldState.tradeCyclesCompleted << ','  
src/SessionProfiles.cpp:499: << worldState.railRunsCompleted << ','  
src/SessionProfiles.cpp:501: << worldState.railFortressDeployments << ','  
src/SessionProfiles.cpp:519: << (worldState.serviceBayActive ? 1 : 0) << ','  
src/SessionProfiles.cpp:520: << worldState.serviceCyclesCompleted << ','  
src/SessionProfiles.cpp:614: else if (key == "lanline_service_hub_known")  
outProfile.lanlineServices.serviceHubKnown = (std::stoi(value) != 0);  
  
src/SessionProfiles.cpp:625: else if (key == "scavenger_runs_completed") outProfile.scavengerRunsCompleted =  
std::stoi(value);  
  
src/SessionProfiles.cpp:635: else if (key == "bt72_hull_moved_to_service_lift")  
outProfile.bt72HullMovedToServiceLift = (std::stoi(value) != 0);  
  
src/SessionProfiles.cpp:684: else if (key == "route_first_service")  
outProfile.firstPlayableRoute.firstServicePerformed = (std::stoi(value) != 0);  
  
src/SessionProfiles.cpp:690: else if (key == "awakening_stress") outProfile.character.awakening.stressSurvivals =  
std::stoi(value);  
  
src/SessionProfiles.cpp:692: else if (key == "awakening_field_service")  
outProfile.character.awakening.fieldServiceUses = std::stoi(value);  
  
src/SessionProfiles.cpp:787: state.caravanRouteActive = std::stoi(value.substr(third + 1, fourth - third - 1)) != 0;  
src/SessionProfiles.cpp:788: state.caravanRunsCompleted = std::stoi(value.substr(fourth + 1, fifth - fourth - 1));  
src/SessionProfiles.cpp:789: state.droneRunsCompleted = std::stoi(value.substr(fifth + 1, sixth - fifth - 1));  
src/SessionProfiles.cpp:807: state.caravanRouteActive = std::stoi(parts[baseOffset]) != 0;  
src/SessionProfiles.cpp:808: state.droneStationsActive = std::stoi(parts[baseOffset + 1]) != 0;  
src/SessionProfiles.cpp:809: state.tradeNetworkActive = std::stoi(parts[baseOffset + 2]) != 0;  
src/SessionProfiles.cpp:812: state.railFreightActive = std::stoi(parts[baseOffset + 3]) != 0;  
src/SessionProfiles.cpp:814: state.railFortressActive = std::stoi(parts[baseOffset + 5]) != 0;
```



```
src/SessionProfiles.cpp:821: state.caravanRunsCompleted = std::stoi(parts[baseIndex + 2]);
src/SessionProfiles.cpp:822: state.droneRunsCompleted = std::stoi(parts[baseIndex + 3]);
src/SessionProfiles.cpp:823: state.tradeCyclesCompleted = std::stoi(parts[baseIndex + 4]);
src/SessionProfiles.cpp:825: state.railRunsCompleted = std::stoi(parts[baseIndex + 6]);
src/SessionProfiles.cpp:827: state.railFortressDeployments = std::stoi(parts[baseIndex + 8]);
src/SessionProfiles.cpp:845: state.serviceBayActive = parts.size() >= baseIndex + 27 ? (std::stoi(parts[baseIndex + 26]) != 0) : false;
src/SessionProfiles.cpp:846: state.serviceCyclesCompleted = parts.size() >= baseIndex + 28 ?
std::stoi(parts[baseIndex + 27]) : 0;
src/SessionProfiles.cpp:874: state.railFreightActive = std::stoi(parts[baseOffset + 3]) != 0;
src/SessionProfiles.cpp:876: state.railFortressActive = std::stoi(parts[baseOffset + 5]) != 0;
src/SessionProfiles.cpp:880: state.caravanRunsCompleted = std::stoi(parts[baseOffset + 8]);
src/SessionProfiles.cpp:881: state.droneRunsCompleted = std::stoi(parts[baseOffset + 9]);
src/SessionProfiles.cpp:882: state.tradeCyclesCompleted = std::stoi(parts[baseOffset + 10]);
src/SessionProfiles.cpp:884: state.railRunsCompleted = std::stoi(parts[baseOffset + 12]);
src/SessionProfiles.cpp:886: state.railFortressDeployments = std::stoi(parts[baseOffset + 14]);
src/SessionProfiles.cpp:904: state.serviceBayActive = false;
src/SessionProfiles.cpp:905: state.serviceCyclesCompleted = 0;
src/SessionProfiles.cpp:909: state.railFreightActive = std::stoi(parts[6]) != 0;
src/SessionProfiles.cpp:913: state.caravanRunsCompleted = std::stoi(parts[9]);
src/SessionProfiles.cpp:914: state.droneRunsCompleted = std::stoi(parts[10]);
src/SessionProfiles.cpp:915: state.tradeCyclesCompleted = std::stoi(parts[11]);
src/SessionProfiles.cpp:917: state.railRunsCompleted = std::stoi(parts[13]);
src/SessionProfiles.cpp:919: state.railFortressDeployments = 0;
src/SessionProfiles.cpp:937: state.serviceBayActive = false;
src/SessionProfiles.cpp:938: state.serviceCyclesCompleted = 0;
src/SessionProfiles.cpp:946: state.caravanRunsCompleted = std::stoi(parts[8]);
src/SessionProfiles.cpp:947: state.droneRunsCompleted = std::stoi(parts[9]);
src/SessionProfiles.cpp:948: state.tradeCyclesCompleted = std::stoi(parts[10]);
src/SessionProfiles.cpp:953: state.railRunsCompleted = 0;
```

```
src/SessionProfiles.cpp:955: state.railFortressDeployments = 0;

src/SessionProfiles.cpp:973: state.serviceBayActive = false;

src/SessionProfiles.cpp:974: state.serviceCyclesCompleted = 0;

src/SkillSystem.cpp:76: profile.character.awakening.stressSurvivals += 1;

src/SkillSystem.cpp:77: if (profile.character.awakening.stressSurvivals >= 1) {

src/StoryRoute.cpp:45: HasCollectedTapelId(profile, "bt72_service_reel_001");

src/StoryRoute.cpp:126: if (routeEventType == "service_call") {

src/StoryRoute.cpp:148: if (worldState.activeRouteEventType == "service_call" ||

src/StoryRoute.cpp:175: return "Restore the industrial rail depot and bring heavy freight back to Shelter 17.";

src/StoryRoute.cpp:211: return "Bring the inner spur service bay online to push BT-72 repairs deeper into the
factory belt.";

src/StoryRoute.cpp:243: return "rail freight";

src/StoryRoute.cpp:279: return "service bay";

src/StoryRoute.cpp:298: return "The rail spur is secured, but salvage is not yet turning into field stock.";

src/StoryRoute.cpp:328: return "Deep BT-72 service is online, but frontier recovery still lacks long-range water
support.";

src/StoryRoute.cpp:330: return "Relay, service, and water loops are all online; Shelter 17 can now sustain a stable
recovery backbone.";

src/StoryRoute.cpp:345: "Upload the debrief to turn the route payoff into honest rail, service, and industrial
work."

src/StoryRoute.cpp:358: const bool railOperational = IsRailFreightOperational(profile, *worldState);

src/StoryRoute.cpp:369: const bool serviceOperational = IsServiceBayOperational(*worldState);

src/StoryRoute.cpp:373: AppendBackboneNodeLabel(activeNodes, railOperational, "rail freight");

src/StoryRoute.cpp:385: AppendBackboneNodeLabel(activeNodes, serviceOperational, "service bay");

src/StoryRoute.cpp:389: (railOperational ? 1 : 0) +

src/StoryRoute.cpp:402: (serviceOperational ? 1 : 0) +

src/StoryRoute.cpp:499: return "Recover a bunker access card and trace the paper trail to the missing Pip-Pad.";

src/StoryRoute.cpp:502: return "Sweep the bunker passage, gather the paper trail, and recover the missing Pip-
Pad.";

src/StoryRoute.cpp:508: ? "Sync the archive and pull the missing personnel trail into the Pip-Pad."

src/StoryRoute.cpp:513: return "Survey the BT-72 hull, recover the starter core, and decode the service notes.";
```

src/StoryRoute.cpp:516: return "Restore hangar power, clear the crane path, and move the BT-72 hull into the service lift cradle.";

src/StoryRoute.cpp:548: return "Take a first service/rest stop before pushing the recovery node.";

src/StoryRoute.cpp:583: return "Sweep the bunker passage, gather the paper trail, and recover the missing Pip-Pad.";

src/StoryRoute.cpp:595: return "Survey the BT-72 hull, recover the starter core, and decode the service notes.";

src/StoryRoute.cpp:598: return "Power the hangar crane, clear the crane path, and lock BT-72 into the service lift cradle.";

src/StoryRoute.cpp:630: return "Run a first service cycle before pushing the relay node.";

src/StoryRoute.cpp:661: return "Handoff: restore rail freight and move heavy salvage off the starter route.";

src/StoryRoute.cpp:697: return "Handoff: bring the service bay online for deep BT-72 support.";

src/StoryRoute.cpp:766: return "contact resolved, service halt pending";

src/StoryRoute.cpp:769: return "service complete, recovery node pending";

src/StoryRoute.cpp:807: "Recover the access card, stabilize the paper trail, and pull the missing Pip-Pad into the route.",

src/StoryRoute.cpp:808: "The archive trail should point cleanly toward the hangar and BT-72."

src/StoryRoute.cpp:814: "Clear the vermin gate and sync the missing personnel trail into the Pip-Pad.",

src/StoryRoute.cpp:821: "Read the BT-72 hull, core, and service notes, move the hull through the crane and service lift cradle, then restore the chassis from bunker salvage.",

src/StoryRoute.cpp:864: "A short service halt should feel earned instead of menu-like."

src/StoryRoute.cpp:870: "Run one field or workshop service cycle to cool BT-72 and reset the push.",

src/StoryRoute.cpp:891: "Route events, recovery state, and BT-72 service now carry progression beyond the starter lane."

src/StoryRoute.cpp:941: {"Decode BT-72 service notes and holo-records.", Bt72ServiceNotesRecovered(profile)},

src/StoryRoute.cpp:942: {"Lock the BT-72 hull in the service lift cradle.",
Bt72HullLockedInRestorationCradle(profile)},

src/StoryRoute.cpp:955: {"Recover the bunker access card and paper trail.", HasAccessCardRecovered(profile) &&

src/StoryRoute.cpp:967: {"Take the first service/rest halt.", FirstServicePerformed(profile)},

src/StoryRoute.cpp:975: const bool railOperational = worldState != nullptr && IsRailFreightOperational(profile,
*worldState);

src/StoryRoute.cpp:986: const bool serviceOperational = worldState != nullptr &&
IsServiceBayOperational(*worldState);

src/StoryRoute.cpp:999: {"Decode BT-72 service notes.", profile.firstPlayableRoute.bt72ServiceNotesRecovered},

```
src/StoryRoute.cpp:1009: {"Take a first service/rest cycle.", profile.firstPlayableRoute.firstServicePerformed},
src/StoryRoute.cpp:1012: {"Restore the rail freight depot.", railOperational},
src/StoryRoute.cpp:1024: {"Bring the inner spur service bay online.", serviceOperational},
src/World.cpp:137: "[%bt72_service_lift_0001]",
src/World.cpp:154: "[%bt72_service_notes_0001]",
src/World.cpp:259: return normalizedTag == "rail_depot" || normalizedTag == "rail_fortress_hub";
src/World.cpp:274: normalizedTag == "workshop_service" ||
src/World.cpp:276: normalizedTag == "drone_station" ||
src/World.cpp:279: normalizedTag == "service_bay" ||
src/World.cpp:281: normalizedTag == "lanline_service_hub" ||
src/World.cpp:283: normalizedTag == "tank_service" ||
src/World.cpp:324: void TrimTrailingEmptyLootEntries(std::vector<LootEntry>& entries) {
src/World.cpp:354: TrimTrailingEmptyLootEntries(object.lootEntries);
src/World.cpp:625: TrimTrailingEmptyLootEntries(persistedLootEntries);
src/World.cpp:690: {"#%it_pippad", "cryo_medkit", "", ""}
src/World.cpp:1010: "[%drone_0001]",
src/World.cpp:1025: "drone_station",
src/World.cpp:1030: "[%rail_0001]",
src/World.cpp:1045: "rail_depot",
src/World.cpp:1085: "rail_fortress_hub",
src/World.cpp:1270: "[%servicebay_0001]",
src/World.cpp:1285: "service_bay",
src/World.cpp:1286: "inner_spur_service"
src/World.cpp:1310: "[%services_0001]",
src/World.cpp:1325: "lanline_service_hub",
src/World.cpp:1326: "shelter17_services"
src/World.cpp:1370: "[%tankservice_0001]",
src/World.cpp:1385: "tank_service",
src/World.cpp:1386: "bt72_service"
src/World.cpp:1556: if (IsStarterScenarioWorld() && !HasObject("[%drone_0001]")) {
```

```
src/World.cpp:1558: "[%drone_0001]",
src/World.cpp:1573: "drone_station",
src/World.cpp:1577: if (IsStarterScenarioWorld() && !HasObject("[%rail_0001]")) {
src/World.cpp:1579: "[%rail_0001]",
src/World.cpp:1594: "rail_depot",
src/World.cpp:1636: "rail_fortress_hub",
src/World.cpp:1829: if (IsStarterScenarioWorld() && !HasObject("[%servicebay_0001]")) {
src/World.cpp:1831: "[%servicebay_0001]",
src/World.cpp:1846: "service_bay",
src/World.cpp:1847: "inner_spur_service"
src/World.cpp:1871: if (IsStarterScenarioWorld() && !HasObject("[%services_0001]")) {
src/World.cpp:1873: "[%services_0001]",
src/World.cpp:1888: "lanline_service_hub",
src/World.cpp:1889: "shelter17_services"
src/World.cpp:1934: if (IsStarterScenarioWorld() && !HasObject("[%tankservice_0001]")) {
src/World.cpp:1936: "[%tankservice_0001]",
src/World.cpp:1951: "tank_service",
src/World.cpp:1952: "bt72_service"

src/WorldEvents.cpp:61: : "LOCKER BAY: 'Emergency recovery locker detected. Mechanical interlock wants a
bunker access card from the core service racks.'";

src/WorldEvents.cpp:96: ? "CONTACT: 'Outer Ghoul charging the cleared lane. Hold BT-72 through the rush, then
take one service halt before syncing the relay.'"

src/WorldEvents.cpp:105: gameState.lastEvent = "DEBRIEF: 'Recovery node recognized. Upload the route
summary and authorize Shelter 17 planning: rail, orbital, fabrication, then the inner spur.'";

src/WorldEvents.cpp:113: gameState.lastEvent = "DEBRIEF: 'Recovery planning remains active. Keep rebuilding
rail freight, orbital support, fabrication, and the inner spur backbone.'";

src/WorldExport.cpp:270: if (result.auditTrailPath.empty()) {
src/WorldExport.cpp:274: std::ofstream auditFile(result.auditTrailPath, std::ios::app);

src/WorldExport.cpp:786: std::filesystem::path ExportAuditTrailPathForWorld(const std::filesystem::path&
worldPath) {
src/WorldExport.cpp:828: std::ifstream file(ExportAuditTrailPathForWorld(worldPath));
```

```
src/WorldExport.cpp:1346: report << "Audit trail: " << result.auditTrailPath.string() << '\n';
src/WorldExport.cpp:1425: result.auditTrailPath = ExportAuditTrailPathForWorld(path);
src/WorldExport.cpp:1439: " Audit: " + result.auditTrailPath.string();
src/WorldExport.cpp:1446: " Audit: " + result.auditTrailPath.string();
src/WorldExport.cpp:1452: " Audit: " + result.auditTrailPath.string();
src/WorldExport.cpp:1459: " Audit: " + result.auditTrailPath.string();
src/WorldExport.cpp:1462: " Audit: " + result.auditTrailPath.string();
src/WorldSemanticAuthoring.cpp:64: } else if (spec.scriptTag == "service_bay") {
src/WorldSemanticAuthoring.cpp:139: if (scriptTag == "lanline_service_hub" || scriptTag == "medical_support") {
src/WorldSemanticAuthoring.cpp:142: if (scriptTag == "tank_service") {
src/WorldSemanticAuthoring.cpp:143: return {"service_bay"};
src/WorldSemanticAuthoring.cpp:148: if (scriptTag == "service_bay") {
src/WorldSemanticAuthoring.cpp:152: return {"service_bay", "relay_substation", "recovery_fabricator"};
src/main.cpp:490: gameState.lastEvent = "Operator fatigue rising. Recover MP before pushing further.";
src/main.cpp:494: if (atCriticalHp && !gameState.stressThresholdTriggered) {
src/main.cpp:495: gameState.stressThresholdTriggered = true;
src/main.cpp:502: gameState.stressThresholdTriggered = false;
src/main.cpp:514: gameState.lastEvent = "Second Wind engaged. Operator stabilized under stress.";
src/main.cpp:674: } else if (bunker::ConsumeInventoryItem(sessionProfile, "cryo_medkit", 1)) {
src/main.cpp:677: gameState.lastEvent = "Cryo medkit applied.";
src/main.cpp:679: gameState.lastEvent = "No cryo medkits available.";
src/main.cpp:864: gameState.stressThresholdTriggered = false;
src/main.cpp:964: ImGui::TextColored(ImVec4(0.95f, 0.7f, 0.22f, 1.0f), "Ammo reserves low. Workshop or field
service recommended.");
tests/SmokeChecks.cpp:201: Check(loaderWorld.HasLinkTarget("inner_spur_service"), "service bay link target
missing after roundtrip") &&
tests/SmokeChecks.cpp:203: Check(loaderWorld.CountObjectsInEditorLayer("Service") >= 1, "world roundtrip
should preserve inferred service layers") &&
tests/SmokeChecks.cpp:231: "world roundtrip should trim trailing empty rows after a filled row 10 entry") &&
tests/SmokeChecks.cpp:237: "world roundtrip should trim all-empty trailing loot rows to zero persisted entries");
tests/SmokeChecks.cpp:344: savedWorldState->activeRouteEventType = "service_call";
```

```
tests/SmokeChecks.cpp:388: Check(bunker::FindWorldFieldState(loadedProfile, loadedProfile.selectedWorld)-
>activeRouteEventType == "service_call",

tests/SmokeChecks.cpp:389: "profile roundtrip active route event mismatch") &&

tests/SmokeChecks.cpp:391: "profile roundtrip active route event timer mismatch") &&

tests/SmokeChecks.cpp:393: "profile roundtrip route event offer timer mismatch") &&

tests/SmokeChecks.cpp:395: "profile roundtrip route event progress mismatch") &&

tests/SmokeChecks.cpp:602: ContainsText(savedText, "bt72_hull_moved_to_service_lift=") &&

tests/SmokeChecks.cpp:779: Check(bunker::CurrentStoryObjectivePreview(profile).find("rail depot") !=
std::string::npos,

tests/SmokeChecks.cpp:821: missingPreview.find("service notes") != std::string::npos,

tests/SmokeChecks.cpp:822: "bt72_objective_requires_knowledge_before_cradle expected hull/core/service
notes objective")) {

tests/SmokeChecks.cpp:832: if (!Check(hasAny(cradlePreview, {"crane", "service lift", "cradle"})) &&

tests/SmokeChecks.cpp:833: hasAny(cradleObjective, {"crane", "service lift", "cradle"})) &&

tests/SmokeChecks.cpp:834: cradlePreview.find("Survey the BT-72 hull, recover the starter core, and decode the
service notes") == std::string::npos &&

tests/SmokeChecks.cpp:835: cradleObjective.find("Survey the BT-72 hull, recover the starter core, and decode the
service notes") == std::string::npos &&

tests/SmokeChecks.cpp:879: return entry.text.find("cradle") != std::string::npos || entry.text.find("service lift") !=
std::string::npos;

tests/SmokeChecks.cpp:885: return entry.text.find("cradle") != std::string::npos || entry.text.find("service lift") !=
std::string::npos;

tests/SmokeChecks.cpp:955: "route beat smoke expected service-halt label after first combat")) {

tests/SmokeChecks.cpp:961: "route beat smoke expected recovery-sync label after first service")) {

tests/SmokeChecks.cpp:978: Check(industrialBeat.cue.find("rail freight") != std::string::npos,

tests/SmokeChecks.cpp:979: "route beat smoke expected industrial handoff cue to point at rail freight");

tests/SmokeChecks.cpp:1036: const auto serviceReadout = bunker::BuildFirstPlayableRouteReadout(profile);

tests/SmokeChecks.cpp:1037: if (!Check(serviceReadout.surfaceStatus == "contact resolved, service halt
pending",

tests/SmokeChecks.cpp:1038: "route readout smoke expected post-contact service-halt surface status")) {

tests/SmokeChecks.cpp:1041: if (!Check(serviceReadout.nextPayoff == "Take the first service/rest halt.",

tests/SmokeChecks.cpp:1042: "route readout smoke expected first service as the next payoff")) {
```

```
tests/SmokeChecks.cpp:1059: Check(industrialReadout.nextPayoff.find("rail freight") != std::string::npos,
tests/SmokeChecks.cpp:1060: "route readout smoke expected industrial handoff to point at rail freight");
tests/SmokeChecks.cpp:1159: Check(gameState.lastEvent.find("service halt") != std::string::npos,
tests/SmokeChecks.cpp:1160: "first combat smoke expected service-halt cue in contact event") &&
tests/SmokeChecks.cpp:1203: "first combat handoff smoke expected service-halt resolution cue") &&
tests/SmokeChecks.cpp:1204: Check(gameState.lastEvent.find("Run a first service cycle before pushing the relay
node.") != std::string::npos,
tests/SmokeChecks.cpp:1205: "first combat handoff smoke expected first-service runtime objective hint") &&
tests/SmokeChecks.cpp:1207: "first combat handoff smoke expected service-halt route beat");
tests/SmokeChecks.cpp:1219: profile.selectedWorld = "service_handoff_smoke.bwld";
tests/SmokeChecks.cpp:1247: return Check(profile.firstPlayableRoute.firstServicePerformed, "workshop handoff
smoke expected first service flag") &&
tests/SmokeChecks.cpp:1248: Check(gameState.lastEvent.find("First service halt logged") != std::string::npos,
tests/SmokeChecks.cpp:1249: "workshop handoff smoke expected first service log copy") &&
tests/SmokeChecks.cpp:1433: Check(gameState.lastEvent.find("rail depot") != std::string::npos,
tests/SmokeChecks.cpp:1434: "debrief handoff smoke expected industrial rail-depot handoff") &&
tests/SmokeChecks.cpp:1468: if (!Check(bunker::CurrentRecoveryHandoffSummary(profile).find("rail freight") !=
std::string::npos,
tests/SmokeChecks.cpp:1469: "recovery handoff smoke expected rail-freight first step")) {
tests/SmokeChecks.cpp:1476: worldState->caravanRouteActive = true;
tests/SmokeChecks.cpp:1477: worldState->railFreightActive = true;
tests/SmokeChecks.cpp:1484: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:1492: worldState->railFortressActive = true;
tests/SmokeChecks.cpp:1544: starterStatus.status.find("rail freight") != std::string::npos,
tests/SmokeChecks.cpp:1545: "recovery backbone smoke expected starter status to point at rail freight: " +
starterStatus.status)) {
tests/SmokeChecks.cpp:1552: worldState->caravanRouteActive = true;
tests/SmokeChecks.cpp:1553: worldState->railFreightActive = true;
tests/SmokeChecks.cpp:1555: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:1574: worldState->railFortressActive = true;
tests/SmokeChecks.cpp:1600: worldState->serviceBayActive = true;
```



```
tests/SmokeChecks.cpp:1616: if (!Check(lockedWorldState != nullptr, "route event smoke expected locked world field state")) {
tests/SmokeChecks.cpp:1622: "route event smoke expected layer to stay locked before onboarding and debrief")) {
tests/SmokeChecks.cpp:1626: "route event smoke expected locked summary before onboarding")) {
tests/SmokeChecks.cpp:1653: if (!Check(worldState != nullptr, "route event smoke expected world field state")) {
tests/SmokeChecks.cpp:1659: if (!Check(worldState->activeRouteEventType == "service_call",
tests/SmokeChecks.cpp:1660: "route event smoke expected deterministic service-call spawn")) {
tests/SmokeChecks.cpp:1664: "route event smoke expected offered state before activation")) {
tests/SmokeChecks.cpp:1668: "route event smoke expected offered route-event summary")) {
tests/SmokeChecks.cpp:1674: "route event smoke expected offer timer to collapse into active state")) {
tests/SmokeChecks.cpp:1678: "route event smoke expected active route-event summary after offer")) {
tests/SmokeChecks.cpp:1688: return Check(!bunker::HasActiveRouteEvent(*worldState), "route event smoke expected service-call resolution") &&
tests/SmokeChecks.cpp:1689: Check(worldState->routeEventsResolved == 1, "route event smoke expected resolved counter increment") &&
tests/SmokeChecks.cpp:1690: Check(worldState->routeEventCooldown > 0.0f, "route event smoke expected post-resolution cooldown") &&
tests/SmokeChecks.cpp:1692: "route event smoke expected resolution event copy") &&
tests/SmokeChecks.cpp:1695: "route event smoke expected success-plus-cooldown summary after resolution");
tests/SmokeChecks.cpp:1729: profile.character.inventory.push_back({"trade_voucher", 1, 0.0f});
tests/SmokeChecks.cpp:1736: if (!Check(worldState != nullptr, "merchant route event smoke expected world field state")) {
tests/SmokeChecks.cpp:1739: worldState->tradeNetworkActive = true;
tests/SmokeChecks.cpp:1740: worldState->serviceBayActive = true;
tests/SmokeChecks.cpp:1741: worldState->serviceCyclesCompleted = 1;
tests/SmokeChecks.cpp:1751: const int vouchersBefore = countInventory(profile, "trade_voucher");
tests/SmokeChecks.cpp:1753: "merchant route event smoke expected merchant-window spawn")) {
tests/SmokeChecks.cpp:1757: "merchant route event smoke expected offered merchant summary")) {
tests/SmokeChecks.cpp:1761: "merchant route event smoke expected merchant exchange resolution")) {
tests/SmokeChecks.cpp:1765: return Check(!bunker::HasActiveRouteEvent(*worldState), "merchant route event smoke expected exchange to close the window") &&
```

tests/SmokeChecks.cpp:1766: Check(worldState->routeEventsResolved == 1, "merchant route event smoke expected resolved counter increment") &&

tests/SmokeChecks.cpp:1767: Check(countInventory(profile, "trade_voucher") == vouchersBefore - 1,

tests/SmokeChecks.cpp:1768: "merchant route event smoke expected voucher to be spent first") &&

tests/SmokeChecks.cpp:1771: "merchant route event smoke expected merchant resolution copy") &&

tests/SmokeChecks.cpp:1773: "merchant route event smoke expected merchant success summary with cooldown");

tests/SmokeChecks.cpp:1797: acidProfile.character.inventory.push_back({"trade_voucher", 1, 0.0f});

tests/SmokeChecks.cpp:1804: if (!Check(acidWorld != nullptr, "weather-weighted route event smoke expected acid world field state")) {

tests/SmokeChecks.cpp:1807: acidWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1808: acidWorld->serviceCyclesCompleted = 0;

tests/SmokeChecks.cpp:1818: if (!Check(acidWorld->activeRouteEventType == "service_call",

tests/SmokeChecks.cpp:1819: "weather-weighted route event smoke expected acid rain to bias into service_call instead of merchant")) {

tests/SmokeChecks.cpp:1823: "weather-weighted route event smoke expected acid-rain offer copy")) {

tests/SmokeChecks.cpp:1847: fogProfile.character.inventory.push_back({"trade_voucher", 1, 0.0f});

tests/SmokeChecks.cpp:1854: if (!Check(fogWorld != nullptr, "weather-weighted route event smoke expected fog world field state")) {

tests/SmokeChecks.cpp:1857: fogWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1858: fogWorld->serviceBayActive = true;

tests/SmokeChecks.cpp:1859: fogWorld->serviceCyclesCompleted = 1;

tests/SmokeChecks.cpp:1872: "weather-weighted route event smoke expected ether fog to bias into relay_instability instead of merchant; got " +

tests/SmokeChecks.cpp:1875: "weather-weighted route event smoke expected ether-fog offer copy");

tests/SmokeChecks.cpp:1915: alphaWorld->caravanRouteActive = true;

tests/SmokeChecks.cpp:1916: alphaWorld->tradeNetworkActive = true;

tests/SmokeChecks.cpp:1917: alphaWorld->railFreightActive = true;

tests/SmokeChecks.cpp:1919: alphaWorld->railFortressActive = true;

tests/SmokeChecks.cpp:1929: alphaWorld->serviceBayActive = true;

tests/SmokeChecks.cpp:1948: Check(betaObjective.find("rail depot") != std::string::npos,

tests/SmokeChecks.cpp:1949: "world-scoped route summary smoke expected beta preview objective to stay at rail depot: " + betaObjective) &&

tests/SmokeChecks.cpp:1952: Check(betaHandoff.find("rail freight") != std::string::npos,

tests/SmokeChecks.cpp:1953: "world-scoped route summary smoke expected beta handoff to point at rail freight: " + betaHandoff) &&

tests/SmokeChecks.cpp:1958: betaBackbone.status.find("rail freight") != std::string::npos,

tests/SmokeChecks.cpp:1959: "world-scoped route summary smoke expected beta backbone status to stay on rail freight: " + betaBackbone.status) &&

tests/SmokeChecks.cpp:1964: "world-scoped route summary smoke expected beta preview to show its blocked-route event");

tests/SmokeChecks.cpp:2626: if (!Check(workshop != nullptr, "service choice smoke expected workshop anchor")) {

tests/SmokeChecks.cpp:2632: profile.selectedWorld = "service_choice_smoke.bwld";

tests/SmokeChecks.cpp:2681: "service choice smoke expected doctrine-weighted service to recover more operator HP") &&

tests/SmokeChecks.cpp:2683: "service choice smoke expected doctrine-weighted service to recover more operator MP") &&

tests/SmokeChecks.cpp:2685: "service choice smoke expected Pilot Sync service pass to recover more BT-72 energy") &&

tests/SmokeChecks.cpp:2687: "service choice smoke expected readable doctrine-weight feedback copy");

tests/SmokeChecks.cpp:2744: if (!Check(worldState != nullptr, "lanline services smoke failed to create world field state")) {

tests/SmokeChecks.cpp:2751: worldState->caravanRouteActive = true;

tests/SmokeChecks.cpp:2755: worldState->tradeNetworkActive = true;

tests/SmokeChecks.cpp:2756: worldState->railFreightActive = true;

tests/SmokeChecks.cpp:2757: worldState->railFortressActive = true;

tests/SmokeChecks.cpp:2764: worldState->serviceBayActive = true;

tests/SmokeChecks.cpp:2771: if (!Check(unlockState.towerSyncRecovered, "lanline services smoke expected tower sync recovery flag")) {

tests/SmokeChecks.cpp:2774: if (!Check(unlockState.relaySubstationActive, "lanline services smoke expected relay substation unlock flag")) {

tests/SmokeChecks.cpp:2777: if (!Check(unlockState.serviceBayActive, "lanline services smoke expected service bay unlock flag")) {

tests/SmokeChecks.cpp:2780: if (!Check(unlockState.waterReclaimerActive, "lanline services smoke expected water reclaimer unlock flag")) {

tests/SmokeChecks.cpp:2783: if (!Check(unlockState.backboneStable, "lanline services smoke expected stable backbone")) {

tests/SmokeChecks.cpp:2786: if (!Check(unlockState.feyRingIntercityUnlocked, "lanline services smoke expected inter-city Fey unlock")) {

tests/SmokeChecks.cpp:2789: if (!Check(unlockState.feyRingInterserverUnlocked, "lanline services smoke expected inter-server Fey unlock")) {

tests/SmokeChecks.cpp:2792: if (!Check(bunker::IsTankServiceUnlocked(unlockState), "lanline services smoke expected tank service unlock")) {

tests/SmokeChecks.cpp:2795: if (!Check(bunker::IsMedicalSupportUnlocked(unlockState), "lanline services smoke expected medical support unlock")) {

tests/SmokeChecks.cpp:2799: "lanline services smoke expected mirrored stable backbone stage")) {

tests/SmokeChecks.cpp:2803: "lanline services smoke expected mirrored full backbone status")) {

tests/SmokeChecks.cpp:2807: "lanline services smoke expected mirrored backbone payoff")) {

tests/SmokeChecks.cpp:2811: "lanline services smoke expected mirrored idle route-event summary")) {

tests/SmokeChecks.cpp:2814: if (!Check(!unlockState.merchantWindowActive, "lanline services smoke expected merchant window to start inactive")) {

tests/SmokeChecks.cpp:2822: order.orderId = "order_service_01";

tests/SmokeChecks.cpp:2835: if (!Check(profileSnapshot.relayCredits == 777, "lanline services smoke expected profile relay credits sync")) {

tests/SmokeChecks.cpp:2838: if (!Check(profileSnapshot.ownedCosmetics.size() == 2, "lanline services smoke expected cosmetic snapshot sync")) {

tests/SmokeChecks.cpp:2842: profileSnapshot.pendingSupportOrders[0] == "order_service_01",

tests/SmokeChecks.cpp:2843: "lanline services smoke expected pending support order sync")) {

tests/SmokeChecks.cpp:2849: "lanline services smoke failed to save services state")) {

tests/SmokeChecks.cpp:2855: "lanline services smoke failed to load services state")) {

tests/SmokeChecks.cpp:2863: "lanline services smoke expected one delivered support order after time advance")) {

tests/SmokeChecks.cpp:2871: if (!Check(merchantUnlockState.merchantWindowActive, "lanline services smoke expected merchant window mirror when active")) {

tests/SmokeChecks.cpp:2875: "lanline services smoke expected merchant route-event summary mirror")) {

tests/SmokeChecks.cpp:2881: "lanline services smoke expected delivered support order claim")) {

tests/SmokeChecks.cpp:2888: return Check(loadedState.relayCredits == 777, "lanline services smoke expected relay credits after roundtrip") &&

tests/SmokeChecks.cpp:2889: Check(loadedState.ownedCosmetics.size() == 2, "lanline services smoke expected owned cosmetics after roundtrip") &&

tests/SmokeChecks.cpp:2890: Check(loadedState.supportOrders.size() == 1, "lanline services smoke expected one support order after roundtrip") &&

tests/SmokeChecks.cpp:2892: "lanline services smoke expected destination node after roundtrip") &&

tests/SmokeChecks.cpp:2894: "lanline services smoke expected claimed support order after runtime claim") &&

tests/SmokeChecks.cpp:2896: "lanline services smoke expected pending support orders to clear after claim") &&

tests/SmokeChecks.cpp:2898: "lanline services smoke expected world relay credit spend mirror after session sync") &&

tests/SmokeChecks.cpp:2900: "lanline services smoke expected claim summary to mention delivered order") &&

tests/SmokeChecks.cpp:2907: "lanline services smoke expected engine service kit delivery to grant engine_seal inventory");

tests/SmokeChecks.cpp:2935: "tank service smoke expected engine kit to apply first")) {

tests/SmokeChecks.cpp:2938: if (!Check(countItem("engine_seal") == 0, "tank service smoke expected engine seal to be consumed")) {

tests/SmokeChecks.cpp:2942: "tank service smoke expected engine kit to restore power core and reserve charge")) {

tests/SmokeChecks.cpp:2945: if (!Check(eventText.find("Engine service kit applied") != std::string::npos,

tests/SmokeChecks.cpp:2946: "tank service smoke expected engine kit event text")) {

tests/SmokeChecks.cpp:2952: "tank service smoke expected suspension kit to apply second")) {

tests/SmokeChecks.cpp:2955: if (!Check(countItem("track_patch") == 0, "tank service smoke expected track patch to be consumed")) {

tests/SmokeChecks.cpp:2959: "tank service smoke expected suspension kit to restore hull carriage and bucket rig")) {

tests/SmokeChecks.cpp:2963: "tank service smoke expected suspension kit event text")) {

tests/SmokeChecks.cpp:2969: "tank service smoke expected sensor kit to apply third")) {

tests/SmokeChecks.cpp:2972: if (!Check(countItem("lens_pack") == 0, "tank service smoke expected lens pack to be consumed")) {

tests/SmokeChecks.cpp:2976: "tank service smoke expected sensor kit to restore optics and cockpit feed")) {

tests/SmokeChecks.cpp:2980: "tank service smoke expected sensor kit event text")) {

tests/SmokeChecks.cpp:2986: "tank service smoke expected turret kit to apply fourth")) {

```
tests/SmokeChecks.cpp:2989: if (!Check(countItem("servo_patch") == 0, "tank service smoke expected servo
patch to be consumed")) {

tests/SmokeChecks.cpp:2993: "tank service smoke expected turret kit to restore turret integrity")) {

tests/SmokeChecks.cpp:2996: if (!Check(eventText.find("Turret service kit applied") != std::string::npos,

tests/SmokeChecks.cpp:2997: "tank service smoke expected turret kit event text")) {

tests/SmokeChecks.cpp:3001: "tank service smoke expected successful service to clear in-repair flag")) {

tests/SmokeChecks.cpp:3007: "tank service smoke expected missing kit fallback after all kits are consumed")) {

tests/SmokeChecks.cpp:3010: if (!Check(eventText.find("Compatible tank service kit not found.") !=
std::string::npos,

tests/SmokeChecks.cpp:3011: "tank service smoke expected missing kit message")) {

tests/SmokeChecks.cpp:3019: "tank service smoke expected idle tank not to consume a valid kit") &&

tests/SmokeChecks.cpp:3021: "tank service smoke expected idle tank to keep the suspension kit") &&

tests/SmokeChecks.cpp:3023: "tank service smoke expected explicit no-damage guidance");

tests/SmokeChecks.cpp:3096: if (!Check(bunker::NormalizeGameplayDescriptorTag("workshop_field_service") ==
"workshop_service", "workshop alias normalization failed")) {

tests/SmokeChecks.cpp:3150: (issue.relatedValue == "service_bay" ||

tests/SmokeChecks.cpp:3172: if (!Check(dependencyTags[0] == "service_bay" &&

tests/SmokeChecks.cpp:3210: if (edge.sourceScriptTag == "water_reclaimer" && edge.dependencyScriptTag ==
"service_bay") {

tests/SmokeChecks.cpp:3219: if (edge.sourceScriptTag == "service_bay" && edge.dependencyScriptTag ==
"foundry_line") {

tests/SmokeChecks.cpp:3230: Check(sawWaterToService, "semantic dependency graph missing water_reclaimer -
> service_bay edge") &&

tests/SmokeChecks.cpp:3233: Check(sawServiceToFoundry, "semantic dependency graph missing service_bay ->
foundry_line edge") &&

tests/SmokeChecks.cpp:3263: const auto* serviceBay = world.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:3271: if (!Check(serviceBay != nullptr &&

tests/SmokeChecks.cpp:3284: const float firstLayerX = serviceBay->x;

tests/SmokeChecks.cpp:3299: Check(serviceBay->y < relaySubstation->y && relaySubstation->y <
recoveryFabricator->y,

tests/SmokeChecks.cpp:3320: bunker::MapObject serviceBay;

tests/SmokeChecks.cpp:3321: serviceBay.registryId = "[%service_manual_0001]";
```

```
tests/SmokeChecks.cpp:3322: serviceBay.displayName = "Service Bay";
tests/SmokeChecks.cpp:3323: serviceBay.interaction = bunker::InteractionType::Terminal;
tests/SmokeChecks.cpp:3324: serviceBay.category = bunker::ObjectCategory::Terminal;
tests/SmokeChecks.cpp:3325: serviceBay.scriptTag = "service_bay";
tests/SmokeChecks.cpp:3326: serviceBay.linkTarget = "inner_spur_service";
tests/SmokeChecks.cpp:3327: serviceBay.x = 27.0f;
tests/SmokeChecks.cpp:3328: serviceBay.y = -17.0f;
tests/SmokeChecks.cpp:3329: world.objects.push_back(serviceBay);
tests/SmokeChecks.cpp:3367: const auto* authoredServiceBay = world.FindObjectByScriptTag("service_bay");
tests/SmokeChecks.cpp:3390: "preserve-manual layout should not move authored service bay")) {
tests/SmokeChecks.cpp:3430: "full semantic reflow should align authored service bay into the first-layer column")
&&
tests/SmokeChecks.cpp:3471: "service_bay",
tests/SmokeChecks.cpp:3618: Check(loaderPrefabs[0].object.lootEntries.size() == 5, "prefab library should
preserve scalable loot entries without trailing empty UI rows") &&
tests/SmokeChecks.cpp:3688: relayPrefab.id = "prefab_relay_service";
tests/SmokeChecks.cpp:3731: brokenObject.prefabSourceId = "prefab_missing_service";
tests/SmokeChecks.cpp:4092: if (!Check(std::filesystem::exists(prototypeExport.auditTrailPath), "prototype
export should append export audit trail")) {
tests/SmokeChecks.cpp:4169: bool RunWorldExportAuditTrailSmoke() {
tests/SmokeChecks.cpp:4171: world.metadata.name = "World Export Audit Trail Smoke";
tests/SmokeChecks.cpp:4184: if (!Check(batch.createdCount == 7, "export audit trail smoke expected seven auto-
created dependency anchors")) {
tests/SmokeChecks.cpp:4193: if (!Check(prototypeExport.ok, "export audit trail smoke expected prototype export
to pass")) {
tests/SmokeChecks.cpp:4202: "export audit trail smoke expected strict export to block by policy")) {
tests/SmokeChecks.cpp:4207: if (!Check(adoptedAnchors == 7, "export audit trail smoke expected adopt-all to
convert seven anchors")) {
tests/SmokeChecks.cpp:4215: if (!Check(shippingExport.ok, "export audit trail smoke expected shipping export to
pass after adopt-all")) {
tests/SmokeChecks.cpp:4218: if (!Check(std::filesystem::exists(shippingExport.auditTrailPath), "export audit trail
smoke expected audit trail file")) {
```

```
tests/SmokeChecks.cpp:4222: const std::string auditTrail =
bunker::LoadTextArtifactPreview(shippingExport.auditTrailPath, 20000);

tests/SmokeChecks.cpp:4223: return Check(auditTrail.find("Policy: prototype / allow warnings") !=
std::string::npos,

tests/SmokeChecks.cpp:4224: "export audit trail should include prototype export entry") &&

tests/SmokeChecks.cpp:4225: Check(auditTrail.find("Policy: shipping-safe") != std::string::npos,

tests/SmokeChecks.cpp:4226: "export audit trail should include shipping export entries") &&

tests/SmokeChecks.cpp:4227: Check(auditTrail.find("Decision: blocked by policy") != std::string::npos,

tests/SmokeChecks.cpp:4228: "export audit trail should record blocked shipping export") &&

tests/SmokeChecks.cpp:4229: Check(auditTrail.find("Auto-created semantic anchors: 0") != std::string::npos,

tests/SmokeChecks.cpp:4230: "export audit trail should record zero semantic debt after adopt-all") &&

tests/SmokeChecks.cpp:4231: Check(auditTrail.find("Decision: exported") != std::string::npos,

tests/SmokeChecks.cpp:4232: "export audit trail should record successful exports");

tests/SmokeChecks.cpp:4353: auto* serviceBay = world.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:4355: if (!Check(serviceBay != nullptr, "shipping baseline object-aware smoke expected
service_bay")) {

tests/SmokeChecks.cpp:4362: const std::string serviceBayRegistryId = serviceBay->registryId;

tests/SmokeChecks.cpp:4363: serviceBay->linkTarget = "service_bay_drift";

tests/SmokeChecks.cpp:4374: serviceBay->linkTarget = "inner_spur_service";

tests/SmokeChecks.cpp:4396: if (!Check(objectAwareDelta.issueImprovements[0].objectId ==
serviceBayRegistryId,

tests/SmokeChecks.cpp:4397: "shipping baseline object-aware smoke expected service_bay baseline drift to be
marked as improved")) {

tests/SmokeChecks.cpp:4404: Check(objectAwareReport.find(serviceBayRegistryId) != std::string::npos,

tests/SmokeChecks.cpp:4405: "shipping baseline object-aware report should mention improved service_bay
object");

tests/SmokeChecks.cpp:4432: auto* serviceBay = world.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:4434: if (!Check(serviceBay != nullptr, "export history selection smoke expected
service_bay")) {

tests/SmokeChecks.cpp:4441: const std::string serviceBayRegistryId = serviceBay->registryId;

tests/SmokeChecks.cpp:4446: std::filesystem::remove(bunker::ExportAuditTrailPathForWorld(exportPath),
cleanupEc);
```


tests/SmokeChecks.cpp:4475: serviceBay->linkTarget = "service_bay_history_drift";

tests/SmokeChecks.cpp:4488: serviceBay->linkTarget = "inner_spur_service";

tests/SmokeChecks.cpp:4670: Check(olderDelta.issueImprovements[0].objectId == serviceBayRegistryId,

tests/SmokeChecks.cpp:4671: "export history selection smoke expected older checkpoint compare to point at service_bay improvement") &&

tests/SmokeChecks.cpp:4676: Check(olderReport.find(serviceBayRegistryId) != std::string::npos,

tests/SmokeChecks.cpp:4677: "export history selection smoke expected report to mention improved service_bay object") &&

tests/SmokeChecks.cpp:4714: WriteRawString(file, "[%service_bay_auto_0001]");

tests/SmokeChecks.cpp:4716: WriteRawString(file, "service_bay");

tests/SmokeChecks.cpp:4717: WriteRawString(file, "inner_spur_service");

tests/SmokeChecks.cpp:4755: const auto* serviceBay = loadedWorld.FindObjectByScriptTag("service_bay");

tests/SmokeChecks.cpp:4756: return Check(serviceBay != nullptr, "legacy semantic inference should load service_bay") &&

tests/SmokeChecks.cpp:4757: Check(bunker::IsAutoGeneratedSemanticAnchor(*serviceBay), "legacy BWL2 auto anchor should infer semanticAutoCreated from registryId") &&

tests/SmokeChecks.cpp:4758: Check(!bunker::IsPinnedSemanticAnchor(*serviceBay), "legacy BWL2 auto anchor should not infer pinned semantic placement");

tests/SmokeChecks.cpp:4848: WriteRawString(file, "[%services_0001]");

tests/SmokeChecks.cpp:4850: WriteRawString(file, "lanline_service_hub");

tests/SmokeChecks.cpp:4851: WriteRawString(file, "shelter17_services");

tests/SmokeChecks.cpp:4893: const auto* servicesHub = loadedWorld.FindObjectByScriptTag("lanline_service_hub");

tests/SmokeChecks.cpp:4895: return Check(servicesHub != nullptr, "legacy editor-layer inference should load service hub") &&

tests/SmokeChecks.cpp:4896: Check(servicesHub->editorLayer == "Service", "legacy BWL3 object should infer Service editor layer from scriptTag") &&

tests/SmokeChecks.cpp:4921: addedObject.registryId = "[%service_0001]";

tests/SmokeChecks.cpp:4925: addedObject.scriptTag = "lanline_service_hub";

tests/SmokeChecks.cpp:4926: addedObject.linkTarget = "shelter17_services";

tests/SmokeChecks.cpp:4928: undoStack.PushObjectAdded("Add service relay", world.objects.back(), 1, addedObject.registryId);

```
tests/SmokeChecks.cpp:4950: undoStack.PushObjectUpdated("Edit service relay", beforeFirstUpdate,
world.objects[1], 1);

tests/SmokeChecks.cpp:4960: undoStack.PushObjectUpdated("Edit service relay", beforeSecondUpdate,
world.objects[1], 1);

tests/SmokeChecks.cpp:5022: undoStack.PushObjectRemoved("Delete service relay", removedObject, 1,
removedObject.registryId, rootObject.registryId);

tests/SmokeChecks.cpp:5176: "service_bay",

tests/SmokeChecks.cpp:5247: "rail_depot",

tests/SmokeChecks.cpp:5249: "rail_fortress_hub",

tests/SmokeChecks.cpp:5259: "service_bay",

tests/SmokeChecks.cpp:5261: "lanline_service_hub",

tests/SmokeChecks.cpp:5264: "tank_service",

tests/SmokeChecks.cpp:5744: "bt72_hull_moved_to_service_lift_before_core_install expected crane transfer to
lock cradle")) {

tests/SmokeChecks.cpp:5762: "bt72_staged_restoration_requires_service_notes expected service notes to gate
final restoration")) {

tests/SmokeChecks.cpp:5767: "bt72_staged_restoration_requires_service_notes expected staged restoration
after notes")) {

tests/SmokeChecks.cpp:5815: addTerminal("[%bt72_service_lift_0001]", "BT-72 Service Lift");

tests/SmokeChecks.cpp:5828: const auto* lift = world.FindObjectByRegistryId("[%bt72_service_lift_0001]");

tests/SmokeChecks.cpp:5870: "bt72_service_lift_requires_attached_hull expected lift to reject unattached hull"))
{

tests/SmokeChecks.cpp:5922: addObject("[%bt72_service_notes_0001]", "BT-72 Service Notes",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);

tests/SmokeChecks.cpp:5934: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world,
player, lockedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5936: !bunker::HasCollectedTapId(lockedProfile, "bt72_service_reel_001"),

tests/SmokeChecks.cpp:5937: "bt72_service_notes_require_pippad expected service notes to stay locked before
Pip-Pad")) {

tests/SmokeChecks.cpp:5944: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world,
player, notesProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5946: bunker::HasCollectedTapId(notesProfile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:5949: "bt72_service_notes_runtime_recovered expected runtime service notes without
tank unlock")) {
```

tests/SmokeChecks.cpp:5963: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, notesProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:5966: bunker::HasCollectedTapeld(notesProfile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:5970: "bt72_service_material_pickups_are_idempotent expected stable duplicate interactions");

tests/SmokeChecks.cpp:5987: "[%bt72_service_lift_0001]",

tests/SmokeChecks.cpp:5988: "[%bt72_service_notes_0001]",

tests/SmokeChecks.cpp:6002: "[%bt72_service_lift_0001]",

tests/SmokeChecks.cpp:6003: "[%bt72_service_notes_0001]",

tests/SmokeChecks.cpp:6041: hasTypedObject("[%bt72_service_lift_0001]", bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal) &&

tests/SmokeChecks.cpp:6042: hasTypedObject("[%bt72_service_notes_0001]", bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal) &&

tests/SmokeChecks.cpp:6078: hasObject("[%bt72_service_lift_0001]"),

tests/SmokeChecks.cpp:6082: return Check(hasObject("[%bt72_service_notes_0001]") &&

tests/SmokeChecks.cpp:6084: "starter_world_contains_bt72_service_material_objects expected service/material runtime objects");

tests/SmokeChecks.cpp:6137: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6144: bunker::HasCollectedTapeld(profile, "bt72_service_reel_001") &&

tests/SmokeChecks.cpp:6156: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6420: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6440: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6563: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), loadedWorld, player, loadedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:6583: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld, player, loadedProfile, staticEraser, gameState);

tests/SmokeChecks.cpp:6757: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:6763: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player, profile, staticEraser, gameState);

```

tests/SmokeChecks.cpp:7086: invalidProfile.pipDeviceCarryMode = "drone";

tests/SmokeChecks.cpp:8132: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"),
loadedWorld, player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8149: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), loadedWorld,
player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8201: addObject("[%bt72_service_lift_0001]", "BT-72 Service Lift",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);

tests/SmokeChecks.cpp:8202: addObject("[%bt72_service_notes_0001]", "BT-72 Service Notes",
bunker::InteractionType::Terminal, bunker::ObjectCategory::Terminal);

tests/SmokeChecks.cpp:8228: "first_route_e2e_pippad_recovered_before_ui expected access card and pre-Pip-
Pad trail")) {

tests/SmokeChecks.cpp:8272: bunker::HandleInteraction(objectById("[%bt72_service_notes_0001]"), world,
player, profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8288: bunker::HandleInteraction(objectById("[%bt72_service_lift_0001]"), world, player,
profile, staticEraser, gameState);

tests/SmokeChecks.cpp:8509: RunWorldExportAuditTrailSmoke() &&

```

Success

Ran command

```
$ git grep -n
```

```

"returnedToBase\|exitedBunker\|surfaceArrivalReached\|outerRoadCleared\|AddInventoryItem\|ConsumeInve
ntoryItem\|PlayerHasActivePipDeviceAccess\|HasActivePipDevice\|TryToggleActivePipDeviceUi\|ApplyActivePip
DeviceCustomizationCommand\|Lanline" -- include src tests

```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```

include/AppPaths.hpp:27:inline fs::path LanlineSessionsDirectory() {

include/AppPaths.hpp:35: fs::create_directories(LanlineSessionsDirectory());

include/AppPaths.hpp:152:inline fs::path LanlineSessionPath() {

include/AppPaths.hpp:156:inline fs::path LanlineSessionSnapshotPath(std::string_view sessionId) {

include/AppPaths.hpp:157: return LanlineSessionsDirectory() / (SanitizeWorldKey(sessionId) + ".state");

include/GameRuntime.hpp:128:void AddInventoryItem(SessionProfile& profile, const std::string& itemId, int
count, float weight);

include/GameRuntime.hpp:130:bool ConsumeInventoryItem(SessionProfile& profile, const std::string& itemId,
int count);

include/GameRuntime.hpp:185:bool HasActivePipDevice(const SessionProfile& profile);

include/GameRuntime.hpp:208:bool ApplyActivePipDeviceCustomizationCommand(SessionProfile& profile,

```

```

include/GameRuntime.hpp:229:bool PlayerHasActivePipDeviceAccess(const SessionProfile& profile);

include/GameRuntime.hpp:231:bool TryToggleActivePipDeviceUi(PlayerState& player, const SessionProfile&
profile, GameState& gameState);

include/LanlineLobbyLogic.hpp:5:#include "LanlineSession.hpp"

include/LanlineLobbyLogic.hpp:9:inline bool IsLanlineAwaitingSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:13:inline bool IsLanlinePendingSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:17:inline bool IsLanlineReservedSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:21:inline bool IsLanlineAcceptedSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:25:inline bool IsLanlineReadyEligibleSlot(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:29:inline const char* LanlineSlotStateLabel(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:30: if (IsLanlineAwaitingSlot(entry)) {

include/LanlineLobbyLogic.hpp:33: if (IsLanlinePendingSlot(entry)) {

include/LanlineLobbyLogic.hpp:36: if (IsLanlineReservedSlot(entry)) {

include/LanlineLobbyLogic.hpp:39: if (IsLanlineAcceptedSlot(entry)) {

include/LanlineLobbyLogic.hpp:45:inline const char* LanlineReadyLabel(const LanlinePlayerEntry& entry) {

include/LanlineLobbyLogic.hpp:46: if (!IsLanlineReadyEligibleSlot(entry)) {

include/LanlineLobbyLogic.hpp:52:inline int FindFirstAwaitingSlotIndex(const LanlineSessionState& state) {

include/LanlineLobbyLogic.hpp:54: if (IsLanlineAwaitingSlot(state.players[static_cast<std::size_t>(index)])) {

include/LanlineLobbyLogic.hpp:61:inline int MaxLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:68:inline int OccupiedLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:71: if (session.mode == "LAN Host" && IsLanlineAwaitingSlot(player)) {

include/LanlineLobbyLogic.hpp:81:inline int AvailableLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:82: return std::max(0, MaxLanlineSessionSlots(session) -
OccupiedLanlineSessionSlots(session));

include/LanlineLobbyLogic.hpp:85:inline int PendingLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:88: if (IsLanlinePendingSlot(player)) {

include/LanlineLobbyLogic.hpp:95:inline int ReservedLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:98: if (IsLanlineReservedSlot(player)) {

include/LanlineLobbyLogic.hpp:105:inline int AcceptedLanlineSessionSlots(const LanlineSessionState& session) {

include/LanlineLobbyLogic.hpp:108: if (IsLanlineAcceptedSlot(player)) {

```

```
include/LanlineLobbyLogic.hpp:115:inline int ReadyLanlineSessionSlots(const LanlineSessionState& session) {  
include/LanlineLobbyLogic.hpp:118: if (!IsLanlineReadyEligibleSlot(player) && player.ready) {  
include/LanlineLobbyLogic.hpp:125:inline bool IsJoinableLanlineSession(const LanlineSessionState& session) {  
include/LanlineLobbyLogic.hpp:137:inline bool IsLanlineMatchStartReady(const LanlineSessionState& session) {  
include/LanlineLobbyLogic.hpp:141: if (ReservedLanlineSessionSlots(session) > 0 ||  
PendingLanlineSessionSlots(session) > 0) {  
include/LanlineLobbyLogic.hpp:147: if (!IsLanlineReadyEligibleSlot(player)) {  
include/LanlineServices.hpp:9:#include "LanlineSession.hpp"  
include/LanlineServices.hpp:16: LanlineLocal,  
include/LanlineServices.hpp:50:inline bool IsLanlineServicesUnlocked(const ServicesUnlockState& state) {  
include/LanlineServices.hpp:183:struct LanlineServicesState {  
include/LanlineServices.hpp:195:struct LanlineServicesSave {  
include/LanlineServices.hpp:206:LanlineServicesState MakeDefaultLanlineServicesState(std::int64_t nowUnix);  
include/LanlineServices.hpp:207:LanlineServicesState MakeLanlineServicesStateFromSave(const  
LanlineServicesSave& save, std::int64_t nowUnix);  
include/LanlineServices.hpp:208:LanlineServicesSave BuildLanlineServicesSave(const LanlineServicesState&  
state);  
include/LanlineServices.hpp:209:void ApplyLanlineServicesProfileSnapshot(LanlineServicesState& state, const  
LanlineServicesProfile& profile);  
include/LanlineServices.hpp:210:void SyncLanlineServicesProfileSnapshot(LanlineServicesProfile& profile, const  
LanlineServicesState& state);  
include/LanlineServices.hpp:211:void SyncLanlineServicesSessionProfile(SessionProfile& profile, const  
LanlineServicesState& state);  
include/LanlineServices.hpp:212:void AdvanceLanlineSupportOrders(LanlineServicesState& state, std::int64_t  
nowUnix);  
include/LanlineServices.hpp:213:int CountSupportOrdersInState(const LanlineServicesState& state,  
SupportOrderState orderState);  
include/LanlineServices.hpp:214:int ClaimDeliveredSupportOrders(LanlineServicesState& state, SessionProfile&  
profile, std::string* summary = nullptr);  
include/LanlineServices.hpp:215:ServiceHubMode ResolveLanlineServicesMode(const ServicesUnlockState&  
unlockState, const LanlineSessionState* sessionState);  
include/LanlineServices.hpp:216:void SyncLanlineServicesPresence(LanlineServicesState& state,  
include/LanlineServices.hpp:217: const LanlineSessionState* sessionState,
```

```
include/LanlineServices.hpp:221:std::filesystem::path DefaultLanlineServicesSavePath();

include/LanlineServices.hpp:222:bool SaveLanlineServicesSave(const LanlineServicesSave& save, const
std::filesystem::path& path);

include/LanlineServices.hpp:223:bool LoadLanlineServicesSave(const std::filesystem::path& path,
LanlineServicesSave& outSave);

include/LanlineServices.hpp:224:void DrawLanlineServicesPanel(LanlineServicesState& state,

include/LanlineSession.hpp:10:struct LanlinePlayerEntry {

include/LanlineSession.hpp:18:struct LanlineRelayMessage {

include/LanlineSession.hpp:25:struct LanlineVoicePresence {

include/LanlineSession.hpp:34:struct LanlineSessionState {

include/LanlineSession.hpp:48: std::vector<LanlinePlayerEntry> players{};

include/LanlineSession.hpp:49: std::vector<LanlineRelayMessage> relayMessages{};

include/LanlineSession.hpp:50: std::vector<LanlineVoicePresence> voicePresence{};

include/LanlineSession.hpp:54:struct LanlineDiagnostics {

include/LanlineSession.hpp:65:bool SaveLanlineSessionState(const LanlineSessionState& state);

include/LanlineSession.hpp:66:bool LoadLanlineSessionState(LanlineSessionState& state);

include/LanlineSession.hpp:67:bool SaveLanlineSessionState(const LanlineSessionState& state, const
std::filesystem::path& path);

include/LanlineSession.hpp:68:bool LoadLanlineSessionState(const std::filesystem::path& path,
LanlineSessionState& state);

include/LanlineSession.hpp:69:std::vector<LanlineSessionState> DiscoverLanlineSessionSnapshots();

include/LanlineSession.hpp:70:LanlineDiagnostics ProbeLanlineHost(const LanlineSessionState& session,
std::string_view runtimeWorldName);

include/SessionProfiles.hpp:70:struct LanlineServicesProfile {

include/SessionProfiles.hpp:169: bool surfaceArrivalReached = false;

include/SessionProfiles.hpp:182: bool exitedBunker = false;

include/SessionProfiles.hpp:183: bool outerRoadCleared = false;

include/SessionProfiles.hpp:185: bool returnedToBase = false;

include/SessionProfiles.hpp:417: LanlineServicesProfile lanlineServices{};

include/SessionProfiles.hpp:636: profile.story.outerRoadCleared &&

include/SessionProfiles.hpp:654: profile.story.outerRoadCleared &&
```

```
include/StoryRoute.hpp:56:bool HasLanlineServicesObjective(const SessionProfile& profile);

src/GameRuntime.cpp:5:#include "../include/LanlineLobbyLogic.hpp"

src/GameRuntime.cpp:6:#include "../include/LanlineServices.hpp"

src/GameRuntime.cpp:7:#include "../include/LanlineSession.hpp"

src/GameRuntime.cpp:1453: return profile.story.returnedToBase ? "Recovery buildout active" : "Starter route";

src/GameRuntime.cpp:1475: {"lanline_service_hub", "Lanline service hub mirrored. Shelter 17 service catalog
now resolves through authored relay infrastructure."},

src/GameRuntime.cpp:1502:const LanlineDiagnostics& CachedLanlineDiagnostics(const LanlineSessionState&
session, std::string_view runtimeWorldName) {

src/GameRuntime.cpp:1506: static LanlineDiagnostics cachedDiagnostics;

src/GameRuntime.cpp:1512: cachedDiagnostics = ProbeLanlineHost(session, normalizedWorldName);

src/GameRuntime.cpp:1520:const std::vector<std::string>& UpdateLanlineRuntimeNotifications(const
LanlineSessionState* session,

src/GameRuntime.cpp:1524: static LanlineSessionState previousSession;

src/GameRuntime.cpp:1540: pushNotification("Lanline session link lost. Waiting for refreshed launcher/session
state.");

src/GameRuntime.cpp:1552: pushNotification("Lanline session linked: " + session->sessionId + " on " +
normalizedSessionWorld + ".");

src/GameRuntime.cpp:1554: pushNotification("Lanline world mismatch: runtime is on " +
normalizedRuntimeWorld +

src/GameRuntime.cpp:1564: pushNotification("Lanline session switched to " + session->sessionId + " (" + session-
>mode + ").");

src/GameRuntime.cpp:1567: pushNotification("Lanline mode changed to " + session->mode + ".");

src/GameRuntime.cpp:1570: pushNotification("Lanline lifecycle advanced to " + session->lifecycleStage + ".");

src/GameRuntime.cpp:1574: ? "Lanline pending peer queue cleared."

src/GameRuntime.cpp:1575: : "Lanline pending peer updated: " + session->pendingPeer + ".");

src/GameRuntime.cpp:1579: ? "Lanline connected peer link cleared."

src/GameRuntime.cpp:1580: : "Lanline peer link confirmed with " + session->connectedPeer + ".");

src/GameRuntime.cpp:1583: pushNotification("Lanline session world updated: " + previousNormalizedWorld + " -
> " + normalizedSessionWorld + ".");

src/GameRuntime.cpp:1589: ? "Lanline session world now matches the active runtime world."

src/GameRuntime.cpp:1590: : "Lanline session world drifted away from the active runtime world.");
```



```
src/GameRuntime.cpp:1595: [&](const LanlinePlayerEntry& previousEntry) {  
  
src/GameRuntime.cpp:1599: pushNotification(playerEntry.displayName + " joined Lanline roster as " +  
playerEntry.role + ".");  
  
src/GameRuntime.cpp:1603: pushNotification(playerEntry.displayName + " moved from " + previousIt->role + " to  
" + playerEntry.role + " in Lanline lobby.");  
  
src/GameRuntime.cpp:1607: (playerEntry.online ? " came online in Lanline." : " went offline in Lanline."));  
  
src/GameRuntime.cpp:1609: if (bunker::IsLanlineReadyEligibleSlot(playerEntry) && playerEntry.ready !=  
previousIt->ready) {  
  
src/GameRuntime.cpp:1611: (playerEntry.ready ? " is now ready for Lanline deployment." : " is no longer ready  
for Lanline deployment."));  
  
src/GameRuntime.cpp:1617: [&](const LanlinePlayerEntry& currentEntry) {  
  
src/GameRuntime.cpp:1621: pushNotification(previousEntry.displayName + " left the Lanline roster.");  
  
src/GameRuntime.cpp:1637: [&](const LanlineVoicePresence& previousVoice) {  
  
src/GameRuntime.cpp:1660: [&](const LanlineVoicePresence& currentVoice) {  
  
src/GameRuntime.cpp:1697: const char* JoinabilityLabel(const LanlineSessionState& session) {  
  
src/GameRuntime.cpp:1776: if (ConsumeInventoryItem(profile, itemId, 1)) {  
  
src/GameRuntime.cpp:2190: if (profile.story.returnedToBase) score += 8.0f;  
  
src/GameRuntime.cpp:2332: ConsumeInventoryItem(profile, "steel_scrap", 1);  
  
src/GameRuntime.cpp:2333: ConsumeInventoryItem(profile, "copper_wire", 1);  
  
src/GameRuntime.cpp:2358: ConsumeInventoryItem(profile, "copper_wire", 1);  
  
src/GameRuntime.cpp:2359: ConsumeInventoryItem(profile, "power_cell", 1);  
  
src/GameRuntime.cpp:2381: ConsumeInventoryItem(profile, "steel_scrap", 1);  
  
src/GameRuntime.cpp:2382: ConsumeInventoryItem(profile, "power_cell", 1);  
  
src/GameRuntime.cpp:2404: ConsumeInventoryItem(profile, "power_cell", 1);  
  
src/GameRuntime.cpp:2405: ConsumeInventoryItem(profile, "copper_wire", 1);  
  
src/GameRuntime.cpp:2427: ConsumeInventoryItem(profile, "old_plate", 1);  
  
src/GameRuntime.cpp:2428: ConsumeInventoryItem(profile, "power_cell", 1);  
  
src/GameRuntime.cpp:2450: ConsumeInventoryItem(profile, "steel_scrap", 1);  
  
src/GameRuntime.cpp:2451: ConsumeInventoryItem(profile, "ether_shard", 1);  
  
src/GameRuntime.cpp:2478: ConsumeInventoryItem(profile, "old_plate", 2);  
  
src/GameRuntime.cpp:2479: ConsumeInventoryItem(profile, "power_cell", 1);
```

```
src/GameRuntime.cpp:2480: ConsumeInventoryItem(profile, "repair_patch", 1);
src/GameRuntime.cpp:2507: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2508: ConsumeInventoryItem(profile, "copper_wire", 1);
src/GameRuntime.cpp:2530: ConsumeInventoryItem(profile, "old_plate", 2);
src/GameRuntime.cpp:2531: ConsumeInventoryItem(profile, "repair_patch", 1);
src/GameRuntime.cpp:2532: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2554: ConsumeInventoryItem(profile, "steel_scrap", 2);
src/GameRuntime.cpp:2555: ConsumeInventoryItem(profile, "old_plate", 1);
src/GameRuntime.cpp:2556: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2578: ConsumeInventoryItem(profile, "steel_scrap", 3);
src/GameRuntime.cpp:2579: ConsumeInventoryItem(profile, "old_plate", 2);
src/GameRuntime.cpp:2580: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2602: ConsumeInventoryItem(profile, "power_cell", 2);
src/GameRuntime.cpp:2603: ConsumeInventoryItem(profile, "ether_shard", 2);
src/GameRuntime.cpp:2604: ConsumeInventoryItem(profile, "copper_wire", 2);
src/GameRuntime.cpp:2626: ConsumeInventoryItem(profile, "power_cell", 2);
src/GameRuntime.cpp:2627: ConsumeInventoryItem(profile, "copper_wire", 2);
src/GameRuntime.cpp:2628: ConsumeInventoryItem(profile, "ether_shard", 1);
src/GameRuntime.cpp:2650: ConsumeInventoryItem(profile, "copper_wire", 3);
src/GameRuntime.cpp:2651: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2652: ConsumeInventoryItem(profile, "repair_patch", 1);
src/GameRuntime.cpp:2676: ConsumeInventoryItem(profile, "old_plate", 2);
src/GameRuntime.cpp:2677: ConsumeInventoryItem(profile, "repair_patch", 1);
src/GameRuntime.cpp:2678: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2700: ConsumeInventoryItem(profile, "copper_wire", 2);
src/GameRuntime.cpp:2701: ConsumeInventoryItem(profile, "old_plate", 1);
src/GameRuntime.cpp:2702: ConsumeInventoryItem(profile, "power_cell", 1);
src/GameRuntime.cpp:2716: gameState.lastEvent = "Lanline Services stay locked until the first tower is
synchronized.";
src/GameRuntime.cpp:2723: ? "Lanline service hub handshake complete."
```

```
src/GameRuntime.cpp:2724: : "Lanline service hub handshake complete -> " + nearest->linkTarget + ".";

src/GameRuntime.cpp:2741: ? "Tank service anchor ready. Bring a compatible service kit from Lanline support or
field salvage."

src/GameRuntime.cpp:2763: gameState.lastEvent = "Fey Ring routing remains sealed until Lanline Services come
online through tower sync.";

src/GameRuntime.cpp:2916: void AddInventoryItem(SessionProfile& profile, const std::string& itemId, int count,
float weight) {

src/GameRuntime.cpp:2944: AddInventoryItem(profile, entry.itemId, RollLootCount(entry), weight);

src/GameRuntime.cpp:2951: AddInventoryItem(profile, lootId, 1, inventoryWeight);

src/GameRuntime.cpp:2998: bool ConsumeInventoryItem(SessionProfile& profile, const std::string& itemId, int
count) {

src/GameRuntime.cpp:3044: if (!ConsumeInventoryItem(profile, "%it_field_ration", 1)) {

src/GameRuntime.cpp:3207: bool HasActivePipDevice(const SessionProfile& profile) {

src/GameRuntime.cpp:3225: AddInventoryItem(profile, selectedDeviceId, 1, spec->inventoryWeight);

src/GameRuntime.cpp:3233: if (!HasActivePipDevice(profile)) {

src/GameRuntime.cpp:3275: if (HasActivePipDevice(profile) && !profile.pipDeviceReselectPending) {

src/GameRuntime.cpp:3283: return profile.story.exitedBunker ||

src/GameRuntime.cpp:3284: profile.firstPlayableRoute.surfaceArrivalReached ||

src/GameRuntime.cpp:3285: profile.story.outerRoadCleared ||

src/GameRuntime.cpp:3287: profile.story.returnedToBase;

src/GameRuntime.cpp:3387: return HasActivePipDevice(profile) &&
IsKnownPipDeviceId(profile.activePipDeviceId);

src/GameRuntime.cpp:3507: bool ApplyActivePipDeviceCustomizationCommand(SessionProfile& profile,

src/GameRuntime.cpp:3551: return HasActivePipDevice(profile);

src/GameRuntime.cpp:3584: bool PlayerHasActivePipDeviceAccess(const SessionProfile& profile) {

src/GameRuntime.cpp:3589: return PlayerHasActivePipDeviceAccess(profile);

src/GameRuntime.cpp:3611: bool TryToggleActivePipDeviceUi(PlayerState& player, const SessionProfile& profile,
GameState& gameState) {

src/GameRuntime.cpp:3630: return TryToggleActivePipDeviceUi(player, profile, gameState);

src/GameRuntime.cpp:3741: profile.firstPlayableRoute.surfaceArrivalReached =

src/GameRuntime.cpp:3742: profile.firstPlayableRoute.surfaceArrivalReached || profile.story.outerRoadCleared;

src/GameRuntime.cpp:3748: profile.firstPlayableRoute.debriefSummaryViewed || profile.story.returnedToBase;
```

```
src/GameRuntime.cpp:3751: profile.story.outerRoadCleared = profile.story.outerRoadCleared ||
staticEraser.IsErased("#%res_scrap_0001");

src/GameRuntime.cpp:3902: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:3966: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:3986: if (profile.story.outerRoadCleared) {

src/GameRuntime.cpp:4012: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
profile.story.exitedBunker) : true;

src/GameRuntime.cpp:4041: if (!world.IsStarterScenarioWorld() || !profile.story.outerRoadCleared) {

src/GameRuntime.cpp:4109: profile.story.outerRoadCleared &&

src/GameRuntime.cpp:4128: AddInventoryItem(profile, "steel_scrap",

src/GameRuntime.cpp:4131: AddInventoryItem(profile, "copper_wire", 1, 0.2f);

src/GameRuntime.cpp:4132: AddInventoryItem(profile, "repair_patch", 1, 0.2f);

src/GameRuntime.cpp:4135: AddInventoryItem(profile, "ether_shard", 1, 0.1f);

src/GameRuntime.cpp:4174: AddInventoryItem(profile, "steel_scrap", std::max(2, steelYield), 0.5f);

src/GameRuntime.cpp:4175: AddInventoryItem(profile, "copper_wire", std::max(1, wireYield), 0.2f);

src/GameRuntime.cpp:4176: AddInventoryItem(profile, "old_plate", 1, 0.5f);

src/GameRuntime.cpp:4178: AddInventoryItem(profile, "%it_ptrs_amm0", 2, 0.7f);

src/GameRuntime.cpp:4213: AddInventoryItem(profile, "steel_scrap", std::max(1, static_cast<int>(std::round(2.0f
* doctrineBoost * infrastructurePenalty * pylonBoost))), 0.5f);

src/GameRuntime.cpp:4214: AddInventoryItem(profile, "ether_shard", 1, 0.1f);

src/GameRuntime.cpp:4216: AddInventoryItem(profile, "copper_wire", 1, 0.2f);

src/GameRuntime.cpp:4250: AddInventoryItem(profile, "trade_voucher", vouchers, 0.0f);

src/GameRuntime.cpp:4284: AddInventoryItem(profile, "steel_scrap", steelYield, 0.5f);

src/GameRuntime.cpp:4285: AddInventoryItem(profile, "old_plate", plateYield, 0.5f);

src/GameRuntime.cpp:4286: AddInventoryItem(profile, "copper_wire", std::max(1,
static_cast<int>(std::round(2.0f * infrastructurePenalty))), 0.2f);

src/GameRuntime.cpp:4287: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4289: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:4324: AddInventoryItem(profile, "ether_shard", 1, 0.1f);

src/GameRuntime.cpp:4325: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
```

src/GameRuntime.cpp:4327: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:4361: AddInventoryItem(profile, "steel_scrap", std::max(4, static_cast<int>(std::round(6.0f * doctrineBoost * infrastructurePenalty))), 0.5f);

src/GameRuntime.cpp:4362: AddInventoryItem(profile, "old_plate", std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty))), 0.5f);

src/GameRuntime.cpp:4363: AddInventoryItem(profile, "%it_ptrs_ammo", 2, 0.7f);

src/GameRuntime.cpp:4364: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4398: AddInventoryItem(profile, "repair_patch", std::max(1, static_cast<int>(std::round(1.0f * doctrineBoost * infrastructurePenalty))), 0.2f);

src/GameRuntime.cpp:4399: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:4401: AddInventoryItem(profile, "old_plate", 1, 0.5f);

src/GameRuntime.cpp:4403: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:4426: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:4427: AddInventoryItem(profile, "steel_scrap", 2, 0.5f);

src/GameRuntime.cpp:4431: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:4432: AddInventoryItem(profile, "repair_patch", 2, 0.2f);

src/GameRuntime.cpp:4436: AddInventoryItem(profile, "trade_voucher", 2, 0.0f);

src/GameRuntime.cpp:4437: AddInventoryItem(profile, "old_plate", 2, 0.5f);

src/GameRuntime.cpp:4438: AddInventoryItem(profile, "clean_water", 2, 0.4f);

src/GameRuntime.cpp:4439: AddInventoryItem(profile, "%it_ptrs_ammo", 2, 0.7f);

src/GameRuntime.cpp:4472: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4473: AddInventoryItem(profile, "ether_shard", 1, 0.1f);

src/GameRuntime.cpp:4474: AddInventoryItem(profile, "copper_wire", 1, 0.2f);

src/GameRuntime.cpp:4507: AddInventoryItem(profile, "repair_patch", 1, 0.2f);

src/GameRuntime.cpp:4508: AddInventoryItem(profile, "%it_ptrs_ammo", 1, 0.7f);

src/GameRuntime.cpp:4509: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

src/GameRuntime.cpp:4542: AddInventoryItem(profile, "repair_patch", 1, 0.2f);

src/GameRuntime.cpp:4543: AddInventoryItem(profile, "old_plate", 1, 0.5f);

src/GameRuntime.cpp:4545: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:4547: AddInventoryItem(profile, "%it_ptrs_ammo", 1, 0.7f);

src/GameRuntime.cpp:4581: AddInventoryItem(profile, "old_plate", 2, 0.5f);

```
src/GameRuntime.cpp:4582: AddInventoryItem(profile, "repair_patch", 1, 0.2f);
src/GameRuntime.cpp:4583: AddInventoryItem(profile, "steel_scrap", 2, 0.5f);
src/GameRuntime.cpp:4619: AddInventoryItem(profile, "power_cell", 2, 0.3f);
src/GameRuntime.cpp:4620: AddInventoryItem(profile, "ether_shard", 1, 0.1f);
src/GameRuntime.cpp:4656: AddInventoryItem(profile, "power_cell", 1, 0.3f);
src/GameRuntime.cpp:4688: AddInventoryItem(profile, "power_cell", 1, 0.3f);
src/GameRuntime.cpp:4689: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
src/GameRuntime.cpp:4761: AddInventoryItem(profile, "clean_water", std::max(1,
static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty))), 0.4f);
src/GameRuntime.cpp:4763: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:4784: profile.story.returnedToBase ||
src/GameRuntime.cpp:5069: AddInventoryItem(profile, "repair_patch", 1, 0.2f);
src/GameRuntime.cpp:5070: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:5077: AddInventoryItem(profile, "power_cell", 1, 0.3f);
src/GameRuntime.cpp:5078: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
src/GameRuntime.cpp:5084: AddInventoryItem(profile, "copper_wire", 1, 0.2f);
src/GameRuntime.cpp:5090: AddInventoryItem(profile, "steel_scrap", 2, 0.5f);
src/GameRuntime.cpp:5091: AddInventoryItem(profile, "old_plate", 1, 0.5f);
src/GameRuntime.cpp:5097: AddInventoryItem(profile, "trade_voucher", 1, 0.0f);
src/GameRuntime.cpp:5098: AddInventoryItem(profile, "repair_patch", 1, 0.2f);
src/GameRuntime.cpp:5099: AddInventoryItem(profile, "steel_scrap", 2, 0.5f);
src/GameRuntime.cpp:5192: if (profile.story.outerRoadCleared &&
src/GameRuntime.cpp:5355: bool paidWithVoucher = ConsumeInventoryItem(profile, "trade_voucher", 1);
src/GameRuntime.cpp:5368: AddInventoryItem(profile, "repair_patch", 1, 0.2f);
src/GameRuntime.cpp:5369: AddInventoryItem(profile, "filter_media", 1, 0.3f);
src/GameRuntime.cpp:5372: AddInventoryItem(profile, "power_cell", 1, 0.3f);
src/GameRuntime.cpp:5373: AddInventoryItem(profile, "field_battery", 1, 0.2f);
src/GameRuntime.cpp:5376: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:5377: AddInventoryItem(profile, "sealant_roll", 1, 0.2f);
```

```
src/GameRuntime.cpp:5711: AddInventoryItem(profile, player.insideTank ? "wreck_scrap" : "ether_tissue", 1, 0.4f);

src/GameRuntime.cpp:5784: if (!ConsumeInventoryItem(profile, "%it_ptr ammo", 1)) {

src/GameRuntime.cpp:5793: AddInventoryItem(profile, "%it_ptr ammo", 1, 0.7f);

src/GameRuntime.cpp:5903: AddInventoryItem(profile, player.insideTank ? "wreck_scrap" : "ether_tissue", 1, 0.4f);

src/GameRuntime.cpp:5967: AddInventoryItem(profile, "%it_emergency_baton", 1, 1.2f);

src/GameRuntime.cpp:5984: AddInventoryItem(profile, "bunker_access_card", 1, 0.1f);

src/GameRuntime.cpp:5996: AddInventoryItem(profile, "power_cell", 1, 0.3f);

src/GameRuntime.cpp:6015: AddInventoryItem(profile, "old_plate", 1, 0.5f);

src/GameRuntime.cpp:6115: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

src/GameRuntime.cpp:6214: AddInventoryItem(profile, "repair_patch", 1, 0.2f);

src/GameRuntime.cpp:6229: ConsumeInventoryItem(profile, "power_cell", 1);

src/GameRuntime.cpp:6230: ConsumeInventoryItem(profile, "repair_patch", 1);

src/GameRuntime.cpp:6231: ConsumeInventoryItem(profile, "old_plate", 1);

src/GameRuntime.cpp:6321: ConsumeInventoryItem(profile, "scrap_steel", 1);

src/GameRuntime.cpp:6322: ConsumeInventoryItem(profile, "hydraulic_seal", 1);

src/GameRuntime.cpp:6339: profile.story.exitedBunker = true;

src/GameRuntime.cpp:6377: profile.story.outerRoadCleared = true;

src/GameRuntime.cpp:6391: if (!profile.story.outerRoadCleared ||
!(profile.firstPlayableRoute.firstTankCombatResolved || staticEraser.IsErased("[%enemy_ghoul_0001]"))) {

src/GameRuntime.cpp:6405: AddInventoryItem(profile, "relay_reconstruction_data", 1, 0.2f);

src/GameRuntime.cpp:6434: if (!profile.story.returnedToBase) {

src/GameRuntime.cpp:6436: profile.story.returnedToBase = true;

src/GameRuntime.cpp:6460: const bool firstSurfaceArrival = profile.story.exitedBunker &&
!profile.firstPlayableRoute.surfaceArrivalReached;

src/GameRuntime.cpp:6461: if (profile.story.exitedBunker) {

src/GameRuntime.cpp:6462: profile.firstPlayableRoute.surfaceArrivalReached = true;

src/GameRuntime.cpp:6684: ConsumeInventoryItem(profile, "steel_scrap", 1);

src/GameRuntime.cpp:6685: ConsumeInventoryItem(profile, "copper_wire", 1);

src/GameRuntime.cpp:6686: AddInventoryItem(profile, "repair_patch", 1, 0.2f);
```

```
src/GameRuntime.cpp:6707: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {
src/GameRuntime.cpp:6708: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);
src/GameRuntime.cpp:6716: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {
src/GameRuntime.cpp:6717: AddInventoryItem(profile, "power_cell", 1, 0.3f);
src/GameRuntime.cpp:6724: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {
src/GameRuntime.cpp:6725: AddInventoryItem(profile, "%it_ptrs_ammo", 2, 0.7f);
src/GameRuntime.cpp:6870: LanlineSessionState sessionState;
src/GameRuntime.cpp:6871: const bool hasSessionState = LoadLanlineSessionState(sessionState);
src/GameRuntime.cpp:6872: const auto& runtimeNotifications = UpdateLanlineRuntimeNotifications(
src/GameRuntime.cpp:6876: ImGui::Text("Lanline - optime");
src/GameRuntime.cpp:6878: ImGui::TextDisabled("No active Lanline session state found. Launch through
BunkerLauncher to seed roster and snapshot data.");
src/GameRuntime.cpp:6891: const auto& diagnostics = CachedLanlineDiagnostics(sessionState,
profile.selectedWorld);
src/GameRuntime.cpp:6898: ImGui::Text("Reserved Slots: %d",
bunker::ReservedLanlineSessionSlots(sessionState));
src/GameRuntime.cpp:6899: ImGui::Text("Pending Slots: %d", bunker::PendingLanlineSessionSlots(sessionState));
src/GameRuntime.cpp:6900: ImGui::Text("Accepted Client Slots: %d",
bunker::AcceptedLanlineSessionSlots(sessionState));
src/GameRuntime.cpp:6901: ImGui::Text("Ready Seats: %d", bunker::ReadyLanlineSessionSlots(sessionState));
src/GameRuntime.cpp:6921: bunker::LanlineSlotStateLabel(playerEntry),
src/GameRuntime.cpp:6922: bunker::LanlineReadyLabel(playerEntry));
src/GameRuntime.cpp:6927: ImGui::TextDisabled("No Lanline events recorded yet.");
src/GameRuntime.cpp:6953: ImGui::TextDisabled("No Lanline voice presence mirrored into this session yet.");
src/GameRuntime.cpp:6967: ImGui::TextDisabled("No runtime Lanline notifications yet.");
src/GameRuntime.cpp:6974: const auto knownSessions = DiscoverLanlineSessionSnapshots();
src/GameRuntime.cpp:6981: const auto& knownDiagnostics = CachedLanlineDiagnostics(knownSession,
profile.selectedWorld);
src/GameRuntime.cpp:6990: bunker::OccupiedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:6991: bunker::MaxLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:6998: if (bunker::IsJoinableLanlineSession(knownSession)) {
```



```
src/GameRuntime.cpp:7000: bunker::AvailableLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7001: bunker::PendingLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7002: bunker::ReservedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7003: bunker::AcceptedLanlineSessionSlots(knownSession),
src/GameRuntime.cpp:7004: bunker::ReadyLanlineSessionSlots(knownSession));

src/GameRuntime.cpp:7009: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and snapshot
trail even without Steam/Xbox auth.");

src/GameRuntime.cpp:7015: static LanlineServicesState lanlineServices =
MakeDefaultLanlineServicesState(std::time(nullptr));

src/GameRuntime.cpp:7017: LanlineServicesSave lanlineSave{};

src/GameRuntime.cpp:7018: if (LoadLanlineServicesSave(DefaultLanlineServicesSavePath(), lanlineSave)) {

src/GameRuntime.cpp:7019: lanlineServices = MakeLanlineServicesStateFromSave(lanlineSave,
std::time(nullptr));

src/GameRuntime.cpp:7021: ApplyLanlineServicesProfileSnapshot(lanlineServices, profile.lanlineServices);

src/GameRuntime.cpp:7025: LanlineSessionState sessionState;

src/GameRuntime.cpp:7026: const bool hasSessionState = LoadLanlineSessionState(sessionState);

src/GameRuntime.cpp:7028: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState :
nullptr, servicesUnlock);

src/GameRuntime.cpp:7034: ImGui::Text("Lanline Services");

src/GameRuntime.cpp:7045: DrawLanlineServicesPanel(lanlineServices, servicesUnlock,
static_cast<std::int64_t>(std::time(nullptr)));

src/GameRuntime.cpp:7046: SyncLanlineServicesProfileSnapshot(profile.lanlineServices, lanlineServices);

src/GameRuntime.cpp:7047: SaveLanlineServicesSave(BuildLanlineServicesSave(lanlineServices),
DefaultLanlineServicesSavePath());

src/GameRuntime.cpp:7058: if (!PlayerHasActivePipDeviceAccess(profile)) {

src/GameRuntime.cpp:7160: } else if (!ConsumeInventoryItem(profile, "steel_scrap", steelCost) ||
!ConsumeInventoryItem(profile, "copper_wire", wireCost)) {

src/GameRuntime.cpp:7247: profile.story.outerRoadCleared &&

src/GameRuntimePipPad.cpp:9:#include "../include/LanlineLobbyLogic.hpp"

src/GameRuntimePipPad.cpp:10:#include "../include/LanlineServices.hpp"

src/GameRuntimePipPad.cpp:17:const LanlineDiagnostics& CachedLanlineDiagnostics(const
LanlineSessionState& session, std::string_view runtimeWorldName) {
```

```
src/GameRuntimePipPad.cpp:21: static LanlineDiagnostics cachedDiagnostics;

src/GameRuntimePipPad.cpp:27: cachedDiagnostics = ProbeLanlineHost(session, normalizedWorldName);

src/GameRuntimePipPad.cpp:35: const std::vector<std::string>& UpdateLanlineRuntimeNotifications(const
LanlineSessionState* session,

src/GameRuntimePipPad.cpp:39: static LanlineSessionState previousSession;

src/GameRuntimePipPad.cpp:55: pushNotification("Lanline session link lost. Waiting for refreshed
launcher/session state.");

src/GameRuntimePipPad.cpp:67: pushNotification("Lanline session linked: " + session->sessionId + " on " +
normalizedSessionWorld + ".");

src/GameRuntimePipPad.cpp:69: pushNotification("Lanline world mismatch: runtime is on " +
normalizedRuntimeWorld +

src/GameRuntimePipPad.cpp:79: pushNotification("Lanline session switched to " + session->sessionId + " (" +
session->mode + ").");

src/GameRuntimePipPad.cpp:82: pushNotification("Lanline mode changed to " + session->mode + ".");

src/GameRuntimePipPad.cpp:85: pushNotification("Lanline lifecycle advanced to " + session->lifecycleStage + ".");

src/GameRuntimePipPad.cpp:89: ? "Lanline pending peer queue cleared."

src/GameRuntimePipPad.cpp:90: : "Lanline pending peer updated: " + session->pendingPeer + ".");

src/GameRuntimePipPad.cpp:94: ? "Lanline connected peer link cleared."

src/GameRuntimePipPad.cpp:95: : "Lanline peer link confirmed with " + session->connectedPeer + ".");

src/GameRuntimePipPad.cpp:98: pushNotification("Lanline session world updated: " + previousNormalizedWorld
+ " -> " + normalizedSessionWorld + ".");

src/GameRuntimePipPad.cpp:104: ? "Lanline session world now matches the active runtime world."

src/GameRuntimePipPad.cpp:105: : "Lanline session world drifted away from the active runtime world.");

src/GameRuntimePipPad.cpp:110: [&](const LanlinePlayerEntry& previousEntry) {

src/GameRuntimePipPad.cpp:114: pushNotification(playerEntry.displayName + " joined Lanline roster as " +
playerEntry.role + ".");

src/GameRuntimePipPad.cpp:118: pushNotification(playerEntry.displayName + " moved from " + previousIt->role
+ " to " + playerEntry.role + " in Lanline lobby.");

src/GameRuntimePipPad.cpp:122: (playerEntry.online ? " came online in Lanline." : " went offline in Lanline."));

src/GameRuntimePipPad.cpp:124: if (IsLanlineReadyEligibleSlot(playerEntry) && playerEntry.ready != previousIt-
>ready) {

src/GameRuntimePipPad.cpp:126: (playerEntry.ready ? " is now ready for Lanline deployment." : " is no longer
ready for Lanline deployment."));
```

```
src/GameRuntimePipPad.cpp:132: [&](const LanlinePlayerEntry& currentEntry) {  
src/GameRuntimePipPad.cpp:136: pushNotification(previousEntry.displayName + " left the Lanline roster.");  
src/GameRuntimePipPad.cpp:150: [&](const LanlineVoicePresence& previousVoice) {  
src/GameRuntimePipPad.cpp:170: [&](const LanlineVoicePresence& currentVoice) {  
src/GameRuntimePipPad.cpp:183: const char* JoinabilityLabel(const LanlineSessionState& session) {  
src/GameRuntimePipPad.cpp:303: void TrustLanlinePeerAsBt72Gunner(SessionProfile& profile, const  
LanlineSessionState* sessionState, GameState& gameState) {  
src/GameRuntimePipPad.cpp:309: gameState.lastEvent = "No Lanline session state found. Launch through  
BunkerLauncher to seed a trusted gunner.";  
src/GameRuntimePipPad.cpp:327: gameState.lastEvent = "No Lanline peer is currently available to trust as BT-72  
gunner.";  
src/GameRuntimePipPad.cpp:374: if (ConsumeInventoryItem(profile, itemId, 1)) {  
src/GameRuntimePipPad.cpp:522: ConsumeInventoryItem(profile, "steel_scrap", 1);  
src/GameRuntimePipPad.cpp:523: ConsumeInventoryItem(profile, "copper_wire", 1);  
src/GameRuntimePipPad.cpp:524: AddInventoryItem(profile, "repair_patch", 1, 0.2f);  
src/GameRuntimePipPad.cpp:545: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {  
src/GameRuntimePipPad.cpp:546: AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);  
src/GameRuntimePipPad.cpp:554: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {  
src/GameRuntimePipPad.cpp:555: AddInventoryItem(profile, "power_cell", 1, 0.3f);  
src/GameRuntimePipPad.cpp:562: if (ConsumeInventoryItem(profile, "trade_voucher", 1)) {  
src/GameRuntimePipPad.cpp:563: AddInventoryItem(profile, "##it_ptrs_amm0", 2, 0.7f);  
src/GameRuntimePipPad.cpp:574: LanlineSessionState sessionState;  
src/GameRuntimePipPad.cpp:575: const bool hasSessionState = LoadLanlineSessionState(sessionState);  
src/GameRuntimePipPad.cpp:625: if (ImGui::Button("Trust Lanline Peer")) {  
src/GameRuntimePipPad.cpp:626: TrustLanlinePeerAsBt72Gunner(profile, hasSessionState ? &sessionState :  
nullptr, gameState);  
src/GameRuntimePipPad.cpp:757: LanlineSessionState sessionState;  
src/GameRuntimePipPad.cpp:758: const bool hasSessionState = LoadLanlineSessionState(sessionState);  
src/GameRuntimePipPad.cpp:759: const auto& runtimeNotifications = UpdateLanlineRuntimeNotifications(  
src/GameRuntimePipPad.cpp:763: ImGui::Text("Lanline - optime");
```

```
src/GameRuntimePipPad.cpp:765: ImGui::TextDisabled("No active Lanline session state found. Launch through  
BunkerLauncher to seed roster and snapshot data.");  
  
src/GameRuntimePipPad.cpp:778: const auto& diagnostics = CachedLanlineDiagnostics(sessionState,  
profile.selectedWorld);  
  
src/GameRuntimePipPad.cpp:785: ImGui::Text("Reserved Slots: %d", ReservedLanlineSessionSlots(sessionState));  
src/GameRuntimePipPad.cpp:786: ImGui::Text("Pending Slots: %d", PendingLanlineSessionSlots(sessionState));  
src/GameRuntimePipPad.cpp:787: ImGui::Text("Accepted Client Slots: %d",  
AcceptedLanlineSessionSlots(sessionState));  
  
src/GameRuntimePipPad.cpp:788: ImGui::Text("Ready Seats: %d", ReadyLanlineSessionSlots(sessionState));  
src/GameRuntimePipPad.cpp:817: LanlineSlotStateLabel(playerEntry),  
src/GameRuntimePipPad.cpp:818: LanlineReadyLabel(playerEntry));  
  
src/GameRuntimePipPad.cpp:823: ImGui::TextDisabled("No Lanline events recorded yet.");  
  
src/GameRuntimePipPad.cpp:849: ImGui::TextDisabled("No Lanline voice presence mirrored into this session  
yet.");  
  
src/GameRuntimePipPad.cpp:863: ImGui::TextDisabled("No runtime Lanline notifications yet.");  
  
src/GameRuntimePipPad.cpp:870: const auto knownSessions = DiscoverLanlineSessionSnapshots();  
  
src/GameRuntimePipPad.cpp:877: const auto& knownDiagnostics = CachedLanlineDiagnostics(knownSession,  
profile.selectedWorld);  
  
src/GameRuntimePipPad.cpp:886: OccupiedLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:887: MaxLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:894: if (IsJoinableLanlineSession(knownSession)) {  
src/GameRuntimePipPad.cpp:896: AvailableLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:897: PendingLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:898: ReservedLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:899: AcceptedLanlineSessionSlots(knownSession),  
src/GameRuntimePipPad.cpp:900: ReadyLanlineSessionSlots(knownSession));  
  
src/GameRuntimePipPad.cpp:905: ImGui::TextWrapped("Lanline - optime keeps a visible session roster and  
snapshot trail even without Steam/Xbox auth.");  
  
src/GameRuntimePipPad.cpp:911: static LanlineServicesState lanlineServices =  
MakeDefaultLanlineServicesState(std::time(nullptr));  
  
src/GameRuntimePipPad.cpp:913: LanlineServicesSave lanlineSave{};  
  
src/GameRuntimePipPad.cpp:914: if (LoadLanlineServicesSave(DefaultLanlineServicesSavePath(), lanlineSave)) {
```

```
src/GameRuntimePipPad.cpp:915: lanlineServices = MakeLanlineServicesStateFromSave(lanlineSave,
std::time(nullptr));

src/GameRuntimePipPad.cpp:920: ApplyLanlineServicesProfileSnapshot(lanlineServices, profile.lanlineServices);

src/GameRuntimePipPad.cpp:921: LanlineSessionState sessionState;

src/GameRuntimePipPad.cpp:922: const bool hasSessionState = LoadLanlineSessionState(sessionState);

src/GameRuntimePipPad.cpp:926: SyncLanlineServicesPresence(lanlineServices, hasSessionState ? &sessionState
: nullptr, servicesUnlock);

src/GameRuntimePipPad.cpp:927: AdvanceLanlineSupportOrders(lanlineServices, nowUnix);

src/GameRuntimePipPad.cpp:933: ImGui::Text("Lanline Services");

src/GameRuntimePipPad.cpp:958: ImGui::TextDisabled("Delivered orders can be claimed after a Lanline service
hub handshake.");

src/GameRuntimePipPad.cpp:977: DrawLanlineServicesPanel(lanlineServices, servicesUnlock, nowUnix);

src/GameRuntimePipPad.cpp:978: SyncLanlineServicesSessionProfile(profile, lanlineServices);

src/GameRuntimePipPad.cpp:979: SaveLanlineServicesSave(BuildLanlineServicesSave(lanlineServices),
DefaultLanlineServicesSavePath());

src/GameplayDescriptorRegistry.cpp:31: {"lanline_service_hub", "Lanline Service Hub",
InteractionType::Terminal, ObjectCategory::Terminal, "shelter17_services", false},

src/LanlineServices.cpp:1:#include "../include/LanlineServices.hpp"

src/LanlineServices.cpp:98:std::string CurrentLanlineTimestamp() {

src/LanlineServices.cpp:176:const SupportCatalogItem* FindSupportCatalogItem(const LanlineServicesState&
state, std::string_view itemId) {

src/LanlineServices.cpp:248:ChatChannel* FindChatChannel(LanlineServicesState& state, const std::string&
channelId) {

src/LanlineServices.cpp:259: {"session", "Session", {"Lanline", "Session relay ready. Presence log restored.",
"boot"}}},

src/LanlineServices.cpp:265:FriendEntry* FindFriendEntry(LanlineServicesState& state, const std::string& handle)
{

src/LanlineServices.cpp:275: LanlineSessionState sessionState;

src/LanlineServices.cpp:276: if (!LoadLanlineSessionState(sessionState)) {

src/LanlineServices.cpp:278: *errorText = "No active Lanline session is available for relay transport.";

src/LanlineServices.cpp:283: sessionState.updatedAt = CurrentLanlineTimestamp();

src/LanlineServices.cpp:288: sessionState.relayMessages.begin() +
static_cast<std::vector<LanlineRelayMessage>::difference_type>(sessionState.relayMessages.size() - 32));
```

```
src/LanlineServices.cpp:291: if (!SaveLanlineSessionState(sessionState) ||  
src/LanlineServices.cpp:292: !SaveLanlineSessionState(sessionState,  
LanlineSessionSnapshotPath(sessionState.sessionId))) {  
src/LanlineServices.cpp:294: *errorText = "Relay transport failed to write Lanline session state.";  
src/LanlineServices.cpp:307: LanlineSessionState sessionState;  
src/LanlineServices.cpp:308: if (!LoadLanlineSessionState(sessionState)) {  
src/LanlineServices.cpp:310: *errorText = "No active Lanline session is available for voice relay.";  
src/LanlineServices.cpp:315: sessionState.updatedAt = CurrentLanlineTimestamp();  
src/LanlineServices.cpp:319: [&](const LanlineVoicePresence& entry) { return entry.handle == handle; });  
src/LanlineServices.cpp:333: sessionState.voicePresence.begin() +  
static_cast<std::vector<LanlineVoicePresence>::difference_type>(sessionState.voicePresence.size() - 16));  
src/LanlineServices.cpp:336: if (!SaveLanlineSessionState(sessionState) ||  
src/LanlineServices.cpp:337: !SaveLanlineSessionState(sessionState,  
LanlineSessionSnapshotPath(sessionState.sessionId))) {  
src/LanlineServices.cpp:339: *errorText = "Voice relay failed to write Lanline session state.";  
src/LanlineServices.cpp:347: void DrawLanlineServicesLockedScreen(const ServicesUnlockState& unlockState) {  
src/LanlineServices.cpp:348: ImGui::Text("Lanline Services: offline");  
src/LanlineServices.cpp:351: ImGui::TextWrapped("Relay access unavailable. Restore and synchronize the first  
tower to open Lanline Services.");  
src/LanlineServices.cpp:375: void DrawFriendsTab(LanlineServicesState& state, char* searchBuffer, std::size_t  
searchBufferSize) {  
src/LanlineServices.cpp:403: void DrawChatTab(LanlineServicesState& state, int& selectedChannelIndex, char*  
messageBuffer, std::size_t messageBufferSize) {  
src/LanlineServices.cpp:421: ImGui::BeginChild("LanlineChatHistory", ImVec2(0.0f, 180.0f), true);  
src/LanlineServices.cpp:443: ImGui::TextDisabled("Relay chat rides on the active Lanline session state and  
snapshot mirror.");  
src/LanlineServices.cpp:447: void DrawVoiceTab(LanlineServicesState& state) {  
src/LanlineServices.cpp:470: ImGui::TextDisabled("Voice activity now relays through the active Lanline session  
state; raw audio transport remains future work.");  
src/LanlineServices.cpp:490: void DrawOperationalSupportCategory(LanlineServicesState& state,  
src/LanlineServices.cpp:556: void DrawCosmeticSupportCategory(LanlineServicesState& state, SupportCategory  
category, const char* title) {
```

```
src/LanlineServices.cpp:602:void DrawSupportOrdersSummary(const LanlineServicesState& state, std::int64_t
nowUnix) {

src/LanlineServices.cpp:624:void DrawSupportOverview(LanlineServicesState& state, std::int64_t nowUnix) {

src/LanlineServices.cpp:632:void DrawFeyGateTab(LanlineServicesState& state, const ServicesUnlockState&
unlockState, std::int64_t nowUnix) {

src/LanlineServices.cpp:786:LanlineServicesState MakeDefaultLanlineServicesState(std::int64_t nowUnix) {

src/LanlineServices.cpp:787: LanlineServicesState state;

src/LanlineServices.cpp:791: {"fortress.dispatch", "Rail Fortress Net", "Patrol routing and support manifests
mirrored through Lanline.", true, false, false, 1.0f},

src/LanlineServices.cpp:799:LanlineServicesState MakeLanlineServicesStateFromSave(const LanlineServicesSave&
save, std::int64_t nowUnix) {

src/LanlineServices.cpp:800: LanlineServicesState state = MakeDefaultLanlineServicesState(nowUnix);

src/LanlineServices.cpp:809:LanlineServicesSave BuildLanlineServicesSave(const LanlineServicesState& state) {

src/LanlineServices.cpp:810: LanlineServicesSave save;

src/LanlineServices.cpp:819:void ApplyLanlineServicesProfileSnapshot(LanlineServicesState& state, const
LanlineServicesProfile& profile) {

src/LanlineServices.cpp:827:void SyncLanlineServicesProfileSnapshot(LanlineServicesProfile& profile, const
LanlineServicesState& state) {

src/LanlineServices.cpp:835:void SyncLanlineServicesSessionProfile(SessionProfile& profile, const
LanlineServicesState& state) {

src/LanlineServices.cpp:837: SyncLanlineServicesProfileSnapshot(profile.lanlineServices, state);

src/LanlineServices.cpp:855:void AdvanceLanlineSupportOrders(LanlineServicesState& state, std::int64_t
nowUnix) {

src/LanlineServices.cpp:867:int CountSupportOrdersInState(const LanlineServicesState& state,
SupportOrderState orderState) {

src/LanlineServices.cpp:874:int ClaimDeliveredSupportOrders(LanlineServicesState& state, SessionProfile&
profile, std::string* summary) {

src/LanlineServices.cpp:911: *summary = "Lanline delivery received: " + claimedOrderSummaries.front() + ".";

src/LanlineServices.cpp:913: *summary = "Lanline deliveries received: " + std::to_string(claimedCount) +

src/LanlineServices.cpp:921:ServiceHubMode ResolveLanlineServicesMode(const ServicesUnlockState&
unlockState, const LanlineSessionState* sessionState) {

src/LanlineServices.cpp:922: if (!IsLanlineServicesUnlocked(unlockState)) {

src/LanlineServices.cpp:926: return ServiceHubMode::LanlineLocal;
```

```
src/LanlineServices.cpp:928: return unlockState.backboneStable ? ServiceHubMode::RelayOnline :
ServiceHubMode::LanlineLocal;

src/LanlineServices.cpp:931:void SyncLanlineServicesPresence(LanlineServicesState& state,

src/LanlineServices.cpp:932: const LanlineSessionState* sessionState,

src/LanlineServices.cpp:934: state.mode = ResolveLanlineServicesMode(unlockState, sessionState);

src/LanlineServices.cpp:942: AddChatMessage(*sessionChannel, "Lanline", "No active session mirror is currently
available.", "now");

src/LanlineServices.cpp:953: AddChatMessage(*sessionChannel, "Lanline", presenceBuffer, sessionState-
>updatedAt);

src/LanlineServices.cpp:972: state.friends.push_back({handle, "Lanline Session", player.role, player.online, true,
false, 1.0f});

src/LanlineServices.cpp:975: friendEntry->nodeLabel = "Lanline Session";

src/LanlineServices.cpp:989: "Lanline Voice",

src/LanlineServices.cpp:997: friendEntry->nodeLabel = "Lanline Voice";

src/LanlineServices.cpp:1019:std::filesystem::path DefaultLanlineServicesSavePath() {

src/LanlineServices.cpp:1023:bool SaveLanlineServicesSave(const LanlineServicesSave& save, const
std::filesystem::path& path) {

src/LanlineServices.cpp:1058:bool LoadLanlineServicesSave(const std::filesystem::path& path,
LanlineServicesSave& outSave) {

src/LanlineServices.cpp:1116:void DrawLanlineServicesPanel(LanlineServicesState& state,

src/LanlineServices.cpp:1119: AdvanceLanlineSupportOrders(state, nowUnix);

src/LanlineServices.cpp:1121: if (!IsLanlineServicesUnlocked(unlockState)) {

src/LanlineServices.cpp:1122: DrawLanlineServicesLockedScreen(unlockState);

src/LanlineServices.cpp:1130: ImGui::Text("Lanline Services Online");

src/LanlineServices.cpp:1161: if (ImGui::BeginTabBar("LanlineServicesTabs")) {

src/LanlineSession.cpp:1:#include "../include/LanlineSession.hpp"

src/LanlineSession.cpp:77:void TrimRelayMessages(LanlineSessionState& state, std::size_t maxMessages) {

src/LanlineSession.cpp:81: state.relayMessages.begin() +
static_cast<std::vector<LanlineRelayMessage>::difference_type>(state.relayMessages.size() - maxMessages));

src/LanlineSession.cpp:85:void TrimVoicePresence(LanlineSessionState& state, std::size_t maxEntries) {

src/LanlineSession.cpp:89: state.voicePresence.begin() +
static_cast<std::vector<LanlineVoicePresence>::difference_type>(state.voicePresence.size() - maxEntries));
```



```
src/LanlineSession.cpp:110:bool SaveLanlineSessionState(const LanlineSessionState& state) {  
src/LanlineSession.cpp:111: return SaveLanlineSessionState(state, LanlineSessionPath());  
src/LanlineSession.cpp:114:bool SaveLanlineSessionState(const LanlineSessionState& state, const  
std::filesystem::path& path) {  
src/LanlineSession.cpp:164:bool LoadLanlineSessionState(LanlineSessionState& state) {  
src/LanlineSession.cpp:165: return LoadLanlineSessionState(LanlineSessionPath(), state);  
src/LanlineSession.cpp:168:bool LoadLanlineSessionState(const std::filesystem::path& path,  
LanlineSessionState& state) {  
src/LanlineSession.cpp:174: state = LanlineSessionState{};  
src/LanlineSession.cpp:215: LanlinePlayerEntry player{};  
src/LanlineSession.cpp:268:std::vector<LanlineSessionState> DiscoverLanlineSessionSnapshots() {  
src/LanlineSession.cpp:269: std::vector<LanlineSessionState> sessions;  
src/LanlineSession.cpp:270: const auto sessionsDir = LanlineSessionsDirectory();  
src/LanlineSession.cpp:280: LanlineSessionState state;  
src/LanlineSession.cpp:281: if (LoadLanlineSessionState(entry.path(), state)) {  
src/LanlineSession.cpp:286: std::sort(sessions.begin(), sessions.end(), [](const LanlineSessionState& left, const  
LanlineSessionState& right) {  
src/LanlineSession.cpp:296:LanlineDiagnostics ProbeLanlineHost(const LanlineSessionState& session,  
std::string_view runtimeWorldName) {  
src/LanlineSession.cpp:297: LanlineDiagnostics out;  
src/LanlineSession.cpp:329: out.lastError = "Unable to resolve Lanline host.";  
src/SessionProfiles.cpp:70: route.surfaceArrivalReached = route.surfaceArrivalReached ||  
src/SessionProfiles.cpp:71: profile.story.outerRoadCleared ||  
src/SessionProfiles.cpp:73: profile.story.returnedToBase;  
src/SessionProfiles.cpp:74: route.firstTankCombatResolved = route.firstTankCombatResolved ||  
profile.story.relayRecovered || profile.story.returnedToBase;  
src/SessionProfiles.cpp:77: profile.story.returnedToBase ||  
src/SessionProfiles.cpp:81: profile.story.returnedToBase ||  
src/SessionProfiles.cpp:94: profile.story.returnedToBase = profile.story.returnedToBase ||  
route.debriefSummaryViewed;  
src/SessionProfiles.cpp:117: route.surfaceArrivalReached = profile.story.outerRoadCleared ||  
profile.story.relayRecovered || profile.story.returnedToBase;
```

```
src/SessionProfiles.cpp:118: route.firstTankCombatResolved = profile.story.relayRecovered ||
profile.story.returnedToBase;

src/SessionProfiles.cpp:120: profile.story.returnedToBase ||

src/SessionProfiles.cpp:123: route.debriefSummaryViewed = profile.story.returnedToBase;

src/SessionProfiles.cpp:136: profile.firstPlayableRoute.surfaceArrivalReached =

src/SessionProfiles.cpp:137: profile.story.outerRoadCleared ||

src/SessionProfiles.cpp:139: profile.story.returnedToBase;

src/SessionProfiles.cpp:432: out << "story_exit=" << (profile.story.exitedBunker ? 1 : 0) << '\n';

src/SessionProfiles.cpp:433: out << "story_debris=" << (profile.story.outerRoadCleared ? 1 : 0) << '\n';

src/SessionProfiles.cpp:435: out << "story_return=" << (profile.story.returnedToBase ? 1 : 0) << '\n';

src/SessionProfiles.cpp:448: out << "route_surface_arrival=" << (profile.firstPlayableRoute.surfaceArrivalReached
? 1 : 0) << '\n';

src/SessionProfiles.cpp:666: else if (key == "story_exit") outProfile.story.exitedBunker = (std::stoi(value) != 0);

src/SessionProfiles.cpp:667: else if (key == "story_debris") outProfile.story.outerRoadCleared = (std::stoi(value) !=
0);

src/SessionProfiles.cpp:669: else if (key == "story_return") outProfile.story.returnedToBase = (std::stoi(value) !=
0);

src/SessionProfiles.cpp:682: else if (key == "route_surface_arrival")
outProfile.firstPlayableRoute.surfaceArrivalReached = (std::stoi(value) != 0);

src/SessionProfiles.cpp:802: const bool hasLanlineFields = parts.size() >= 37;

src/SessionProfiles.cpp:803: const std::size_t baseOffset = hasLanlineFields ? 6 : 3;

src/SessionProfiles.cpp:804: state.towerSyncRecovered = hasLanlineFields ? (std::stoi(parts[3]) != 0) : false;

src/SessionProfiles.cpp:805: state.localRelayAvailable = hasLanlineFields ? (std::stoi(parts[4]) != 0) : false;

src/SessionProfiles.cpp:806: state.regionalGridOnline = hasLanlineFields ? (std::stoi(parts[5]) != 0) : false;

src/StoryRoute.cpp:65: profile.story.exitedBunker;

src/StoryRoute.cpp:71: profile.story.exitedBunker;

src/StoryRoute.cpp:77: profile.story.exitedBunker;

src/StoryRoute.cpp:81: return profile.firstPlayableRoute.surfaceArrivalReached ||

src/StoryRoute.cpp:82: profile.story.outerRoadCleared ||

src/StoryRoute.cpp:84: profile.story.returnedToBase;

src/StoryRoute.cpp:90: profile.story.returnedToBase;
```

```
src/StoryRoute.cpp:100: profile.story.returnedToBase ||
src/StoryRoute.cpp:110: profile.story.returnedToBase ||
src/StoryRoute.cpp:428: bool HasLanlineServicesObjective(const SessionProfile& profile) {
src/StoryRoute.cpp:466: if (!profile.story.exitedBunker) {
src/StoryRoute.cpp:472: if (!profile.story.outerRoadCleared) {
src/StoryRoute.cpp:535: if (!profile.story.exitedBunker) {
src/StoryRoute.cpp:541: if (!profile.story.outerRoadCleared) {
src/StoryRoute.cpp:617: if (!profile.story.exitedBunker) {
src/StoryRoute.cpp:623: if (!profile.story.outerRoadCleared) {
src/StoryRoute.cpp:722: if (!(DebriefSummaryViewed(profile) || profile.story.returnedToBase ||
HasCollectedTapeId(profile, "debrief_shelter17"))) {
src/StoryRoute.cpp:753: if (!profile.story.exitedBunker) {
src/StoryRoute.cpp:759: if (!profile.story.outerRoadCleared) {
src/StoryRoute.cpp:839: if (!profile.story.exitedBunker) {
src/StoryRoute.cpp:853: if (!profile.story.outerRoadCleared) {
src/StoryRoute.cpp:963: {"Open the outer bulkhead.", profile.story.exitedBunker},
src/StoryRoute.cpp:965: {"Clear the outer debris barrier.", profile.story.outerRoadCleared},
src/StoryRoute.cpp:1005: {"Open the outer bulkhead.", profile.story.exitedBunker},
src/StoryRoute.cpp:1006: {"Reach the first surface arrival foothold.",
profile.firstPlayableRoute.surfaceArrivalReached},
src/StoryRoute.cpp:1007: {"Clear the outer debris barrier.", profile.story.outerRoadCleared ||
staticEraser.IsErased("#%res_scrap_0001")},
src/World.cpp:1311: "Shelter 17 Lanline Service Hub",
src/World.cpp:1874: "Shelter 17 Lanline Service Hub",
src/WorldEvents.cpp:79: if (profile.story.exitedBunker &&
src/WorldEvents.cpp:80: !profile.firstPlayableRoute.surfaceArrivalReached &&
src/WorldEvents.cpp:82: profile.firstPlayableRoute.surfaceArrivalReached = true;
src/WorldEvents.cpp:90: if (profile.story.outerRoadCleared &&
src/WorldEvents.cpp:102: if (profile.story.relayRecovered && !profile.story.returnedToBase &&
!gameState.zoneEventReturn &&
src/WorldEvents.cpp:110: if (profile.story.returnedToBase && !gameState.zoneEventReturn &&
```

```
src/main.cpp:15:#include "../include/LanlineLobbyLogic.hpp"

src/main.cpp:16:#include "../include/LanlineSession.hpp"

src/main.cpp:72:void TrimLanlineEventLog(bunker::LanlineSessionState& state, std::size_t maxEntries) {

src/main.cpp:78:void UpsertLanlinePlayer(bunker::LanlineSessionState& state,

src/main.cpp:85: [&](const bunker::LanlinePlayerEntry& entry) {

src/main.cpp:98:void AcceptLanlineLobbySlot(bunker::LanlineSessionState& state, const std::string& peerName) {

src/main.cpp:118: UpsertLanlinePlayer(state, peerName, "Client", true, false);

src/main.cpp:128:void SyncLanlineRuntimeLaunchState(const bunker::LaunchTicketInfo& launchTicket, const
bunker::SessionProfile& sessionProfile) {

src/main.cpp:129: bunker::LanlineSessionState state;

src/main.cpp:130: if (!bunker::LoadLanlineSessionState(state)) {

src/main.cpp:131: state = bunker::LanlineSessionState{};

src/main.cpp:174: AcceptLanlineLobbySlot(state, state.connectedPeer);

src/main.cpp:175: state.eventLog.push_back("Host accepted Lanline peer " + state.connectedPeer + ".");

src/main.cpp:187: if (player.displayName == actorName && bunker::IsLanlineReservedSlot(player)) {

src/main.cpp:211: UpsertLanlinePlayer(state, actorName, actorRole, true, currentReady, seatAssignment);

src/main.cpp:214: state.eventLog.push_back("Lanline lifecycle advanced to " + state.lifecycleStage + ".");

src/main.cpp:218: TrimLanlineEventLog(state, 12);

src/main.cpp:219: bunker::SaveLanlineSessionState(state);

src/main.cpp:220: bunker::SaveLanlineSessionState(state, bunker::LanlineSessionSnapshotPath(state.sessionId));

src/main.cpp:223:void SyncLanlineRuntimePresence(const bunker::PlayerState& player, const
bunker::SessionProfile& sessionProfile) {

src/main.cpp:246: bunker::LanlineSessionState state;

src/main.cpp:247: if (!bunker::LoadLanlineSessionState(state)) {

src/main.cpp:269: UpsertLanlinePlayer(state, actorName, actorRole, true, currentReady, seatAssignment);

src/main.cpp:275: TrimLanlineEventLog(state, 12);

src/main.cpp:276: bunker::SaveLanlineSessionState(state);

src/main.cpp:277: bunker::SaveLanlineSessionState(state, bunker::LanlineSessionSnapshotPath(state.sessionId));

src/main.cpp:365: SyncLanlineRuntimeLaunchState(launchTicket, sessionProfile);

src/main.cpp:415: player.uiVisible = bunker::PlayerHasActivePipDeviceAccess(sessionProfile) && player.uiVisible;
```

```
src/main.cpp:453: SyncLanlineRuntimePresence(player, sessionProfile);

src/main.cpp:637: bunker::TryToggleActivePipDeviceUi(player, sessionProfile, gameState);

src/main.cpp:643: bunker::ApplyActivePipDeviceCustomizationCommand(sessionProfile, "theme_next",
&gameState);

src/main.cpp:649: bunker::ApplyActivePipDeviceCustomizationCommand(sessionProfile, "display_next",
&gameState);

src/main.cpp:655: bunker::ApplyActivePipDeviceCustomizationCommand(sessionProfile, "carry_next",
&gameState);

src/main.cpp:674: } else if (bunker::ConsumeInventoryItem(sessionProfile, "cryo_medkit", 1)) {

src/main.cpp:699: const bool outdoors = world.IsStarterScenarioWorld() ? (player.x >= 13.0f ||
sessionProfile.story.exitedBunker) : true;

tests/SmokeChecks.cpp:8:#include "../include/LanlineServices.hpp"

tests/SmokeChecks.cpp:334: savedProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:382: Check(loaderProfile.firstPlayableRoute.surfaceArrivalReached, "profile first route
surface arrival flag mismatch") &&

tests/SmokeChecks.cpp:542: bunker::AddInventoryItem(currentProfile, "#%it_pippad", 1, 0.8f);

tests/SmokeChecks.cpp:743: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:757: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:758: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:772: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:809: bunker::AddInventoryItem(profile, "power_cell", 1, 0.3f);

tests/SmokeChecks.cpp:810: bunker::AddInventoryItem(profile, "repair_patch", 1, 0.2f);

tests/SmokeChecks.cpp:811: bunker::AddInventoryItem(profile, "old_plate", 1, 0.5f);

tests/SmokeChecks.cpp:846: bunker::AddInventoryItem(materialMissing, "power_cell", 1, 0.3f);

tests/SmokeChecks.cpp:847: bunker::AddInventoryItem(materialMissing, "old_plate", 1, 0.5f);

tests/SmokeChecks.cpp:930: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:941: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:947: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:972: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1014: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1033: profile.firstPlayableRoute.surfaceArrivalReached = true;
```

tests/SmokeChecks.cpp:1034: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1106: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1115: return Check(profile.firstPlayableRoute.surfaceArrivalReached, "surface arrival smoke expected route flag to flip at camp anchor") &&

tests/SmokeChecks.cpp:1140: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1141: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1142: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1187: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1188: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1189: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1228: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1229: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1230: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1412: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1413: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1414: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1429: return Check(profile.story.returnedToBase, "debrief handoff smoke expected returned-to-base flag") &&

tests/SmokeChecks.cpp:1452: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1453: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1454: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1459: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1515: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1516: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1517: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1529: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1640: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1641: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1642: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1647: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1719: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1720: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1721: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1726: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1787: acidProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1788: acidProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1789: acidProfile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1794: acidProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1837: fogProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1838: fogProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1839: fogProfile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1844: fogProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:1889: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:1890: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:1891: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:1896: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:2719: bool RunLanlineServicesRoundtripSmoke() {

tests/SmokeChecks.cpp:2732: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:2733: profile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:2734: profile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:2739: profile.story.returnedToBase = true;

tests/SmokeChecks.cpp:2818: bunker::LanlineServicesState state =
bunker::MakeDefaultLanlineServicesState(1000);

tests/SmokeChecks.cpp:2833: bunker::LanlineServicesProfile profileSnapshot{};

tests/SmokeChecks.cpp:2834: bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, state);

tests/SmokeChecks.cpp:2847: const bunker::LanlineServicesSave save = bunker::BuildLanlineServicesSave(state);

tests/SmokeChecks.cpp:2848: if (!Check(bunker::SaveLanlineServicesSave(save,
bunker::DefaultLanlineServicesSavePath()),

tests/SmokeChecks.cpp:2853: bunker::LanlineServicesSave loadedSave{};

tests/SmokeChecks.cpp:2854: if
(!Check(bunker::LoadLanlineServicesSave(bunker::DefaultLanlineServicesSavePath(), loadedSave),

```
tests/SmokeChecks.cpp:2859: bunker::LanlineServicesState loadedState =
bunker::MakeLanlineServicesStateFromSave(loadedSave, 1200);

tests/SmokeChecks.cpp:2860: bunker::ApplyLanlineServicesProfileSnapshot(loadedState, profileSnapshot);

tests/SmokeChecks.cpp:2861: bunker::AdvanceLanlineSupportOrders(loadedState, 1500);

tests/SmokeChecks.cpp:2886: bunker::SyncLanlineServicesSessionProfile(profile, loadedState);

tests/SmokeChecks.cpp:2887: bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, loadedState);

tests/SmokeChecks.cpp:3026: bool RunLanlineSeatRoundtripSmoke() {

tests/SmokeChecks.cpp:3027: bunker::LanlineSessionState savedState;

tests/SmokeChecks.cpp:3041: if (!Check(bunker::SaveLanlineSessionState(savedState), "lanline seat smoke failed
to save session state")) {

tests/SmokeChecks.cpp:3045: bunker::LanlineSessionState loadedState;

tests/SmokeChecks.cpp:3046: if (!Check(bunker::LoadLanlineSessionState(loadedState), "lanline seat smoke
failed to load session state")) {

tests/SmokeChecks.cpp:4849: WriteRawString(file, "Lanline Services Relay");

tests/SmokeChecks.cpp:5615: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:5715: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:6119: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:6129: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:6795: bunker::HasActivePipDevice(profile) &&

tests/SmokeChecks.cpp:6804: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6816: bunker::HasActivePipDevice(profile) &&

tests/SmokeChecks.cpp:6824: surfaceProfile.firstPlayableRoute.surfaceArrivalReached = true;

tests/SmokeChecks.cpp:6826: lateRouteProfile.story.outerRoadCleared = true;

tests/SmokeChecks.cpp:6830: returnedProfile.story.returnedToBase = true;

tests/SmokeChecks.cpp:6861: bunker::HasActivePipDevice(loadedProfile) &&

tests/SmokeChecks.cpp:6879: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6882: bunker::HasActivePipDevice(pippadProfile),

tests/SmokeChecks.cpp:6912: pipBoy10Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6925: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6937: noDeviceProfile.story.exitedBunker = true;
```


tests/SmokeChecks.cpp:6965: const bool activeOpen = bunker::TryToggleActivePipDeviceUi(activePlayer, profile, activeGameState);

tests/SmokeChecks.cpp:6973: const bool activeClose = bunker::TryToggleActivePipDeviceUi(activePlayer, profile, activeGameState);

tests/SmokeChecks.cpp:6982: pipPadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6990: pipBoy10Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:6998: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7005: noDeviceProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7012: const bool activeResult = bunker::TryToggleActivePipDeviceUi(activePlayer, noDeviceProfile, activeGameState);

tests/SmokeChecks.cpp:7025: if (!Check(!bunker::PlayerHasActivePipDeviceAccess(freshProfile) &&

tests/SmokeChecks.cpp:7033: if (!Check(bunker::PlayerHasActivePipDeviceAccess(pipPadProfile) &&

tests/SmokeChecks.cpp:7041: if (!Check(bunker::PlayerHasActivePipDeviceAccess(pipBoy10Profile) &&

tests/SmokeChecks.cpp:7049: if (!Check(bunker::PlayerHasActivePipDeviceAccess(pipBoy3000Profile) &&

tests/SmokeChecks.cpp:7055: pipBoy3000Profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7057: bunker::PlayerHasActivePipDeviceAccess(pipBoy3000Profile) &&

tests/SmokeChecks.cpp:7067: !bunker::PlayerHasActivePipDeviceAccess(invalidProfile) &&

tests/SmokeChecks.cpp:7146: loadedProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7160: if (!Check(bunker::TryToggleActivePipDeviceUi(player, loadedProfile, gameState) &&

tests/SmokeChecks.cpp:7199: if (!Check(bunker::ApplyActivePipDeviceCustomizationCommand(profile, "display_next", &gameState) &&

tests/SmokeChecks.cpp:7202: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "display_next") &&

tests/SmokeChecks.cpp:7204: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "display_next") &&

tests/SmokeChecks.cpp:7210: if (!Check(bunker::ApplyActivePipDeviceCustomizationCommand(profile, "carry_next", &gameState) &&

tests/SmokeChecks.cpp:7213: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "carry_next") &&

tests/SmokeChecks.cpp:7215: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "carry_next") &&

tests/SmokeChecks.cpp:7224: if (!Check(!bunker::ApplyActivePipDeviceCustomizationCommand(profile, "rgb_party", &gameState) &&

tests/SmokeChecks.cpp:7238: profile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7248: if (!Check(bunker::ApplyActivePipDeviceCustomizationCommand(profile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7249: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "display_next", &gameState) &&

tests/SmokeChecks.cpp:7250: bunker::ApplyActivePipDeviceCustomizationCommand(profile, "carry_next", &gameState) &&

tests/SmokeChecks.cpp:7257: if (!Check(bunker::TryToggleActivePipDeviceUi(player, profile, gameState) &&

tests/SmokeChecks.cpp:7357: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "rgb_party", &gameState) &&

tests/SmokeChecks.cpp:7406: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7408: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "display_next", &gameState) &&

tests/SmokeChecks.cpp:7410: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "carry_next", &gameState) &&

tests/SmokeChecks.cpp:7422: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7424: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile, "display_next", &gameState) &&

tests/SmokeChecks.cpp:7426: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile, "carry_next", &gameState) &&

tests/SmokeChecks.cpp:7433: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7436: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7453: !bunker::ApplyActivePipDeviceCustomizationCommand(invalidActiveProfile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7467: if (!Check(!bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "theme_next", &gameState) &&

tests/SmokeChecks.cpp:7468: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "display_next", &gameState) &&

tests/SmokeChecks.cpp:7469: !bunker::ApplyActivePipDeviceCustomizationCommand(noDeviceProfile, "carry_next", &gameState) &&

tests/SmokeChecks.cpp:7480: if (!Check(bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile, "theme_next", &gameState) &&

```
tests/SmokeChecks.cpp:7482: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile,
"display_next", &gameState) &&

tests/SmokeChecks.cpp:7484: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile,
"carry_next", &gameState) &&

tests/SmokeChecks.cpp:7490: if (!Check(bunker::TryToggleActivePipDeviceUi(player, pippadProfile, gameState)
&&

tests/SmokeChecks.cpp:7496: pippadProfile.story.exitedBunker = true;

tests/SmokeChecks.cpp:7498: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile,
"theme_next", &gameState) &&

tests/SmokeChecks.cpp:7500: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile,
"display_next", &gameState) &&

tests/SmokeChecks.cpp:7502: bunker::ApplyActivePipDeviceCustomizationCommand(pippadProfile,
"carry_next", &gameState) &&

tests/SmokeChecks.cpp:7537: if (!Check(bunker::ApplyActivePipDeviceCustomizationCommand(pipBoy10Profile,
"display_next", &gameState) &&

tests/SmokeChecks.cpp:7539: bunker::ApplyActivePipDeviceCustomizationCommand(pipBoy10Profile,
"carry_next", &gameState) &&

tests/SmokeChecks.cpp:7541: bunker::ApplyActivePipDeviceCustomizationCommand(pipBoy10Profile,
"carry_next", &gameState) &&

tests/SmokeChecks.cpp:7555: if (!Check(!bunker::HasActivePipDevice(profile) &&

tests/SmokeChecks.cpp:7564: bunker::HasActivePipDevice(profile) &&

tests/SmokeChecks.cpp:7625: bunker::HasActivePipDevice(loadedProfile) &&

tests/SmokeChecks.cpp:8256: !profile.story.returnedToBase,

tests/SmokeChecks.cpp:8491: RunLanlineServicesRoundtripSmoke() &&

tests/SmokeChecks.cpp:8493: RunLanlineSeatRoundtripSmoke() &&

Success

Ran command

$ git grep -n
"Run.*Field\|Run.*Scav\|Run.*Lanline\|Run.*Recovery\|Run.*Fatigue\|Run.*Medicine\|Run.*Route\|RunExisti
ng" -- tests/SmokeChecks.cpp

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

tests/SmokeChecks.cpp:685:bool RunFirstPlayableRouteStorySmoke() {

tests/SmokeChecks.cpp:910:bool RunRouteBeatPresentationSmoke() {
```

tests/SmokeChecks.cpp:982:bool RunFirstPlayableRouteReadoutSmoke() {
tests/SmokeChecks.cpp:1210:bool RunWorkshopServiceRouteHandoffSmoke() {
tests/SmokeChecks.cpp:1441:bool RunRecoveryHandoffSummarySmoke() {
tests/SmokeChecks.cpp:1497:bool RunRecoveryBackboneStatusSmoke() {
tests/SmokeChecks.cpp:1612:bool RunRouteEventLifecycleSmoke() {
tests/SmokeChecks.cpp:1698:bool RunMerchantRouteEventSmoke() {
tests/SmokeChecks.cpp:1776:bool RunWeatherWeightedRouteEventSmoke() {
tests/SmokeChecks.cpp:1878:bool RunWorldScopedRouteSummarySmoke() {
tests/SmokeChecks.cpp:2363:bool RunFieldReflexRpgWeightSmoke() {
tests/SmokeChecks.cpp:2719:bool RunLanlineServicesRoundtripSmoke() {
tests/SmokeChecks.cpp:3026:bool RunLanlineSeatRoundtripSmoke() {
tests/SmokeChecks.cpp:6047:bool RunStarterWorldFirstRouteObjectAvailabilitySmoke() {
tests/SmokeChecks.cpp:6087:bool RunStarterWorldFirstPlayableRouteSmoke() {
tests/SmokeChecks.cpp:6183:bool RunStarterWorldRouteObjectsPersistAfterSaveLoadSmoke() {
tests/SmokeChecks.cpp:6347:bool RunLoadedStarterWorldContinuesRouteAfterPipPadPickupSmoke() {
tests/SmokeChecks.cpp:7971:bool RunLegacyStarterWorldGetsV940RouteInfrastructureSmoke() {
tests/SmokeChecks.cpp:8082:bool RunLoadedStarterWorldFirstPlayableRouteSmoke() {
tests/SmokeChecks.cpp:8176:bool RunFirstPlayableRouteEndToEndSmoke() {
tests/SmokeChecks.cpp:8461: RunFirstPlayableRouteStorySmoke() &&
tests/SmokeChecks.cpp:8462: RunRouteBeatPresentationSmoke() &&
tests/SmokeChecks.cpp:8463: RunFirstPlayableRouteReadoutSmoke() &&
tests/SmokeChecks.cpp:8468: RunWorkshopServiceRouteHandoffSmoke() &&
tests/SmokeChecks.cpp:8472: RunRecoveryHandoffSummarySmoke() &&
tests/SmokeChecks.cpp:8473: RunRecoveryBackboneStatusSmoke() &&
tests/SmokeChecks.cpp:8474: RunRouteEventLifecycleSmoke() &&
tests/SmokeChecks.cpp:8475: RunMerchantRouteEventSmoke() &&
tests/SmokeChecks.cpp:8476: RunWeatherWeightedRouteEventSmoke() &&
tests/SmokeChecks.cpp:8477: RunWorldScopedRouteSummarySmoke() &&
tests/SmokeChecks.cpp:8485: RunFieldReflexRpgWeightSmoke() &&
tests/SmokeChecks.cpp:8491: RunLanlineServicesRoundtripSmoke() &&

```
tests/SmokeChecks.cpp:8493: RunLanlineSeatRoundtripSmoke() &&  
tests/SmokeChecks.cpp:8532: RunStarterWorldFirstRouteObjectAvailabilitySmoke() &&  
tests/SmokeChecks.cpp:8533: RunStarterWorldFirstPlayableRouteSmoke() &&  
tests/SmokeChecks.cpp:8537: RunLoadedStarterWorldContinuesRouteAfterPipPadPickupSmoke() &&  
tests/SmokeChecks.cpp:8555: RunStarterWorldRouteObjectsPersistAfterSaveLoadSmoke() &&  
tests/SmokeChecks.cpp:8556: RunLegacyStarterWorldGetsV940RouteInfrastructureSmoke() &&  
tests/SmokeChecks.cpp:8559: RunLoadedStarterWorldFirstPlayableRouteSmoke() &&  
tests/SmokeChecks.cpp:8560: RunFirstPlayableRouteEndToEndSmoke() &&
```

Success

Нашёл, что базовые куски уже есть: surface/route flags, scavenger/caravan/drone/trade/rail timers, Lanline services, field service, inventory rewards и active Pip device helpers. Сейчас читаю конкретные функции вокруг готовых циклов, чтобы smoke опирался на существующее поведение, а не на новый слой.

Ran 26 commands

Ran command

```
$ $lines=Get-Content src\GameRuntime.cpp; $lines[4030..4305]
```

[output truncated]

```
8f,  
0.0f,  
1.5f,  
1.2f,  
1.7f,  
50.0f,  
true,  
true,  
false,  
{},  
"ghoul_rush"  
});  
}
```

```
gameState.lastEvent = "Outer route contamination surged. Ether barricades and fresh nests are reforming beyond the bulkhead.";
```

```

} else if (worldState.routeOverrun && worldState.routeContamination <= 18.0f) {
worldState.routeOverrun = false;

gameState.lastEvent = "Outer route stabilized again. Contamination pressure has receded.";
}
}

void UpdateScavengerTeams(SessionProfile& profile, GameState& gameState, float dt) {
const bool scavengerReady =
HasActiveFieldCheckpoint(profile) &&
profile.story.outerRoadCleared &&
IsRegionalGridOnline(profile) &&
HasAwakenedSpecialistRole(profile, "engineer");
if (!scavengerReady) {
gameState.scavengerTimer = 180.0f;
return;
}

const bool engineerAssignedToScavengers = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");
gameState.scavengerTimer -= dt;
if (gameState.scavengerTimer > 0.0f) {
return;
}

const auto* worldState = FindWorldFieldState(profile, profile.selectedWorld);
const float infrastructurePenalty = worldState != nullptr
? std::max(0.65f, 1.0f - worldState->infrastructureDecay / 180.0f)
: 1.0f;

const float doctrineScavengerBoost = DoctrineScavengerBoost(profile) * infrastructurePenalty *
PylonGridBoost(profile) * TowLogisticsBoost(profile);
AddInventoryItem(profile, "steel_scrap",
static_cast<int>(std::round((engineerAssignedToScavengers ? 3.0f : 2.0f) * doctrineScavengerBoost)),
0.5f);
AddInventoryItem(profile, "copper_wire", 1, 0.2f);

```

```

AddInventoryItem(profile, "repair_patch", 1, 0.2f);

const int erosionCleared = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 6.0f, true)));

if (erosionCleared > 0) {

AddInventoryItem(profile, "ether_shard", 1, 0.1f);

}

profile.scavengerRunsCompleted += 1;

gameState.scavengerTimer = (engineerAssignedToScavengers ? 150.0f : 180.0f) / doctrineScavengerBoost;

gameState.lastEvent = engineerAssignedToScavengers

? "Scavenger team returned under engineer supervision with boosted salvage intake."

: (erosionCleared > 0

? "Scavenger team returned with salvage and cut back nearby ether bloom."

: "Scavenger team returned to camp with fresh salvage from cleared routes.");

}

void UpdateCaravanRoute(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->caravanRouteActive) {

gameState.caravanTimer = 260.0f;

return;

}

const bool caravanReady = CaravanRouteReady(profile);

if (!caravanReady) {

gameState.caravanTimer = 260.0f;

return;

}

const float infrastructurePenalty = std::max(0.6f, 1.0f - worldState->infrastructureDecay / 170.0f);

const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.22f :

(profile.doctrine == ShelterDoctrine::Defense ? 1.08f : 1.0f)) *

PylonGridBoost(profile) *

TowLogisticsBoost(profile);

```

```

const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");

gameState.caravanTimer -= dt;

if (gameState.caravanTimer > 0.0f) {

return;

}

const int steelYield = static_cast<int>(std::round((engineerSupport ? 5.0f : 4.0f) * doctrineBoost *
infrastructurePenalty));

const int wireYield = static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty));

AddInventoryItem(profile, "steel_scrap", std::max(2, steelYield), 0.5f);

AddInventoryItem(profile, "copper_wire", std::max(1, wireYield), 0.2f);

AddInventoryItem(profile, "old_plate", 1, 0.5f);

if (profile.doctrine == ShelterDoctrine::Defense) {

AddInventoryItem(profile, "%it_ptrs_ammo", 2, 0.7f);

}

if (worldState->infrastructureDecay >= 12.0f) {

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 3.0f);

}

worldState->caravanRunsCompleted += 1;

gameState.caravanTimer = 260.0f / doctrineBoost;

gameState.lastEvent = engineerSupport

? "Autopilot caravan returned under escort support with reinforced cargo intake."

: "Autopilot caravan returned from the bunker route with bulk salvage.";

}

void UpdateDroneStations(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->droneStationsActive) {

gameState.droneTimer = 210.0f;

return;

}

if (!IsRegionalGridOnline(profile)) {

```



```

gameState.droneTimer = 210.0f;

return;
}

const float infrastructurePenalty = std::max(0.62f, 1.0f - worldState->infrastructureDecay / 175.0f);

const float doctrineBoost = (profile.doctrine == ShelterDoctrine::Industry ? 1.28f :
(profile.doctrine == ShelterDoctrine::Defense ? 1.05f : 1.0f)) *
TowLogisticsBoost(profile);

const float pylonBoost = PylonGridBoost(profile);

gameState.droneTimer -= dt;

if (gameState.droneTimer > 0.0f) {

return;

}

AddInventoryItem(profile, "steel_scrap", std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost *
infrastructurePenalty * pylonBoost))), 0.5f);

AddInventoryItem(profile, "ether_shard", 1, 0.1f);

if (profile.doctrine == ShelterDoctrine::Industry) {

AddInventoryItem(profile, "copper_wire", 1, 0.2f);

}

worldState->droneRunsCompleted += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 1.5f);

gameState.droneTimer = 210.0f / (doctrineBoost * std::max(1.0f, pylonBoost * 0.95f));

gameState.lastEvent = "Automated drone sweep completed. Salvage and ether traces transferred to shelter
stores.";

}

void UpdateTradeNetwork(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->tradeNetworkActive) {

gameState.tradeTimer = 240.0f;

return;

}

```

```

const bool tradeReady = TradeNetworkReady(profile, *worldState);

if (!tradeReady) {

gameState.tradeTimer = 240.0f;

return;

}

const float infrastructurePenalty = std::max(0.65f, 1.0f - worldState->infrastructureDecay / 185.0f);

const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.24f :

(profile.doctrine == ShelterDoctrine::Medical ? 1.1f : 1.0f)) *

PylonGridBoost(profile) *

TowLogisticsBoost(profile);

gameState.tradeTimer -= dt;

if (gameState.tradeTimer > 0.0f) {

return;

}

const int vouchers = std::max(1, static_cast<int>(std::round(doctrineBoost * infrastructurePenalty)));

AddInventoryItem(profile, "trade_voucher", vouchers, 0.0f);

worldState->tradeCyclesCompleted += 1;

gameState.tradeTimer = 240.0f / doctrineBoost;

gameState.lastEvent = "Trade network convoy synchronized. New vouchers and exchange stock entered the camp ledger.";

}

void UpdateRailFreight(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->railFreightActive) {

gameState.railTimer = 320.0f;

return;

}

const bool railReady = RailFreightReady(profile, *worldState);

if (!railReady) {

```

```

gameState.railTimer = 320.0f;

return;
}

const float infrastructurePenalty = std::max(0.58f, 1.0f - worldState->infrastructureDecay / 190.0f);

const float doctrineBoost =
(profile.doctrine == ShelterDoctrine::Industry ? 1.3f :
(profile.doctrine == ShelterDoctrine::Defense ? 1.08f : 1.0f)) *
PylonGridBoost(profile) *
TowLogisticsBoost(profile);

const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");

gameState.railTimer -= dt;

if (gameState.railTimer > 0.0f) {
return;
}

const int steelYield = std::max(3, static_cast<int>(std::round((engineerSupport ? 8.0f : 6.0f) * doctrineBoost *
infrastructurePenalty)));

const int plateYield = std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty)));

AddInventoryItem(profile, "steel_scrap", steelYield, 0.5f);

AddInventoryItem(profile, "old_plate", plateYield, 0.5f);

AddInventoryItem(profile, "copper_wire", std::max(1, static_cast<int>(std::round(2.0f * infrastructurePenalty))),
0.2f);

AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

if (profile.doctrine == ShelterDoctrine::Industry) {
AddInventoryItem(profile, "power_cell", 1, 0.3f);
}

worldState->railRunsCompleted += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 2.5f);

gameState.railTimer = 320.0f / doctrineBoost;

gameState.lastEvent = engineerSupport
? "Rail freight link returned with reinforced salvage under engineer-backed logistics."

```

```

: "Rail freight link delivered heavy salvage from the restored industrial spur.";
}

void UpdateOrbitalUplink(SessionProfile& profile, GameState& gameState, float dt) {
    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);
    if (worldState == nullptr || !worldState->orbitalUplinkActive) {
        gameState.orbitalTimer = 420.0f;
        return;
    }

    const bool uplinkReady = OrbitalUplinkReady(profile, *worldState);

    gameState.lastEvent = "Outer route contamination surged. Ether barricades and fresh nests are reforming beyond the bulkhead.";

    } else if (worldState.routeOverrun && worldState.routeContamination <= 18.0f) {
        worldState.routeOverrun = false;

        gameState.lastEvent = "Outer route stabilized again. Contamination pressure has receded.";
    }
}

void UpdateScavengerTeams(SessionProfile& profile, GameState& gameState, float dt) {
    const bool scavengerReady =
        HasActiveFieldCheckpoint(profile) &&
        profile.story.outerRoadCleared &&
        IsRegionalGridOnline(profile) &&
        HasAwakenedSpecialistRole(profile, "engineer");
    if (!scavengerReady) {
        gameState.scavengerTimer = 180.0f;
        return;
    }

    const bool engineerAssignedToScavengers = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");
    gameState.scavengerTimer -= dt;
    if (gameState.scavengerTimer > 0.0f) {
        return;
    }
}

```

```

}

const auto* worldState = FindWorldFieldState(profile, profile.selectedWorld);

const float infrastructurePenalty = worldState != nullptr
? std::max(0.65f, 1.0f - worldState->infrastructureDecay / 180.0f)
: 1.0f;

const float doctrineScavengerBoost = DoctrineScavengerBoost(profile) * infrastructurePenalty *
PylonGridBoost(profile) * TowLogisticsBoost(profile);

AddInventoryItem(profile, "steel_scrap",
static_cast<int>(std::round((engineerAssignedToScavengers ? 3.0f : 2.0f) * doctrineScavengerBoost)),
0.5f);

AddInventoryItem(profile, "copper_wire", 1, 0.2f);

AddInventoryItem(profile, "repair_patch", 1, 0.2f);

const int erosionCleared = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 6.0f, true)));
if (erosionCleared > 0) {
AddInventoryItem(profile, "ether_shard", 1, 0.1f);
}

profile.scavengerRunsCompleted += 1;

gameState.scavengerTimer = (engineerAssignedToScavengers ? 150.0f : 180.0f) / doctrineScavengerBoost;
gameState.lastEvent = engineerAssignedToScavengers
? "Scavenger team returned under engineer supervision with boosted salvage intake."
: (erosionCleared > 0
? "Scavenger team returned with salvage and cut back nearby ether bloom."
: "Scavenger team returned to camp with fresh salvage from cleared routes.");
}

void UpdateCaravanRoute(SessionProfile& profile, GameState& gameState, float dt) {
auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);
if (worldState == nullptr || !worldState->caravanRouteActive) {
gameState.caravanTimer = 260.0f;

return;
}

```

```

const bool caravanReady = CaravanRouteReady(profile);

if (!caravanReady) {

gameState.caravanTimer = 260.0f;

return;

}

const float infrastructurePenalty = std::max(0.6f, 1.0f - worldState->infrastructureDecay / 170.0f);

const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.22f :

(profile.doctrine == ShelterDoctrine::Defense ? 1.08f : 1.0f)) *

PylonGridBoost(profile) *

TowLogisticsBoost(profile);

const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");

gameState.caravanTimer -= dt;

if (gameState.caravanTimer > 0.0f) {

return;

}

const int steelYield = static_cast<int>(std::round((engineerSupport ? 5.0f : 4.0f) * doctrineBoost *

infrastructurePenalty));

const int wireYield = static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty));

AddInventoryItem(profile, "steel_scrap", std::max(2, steelYield), 0.5f);

AddInventoryItem(profile, "copper_wire", std::max(1, wireYield), 0.2f);

AddInventoryItem(profile, "old_plate", 1, 0.5f);

if (profile.doctrine == ShelterDoctrine::Defense) {

AddInventoryItem(profile, "%it_ptr ammo", 2, 0.7f);

}

if (worldState->infrastructureDecay >= 12.0f) {

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 3.0f);

}

worldState->caravanRunsCompleted += 1;

gameState.caravanTimer = 260.0f / doctrineBoost;

```

```

gameState.lastEvent = engineerSupport

? "Autopilot caravan returned under escort support with reinforced cargo intake."

: "Autopilot caravan returned from the bunker route with bulk salvage.";

}

void UpdateDroneStations(SessionProfile& profile, GameState& gameState, float dt) {

    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

    if (worldState == nullptr || !worldState->droneStationsActive) {

        gameState.droneTimer = 210.0f;

        return;

    }

    if (!IsRegionalGridOnline(profile)) {

        gameState.droneTimer = 210.0f;

        return;

    }

    const float infrastructurePenalty = std::max(0.62f, 1.0f - worldState->infrastructureDecay / 175.0f);

    const float doctrineBoost = (profile.doctrine == ShelterDoctrine::Industry ? 1.28f :

    (profile.doctrine == ShelterDoctrine::Defense ? 1.05f : 1.0f)) *

    TowLogisticsBoost(profile);

    const float pylonBoost = PylonGridBoost(profile);

    gameState.droneTimer -= dt;

    if (gameState.droneTimer > 0.0f) {

        return;

    }

    AddInventoryItem(profile, "steel_scrap", std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost *

    infrastructurePenalty * pylonBoost))), 0.5f);

    AddInventoryItem(profile, "ether_shard", 1, 0.1f);

    if (profile.doctrine == ShelterDoctrine::Industry) {

        AddInventoryItem(profile, "copper_wire", 1, 0.2f);

    }

    worldState->droneRunsCompleted += 1;

```

```

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 1.5f);

gameState.droneTimer = 210.0f / (doctrineBoost * std::max(1.0f, pylonBoost * 0.95f));

gameState.lastEvent = "Automated drone sweep completed. Salvage and ether traces transferred to shelter stores.";

}

void UpdateTradeNetwork(SessionProfile& profile, GameState& gameState, float dt) {

    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

    if (worldState == nullptr || !worldState->tradeNetworkActive) {

        gameState.tradeTimer = 240.0f;

        return;

    }

    const bool tradeReady = TradeNetworkReady(profile, *worldState);

    if (!tradeReady) {

        gameState.tradeTimer = 240.0f;

        return;

    }

    const float infrastructurePenalty = std::max(0.65f, 1.0f - worldState->infrastructureDecay / 185.0f);

    const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.24f :

(profile.doctrine == ShelterDoctrine::Medical ? 1.1f : 1.0f)) *

PylonGridBoost(profile) *

TowLogisticsBoost(profile);

gameState.tradeTimer -= dt;

if (gameState.tradeTimer > 0.0f) {

    return;

}

const int vouchers = std::max(1, static_cast<int>(std::round(doctrineBoost * infrastructurePenalty)));

AddInventoryItem(profile, "trade_voucher", vouchers, 0.0f);

worldState->tradeCyclesCompleted += 1;

gameState.tradeTimer = 240.0f / doctrineBoost;

```



```

gameState.lastEvent = "Trade network convoy synchronized. New vouchers and exchange stock entered the camp ledger.";
}

void UpdateRailFreight(SessionProfile& profile, GameState& gameState, float dt) {

    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

    if (worldState == nullptr || !worldState->railFreightActive) {

        gameState.railTimer = 320.0f;

        return;

    }

    const bool railReady = RailFreightReady(profile, *worldState);

    if (!railReady) {

        gameState.railTimer = 320.0f;

        return;

    }

    const float infrastructurePenalty = std::max(0.58f, 1.0f - worldState->infrastructureDecay / 190.0f);

    const float doctrineBoost =

        (profile.doctrine == ShelterDoctrine::Industry ? 1.3f :

        (profile.doctrine == ShelterDoctrine::Defense ? 1.08f : 1.0f)) *

        PylonGridBoost(profile) *

        TowLogisticsBoost(profile);

    const bool engineerSupport = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");

    gameState.railTimer -= dt;

    if (gameState.railTimer > 0.0f) {

        return;

    }

    const int steelYield = std::max(3, static_cast<int>(std::round((engineerSupport ? 8.0f : 6.0f) * doctrineBoost * infrastructurePenalty)));

    const int plateYield = std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost * infrastructurePenalty)));

    AddInventoryItem(profile, "steel_scrap", steelYield, 0.5f);

    AddInventoryItem(profile, "old_plate", plateYield, 0.5f);

```

```

AddInventoryItem(profile, "copper_wire", std::max(1, static_cast<int>(std::round(2.0f * infrastructurePenalty))),
0.2f);

AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

if (profile.doctrine == ShelterDoctrine::Industry) {

AddInventoryItem(profile, "power_cell", 1, 0.3f);

}

worldState->railRunsCompleted += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 2.5f);

gameState.railTimer = 320.0f / doctrineBoost;

gameState.lastEvent = engineerSupport

? "Rail freight link returned with reinforced salvage under engineer-backed logistics."

: "Rail freight link delivered heavy salvage from the restored industrial spur.";

}

void UpdateOrbitalUplink(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->orbitalUplinkActive) {

gameState.orbitalTimer = 420.0f;

return;

}

const bool uplinkReady = OrbitalUplinkReady(profile, *worldState);

Success

Ran command

$ $lines=Get-Content src\GameRuntime.cpp; $lines[3020..3065]; $lines[3550..3640]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

totalWeight += item.unitWeight * static_cast<float>(item.count);

}

return totalWeight;

}

int EffectiveStatValue(const SessionProfile& profile, const GameState& gameState, char statCode) {

int value = profile.character.StatValue(statCode);

```

```

if (gameState.rationEffectTimer > 0.0f) {
    switch (static_cast<char>(std::toupper(static_cast<unsigned char>(statCode)))) {
        case 'S':
            value += gameState.rationStrengthBonus;
            break;
        case 'I':
            value -= gameState.rationIntelligencePenalty;
            break;
        default:
            break;
    }
}

return std::max(1, value);
}

bool TryConsumeFieldRation(SessionProfile& profile, GameState& gameState) {
    if (!ConsumeInventoryItem(profile, "%it_field_ration", 1)) {
        return false;
    }

    gameState.rationEffectTimer = 150.0f;
    gameState.rationStrengthBonus = 2;
    gameState.rationIntelligencePenalty = 1;
    gameState.lastEvent = "Old ration consumed. Toxic boost applied: +2 STR, -1 INT for a while.";
    return true;
}

bool CanAttachBt72HullToCrane(const SessionProfile& profile) {
    return profile.firstPlayableRoute.bt72HullInspected &&
        profile.hangarPowerRestored &&
        profile.bt72CraneControlOnline &&
        profile.bt72CranePathClear &&
        !profile.bt72HullMovedToServiceLift &&

```

```

!profile.bt72HullLockedInRestorationCradle;
}

bool AttachBt72HullToCrane(SessionProfile& profile) {
    if (!CanAttachBt72HullToCrane(profile)) {
        return false;
    }

    return HasActivePipDevice(profile);
}

bool RecoverPipPad(SessionProfile& profile) {
    return SelectPipDevice(profile, ActivePipDeviceIdOrDefault(profile));
}

bool HasBlueLinkModule(const SessionProfile& profile) {
    return profile.blueLinkModuleRecovered || profile.blueLinkModuleInstalled;
}

bool IsBlueLinkInstalled(const SessionProfile& profile) {
    return HasPipPad(profile) &&
        profile.blueLinkModuleInstalled &&
        !profile.pipPadExpansionCoverPresent;
}

bool InstallBlueLinkModule(SessionProfile& profile) {
    if (!HasPipPad(profile) || !HasBlueLinkModule(profile)) {
        return false;
    }

    const bool alreadyInstalled = IsBlueLinkInstalled(profile);
    profile.blueLinkModuleRecovered = true;
    profile.blueLinkModuleInstalled = true;
    profile.pipPadExpansionCoverPresent = false;
    return !alreadyInstalled;
}

bool CanUsePipPadMediaIndex(const SessionProfile& profile) {

```

```

return IsBlueLinkInstalled(profile) &&
ActivePipDeviceSupportsMediaIndex(profile);
}

bool PlayerHasActivePipDeviceAccess(const SessionProfile& profile) {
return HasPipPad(profile);
}

bool PlayerHasPipPadAccess(const SessionProfile& profile) {
return PlayerHasActivePipDeviceAccess(profile);
}

std::string ActivePipDeviceUiCapabilitySummary(const SessionProfile& profile) {
if (ActivePipDeviceSupportsFullWorkspace(profile)) {
return "Full field workspace online.";
}

if (ActivePipDeviceSupportsMediaIndex(profile) && ActivePipDeviceHasDigitalMap(profile)) {
return "Map and media-capable field interface online.";
}

if (ActivePipDeviceSupportsMediaIndex(profile)) {
return "Media-capable physical-navigation interface online.";
}

if (ActivePipDeviceHasDigitalMap(profile)) {
return "Digital map interface online.";
}

if (ActivePipDeviceUsesPhysicalNavigation(profile)) {
return "Physical-navigation interface online.";
}

return "Field interface online.";
}

bool TryToggleActivePipDeviceUi(PlayerState& player, const SessionProfile& profile, GameState& gameState) {
if (!HasPipPad(profile)) {
player.uiVisible = false;

```

```

gameState.lastEvent = "No Pip-Pad linked yet. No active Pip device assigned; recover a Pip-Boy or Pip-Pad first.";
return false;
}

player.uiVisible = !player.uiVisible;

if (player.uiVisible) {

gameState.lastEvent = ActivePipDeviceDisplayName(profile) + " opened. " +
ActivePipDeviceUiCapabilitySummary(profile) + " " +
ActivePipDeviceCustomizationSummary(profile);

} else {

gameState.lastEvent = ActivePipDeviceDisplayName(profile) + " closed.";

}

return true;

}

bool TryTogglePipPadUi(PlayerState& player, const SessionProfile& profile, GameState& gameState) {
return TryToggleActivePipDeviceUi(player, profile, gameState);
}

void AdvanceViewMode(PlayerState& player) {

switch (player.viewMode) {

case ViewMode::FirstPerson:

player.viewMode = ViewMode::ThirdPerson;

break;

case ViewMode::ThirdPerson:

player.viewMode = ViewMode::Cockpit;

break;

case ViewMode::Cockpit:

Success

Ran command

$ $lines=Get-Content src\WorldEvents.cpp; $lines[0..140]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

#include "../include/WorldEvents.hpp"

```

```

#include <cmath>

namespace bunker {

namespace {

bool IsNear(const PlayerState& player, float x, float y, float radius) {

const float dx = player.x - x;

const float dy = player.y - y;

return (dx * dx) + (dy * dy) <= radius * radius;

}

void AppendObjectiveHint(std::string& message, const std::string& objective) {

if (!objective.empty() && message.find(objective) == std::string::npos) {

message += " Next: " + objective;

}

}

void AppendObjectivePreviewHint(std::string& message, const SessionProfile& profile) {

AppendObjectiveHint(message, CurrentStoryObjectivePreview(profile));

}

void AppendRouteBeatReadabilityHint(std::string& message, const SessionProfile& profile) {

const auto beat = CurrentFirstPlayableRouteBeat(profile);

if (!beat.label.empty() && message.find("Route beat: " + beat.label) == std::string::npos) {

message += " Route beat: " + beat.label + ".";

}

if (!beat.payoff.empty() && message.find(beat.payoff) == std::string::npos) {

message += " Readable payoff: " + beat.payoff;

}

}

bool IsAtSurfaceArrivalAnchor(const World& world, const PlayerState& player) {

if (const auto* campMarker = world.FindObjectByRegistryId("[%camp_0001]"); campMarker != nullptr) {

return IsNear(player, campMarker->x, campMarker->y, 5.0f);

}

return player.x >= 18.0f;

```

```

}

bool IsAtFirstCombatAnchor(const World& world, const PlayerState& player) {

if (const auto* firstContact = world.FindObjectByRegistryId("[%enemy_ghoul_0001]"); firstContact != nullptr) {

return IsNear(player, firstContact->x, firstContact->y, 6.5f);

}

return player.x >= 17.0f;

}

} // namespace

void ProcessScriptedWorldEvents(const World& world, const PlayerState& player, SessionProfile& profile,
GameState& gameState) {

if (!world.IsStarterScenarioWorld()) {

return;

}

if (profile.story.awakenedFromCryo && !profile.story.pipPadRecovered && !gameState.zoneEventCryoLocker &&
IsNear(player, -9.0f, -6.0f, 2.5f)) {

gameState.zoneEventCryoLocker = true;

gameState.lastEvent = profile.firstPlayableRoute.accessCardRecovered

? "LOCKER BAY: 'Emergency recovery locker detected. Card interlock accepts the bunker access card. Pip-Pad
signature nearby.'"

: "LOCKER BAY: 'Emergency recovery locker detected. Mechanical interlock wants a bunker access card from the
core service racks.'";

return;

}

if (profile.story.pipPadRecovered && !profile.story.archiveRecovered && !gameState.zoneEventArchive &&
IsNear(player, -7.0f, -1.0f, 3.0f)) {

gameState.zoneEventArchive = true;

gameState.lastEvent = "ARCHIVE: 'Missing personnel logs available. Expect vermin contamination in the corridor
and garage references in the recovered notes.'";

return;

}

```



```

if (profile.story.archiveRecovered && !profile.firstPlayableRoute.bt72Restored && !gameState.zoneEventGarage
&&
IsNear(player, 4.0f, -1.5f, 4.0f)) {

gameState.zoneEventGarage = true;

gameState.lastEvent = "GARAGE: 'BT-72 hull located. Pairing is still locked: inspect the hull, recover the core, and
pull the maintenance echo first.'";

return;

}

if (profile.story.exitedBunker &&
!profile.firstPlayableRoute.surfaceArrivalReached &&
IsAtSurfaceArrivalAnchor(world, player)) {

profile.firstPlayableRoute.surfaceArrivalReached = true;

gameState.zoneEventExterior = true;

gameState.lastEvent = "SURFACE ARRIVAL: 'BT-72 reached the first exterior foothold. Skyline active, debris barrier
ahead, and the first hostile contact is forming beyond the lift route.'";

AppendObjectivePreviewHint(gameState.lastEvent, profile);

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

}

if (profile.story.outerRoadCleared &&
!profile.firstPlayableRoute.firstTankCombatResolved &&
!gameState.zoneEventFirstCombat &&
IsAtFirstCombatAnchor(world, player)) {

gameState.zoneEventFirstCombat = true;

gameState.lastEvent = player.insideTank
? "CONTACT: 'Outer Ghoul charging the cleared lane. Hold BT-72 through the rush, then take one service halt
before syncing the relay.'"
: "CONTACT: 'Outer Ghoul detected beyond the cleared debris lane. BT-72 is the intended answer for the first
surface contact.'";

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

```

```

}

if (profile.story.relayRecovered && !profile.story.returnedToBase && !gameState.zoneEventReturn &&
IsNear(player, -2.5f, 3.2f, 3.0f)) {

gameState.zoneEventReturn = true;

gameState.lastEvent = "DEBRIEF: 'Recovery node recognized. Upload the route summary and authorize Shelter 17
planning: rail, orbital, fabrication, then the inner spur.'";

AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);

return;

}

if (profile.story.returnedToBase && !gameState.zoneEventReturn &&
IsNear(player, -2.5f, 3.2f, 3.0f)) {

gameState.zoneEventReturn = true;

gameState.lastEvent = "DEBRIEF: 'Recovery planning remains active. Keep rebuilding rail freight, orbital support,
fabrication, and the inner spur backbone.'";

}

}

} // namespace bunker

Success

Ran command

$ $lines=Get-Content include\GameRuntime.hpp; $lines[60..240]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

float radioTimer = 12.0f;

float attackCooldown = 0.0f;

float specialAttackCooldown = 0.0f;

float damageCooldown = 0.0f;

float damageFlashTimer = 0.0f;

float weatherTimer = 70.0f;

float weatherEventTimer = 0.0f;

float weatherIntensity = 0.0f;

std::size_t radioPhase = 0;

```

```
bool zoneEventCryoLocker = false;

bool zoneEventArchive = false;

bool zoneEventGarage = false;

bool zoneEventExterior = false;

bool zoneEventFirstCombat = false;

bool zoneEventReturn = false;

bool stressThresholdTriggered = false;

bool secondWindTriggered = false;

bool soulLineTriggered = false;

float rationEffectTimer = 0.0f;

int rationStrengthBonus = 0;

int rationIntelligencePenalty = 0;

float scavengerTimer = 180.0f;

float caravanTimer = 260.0f;

float droneTimer = 210.0f;

float tradeTimer = 240.0f;

float railTimer = 320.0f;

float orbitalTimer = 420.0f;

float railFortressTimer = 520.0f;

float fabricatorTimer = 280.0f;

float surveyTimer = 360.0f;

float outpostTimer = 300.0f;

float assemblyTimer = 340.0f;

float foundryTimer = 420.0f;

float reactorTimer = 460.0f;

float capacitorTimer = 390.0f;

float relaySubstationTimer = 430.0f;

float serviceBayTimer = 300.0f;

float waterReclaimerTimer = 260.0f;

float etherErosionEventTimer = 0.0f;
```

```
float fieldWorkbenchCooldown = 0.0f;

float workshopServiceCooldown = 0.0f;

float heavyCarryTimer = 0.0f;

float supportRefreshTimer = 0.0f;

float feyRingRefreshTimer = 0.0f;

float tankThermalLoad = 12.0f;

bool lanlineServicesVisible = false;

bool feyRingScheduleVisible = false;

bool supportTerminalNearby = false;

bool tankServiceNearby = false;

bool medicalSupportNearby = false;

WeatherAnomaly weather = WeatherAnomaly::Clear;

OverlayState overlay{};

std::vector<HostileAwarenessState> hostileAwareness{};

std::vector<MechanicalHostileDamageState> mechanicalHostileDamage{};

std::string lastEvent = "Cryostasis breached. Reorient, secure an access card, recover the Pip-Pad, and trace the BT-72 berth.";

std::string lastSupportAction = "No support activity.";

std::string lastPortalAction = "No portal updates.";

std::vector<std::string> radioMessages = {

"SYSTEM: 'Cryo wing unstable. Sweep the shared cryo tier, recover an access card, and follow the paper trail to the Pip-Pad.'",

"ARCHIVE: 'One reactor core is missing. One body is missing. Garage service traces remain incomplete.'",

"BT-72: 'Hull detected. Restore the chassis before any cockpit link attempt.'",

"HQ: 'The outer bulkhead is blocked. Install a clearance module before forcing that route.'",

"ARCHIVE: 'Relay and city-fringe schematics remain fragmented. Continue recovery.'"

};

};

const char* RuntimeGamePhaseLabel(RuntimeGamePhase phase);

void AddInventoryItem(SessionProfile& profile, const std::string& itemId, int count, float weight);
```

```
bool HasInventoryItem(const SessionProfile& profile, const std::string& itemId);

bool ConsumeInventoryItem(SessionProfile& profile, const std::string& itemId, int count);

float CurrentInventoryWeight(const SessionProfile& profile);

int EffectiveStatValue(const SessionProfile& profile, const GameState& gameState, char statCode);

bool TryConsumeFieldRation(SessionProfile& profile, GameState& gameState);

enum class PipDeviceModel {

    PipBoy01,

    PipBoy10,

    PipBoy2000Classic,

    PipBoy2000MarkVI,

    PipBoy3000MarkIV,

    PipBoy3000 = PipBoy3000MarkIV,

    PipBoy3000MarkV,

    PipPad3500

};

struct PipDeviceCapabilities {

    const char* displayName = "";

    bool selectable = false;

    bool propOnly = false;

    bool hasDigitalMap = false;

    bool physicalNavigationOnly = false;

    bool supportsMediaIndex = false;

    bool supportsFullPipPadWorkspace = false;

    bool ruggedMilitaryPipBoyDerived = false;

    bool userPreferredComfortShell = false;

};

struct PipDeviceSelectionOption {

    std::string itemId;

    PipDeviceModel model = PipDeviceModel::PipPad3500;

    std::string displayName;
```

```
bool currentlyActive = false;

bool supportsMediaIndex = false;

bool supportsFullWorkspace = false;

bool hasDigitalMap = false;

bool usesPhysicalNavigation = false;

bool userPreferredComfortShell = false;

};

enum class PipDeviceCustomizationSlot {

Theme,

DisplayMode,

CarryMode

};

PipDeviceCapabilities GetPipDeviceCapabilities(PipDeviceModel model);

const char* DeviceDisplayName(PipDeviceModel model);

bool IsKnownPipDeviceId(std::string_view deviceId);

bool TryGetPipDeviceModelForId(std::string_view deviceId, PipDeviceModel& outModel);

PipDeviceCapabilities ActivePipDeviceCapabilities(const SessionProfile& profile);

bool ActivePipDeviceSupportsMediaIndex(const SessionProfile& profile);

bool ActivePipDeviceSupportsFullWorkspace(const SessionProfile& profile);

bool ActivePipDeviceHasDigitalMap(const SessionProfile& profile);

bool ActivePipDeviceUsesPhysicalNavigation(const SessionProfile& profile);

std::string ActivePipDeviceIdOrDefault(const SessionProfile& profile);

bool HasActivePipDevice(const SessionProfile& profile);

bool SelectPipDevice(SessionProfile& profile, std::string_view deviceId);

bool BeginPipDeviceReselect(SessionProfile& profile);

std::string ActivePipDeviceDisplayName(const SessionProfile& profile);

std::vector<PipDeviceSelectionOption> BuildPipDeviceSelectionOptions(const SessionProfile& profile);

bool CompletePipDeviceReselect(SessionProfile& profile, std::string_view deviceId);

bool IsPipDeviceSelectionStationRetired(const SessionProfile& profile);

std::string ActivePipDeviceCustomizationSummary(const SessionProfile& profile);
```

```
bool ActivePipDeviceSupportsTheme(const SessionProfile& profile, std::string_view themeld);

bool ActivePipDeviceSupportsDisplayMode(const SessionProfile& profile, std::string_view displayMode);

bool ActivePipDeviceSupportsCarryMode(const SessionProfile& profile, std::string_view carryMode);

bool ActivePipDeviceSupportsCustomization(const SessionProfile& profile,
std::string_view themeld,
std::string_view displayMode,
std::string_view carryMode);

bool CanCustomizeActivePipDevice(const SessionProfile& profile);

bool SetActivePipDeviceCustomization(SessionProfile& profile,
std::string_view themeld,
std::string_view displayMode,
std::string_view carryMode);

bool CycleActivePipDeviceCustomization(SessionProfile& profile,
PipDeviceCustomizationSlot slot,
GameState* gameState = nullptr);

bool ApplyActivePipDeviceCustomizationCommand(SessionProfile& profile,
std::string_view command,
GameState* gameState = nullptr);

bool IsSelectablePipDevice(PipDeviceModel model);

bool IsPropOnlyPipDevice(PipDeviceModel model);

bool HasDigitalMap(PipDeviceModel model);

bool UsesPhysicalNavigation(PipDeviceModel model);

bool SupportsMediaIndex(PipDeviceModel model);

bool SupportsFullPipPadWorkspace(PipDeviceModel model);

bool HasPipPad(const SessionProfile& profile);

bool RecoverPipPad(SessionProfile& profile);

bool HasBlueLinkModule(const SessionProfile& profile);

bool IsBlueLinkInstalled(const SessionProfile& profile);

bool InstallBlueLinkModule(SessionProfile& profile);

bool CanUsePipPadMediaIndex(const SessionProfile& profile);
```

```

bool CanAttachBt72HullToCrane(const SessionProfile& profile);

bool AttachBt72HullToCrane(SessionProfile& profile);

bool CanMoveBt72HullToServiceLift(const SessionProfile& profile);

bool MoveBt72HullToServiceLift(SessionProfile& profile);

bool CanInstallBt72Core(const SessionProfile& profile);

bool CanCompleteBt72StagedRestoration(const SessionProfile& profile);

bool PlayerHasActivePipDeviceAccess(const SessionProfile& profile);

bool PlayerHasPipPadAccess(const SessionProfile& profile);

bool TryToggleActivePipDeviceUi(PlayerState& player, const SessionProfile& profile, GameState& gameState);

bool TryTogglePipPadUi(PlayerState& player, const SessionProfile& profile, GameState& gameState);

void AdvanceViewMode(PlayerState& player);

void TryToggleBt72CrewSeat(PlayerState& player, SessionProfile& profile, GameState& gameState);

void ApplyStaticEraser(World& world, const StaticEraser& staticEraser);

bool ShouldUseStarterStoryFlow(const World& world);

void SyncStoryFlagsFromWorld(SessionProfile& profile, const StaticEraser& staticEraser);

void UpdateWorldMetadata(World& world, const SessionProfile& profile, const StaticEraser& staticEraser);

void UpdateWindowTitle(GLFWwindow* window, const PlayerState& player, const World& world, const SessionProfile& sessionProfile);

void UpdateRadio(GameState& gameState, const World& world, const SessionProfile& profile, const StaticEraser& staticEraser, float dt);

const MapObject* FindNearestHostile(const World& world, float x, float y, float radius);

```

Success

Ran command

```
$ $lines=Get-Content include\GameRuntime.hpp; $lines[240..360]
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
const MapObject* FindNearestHostile(const World& world, float x, float y, float radius);
```

```
const char* TankIntegrityBand(float integrity);
```

```
std::string DescribeHostileReadability(const MapObject& object, const GameState& gameState);
```

```
void UpdateHostiles(World& world,
```

```
PlayerState& player,
```



```
SessionProfile& profile,  
StaticEraser& staticEraser,  
GameState& gameState,  
float dt);  
  
void SyncPartnerTankAnchor(World& world,  
const PlayerState& player,  
SessionProfile& profile);  
  
void UpdateAmbientTankCharging(const World& world,  
SessionProfile& profile,  
GameState& gameState,  
float dt);  
  
void UpdateWeatherAnomaly(const World& world,  
PlayerState& player,  
SessionProfile& profile,  
GameState& gameState,  
float dt);  
  
void UpdateEtherErosion(const World& world,  
const PlayerState& player,  
SessionProfile& profile,  
GameState& gameState,  
float dt);  
  
void UpdateInfrastructureDecay(const World& world,  
const PlayerState& player,  
SessionProfile& profile,  
GameState& gameState,  
float dt);  
  
void UpdateRouteContamination(World& world,  
SessionProfile& profile,  
StaticEraser& staticEraser,  
GameState& gameState,
```

```
float dt);

void UpdateScavengerTeams(SessionProfile& profile, GameState& gameState, float dt);

void UpdateCaravanRoute(SessionProfile& profile, GameState& gameState, float dt);

void UpdateDroneStations(SessionProfile& profile, GameState& gameState, float dt);

void UpdateTradeNetwork(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRailFreight(SessionProfile& profile, GameState& gameState, float dt);

void UpdateOrbitalUplink(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRailFortress(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRecoveryFabricator(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRecoveryMilestones(SessionProfile& profile, GameState& gameState);

void UpdateIndustrialSurvey(SessionProfile& profile, GameState& gameState, float dt);

void UpdateIndustrialOutpost(SessionProfile& profile, GameState& gameState, float dt);

void UpdateAssemblyCell(SessionProfile& profile, GameState& gameState, float dt);

void UpdateFoundryLine(SessionProfile& profile, GameState& gameState, float dt);

void UpdateReactorYard(SessionProfile& profile, GameState& gameState, float dt);

void UpdateCapacitorBank(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRelaySubstation(SessionProfile& profile, GameState& gameState, float dt);

void UpdateServiceBay(SessionProfile& profile, GameState& gameState, float dt);

void UpdateWaterReclaimer(SessionProfile& profile, GameState& gameState, float dt);

void UpdateRouteRandomEvents(SessionProfile& profile, GameState& gameState, float dt);

bool TryResolveMerchantRouteEvent(SessionProfile& profile, GameState& gameState);

void HandleAttack(World& world,

PlayerState& player,

SessionProfile& profile,

StaticEraser& staticEraser,

GameState& gameState);

void HandleSpecialAttack(World& world,

PlayerState& player,

SessionProfile& profile,

StaticEraser& staticEraser,
```

```

GameState& gameState);

void HandleInteraction(const MapObject* nearest,
World& world,
PlayerState& player,
SessionProfile& profile,
StaticEraser& staticEraser,
GameState& gameState,
const WorldExecutionContext* executionContext = nullptr);

bool WantsUseKey(const MapObject* nearest);

bool WantsContextKey(const MapObject* nearest);

void DrawPipPad(const World& world,
PlayerState& player,
SessionProfile& profile,
StaticEraser& staticEraser,
GameState& gameState);

bool SweepMovePlayerAgainstWorld(const World& world, PlayerState& player, float deltaX, float deltaY);
} // namespace bunker

Success

Ran command

$ $lines=Get-Content src\GameRuntime.cpp; $lines[4300..4485]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

if (worldState == nullptr || !worldState->orbitalUplinkActive) {

gameState.orbitalTimer = 420.0f;

return;

}

const bool uplinkReady = OrbitalUplinkReady(profile, *worldState);

if (!uplinkReady) {

gameState.orbitalTimer = 420.0f;

return;

}

```

```

const float doctrineBoost =
(profile.doctrine == ShelterDoctrine::Industry ? 1.18f :
(profile.doctrine == ShelterDoctrine::Medical ? 1.08f : 1.0f)) *
PylonGridBoost(profile);

const float infrastructurePenalty = std::max(0.64f, 1.0f - worldState->infrastructureDecay / 200.0f);

gameState.orbitalTimer -= dt;

if (gameState.orbitalTimer > 0.0f) {

return;

}

const int purged = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 10.0f, true)));

AddInventoryItem(profile, "ether_shard", 1, 0.1f);

AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

if (profile.doctrine == ShelterDoctrine::Industry) {

AddInventoryItem(profile, "power_cell", 1, 0.3f);

}

worldState->orbitalScansCompleted += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - (2.0f *
infrastructurePenalty));

gameState.orbitalTimer = 420.0f / (doctrineBoost * infrastructurePenalty);

gameState.lastEvent = purged > 0

? "Orbital scan completed. Ether bloom pockets mapped and reduced across the route."
: "Orbital scan completed. New salvage and relay traces added to the camp network.";

}

void UpdateRailFortress(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->railFortressActive) {

gameState.railFortressTimer = 520.0f;

return;

}

const bool fortressReady = RailFortressReady(profile, *worldState);

```

```

if (!fortressReady) {
    gameState.railFortressTimer = 520.0f;
    return;
}

const float doctrineBoost =
(profile.doctrine == ShelterDoctrine::Defense ? 1.24f :
(profile.doctrine == ShelterDoctrine::Industry ? 1.16f : 1.0f)) *
PylonGridBoost(profile);

const float infrastructurePenalty = std::max(0.68f, 1.0f - worldState->infrastructureDecay / 210.0f);

gameState.railFortressTimer -= dt;

if (gameState.railFortressTimer > 0.0f) {
    return;
}

AddInventoryItem(profile, "steel_scrap", std::max(4, static_cast<int>(std::round(6.0f * doctrineBoost *
infrastructurePenalty))), 0.5f);

AddInventoryItem(profile, "old_plate", std::max(1, static_cast<int>(std::round(2.0f * doctrineBoost *
infrastructurePenalty))), 0.5f);

AddInventoryItem(profile, "%it_ptrs_amm0", 2, 0.7f);

AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

worldState->railFortressDeployments += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 3.0f);

const int purged = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 5.0f, true)));

gameState.railFortressTimer = 520.0f / (doctrineBoost * infrastructurePenalty);

gameState.lastEvent = purged > 0
? "Rail fortress patrol returned with heavy salvage and suppressed outer ether pressure."
: "Rail fortress patrol returned with armored cargo and route security supplies.";
}

void UpdateRecoveryFabricator(SessionProfile& profile, GameState& gameState, float dt) {
    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

    if (worldState == nullptr || !worldState->recoveryFabricatorActive) {

```

```

gameState.fabricatorTimer = 280.0f;

return;

}

const bool fabricatorReady = RecoveryFabricatorReady(profile, *worldState);

if (!fabricatorReady) {

gameState.fabricatorTimer = 280.0f;

return;

}

const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.28f :

(profile.doctrine == ShelterDoctrine::Medical ? 1.08f : 1.0f)) *

PylonGridBoost(profile);

const float infrastructurePenalty = std::max(0.66f, 1.0f - worldState->infrastructureDecay / 195.0f);

gameState.fabricatorTimer -= dt;

if (gameState.fabricatorTimer > 0.0f) {

return;

}

AddInventoryItem(profile, "repair_patch", std::max(1, static_cast<int>(std::round(1.0f * doctrineBoost *

infrastructurePenalty))), 0.2f);

AddInventoryItem(profile, "power_cell", 1, 0.3f);

if (profile.doctrine == ShelterDoctrine::Industry) {

AddInventoryItem(profile, "old_plate", 1, 0.5f);

} else {

AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);

}

worldState->fabricatorCyclesCompleted += 1;

worldState->infrastructureDecay = std::max(0.0f, worldState->infrastructureDecay - 1.5f);

gameState.fabricatorTimer = 280.0f / (doctrineBoost * infrastructurePenalty);

gameState.lastEvent = "Recovery fabricator completed a refinement cycle and issued fresh field supplies.";

}

```

```
void UpdateRecoveryMilestones(SessionProfile& profile, GameState& gameState) {  
    auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);  
    if (worldState == nullptr) {  
        return;  
    }  
    const int currentTier = ShelterRecoveryMilestoneTier(ShelterRecoveryIndex(profile));  
    if (currentTier <= worldState->recoveryMilestonesClaimed) {  
        return;  
    }  
    while (worldState->recoveryMilestonesClaimed < currentTier) {  
        const int nextTier = worldState->recoveryMilestonesClaimed + 1;  
        std::string xpEvent;  
        if (nextTier == 1) {  
            AddInventoryItem(profile, "cryo_medkit", 1, 0.5f);  
            AddInventoryItem(profile, "steel_scrap", 2, 0.5f);  
            AwardExperience(profile, 35, &xpEvent);  
            gameState.lastEvent = "Shelter Recovery Checkpoint I secured. Emergency stores unlocked for the crew. " +  
                xpEvent;  
        } else if (nextTier == 2) {  
            AddInventoryItem(profile, "power_cell", 1, 0.3f);  
            AddInventoryItem(profile, "repair_patch", 2, 0.2f);  
            AwardExperience(profile, 55, &xpEvent);  
            gameState.lastEvent = "Shelter Recovery Checkpoint II secured. Grid and field service reserves expanded. " +  
                xpEvent;  
        } else {  
            AddInventoryItem(profile, "trade_voucher", 2, 0.0f);  
            AddInventoryItem(profile, "old_plate", 2, 0.5f);  
            AddInventoryItem(profile, "clean_water", 2, 0.4f);  
            AddInventoryItem(profile, "%it_ptrs_ammo", 2, 0.7f);  
            AwardExperience(profile, 75, &xpEvent);  
        }  
    }  
}
```

```

gameState.lastEvent = "Shelter Recovery Checkpoint III secured. Shelter 17 now has a stable recovery backbone
and sustained water reserves. " + xpEvent;
}

worldState->recoveryMilestonesClaimed = nextTier;

}

}

void UpdateIndustrialSurvey(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->industrialSurveyActive) {

gameState.surveyTimer = 360.0f;

return;

}

const bool surveyReady = IndustrialSurveyReady(*worldState);

if (!surveyReady) {

gameState.surveyTimer = 360.0f;

return;

}

const float doctrineBoost =

(profile.doctrine == ShelterDoctrine::Industry ? 1.16f :

(profile.doctrine == ShelterDoctrine::Defense ? 1.08f : 1.0f)) *

PylonGridBoost(profile) *

CampFortificationBoost(profile);

const float infrastructurePenalty = std::max(0.68f, 1.0f - worldState->infrastructureDecay / 210.0f);

gameState.surveyTimer -= dt;

if (gameState.surveyTimer > 0.0f) {

return;

}

AddInventoryItem(profile, "trade_voucher", 1, 0.0f);

AddInventoryItem(profile, "ether_shard", 1, 0.1f);

AddInventoryItem(profile, "copper_wire", 1, 0.2f);

```



```

worldState->surveyRunsCompleted += 1;

worldState->routeContamination = std::max(0.0f, worldState->routeContamination - 2.5f);

ReduceSelectedWorldEtherErosion(profile, 4.0f, false);

gameState.surveyTimer = 360.0f / (doctrineBoost * infrastructurePenalty);

gameState.lastEvent = "Industrial survey sweep returned with route intel, trace resources, and inner spur
markers.";

}

void UpdateIndustrialOutpost(SessionProfile& profile, GameState& gameState, float dt) {

auto* worldState = FindWorldFieldState(profile, profile.selectedWorld, true);

if (worldState == nullptr || !worldState->industrialOutpostActive) {

gameState.outpostTimer = 300.0f;

return;

Success

Ran command

$ $lines=Get-Content include\SessionProfiles.hpp; $lines[40..190]; $lines[400..455]; $lines[620..750]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

inline bool IsCanonicalPipDeviceItemId(std::string_view itemId) {

return std::find(kCanonicalPipDeviceItemIds.begin(), kCanonicalPipDeviceItemIds.end(), itemId) !=

kCanonicalPipDeviceItemIds.end();

}

struct TapeEntry {

std::string tapeId;

std::string title;

bool played = false;

bool damaged = false;

bool reconstructed = false;

};

struct PassiveSkill {

std::string skillId;

std::string displayName;

```

```
bool unlocked = false;

bool equipped = false;

};

struct SpecialistEntry {

    std::string specialistId;

    std::string displayName;

    std::string role;

    std::string assignment = "unassigned";

    bool awakened = false;

};

struct LanlineServicesProfile {

    int relayCredits = 420;

    std::vector<std::string> ownedCosmetics{};

    std::vector<std::string> pendingSupportOrders{};

    bool serviceHubKnown = false;

    bool cosmeticsShopSeen = false;

};

struct LauncherAnnouncementState {

    int lastSeenBuildNumber = 0;

    std::string lastSeenAnnouncementId{};

    std::string lastSeenVersionLabel{};

};

struct WorldFieldState {

    std::string worldName;

    float etherErosion = 0.0f;

    float infrastructureDecay = 0.0f;

    bool towerSyncRecovered = false;

    bool localRelayAvailable = false;

    bool regionalGridOnline = false;

    bool caravanRouteActive = false;
```

```
bool droneStationsActive = false;

bool tradeNetworkActive = false;

bool railFreightActive = false;

bool orbitalUplinkActive = false;

bool railFortressActive = false;

bool recoveryFabricatorActive = false;

bool industrialGateUnlocked = false;

float routeContamination = 0.0f;

bool routeOverrun = false;

int caravanRunsCompleted = 0;

int droneRunsCompleted = 0;

int tradeCyclesCompleted = 0;

int purgeCycles = 0;

int railRunsCompleted = 0;

int orbitalScansCompleted = 0;

int railFortressDeployments = 0;

int fabricatorCyclesCompleted = 0;

int recoveryMilestonesClaimed = 0;

int campFortificationLevel = 0;

bool industrialSurveyActive = false;

int surveyRunsCompleted = 0;

bool industrialOutpostActive = false;

int outpostSupplyRuns = 0;

bool assemblyCellActive = false;

int assemblyCyclesCompleted = 0;

bool foundryLineActive = false;

int foundryCyclesCompleted = 0;

bool reactorYardActive = false;

int reactorCyclesCompleted = 0;

bool capacitorBankActive = false;
```

```
int capacitorDischargeCycles = 0;

bool relaySubstationActive = false;

int relaySyncCycles = 0;

bool serviceBayActive = false;

int serviceCyclesCompleted = 0;

bool waterReclaimerActive = false;

int waterCyclesCompleted = 0;

bool feyRingIntercityUnlocked = false;

bool feyRingInterserverUnlocked = false;

int relayCreditsEarned = 0;

int relayCreditsSpent = 0;

std::string activeRouteEventType{};

float routeEventTimeRemaining = 0.0f;

float routeEventCooldown = 0.0f;

float routeEventOfferTimeRemaining = 0.0f;

int routeEventProgress = 0;

int routeEventStage = 0;

int routeEventSerial = 0;

int routeEventsResolved = 0;

int routeEventsFailed = 0;

int routeEventsExpired = 0;

std::string lastRouteEventType{};

std::string lastRouteEventOutcome{};

};

struct SkillAwakeningProgress {

int archiveSyncs = 0;

int footKills = 0;

int tankActions = 0;

int stressSurvivals = 0;

int heavyCarryDrills = 0;
```

```
int fieldServiceUses = 0;

};

struct FirstPlayableRouteProgress {

bool introSeen = false;

bool emergencyMeleeRecovered = false;

bool earlyVerminEncounterResolved = false;

bool accessCardRecovered = false;

int prePipPadClueCount = 0;

bool bt72HullInspected = false;

bool bt72CoreRecovered = false;

bool bt72ServiceNotesRecovered = false;

bool bt72Restored = false;

bool clearanceBlueprintRecovered = false;

bool clearanceMaterialsRecovered = false;

bool clearanceModuleInstalled = false;

bool surfaceArrivalReached = false;

bool firstTankCombatResolved = false;

bool firstServicePerformed = false;

bool firstRecoveryNodeActivated = false;

bool debriefSummaryViewed = false;

};

struct StoryProgress {

bool awakenedFromCryo = false;

bool pipPadRecovered = false;

bool archiveRecovered = false;

bool tankLinked = false;

bool bucketRecovered = false;

bool exitedBunker = false;

bool outerRoadCleared = false;

bool relayRecovered = false;
```

```
bool returnedToBase = false;

};

enum class VehicleType {

    Motorcycle,

    Car,

    Truck,

    return "BT-72 Gunner";

}

if (normalized == "pilot") {

    return "BT-72 Pilot";

}

return "On Foot";

}

struct SessionProfile {

    AccountProfile account{};

    CharacterProfile character{};

    PartnerTankProfile partnerTank{};

    std::vector<VehicleProfile> ownedVehicles{};

    std::vector<SpecialistEntry> rescuedSpecialists{};

    std::vector<WorldFieldState> worldFieldStates{};

    LanlineServicesProfile lanlineServices{};

    LauncherAnnouncementState launcherAnnouncements{};

    ShelterDoctrine doctrine = ShelterDoctrine::Balanced;

    std::string selectedWorld = "start_zone.bwld";

    std::string sessionMode = "Solo";

    bool fieldCheckpointKnown = false;

    float fieldCheckpointX = 0.0f;

    float fieldCheckpointY = 0.0f;

    std::string fieldCheckpointWorld{};

    std::string fieldCheckpointLabel{};
```

```

int scavengerRunsCompleted = 0;

bool continuityAnchorSeeded = false;

float continuityAnchorVariance = 0.0f;

std::string activePipDeviceId{};

bool pipDeviceReselectPending = false;

std::string pipDeviceThemeld = "classic_green";

std::string pipDeviceDisplayMode = "standard";

std::string pipDeviceCarryMode = "auto";

bool pipPadExpansionCoverPresent = true;

bool blueLinkModuleRecovered = false;

bool blueLinkModuleInstalled = false;

bool hangarPowerRestored = false;

bool bt72CraneControlOnline = false;

bool bt72CranePathClear = false;

bool bt72HullAttachedToCrane = false;

bool bt72HullMovedToServiceLift = false;

bool bt72HullLockedInRestorationCradle = false;

FirstPlayableRouteProgress firstPlayableRoute{};

StoryProgress story{};

};

// Legacy/internal alias: SoulLine / Линейка Сознания.

inline bool SoulLineSeeded(const SessionProfile& profile) {

return profile.continuityAnchorSeeded;

}

inline float SoulLineVariance(const SessionProfile& profile) {

return profile.continuityAnchorVariance;

}

}

inline int CountRestoredPylonsInProfile(const SessionProfile& profile) {

int restored = 0;

```

```

for (const auto& tape : profile.character.collectedTapes) {
    if (tape.tapeld.size() >= 6 && tape.tapeld.find("_pylon") != std::string::npos) {
        restored += 1;
    }
}

return restored;
}

inline bool IsCaravanOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.caravanRouteActive &&
        HasActiveFieldCheckpoint(profile) &&
        profile.story.outerRoadCleared &&
        HasRegionalGridOnline(profile);
}

inline bool IsDroneStationOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.droneStationsActive && HasRegionalGridOnline(profile);
}

inline bool IsTradeNetworkOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.tradeNetworkActive &&
        HasActiveFieldCheckpoint(profile) &&
        HasRegionalGridOnline(profile) &&
        (worldState.caravanRouteActive || worldState.droneStationsActive);
}

inline bool IsRailFreightOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.railFreightActive &&
        HasActiveFieldCheckpoint(profile) &&
        profile.story.outerRoadCleared &&
        HasRegionalGridOnline(profile) &&
        CountRestoredPylonsInProfile(profile) >= 1 &&
        (worldState.caravanRouteActive || worldState.droneStationsActive);
}

```



```

inline bool IsOrbitalUplinkOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.orbitalUplinkActive &&
        HasRegionalGridOnline(profile) &&
        worldState.railFreightActive &&
        CountRestoredPylonsInProfile(profile) >= 2 &&
        (worldState.tradeNetworkActive || worldState.droneStationsActive);
}

inline bool IsRailFortressOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.railFortressActive &&
        worldState.railFreightActive &&
        worldState.orbitalUplinkActive &&
        HasRegionalGridOnline(profile) &&
        CountRestoredPylonsInProfile(profile) >= 2;
}

inline bool IsRecoveryFabricatorOperational(const SessionProfile& profile, const WorldFieldState& worldState) {
    return worldState.recoveryFabricatorActive &&
        HasRegionalGridOnline(profile) &&
        (worldState.railFreightActive || worldState.droneStationsActive) &&
        (worldState.tradeNetworkActive || worldState.orbitalUplinkActive);
}

inline bool IsIndustrialSurveyOperational(const WorldFieldState& worldState) {
    return worldState.industrialSurveyActive &&
        worldState.industrialGateUnlocked &&
        worldState.orbitalUplinkActive &&
        worldState.tradeNetworkActive;
}

inline bool IsIndustrialOutpostOperational(const WorldFieldState& worldState) {
    return worldState.industrialOutpostActive &&
        worldState.industrialGateUnlocked &&
        worldState.industrialSurveyActive &&

```

```
worldState.tradeNetworkActive;
}

inline bool IsAssemblyCellOperational(const WorldFieldState& worldState) {
return worldState.assemblyCellActive &&
worldState.industrialOutpostActive &&
worldState.industrialSurveyActive &&
worldState.recoveryFabricatorActive;
}

inline bool IsFoundryLineOperational(const WorldFieldState& worldState) {
return worldState.foundryLineActive &&
worldState.assemblyCellActive &&
worldState.industrialOutpostActive &&
worldState.railFortressActive;
}

inline bool IsReactorYardOperational(const WorldFieldState& worldState) {
return worldState.reactorYardActive &&
worldState.foundryLineActive &&
worldState.assemblyCellActive &&
worldState.orbitalUplinkActive;
}

inline bool IsCapacitorBankOperational(const WorldFieldState& worldState) {
return worldState.capacitorBankActive &&
worldState.reactorYardActive &&
worldState.foundryLineActive &&
worldState.orbitalUplinkActive;
}

inline bool IsRelaySubstationOperational(const WorldFieldState& worldState) {
return worldState.relaySubstationActive &&
worldState.capacitorBankActive &&
worldState.reactorYardActive &&
```

```

worldState.industrialOutpostActive;
}

inline bool IsServiceBayOperational(const WorldFieldState& worldState) {
return worldState.serviceBayActive &&
worldState.relaySubstationActive &&
worldState.foundryLineActive &&
worldState.industrialOutpostActive;
}

inline bool IsWaterReclaimerOperational(const WorldFieldState& worldState) {
return worldState.waterReclaimerActive &&
worldState.serviceBayActive &&
worldState.relaySubstationActive &&
worldState.recoveryFabricatorActive;
}

inline bool IsStableRecoveryBackbone(const SessionProfile& profile, const WorldFieldState& worldState) {
return IsRelaySubstationOperational(worldState) &&
IsServiceBayOperational(worldState) &&
IsWaterReclaimerOperational(worldState) &&
IsRecoveryFabricatorOperational(profile, worldState);
}

Success

Ran command

$ $lines=Get-Content tests\SmokeChecks.cpp; $lines[1490..1745]

[output truncated]

"Starter Backbone",
"recovery backbone smoke expected starter stage after debrief")) {
return false;
}

if (!Check(starterStatus.status.find("0/4") != std::string::npos &&
starterStatus.status.find("rail freight") != std::string::npos,

```

```

"recovery backbone smoke expected starter status to point at rail freight: " + starterStatus.status)) {
return false;
}

worldState->towerSyncRecovered = true;

worldState->localRelayAvailable = true;

worldState->regionalGridOnline = true;

worldState->caravanRouteActive = true;

worldState->railFreightActive = true;

profile.character.collectedTapes.push_back({"grid_pylon_01", "Pylon 01", false, false, false});

worldState->tradeNetworkActive = true;

worldState->orbitalUplinkActive = true;

profile.character.collectedTapes.push_back({"grid_pylon_02", "Pylon 02", false, false, false});

const auto freightStatus = bunker::CurrentRecoveryBackboneStatus(profile);

if (!Check(freightStatus.stage == "Starter Backbone",

"recovery backbone smoke expected starter stage during starter logistics")) {
return false;
}

if (!Check(freightStatus.status.find("2/4") != std::string::npos &&

freightStatus.status.find("Rail Fortress") != std::string::npos,

"recovery backbone smoke expected starter status to point at Rail Fortress: " + freightStatus.status)) {
return false;
}

if (!Check(freightStatus.payoff.find("restored spur still lacks hardened control") != std::string::npos,

"recovery backbone smoke expected starter payoff to explain missing hardened control: " + freightStatus.payoff))
{
return false;
}

worldState->railFortressActive = true;

worldState->recoveryFabricatorActive = true;

worldState->industrialGateUnlocked = true;

```

```
worldState->industrialSurveyActive = true;

worldState->industrialOutpostActive = true;

worldState->assemblyCellActive = true;

const auto innerSpurStatus = bunker::CurrentRecoveryBackboneStatus(profile);

if (!Check(innerSpurStatus.stage == "Inner Spur Expansion",
"recovery backbone smoke expected inner-spur stage after gate unlock")) {

return false;

}

if (!Check(innerSpurStatus.status.find("4/10") != std::string::npos &&
innerSpurStatus.status.find("foundry line") != std::string::npos,
"recovery backbone smoke expected inner-spur status to point at foundry line: " + innerSpurStatus.status)) {

return false;

}

if (!Check(innerSpurStatus.payoff.find("heavy plate output is still dark") != std::string::npos,
"recovery backbone smoke expected inner-spur payoff to explain foundry gap: " + innerSpurStatus.payoff)) {

return false;

}

worldState->foundryLineActive = true;

worldState->reactorYardActive = true;

worldState->capacitorBankActive = true;

worldState->relaySubstationActive = true;

worldState->serviceBayActive = true;

worldState->waterReclaimerActive = true;

const auto stableStatus = bunker::CurrentRecoveryBackboneStatus(profile);

return Check(stableStatus.stage == "Backbone Stable",
"recovery backbone smoke expected stable stage after full backbone") &&
Check(stableStatus.status.find("14/14") != std::string::npos,
"recovery backbone smoke expected stable status count after full backbone: " + stableStatus.status) &&
Check(stableStatus.payoff.find("stable recovery backbone") != std::string::npos,
"recovery backbone smoke expected stable payoff after full backbone: " + stableStatus.payoff);
```

```

}

bool RunRouteEventLifecycleSmoke() {
    bunker::SessionProfile lockedProfile = bunker::MakeDefaultSessionProfile();

    lockedProfile.selectedWorld = "route_event_locked_smoke.bwld";

    auto* lockedWorldState = bunker::FindWorldFieldState(lockedProfile, lockedProfile.selectedWorld, true);

    if (!Check(lockedWorldState != nullptr, "route event smoke expected locked world field state")) {
        return false;
    }

    bunker::GameState lockedState;

    bunker::UpdateRouteRandomEvents(lockedProfile, lockedState, 1.0f);

    if (!Check(!bunker::HasActiveRouteEvent(*lockedWorldState),
        "route event smoke expected layer to stay locked before onboarding and debrief")) {
        return false;
    }

    if (!Check(bunker::ActiveRouteEventSummary(lockedProfile).find("locked before onboarding") !=
        std::string::npos,
        "route event smoke expected locked summary before onboarding")) {
        return false;
    }

    bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();

    profile.selectedWorld = "route_event_lifecycle_smoke.bwld";

    profile.story.awakenedFromCryo = true;

    profile.story.pipPadRecovered = true;

    profile.story.archiveRecovered = true;

    profile.firstPlayableRoute.bt72Restored = true;

    profile.story.tankLinked = true;

    profile.firstPlayableRoute.clearanceBlueprintRecovered = true;

    profile.firstPlayableRoute.clearanceMaterialsRecovered = true;

    profile.firstPlayableRoute.clearanceModuleInstalled = true;

    profile.story.exitedBunker = true;

```

```
profile.firstPlayableRoute.surfaceArrivalReached = true;

profile.story.outerRoadCleared = true;

profile.firstPlayableRoute.firstTankCombatResolved = true;

profile.firstPlayableRoute.firstServicePerformed = true;

profile.story.relayRecovered = true;

profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

profile.story.returnedToBase = true;

profile.firstPlayableRoute.debriefSummaryViewed = true;

profile.partnerTank.damage.hull = 58.0f;

profile.partnerTank.energyReserve = 78.0f;

profile.partnerTank.ammoReserve = 72.0f;

auto* worldState = bunker::FindWorldFieldState(profile, profile.selectedWorld, true);

if (!Check(worldState != nullptr, "route event smoke expected world field state")) {

return false;

}

bunker::GameState gameState;

bunker::UpdateRouteRandomEvents(profile, gameState, 1.0f);

if (!Check(worldState->activeRouteEventType == "service_call",

"route event smoke expected deterministic service-call spawn")) {

return false;

}

if (!Check(worldState->routeEventOfferTimeRemaining > 0.0f,

"route event smoke expected offered state before activation")) {

return false;

}

if (!Check(bunker::ActiveRouteEventSummary(profile).find("offered") != std::string::npos,

"route event smoke expected offered route-event summary")) {

return false;

}

bunker::UpdateRouteRandomEvents(profile, gameState, 20.0f);
```

```

if (!Check(worldState->routeEventOfferTimeRemaining <= 0.0f,
"route event smoke expected offer timer to collapse into active state")) {
return false;
}

if (!Check(bunker::ActiveRouteEventSummary(profile).find("active") != std::string::npos,
"route event smoke expected active route-event summary after offer")) {
return false;
}

profile.character.awakening.fieldServiceUses = 1;

profile.partnerTank.damage.hull = 84.0f;

profile.partnerTank.energyReserve = 81.0f;

profile.partnerTank.ammoReserve = 77.0f;

bunker::UpdateRouteRandomEvents(profile, gameState, 1.0f);

return Check(!bunker::HasActiveRouteEvent(*worldState), "route event smoke expected service-call resolution")
&&

Check(worldState->routeEventsResolved == 1, "route event smoke expected resolved counter increment") &&

Check(worldState->routeEventCooldown > 0.0f, "route event smoke expected post-resolution cooldown") &&

Check(gameState.lastEvent.find("ROUTE EVENT RESOLVED") != std::string::npos,
"route event smoke expected resolution event copy") &&

Check(bunker::ActiveRouteEventSummary(profile).find("success") != std::string::npos &&
bunker::ActiveRouteEventSummary(profile).find("cooldown") != std::string::npos,
"route event smoke expected success-plus-cooldown summary after resolution");
}

bool RunMerchantRouteEventSmoke() {

auto countInventory = [](const bunker::SessionProfile& profile, const std::string& itemId) {

int total = 0;

for (const auto& entry : profile.character.inventory) {

if (entry.itemId == itemId) {

total += entry.count;

}

}
}

```



```
}  
  
return total;  
  
};  
  
bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();  
  
profile.selectedWorld = "merchant_route_event_smoke.bwld";  
  
profile.story.awakenedFromCryo = true;  
  
profile.story.pipPadRecovered = true;  
  
profile.story.archiveRecovered = true;  
  
profile.firstPlayableRoute.bt72Restored = true;  
  
profile.story.tankLinked = true;  
  
profile.firstPlayableRoute.clearanceBlueprintRecovered = true;  
  
profile.firstPlayableRoute.clearanceMaterialsRecovered = true;  
  
profile.firstPlayableRoute.clearanceModuleInstalled = true;  
  
profile.story.exitedBunker = true;  
  
profile.firstPlayableRoute.surfaceArrivalReached = true;  
  
profile.story.outerRoadCleared = true;  
  
profile.firstPlayableRoute.firstTankCombatResolved = true;  
  
profile.firstPlayableRoute.firstServicePerformed = true;  
  
profile.story.relayRecovered = true;  
  
profile.firstPlayableRoute.firstRecoveryNodeActivated = true;  
  
profile.story.returnedToBase = true;  
  
profile.firstPlayableRoute.debriefSummaryViewed = true;  
  
profile.lanlineServices.relayCredits = 150;  
  
profile.character.inventory.push_back({"trade_voucher", 1, 0.0f});  
  
profile.character.inventory.push_back({"power_cell", 1, 0.3f});  
  
profile.partnerTank.damage.hull = 100.0f;  
  
profile.partnerTank.energyReserve = 94.0f;  
  
profile.partnerTank.ammoReserve = 92.0f;  
  
auto* worldState = bunker::FindWorldFieldState(profile, profile.selectedWorld, true);  
  
if (!Check(worldState != nullptr, "merchant route event smoke expected world field state")) {
```

```

return false;
}

worldState->tradeNetworkActive = true;

worldState->serviceBayActive = true;

worldState->serviceCyclesCompleted = 1;

worldState->localRelayAvailable = true;

worldState->regionalGridOnline = true;

worldState->routeContamination = 4.0f;

worldState->infrastructureDecay = 6.0f;

worldState->routeOverrun = false;
}

if (!Check(freightStatus.status.find("2/4") != std::string::npos &&
freightStatus.status.find("Rail Fortress") != std::string::npos,
"recovery backbone smoke expected starter status to point at Rail Fortress: " + freightStatus.status)) {
return false;
}

if (!Check(freightStatus.payoff.find("restored spur still lacks hardened control") != std::string::npos,
"recovery backbone smoke expected starter payoff to explain missing hardened control: " + freightStatus.payoff))
{
return false;
}

worldState->railFortressActive = true;

worldState->recoveryFabricatorActive = true;

worldState->industrialGateUnlocked = true;

worldState->industrialSurveyActive = true;

worldState->industrialOutpostActive = true;

worldState->assemblyCellActive = true;

const auto innerSpurStatus = bunker::CurrentRecoveryBackboneStatus(profile);

if (!Check(innerSpurStatus.stage == "Inner Spur Expansion",
"recovery backbone smoke expected inner-spur stage after gate unlock")) {

```

```

return false;
}

if (!Check(innerSpurStatus.status.find("4/10") != std::string::npos &&
innerSpurStatus.status.find("foundry line") != std::string::npos,
"recovery backbone smoke expected inner-spur status to point at foundry line: " + innerSpurStatus.status)) {
return false;
}

if (!Check(innerSpurStatus.payoff.find("heavy plate output is still dark") != std::string::npos,
"recovery backbone smoke expected inner-spur payoff to explain foundry gap: " + innerSpurStatus.payoff)) {
return false;
}

worldState->foundryLineActive = true;
worldState->reactorYardActive = true;
worldState->capacitorBankActive = true;
worldState->relaySubstationActive = true;
worldState->serviceBayActive = true;
worldState->waterReclaimerActive = true;

const auto stableStatus = bunker::CurrentRecoveryBackboneStatus(profile);
return Check(stableStatus.stage == "Backbone Stable",
"recovery backbone smoke expected stable stage after full backbone") &&
Check(stableStatus.status.find("14/14") != std::string::npos,
"recovery backbone smoke expected stable status count after full backbone: " + stableStatus.status) &&
Check(stableStatus.payoff.find("stable recovery backbone") != std::string::npos,
"recovery backbone smoke expected stable payoff after full backbone: " + stableStatus.payoff);
}

bool RunRouteEventLifecycleSmoke() {
bunker::SessionProfile lockedProfile = bunker::MakeDefaultSessionProfile();
lockedProfile.selectedWorld = "route_event_locked_smoke.bwld";
auto* lockedWorldState = bunker::FindWorldFieldState(lockedProfile, lockedProfile.selectedWorld, true);
if (!Check(lockedWorldState != nullptr, "route event smoke expected locked world field state")) {

```

```
return false;
}

bunker::GameState lockedState;

bunker::UpdateRouteRandomEvents(lockedProfile, lockedState, 1.0f);

if (!Check(!bunker::HasActiveRouteEvent(*lockedWorldState),
"route event smoke expected layer to stay locked before onboarding and debrief")) {
return false;
}

if (!Check(bunker::ActiveRouteEventSummary(lockedProfile).find("locked before onboarding") !=
std::string::npos,
"route event smoke expected locked summary before onboarding")) {
return false;
}

bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();
profile.selectedWorld = "route_event_lifecycle_smoke.bwld";
profile.story.awakenedFromCryo = true;
profile.story.pipPadRecovered = true;
profile.story.archiveRecovered = true;
profile.firstPlayableRoute.bt72Restored = true;
profile.story.tankLinked = true;
profile.firstPlayableRoute.clearanceBlueprintRecovered = true;
profile.firstPlayableRoute.clearanceMaterialsRecovered = true;
profile.firstPlayableRoute.clearanceModuleInstalled = true;
profile.story.exitedBunker = true;
profile.firstPlayableRoute.surfaceArrivalReached = true;
profile.story.outerRoadCleared = true;
profile.firstPlayableRoute.firstTankCombatResolved = true;
profile.firstPlayableRoute.firstServicePerformed = true;
profile.story.relayRecovered = true;
profile.firstPlayableRoute.firstRecoveryNodeActivated = true;
```

```
profile.story.returnedToBase = true;

profile.firstPlayableRoute.debriefSummaryViewed = true;

profile.partnerTank.damage.hull = 58.0f;

profile.partnerTank.energyReserve = 78.0f;

profile.partnerTank.ammoReserve = 72.0f;

auto* worldState = bunker::FindWorldFieldState(profile, profile.selectedWorld, true);

if (!Check(worldState != nullptr, "route event smoke expected world field state")) {

return false;

}

bunker::GameState gameState;

bunker::UpdateRouteRandomEvents(profile, gameState, 1.0f);

if (!Check(worldState->activeRouteEventType == "service_call",

"route event smoke expected deterministic service-call spawn")) {

return false;

}

if (!Check(worldState->routeEventOfferTimeRemaining > 0.0f,

"route event smoke expected offered state before activation")) {

return false;

}

if (!Check(bunker::ActiveRouteEventSummary(profile).find("offered") != std::string::npos,

"route event smoke expected offered route-event summary")) {

return false;

}

bunker::UpdateRouteRandomEvents(profile, gameState, 20.0f);

if (!Check(worldState->routeEventOfferTimeRemaining <= 0.0f,

"route event smoke expected offer timer to collapse into active state")) {

return false;

}

if (!Check(bunker::ActiveRouteEventSummary(profile).find("active") != std::string::npos,

"route event smoke expected active route-event summary after offer")) {
```

```

return false;
}

profile.character.awakening.fieldServiceUses = 1;

profile.partnerTank.damage.hull = 84.0f;

profile.partnerTank.energyReserve = 81.0f;

profile.partnerTank.ammoReserve = 77.0f;

bunker::UpdateRouteRandomEvents(profile, gameState, 1.0f);

return Check(!bunker::HasActiveRouteEvent(*worldState), "route event smoke expected service-call resolution")
&&

Check(worldState->routeEventsResolved == 1, "route event smoke expected resolved counter increment") &&

Check(worldState->routeEventCooldown > 0.0f, "route event smoke expected post-resolution cooldown") &&

Check(gameState.lastEvent.find("ROUTE EVENT RESOLVED") != std::string::npos,

"route event smoke expected resolution event copy") &&

Check(bunker::ActiveRouteEventSummary(profile).find("success") != std::string::npos &&

bunker::ActiveRouteEventSummary(profile).find("cooldown") != std::string::npos,

"route event smoke expected success-plus-cooldown summary after resolution");

}

bool RunMerchantRouteEventSmoke() {

auto countInventory = [](const bunker::SessionProfile& profile, const std::string& itemId) {

int total = 0;

for (const auto& entry : profile.character.inventory) {

if (entry.itemId == itemId) {

total += entry.count;

}

}

}

return total;

};

bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();

profile.selectedWorld = "merchant_route_event_smoke.bwld";

profile.story.awakenedFromCryo = true;

```

```
profile.story.pipPadRecovered = true;

profile.story.archiveRecovered = true;

profile.firstPlayableRoute.bt72Restored = true;

profile.story.tankLinked = true;

profile.firstPlayableRoute.clearanceBlueprintRecovered = true;

profile.firstPlayableRoute.clearanceMaterialsRecovered = true;

profile.firstPlayableRoute.clearanceModuleInstalled = true;

profile.story.exitedBunker = true;

profile.firstPlayableRoute.surfaceArrivalReached = true;

profile.story.outerRoadCleared = true;

profile.firstPlayableRoute.firstTankCombatResolved = true;

profile.firstPlayableRoute.firstServicePerformed = true;

profile.story.relayRecovered = true;

profile.firstPlayableRoute.firstRecoveryNodeActivated = true;

profile.story.returnedToBase = true;

profile.firstPlayableRoute.debriefSummaryViewed = true;

profile.lanlineServices.relayCredits = 150;

profile.character.inventory.push_back({"trade_voucher", 1, 0.0f});

profile.character.inventory.push_back({"power_cell", 1, 0.3f});

profile.partnerTank.damage.hull = 100.0f;

profile.partnerTank.energyReserve = 94.0f;

profile.partnerTank.ammoReserve = 92.0f;

auto* worldState = bunker::FindWorldFieldState(profile, profile.selectedWorld, true);

if (!Check(worldState != nullptr, "merchant route event smoke expected world field state")) {

return false;

}

worldState->tradeNetworkActive = true;

worldState->serviceBayActive = true;

worldState->serviceCyclesCompleted = 1;

worldState->localRelayAvailable = true;
```

worldState->regionalGridOnline = true;

worldState->routeContamination = 4.0f;

worldState->infrastructureDecay = 6.0f;

worldState->routeOverrun = false;

Success

Ran command

```
$ git grep -n "fieldServiceUses\|firstServicePerformed\|Try.*Service\|Field service" -- include src tests
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
include/HangarSystem.hpp:218:inline bool TryConsumeBestTankServiceKit(SessionProfile& profile, std::string*  
eventText = nullptr) {
```

```
include/SessionProfiles.hpp:153: int fieldServiceUses = 0;
```

```
include/SessionProfiles.hpp:171: bool firstServicePerformed = false;
```

```
src/GameRuntime.cpp:1881: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";
```

```
src/GameRuntime.cpp:1885: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the gunner  
station.";
```

```
src/GameRuntime.cpp:1889: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";
```

```
src/GameRuntime.cpp:1898: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable  
condition.";
```

```
src/GameRuntime.cpp:1902: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a repair  
patch.";
```

```
src/GameRuntime.cpp:1923: ? "Field service rack cycled. Scavenger-side engineer tuning improved the repair  
pass."
```

```
src/GameRuntime.cpp:1924: : "Field service rack cycled. BT-72 patched and partially resupplied in the field.";
```

```
src/GameRuntime.cpp:1926: if (profile.firstPlayableRoute.firstTankCombatResolved &&  
!profile.firstPlayableRoute.firstServicePerformed) {
```

```
src/GameRuntime.cpp:1927: profile.firstPlayableRoute.firstServicePerformed = true;
```

```
src/GameRuntime.cpp:2736: if (TryConsumeBestTankServiceKit(profile, &repairEvent)) {
```

```
src/GameRuntime.cpp:4966: if (profile.character.awakening.fieldServiceUses > 0 ||
```

```
src/GameRuntime.cpp:5029: profile.firstPlayableRoute.firstServicePerformed) {
```

```
src/GameRuntime.cpp:5174: if (profile.firstPlayableRoute.firstServicePerformed &&
```

```
src/GameRuntime.cpp:6395: if (!profile.firstPlayableRoute.firstServicePerformed) {
```



```
src/GameRuntime.cpp:6579: if (profile.firstPlayableRoute.firstTankCombatResolved &&
!profile.firstPlayableRoute.firstServicePerformed) {

src/GameRuntime.cpp:6580: profile.firstPlayableRoute.firstServicePerformed = true;

src/GameRuntime.cpp:6694: const int fieldServiceUses = profile.character.awakening.fieldServiceUses;

src/GameRuntime.cpp:6695: const int cyclesRemaining = std::max(0, 3 - fieldServiceUses);

src/GameRuntime.cpp:6696: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service cycles
logged", std::min(fieldServiceUses, 3));

src/GameRuntime.cpp:6698: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the recipe.",
cyclesRemaining, cyclesRemaining == 1 ? "" : "s");

src/GameRuntime.cpp:7190: : specialist.displayName + " reassigned to scavenger support. Field service and
salvage teams now get the engineer boost.";

src/GameRuntimePipPad.cpp:385: gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";

src/GameRuntimePipPad.cpp:389: gameState.lastEvent = "Field service requires BT-72 pilot controls, not the
gunner station.";

src/GameRuntimePipPad.cpp:393: gameState.lastEvent = "Field service rack still cycling. Give it a moment.";

src/GameRuntimePipPad.cpp:402: gameState.lastEvent = "Field service rack reports BT-72 already in acceptable
condition.";

src/GameRuntimePipPad.cpp:406: gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a
repair patch.";

src/GameRuntimePipPad.cpp:425: ? "Field service rack cycled. Scavenger-side engineer tuning improved the
repair pass."

src/GameRuntimePipPad.cpp:426: : "Field service rack cycled. BT-72 patched and partially resupplied in the
field.";

src/GameRuntimePipPad.cpp:427: if (profile.firstPlayableRoute.firstTankCombatResolved &&
!profile.firstPlayableRoute.firstServicePerformed) {

src/GameRuntimePipPad.cpp:428: profile.firstPlayableRoute.firstServicePerformed = true;

src/GameRuntimePipPad.cpp:532: const int fieldServiceUses = profile.character.awakening.fieldServiceUses;

src/GameRuntimePipPad.cpp:533: const int cyclesRemaining = std::max(0, 3 - fieldServiceUses);

src/GameRuntimePipPad.cpp:534: ImGui::TextDisabled("Awakened repair recipe dormant: %d/3 field service
cycles logged", std::min(fieldServiceUses, 3));

src/GameRuntimePipPad.cpp:536: ImGui::TextDisabled("Field service needs %d more cycle%s to surface the
recipe.", cyclesRemaining, cyclesRemaining == 1 ? "" : "s");

src/SessionProfiles.cpp:75: route.firstServicePerformed = route.firstServicePerformed ||

src/SessionProfiles.cpp:78: profile.character.awakening.fieldServiceUses > 0;
```

```
src/SessionProfiles.cpp:119: route.firstServicePerformed = profile.story.relayRecovered ||
src/SessionProfiles.cpp:121: profile.character.awakening.fieldServiceUses > 0;
src/SessionProfiles.cpp:450: out << "route_first_service=" << (profile.firstPlayableRoute.firstServicePerformed ? 1
: 0) << '\n';
src/SessionProfiles.cpp:458: out << "awakening_field_service=" << profile.character.awakening.fieldServiceUses
<< '\n';
src/SessionProfiles.cpp:684: else if (key == "route_first_service")
outProfile.firstPlayableRoute.firstServicePerformed = (std::stoi(value) != 0);
src/SessionProfiles.cpp:692: else if (key == "awakening_field_service")
outProfile.character.awakening.fieldServiceUses = std::stoi(value);
src/SkillSystem.cpp:90: profile.character.awakening.fieldServiceUses += 1;
src/SkillSystem.cpp:91: if (profile.character.awakening.fieldServiceUses >= 3 && !HasRecipeItem(profile,
"recipe_repair_patch")) {
src/StoryRoute.cpp:98: return profile.firstPlayableRoute.firstServicePerformed ||
src/StoryRoute.cpp:101: profile.character.awakening.fieldServiceUses > 0;
src/StoryRoute.cpp:1009: {"Take a first service/rest cycle.", profile.firstPlayableRoute.firstServicePerformed},
tests/SmokeChecks.cpp:760: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:959: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1046: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1247: return Check(profile.firstPlayableRoute.firstServicePerformed, "workshop handoff
smoke expected first service flag") &&
tests/SmokeChecks.cpp:1416: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1456: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1519: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1644: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1682: profile.character.awakening.fieldServiceUses = 1;
tests/SmokeChecks.cpp:1723: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1791: acidProfile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1841: fogProfile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:1893: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:2736: profile.firstPlayableRoute.firstServicePerformed = true;
tests/SmokeChecks.cpp:2934: if (!Check(bunker::TryConsumeBestTankServiceKit(profile, &eventText),
```

```
tests/SmokeChecks.cpp:2951: if (!Check(bunker::TryConsumeBestTankServiceKit(profile, &eventText),
tests/SmokeChecks.cpp:2968: if (!Check(bunker::TryConsumeBestTankServiceKit(profile, &eventText),
tests/SmokeChecks.cpp:2985: if (!Check(bunker::TryConsumeBestTankServiceKit(profile, &eventText),
tests/SmokeChecks.cpp:3006: if (!Check(!bunker::TryConsumeBestTankServiceKit(profile, &eventText),
tests/SmokeChecks.cpp:3018: return Check(!bunker::TryConsumeBestTankServiceKit(idleProfile, &idleEvent),
Success
```

Ran command

```
$ git grep -n
```

```
"HasActiveFieldCheckpoint\|fieldCheckpointKnown\|ActiveFieldCheckpoint\|scavengerReady\|regionalGridOnline" -- include src tests
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
include/SessionProfiles.hpp:90: bool regionalGridOnline = false;
```

```
include/SessionProfiles.hpp:422: bool fieldCheckpointKnown = false;
```

```
include/SessionProfiles.hpp:457:inline bool HasActiveFieldCheckpoint(const SessionProfile& profile) {
```

```
include/SessionProfiles.hpp:458: return profile.fieldCheckpointKnown && !profile.fieldCheckpointWorld.empty()
&&
```

```
include/SessionProfiles.hpp:520: target.regionalGridOnline = target.regionalGridOnline ||
source.regionalGridOnline;
```

```
include/SessionProfiles.hpp:616: if (state->regionalGridOnline || state->towerSyncRecovered) {
```

```
include/SessionProfiles.hpp:635: HasActiveFieldCheckpoint(profile) &&
```

```
include/SessionProfiles.hpp:646: HasActiveFieldCheckpoint(profile) &&
```

```
include/SessionProfiles.hpp:653: HasActiveFieldCheckpoint(profile) &&
```

```
src/GameRuntime.cpp:1823: profile.fieldCheckpointKnown = true;
```

```
src/GameRuntime.cpp:2191: if (HasActiveFieldCheckpoint(profile)) score += 8.0f;
```

```
src/GameRuntime.cpp:2302: worldState.regionalGridOnline = true;
```

```
src/GameRuntime.cpp:3970: if (IsRegionalGridOnline(profile) && HasActiveFieldCheckpoint(profile) &&
worldState.etherErosion > 0.0f) {
```

```
src/GameRuntime.cpp:3983: if (HasActiveFieldCheckpoint(profile)) {
```

```
src/GameRuntime.cpp:3991: const float decayRate = HasActiveFieldCheckpoint(profile) ? 0.08f : 0.04f;
```

```
src/GameRuntime.cpp:4020: float recoveryRate = HasActiveFieldCheckpoint(profile) ? 0.08f : 0.05f;
```

```
src/GameRuntime.cpp:4107: const bool scavengerReady =
```

```
src/GameRuntime.cpp:4108: HasActiveFieldCheckpoint(profile) &&
src/GameRuntime.cpp:4112: if (!scavengerReady) {
src/GameRuntime.cpp:4995: worldState.regionalGridOnline ||
src/GameRuntime.cpp:5189: !worldState.regionalGridOnline)) {
src/GameRuntime.cpp:7146: if (HasActiveFieldCheckpoint(profile)) {
src/GameRuntime.cpp:7245: const bool scavengerReady =
src/GameRuntime.cpp:7246: HasActiveFieldCheckpoint(profile) &&
src/GameRuntime.cpp:7251: if (scavengerReady) {
src/SessionProfiles.cpp:99: if (profile.fieldCheckpointKnown && profile.fieldCheckpointWorld.empty()) {
src/SessionProfiles.cpp:189: if (!state.regionalGridOnline && !state.towerSyncRecovered) {
src/SessionProfiles.cpp:313: if (profile.fieldCheckpointKnown && profile.fieldCheckpointWorld.empty()) {
src/SessionProfiles.cpp:381: out << "field_checkpoint_known=" << (profile.fieldCheckpointKnown ? 1 : 0) << '\n';
src/SessionProfiles.cpp:484: << (worldState.regionalGridOnline ? 1 : 0) << ','
src/SessionProfiles.cpp:620: else if (key == "field_checkpoint_known") outProfile.fieldCheckpointKnown =
(std::stoi(value) != 0);
src/SessionProfiles.cpp:806: state.regionalGridOnline = hasLanlineFields ? (std::stoi(parts[5]) != 0) : false;
src/main.cpp:416: if (bunker::HasActiveFieldCheckpoint(sessionProfile)) {
src/main.cpp:573: const bool cooledByGrid = bunker::HasActiveFieldCheckpoint(sessionProfile) &&
src/main.cpp:839: if (bunker::HasActiveFieldCheckpoint(sessionProfile)) {
src/main.cpp:866: gameState.lastEvent = bunker::HasActiveFieldCheckpoint(sessionProfile)
tests/SmokeChecks.cpp:312: savedProfile.fieldCheckpointKnown = true;
tests/SmokeChecks.cpp:1461: profile.fieldCheckpointKnown = true;
tests/SmokeChecks.cpp:1475: worldState->regionalGridOnline = true;
tests/SmokeChecks.cpp:1531: profile.fieldCheckpointKnown = true;
tests/SmokeChecks.cpp:1551: worldState->regionalGridOnline = true;
tests/SmokeChecks.cpp:1743: worldState->regionalGridOnline = true;
tests/SmokeChecks.cpp:1863: fogWorld->regionalGridOnline = false;
tests/SmokeChecks.cpp:1898: profile.fieldCheckpointKnown = true;
tests/SmokeChecks.cpp:1914: alphaWorld->regionalGridOnline = true;
tests/SmokeChecks.cpp:2722: profile.fieldCheckpointKnown = true;
```

```
tests/SmokeChecks.cpp:2750: worldState->regionalGridOnline = true;
```

Success

Ran command

```
$ git grep -n "SaveProfileAtomically\|ContainsText\|ReadTextFile\|WriteTextFile\|current_path" --  
tests/SmokeChecks.cpp
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
tests/SmokeChecks.cpp:40: : original(fs::current_path()) {
```

```
tests/SmokeChecks.cpp:41: fs::current_path(target);
```

```
tests/SmokeChecks.cpp:46: fs::current_path(original, ec);
```

```
tests/SmokeChecks.cpp:83:bool WriteTextFile(const fs::path& path, const std::string& text) {
```

```
tests/SmokeChecks.cpp:92:std::string ReadTextFile(const fs::path& path) {
```

```
tests/SmokeChecks.cpp:103:bool ContainsText(const std::string& text, const std::string& needle) {
```

```
tests/SmokeChecks.cpp:357: const auto saveStatus = bunker::SaveProfileAtomically(savedProfile,  
bunker::DefaultSessionProfilePath());
```

```
tests/SmokeChecks.cpp:409: const fs::path tempRoot = fs::current_path() / "profile_migration_contract_smoke";
```

```
tests/SmokeChecks.cpp:418: if (!Check(WriteTextFile(v9Path,
```

```
tests/SmokeChecks.cpp:461: if (!Check(WriteTextFile(v10Path,
```

```
tests/SmokeChecks.cpp:503: if (!Check(WriteTextFile(restoredPath,
```

```
tests/SmokeChecks.cpp:562: const auto saveStatus = bunker::SaveProfileAtomically(currentProfile, currentPath);
```

```
tests/SmokeChecks.cpp:587: const std::string savedText = ReadTextFile(currentPath);
```

```
tests/SmokeChecks.cpp:588: const bool ok = Check(ContainsText(savedText, "profile_format=BPF1\n") &&
```

```
tests/SmokeChecks.cpp:589: ContainsText(savedText, "profile_version=13\n") &&
```

```
tests/SmokeChecks.cpp:590: ContainsText(savedText, "active_pip_device_id=") &&
```

```
tests/SmokeChecks.cpp:591: ContainsText(savedText, "pip_device_reselect_pending=") &&
```

```
tests/SmokeChecks.cpp:592: ContainsText(savedText, "pip_device_theme_id=") &&
```

```
tests/SmokeChecks.cpp:593: ContainsText(savedText, "pip_device_display_mode=") &&
```

```
tests/SmokeChecks.cpp:594: ContainsText(savedText, "pip_device_carry_mode=") &&
```

```
tests/SmokeChecks.cpp:595: ContainsText(savedText, "pippad_expansion_cover_present=") &&
```

```
tests/SmokeChecks.cpp:596: ContainsText(savedText, "bluelink_module_recovered=") &&
```

```
tests/SmokeChecks.cpp:597: ContainsText(savedText, "bluelink_module_installed=") &&
```

tests/SmokeChecks.cpp:598: ContainsText(savedText, "hangar_power_restored=") &&

tests/SmokeChecks.cpp:599: ContainsText(savedText, "bt72_crane_control_online=") &&

tests/SmokeChecks.cpp:600: ContainsText(savedText, "bt72_crane_path_clear=") &&

tests/SmokeChecks.cpp:601: ContainsText(savedText, "bt72_hull_attached_to_crane=") &&

tests/SmokeChecks.cpp:602: ContainsText(savedText, "bt72_hull_moved_to_service_lift=") &&

tests/SmokeChecks.cpp:603: ContainsText(savedText, "bt72_hull_locked_in_restoration_cradle="),

tests/SmokeChecks.cpp:633: const fs::path tempRoot = fs::current_path() / "continuity_anchor_contract_smoke";

tests/SmokeChecks.cpp:642: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:1374: const auto profileSave = bunker::SaveProfileAtomically(runtimeProfile, bunker::DefaultSessionProfilePath());

tests/SmokeChecks.cpp:5324: const fs::path tempRoot = fs::current_path() / "game_execution_resource_lookup";

tests/SmokeChecks.cpp:5446: const fs::path tempRoot = fs::current_path() / "game_execution_script_bridge";

tests/SmokeChecks.cpp:5620: const fs::path tempRoot = fs::current_path() / "bluelink_expansion_contract_smoke";

tests/SmokeChecks.cpp:5628: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:5771: const fs::path tempRoot = fs::current_path() / "bt72_crane_contract_smoke";

tests/SmokeChecks.cpp:5779: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:6184: const fs::path tempRoot = fs::current_path() / "starter_world_route_persistence_smoke";

tests/SmokeChecks.cpp:6293: const fs::path tempRoot = fs::current_path() / "starter_identity_after_pippad_pickup_smoke";

tests/SmokeChecks.cpp:6348: const fs::path tempRoot = fs::current_path() / "loaded_starter_route_after_pippad_pickup_smoke";

tests/SmokeChecks.cpp:6475: const fs::path tempRoot = fs::current_path() / "loaded_profile_and_world_after_pippad_pickup_smoke";

tests/SmokeChecks.cpp:6516: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:6611: const auto finalSaveStatus = bunker::SaveProfileAtomically(loadedProfile, profilePath);

tests/SmokeChecks.cpp:6665: const fs::path tempRoot = fs::current_path() / "pip_device_selection_station_roundtrip_smoke";

tests/SmokeChecks.cpp:6692: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

```
tests/SmokeChecks.cpp:6839: const fs::path tempRoot = fs::current_path() /
"pip_device_station_retired_after_exit_smoke";

tests/SmokeChecks.cpp:6847: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:7112: const fs::path tempRoot = fs::current_path() /
"active_pip_device_customization_state_smoke";

tests/SmokeChecks.cpp:7120: const auto saveStatus = bunker::SaveProfileAtomically(configuredProfile,
profilePath);

tests/SmokeChecks.cpp:7263: const fs::path tempRoot = fs::current_path() /
"active_pip_device_customization_command_smoke";

tests/SmokeChecks.cpp:7271: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:7366: const fs::path tempRoot = fs::current_path() /
"active_pip_device_customization_capability_rules_smoke";

tests/SmokeChecks.cpp:7374: const auto saveStatus = bunker::SaveProfileAtomically(pipBoy3000Profile,
profilePath);

tests/SmokeChecks.cpp:7508: const fs::path tempRoot = fs::current_path() /
"active_pip_device_customization_runtime_binding_smoke";

tests/SmokeChecks.cpp:7516: const auto saveStatus = bunker::SaveProfileAtomically(pippadProfile, profilePath);

tests/SmokeChecks.cpp:7604: const fs::path tempRoot = fs::current_path() /
"pip_device_selection_helper_contract_smoke";

tests/SmokeChecks.cpp:7612: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:7780: const fs::path tempRoot = fs::current_path() /
"pip_device_item_registry_station_options_smoke";

tests/SmokeChecks.cpp:7789: auto classicSaveStatus = bunker::SaveProfileAtomically(classicProfile, classicPath);

tests/SmokeChecks.cpp:7790: auto markViSaveStatus = bunker::SaveProfileAtomically(markViProfile,
markViPath);

tests/SmokeChecks.cpp:7854: const fs::path tempRoot = fs::current_path() /
"pip_device_profile_normalization_registry_smoke";

tests/SmokeChecks.cpp:7869: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:7941: const fs::path tempRoot = fs::current_path() /
"active_pip_device_capability_rules_smoke";

tests/SmokeChecks.cpp:7949: const auto saveStatus = bunker::SaveProfileAtomically(pipBoy3000Profile,
profilePath);

tests/SmokeChecks.cpp:8043: const fs::path tempRoot = fs::current_path() /
"starter_world_registry_identity_roundtrip_smoke";
```

```
tests/SmokeChecks.cpp:8083: const fs::path tempRoot = fs::current_path() /
"loaded_starter_world_route_smoke";

tests/SmokeChecks.cpp:8321: const fs::path tempRoot = fs::current_path() / "first_playable_route_e2e_smoke";

tests/SmokeChecks.cpp:8329: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:8407: const fs::path tempRoot = fs::current_path() / "pip_pad_access_gating_smoke";

tests/SmokeChecks.cpp:8415: const auto saveStatus = bunker::SaveProfileAtomically(profile, profilePath);

tests/SmokeChecks.cpp:8444: const fs::path sandboxRoot = fs::current_path() / ".bunker_smoke_checks";
```

Success

Ran command

```
$ $lines=Get-Content src\GameRuntime.cpp; $lines[1790..1935]
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
bool IsNearTaggedObject(const World& world,
```

```
float playerX,
```

```
float playerY,
```

```
const std::string& scriptTag,
```

```
float maxDistance) {
```

```
const auto* object = world.FindObjectByScriptTag(scriptTag);
```

```
if (object == nullptr) {
```

```
return false;
```

```
}
```

```
const float dx = object->x - playerX;
```

```
const float dy = object->y - playerY;
```

```
return (dx * dx + dy * dy) <= (maxDistance * maxDistance);
```

```
}
```

```
float DistanceSqToTankAnchor(const SessionProfile& profile, const MapObject& object) {
```

```
const float dx = object.x - profile.partnerTank.worldX;
```

```
const float dy = object.y - profile.partnerTank.worldY;
```

```
return (dx * dx) + (dy * dy);
```

```
}
```

```
bool IsTankNearServicePoint(const SessionProfile& profile, const MapObject& object, float radius) {
```



```

if (!profile.partnerTank.worldPositionKnown) {
    return false;
}

return DistanceSqToTankAnchor(profile, object) <= (radius * radius);
}

void ActivateFieldCheckpoint(const MapObject& campObject,
    SessionProfile& profile,
    const std::string& worldName) {
    profile.fieldCheckpointKnown = true;
    profile.fieldCheckpointX = campObject.x;
    profile.fieldCheckpointY = campObject.y;
    profile.fieldCheckpointWorld = NormalizeWorldReference(worldName);
    profile.fieldCheckpointLabel = campObject.displayName;
}

void AddRescuedSpecialist(SessionProfile& profile,
    const std::string& specialistId,
    const std::string& displayName,
    const std::string& role) {
    for (auto& specialist : profile.rescuedSpecialists) {
        if (specialist.specialistId == specialistId) {
            specialist.awakened = true;

            if (specialist.displayName.empty()) {
                specialist.displayName = displayName;
            }

            if (specialist.role.empty()) {
                specialist.role = role;
            }

            if (specialist.assignment.empty() || specialist.assignment == "unassigned") {
                specialist.assignment = role == "engineer" ? "workshop" : "field";
            }
        }
    }
}

```

```

return;
}
}

profile.rescuedSpecialists.push_back({specialistId, displayName, role, role == "engineer" ? "workshop" : "field",
true});

}

bool TryApplyTankEnergySiphon(const MapObject& object,
PlayerState& player,
SessionProfile& profile,
GameState& gameState,
float radius,
float burstCharge,
const char* successMessage,
const char* fullMessage) {
if (!player.insideTank || !IsTankNearServicePoint(profile, object, radius)) {
return false;
}

if (profile.partnerTank.energyReserve >= 100.0f) {
gameState.lastEvent = fullMessage;
return true;
}

profile.partnerTank.energyReserve = std::min(100.0f, profile.partnerTank.energyReserve + burstCharge);
gameState.lastEvent = successMessage;
return true;
}

bool IsContextualAction(const MapObject& object) {
return object.interaction == InteractionType::VehicleAnchor ||
object.interaction == InteractionType::Transition;
}

bool TryRunFieldWorkbench(PlayerState& player,

```

```

SessionProfile& profile,
GameState& gameState) {
    if (!player.insideTank) {
        gameState.lastEvent = "Field service requires an active BT-72 cockpit link.";
        return false;
    }

    if (player.bt72GunnerSeat) {
        gameState.lastEvent = "Field service requires BT-72 pilot controls, not the gunner station.";
        return false;
    }

    if (gameState.fieldWorkbenchCooldown > 0.0f) {
        gameState.lastEvent = "Field service rack still cycling. Give it a moment.";
        return false;
    }

    const float momentum = std::sqrt((player.velocityX * player.velocityX) + (player.velocityY * player.velocityY));
    if (momentum > 1.0f) {
        gameState.lastEvent = "BT-72 is moving too fast for field service. Stop the hull first.";
        return false;
    }

    if (!TankNeedsRepair(profile) && profile.partnerTank.energyReserve >= 92.0f &&
        profile.partnerTank.ammoReserve >= 90.0f) {
        gameState.lastEvent = "Field service rack reports BT-72 already in acceptable condition.";
        return false;
    }

    if (!ConsumeAnyRepairMaterial(profile)) {
        gameState.lastEvent = "Field service needs scrap, seals, plates, wire, or a repair patch.";
        return false;
    }

    const bool hasEngineer = HasAssignedSpecialistRole(profile, "engineer", "scavenger_support");
    const float intelligence = static_cast<float>(EffectiveStatValue(profile, gameState, 'I'));

```

```

const float boost = (hasEngineer ? 1.12f : 1.0f) * Bt72ServiceSkillBoost(profile, gameState) *
(1.0f + std::max(0.0f, intelligence - 5.0f) * 0.015f);

profile.partnerTank.damage.hull = std::min(100.0f, profile.partnerTank.damage.hull + 8.0f * boost);
profile.partnerTank.damage.bucket = std::min(100.0f, profile.partnerTank.damage.bucket + 10.0f * boost);
profile.partnerTank.damage.sensors = std::min(100.0f, profile.partnerTank.damage.sensors + 9.0f * boost);
profile.partnerTank.damage.turret = std::min(100.0f, profile.partnerTank.damage.turret + 6.0f * boost);
profile.partnerTank.damage.cockpit = std::min(100.0f, profile.partnerTank.damage.cockpit + 5.0f * boost);
profile.partnerTank.damage.powerCore = std::min(100.0f, profile.partnerTank.damage.powerCore + 7.0f * boost);
profile.partnerTank.energyReserve = std::min(100.0f, profile.partnerTank.energyReserve + 7.0f);
profile.partnerTank.ammoReserve = std::min(100.0f, profile.partnerTank.ammoReserve + 4.0f);

gameState.tankThermalLoad = std::max(0.0f, gameState.tankThermalLoad - 16.0f);

gameState.fieldWorkbenchCooldown = 45.0f;

std::string recipeEvent;

RegisterFieldServiceUse(profile, &recipeEvent);

gameState.lastEvent = hasEngineer
? "Field service rack cycled. Scavenger-side engineer tuning improved the repair pass."
: "Field service rack cycled. BT-72 patched and partially resupplied in the field.";

ApplyServiceChoiceBonuses(profile, gameState, true, gameState.lastEvent);

if (profile.firstPlayableRoute.firstTankCombatResolved && !profile.firstPlayableRoute.firstServicePerformed) {
    profile.firstPlayableRoute.firstServicePerformed = true;

    gameState.lastEvent += " First service halt logged for the route.";

    AppendObjectivePreviewHint(gameState.lastEvent, profile);

    AppendRouteBeatReadabilityHint(gameState.lastEvent, profile);
}

if (!recipeEvent.empty()) {
    gameState.lastEvent += " " + recipeEvent;
}

return true;
}

Success

```

Ran command

```
$ $lines=Get-Content src\GameRuntime.cpp; $lines[2280..2748]
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
if (scriptTag == "tower_sync") {
```

```
if (TryApplyTankEnergySiphon(
```

```
*nearest,
```

```
player,
```

```
profile,
```

```
gameState,
```

```
3.8f,
```

```
18.0f,
```

```
"Tower hardline latched. BT-72 batteries fast-charged from the relay spine.",
```

```
"BT-72 batteries already topped off. Tower hardline not needed.")) {
```

```
return true;
```

```
}
```

```
AddCollectedTapIfMissing(profile, nearest->registryId + "_tower", nearest->displayName + " // Tower Sync");
```

```
std::string progressionEvent;
```

```
AwardExperience(profile, 20, &progressionEvent);
```

```
const std::string routeTarget = nearest->linkTarget.empty() ? "regional_grid" : nearest->linkTarget;
```

```
const int purged = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 14.0f, true)));
```

```
auto& worldState = EnsureSelectedWorldFieldState(profile);
```

```
worldState.towerSyncRecovered = true;
```

```
worldState.localRelayAvailable = true;
```

```
worldState.regionalGridOnline = true;
```

```
gameState.lastEvent = "Tower relay synchronized. Regional scan and power route tagged -> " + routeTarget + ". "  
+ progressionEvent;
```

```
if (purged > 0) {
```

```
gameState.lastEvent += " Ether bloom purged by " + std::to_string(purged) + "%.";
```

```
}
```

```
return true;
```

```

}

if (scriptTag == "remote_link") {

const std::string target = nearest->linkTarget.empty() ? "gate_control" : nearest->linkTarget;

gameState.lastEvent = player.insideTank

? "Remote link established from cockpit. Control target -> " + target + "."

: "Remote link available. Use from cockpit or field terminal. Control target -> " + target + ".";

return true;

}

if (scriptTag == "power_pylon") {

if (!IsRegionalGridOnline(profile)) {

gameState.lastEvent = "Pylon restoration requires a live relay backbone first.";

return true;

}

const std::string markerId = nearest->registryId + "_pylon";

if (HasCollectedTape(profile, markerId)) {

gameState.lastEvent = "Power pylon already restored and feeding the regional line.";

return true;

}

if (!HasInventoryItem(profile, "steel_scrap") || !HasInventoryItem(profile, "copper_wire")) {

gameState.lastEvent = "Pylon restoration needs steel_scrap and copper_wire.";

return true;

}

ConsumeInventoryItem(profile, "steel_scrap", 1);

ConsumeInventoryItem(profile, "copper_wire", 1);

AddCollectedTapIfMissing(profile, markerId, nearest->displayName + " // Pylon Restored");

std::string progressionEvent;

AwardExperience(profile, 25, &progressionEvent);

const int purged = static_cast<int>(std::round(ReduceSelectedWorldEtherErosion(profile, 6.0f, true)));

auto& worldState = EnsureSelectedWorldFieldState(profile);

worldState.infrastructureDecay = std::max(0.0f, worldState.infrastructureDecay - 5.0f);

```

```
gameState.lastEvent = "Power pylon restored. Grid reach extended. " + progressionEvent;

if (purged > 0) {

gameState.lastEvent += " Ether bloom reduced by " + std::to_string(purged) + "%.";

}

return true;

}

if (scriptTag == "drone_station") {

if (!IsRegionalGridOnline(profile)) {

gameState.lastEvent = "Drone station needs a live grid before boot sequence can begin.";

return true;

}

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.droneStationsActive) {

if (!HasInventoryItem(profile, "copper_wire") || !HasInventoryItem(profile, "power_cell")) {

gameState.lastEvent = "Drone station activation needs copper_wire and power_cell.";

return true;

}

ConsumeInventoryItem(profile, "copper_wire", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.droneStationsActive = true;

gameState.droneTimer = 210.0f;

gameState.lastEvent = "Drone station brought online. Automated scavenger drones launched to local sweep routes.";

} else {

worldState.droneStationsActive = false;

gameState.lastEvent = "Drone station returned to standby. Automated sweep routes suspended.";

}

return true;

}

if (scriptTag == "rail_depot") {
```

```
auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.railFreightActive) {

if (!IsRegionalGridOnline(profile)) {

gameState.lastEvent = "Rail freight link needs a live grid before depot motors can wake.";

return true;

}

if (!HasInventoryItem(profile, "steel_scrap") || !HasInventoryItem(profile, "power_cell")) {

gameState.lastEvent = "Rail depot activation needs steel_scrap and power_cell.";

return true;

}

ConsumeInventoryItem(profile, "steel_scrap", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.railFreightActive = true;

gameState.railTimer = 320.0f;

gameState.lastEvent = "Rail freight spur restored. Heavy salvage trains can now service the bunker route.";

} else {

worldState.railFreightActive = false;

gameState.lastEvent = "Rail freight spur placed on standby. Depot traffic suspended.";

}

return true;

}

if (scriptTag == "orbital_uplink") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.orbitalUplinkActive) {

if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {

gameState.lastEvent = "Orbital uplink requires a live grid and active rail freight backbone.";

return true;

}

if (!HasInventoryItem(profile, "power_cell") || !HasInventoryItem(profile, "copper_wire")) {

gameState.lastEvent = "Orbital uplink boot needs power_cell and copper_wire.";
```



```
return true;
}
ConsumeInventoryItem(profile, "power_cell", 1);
ConsumeInventoryItem(profile, "copper_wire", 1);
worldState.orbitalUplinkActive = true;
gameState.orbitalTimer = 420.0f;
gameState.lastEvent = "Orbital uplink aligned. Low-orbit scan window acquired for Shelter 17.";
} else {
worldState.orbitalUplinkActive = false;
gameState.lastEvent = "Orbital uplink placed on standby. Scan uplink suspended.";
}
return true;
}
if (scriptTag == "rail_fortress_hub") {
auto& worldState = EnsureSelectedWorldFieldState(profile);
if (!worldState.railFortressActive) {
if (!worldState.railFreightActive || !worldState.orbitalUplinkActive) {
gameState.lastEvent = "Rail fortress requires active rail freight and orbital uplink support.";
return true;
}
if (!HasInventoryItem(profile, "old_plate") || !HasInventoryItem(profile, "power_cell")) {
gameState.lastEvent = "Rail fortress deployment needs old_plate and power_cell.";
return true;
}
ConsumeInventoryItem(profile, "old_plate", 1);
ConsumeInventoryItem(profile, "power_cell", 1);
worldState.railFortressActive = true;
gameState.railFortressTimer = 520.0f;
gameState.lastEvent = "Rail fortress marshaled. Armored train now anchors heavy recovery across the spur.";
} else {
```

```
worldState.railFortressActive = false;

gameState.lastEvent = "Rail fortress returned to depot standby.";

}

return true;

}

if (scriptTag == "recovery_fabricator") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.recoveryFabricatorActive) {

if (!IsRegionalGridOnline(profile) || !worldState.railFreightActive) {

gameState.lastEvent = "Recovery fabricator requires a live grid and rail freight support.";

return true;

}

if (!HasInventoryItem(profile, "steel_scrap") || !HasInventoryItem(profile, "ether_shard")) {

gameState.lastEvent = "Recovery fabricator boot needs steel_scrap and ether_shard.";

return true;

}

ConsumeInventoryItem(profile, "steel_scrap", 1);

ConsumeInventoryItem(profile, "ether_shard", 1);

worldState.recoveryFabricatorActive = true;

gameState.fabricatorTimer = 280.0f;

gameState.lastEvent = "Recovery fabricator primed. Shelter industry can now refine salvage into field supplies.";

} else {

worldState.recoveryFabricatorActive = false;

gameState.lastEvent = "Recovery fabricator returned to standby.";

}

return true;

}

if (scriptTag == "industrial_gate") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (worldState.industrialGateUnlocked) {
```

```

gameState.lastEvent = nearest->linkTarget.empty()

? "Industrial gate already unlocked. Inner spur route is open."

: "Industrial gate already unlocked. Route open -> " + nearest->linkTarget + ".";

return true;

}

if (!worldState.recoveryFabricatorActive || !worldState.railFortressActive || !worldState.orbitalUplinkActive) {

gameState.lastEvent = "Industrial gate requires the full recovery backbone: fabricator, fortress, and uplink.";

return true;

}

if (InventoryCount(profile, "old_plate") < 2 || InventoryCount(profile, "power_cell") < 1 ||
InventoryCount(profile, "repair_patch") < 1) {

gameState.lastEvent = "Industrial gate override needs 2 old_plate, 1 power_cell, and 1 repair_patch.";

return true;

}

ConsumeInventoryItem(profile, "old_plate", 2);

ConsumeInventoryItem(profile, "power_cell", 1);

ConsumeInventoryItem(profile, "repair_patch", 1);

worldState.industrialGateUnlocked = true;

worldState.routeContamination = std::max(0.0f, worldState.routeContamination - 8.0f);

worldState.infrastructureDecay = std::max(0.0f, worldState.infrastructureDecay - 6.0f);

std::string progressionEvent;

AwardExperience(profile, 90, &progressionEvent);

gameState.lastEvent = nearest->linkTarget.empty()

? "Industrial gate override accepted. Inner spur route unlocked for Shelter 17. " + progressionEvent

: "Industrial gate override accepted. Route open -> " + nearest->linkTarget + ". " + progressionEvent;

return true;

}

if (scriptTag == "industrial_survey") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.industrialGateUnlocked) {

```

```

gameState.lastEvent = "Industrial survey beacon is outside the current recovery perimeter. Unlock the gate first.";
return true;
}

if (!worldState.industrialSurveyActive) {
    if (!worldState.orbitalUplinkActive || !worldState.tradeNetworkActive) {
        gameState.lastEvent = "Industrial survey needs orbital uplink and trade network support.";
        return true;
    }

    if (InventoryCount(profile, "power_cell") < 1 || InventoryCount(profile, "copper_wire") < 1) {
        gameState.lastEvent = "Industrial survey startup needs 1 power_cell and 1 copper_wire.";
        return true;
    }

    ConsumeInventoryItem(profile, "power_cell", 1);
    ConsumeInventoryItem(profile, "copper_wire", 1);
    worldState.industrialSurveyActive = true;
    gameState.surveyTimer = 360.0f;
    gameState.lastEvent = "Industrial survey beacon aligned. Inner spur reconnaissance is now feeding Shelter 17.";
} else {
    worldState.industrialSurveyActive = false;
    gameState.lastEvent = "Industrial survey beacon returned to standby.";
}

return true;
}

if (scriptTag == "industrial_outpost") {
    auto& worldState = EnsureSelectedWorldFieldState(profile);
    if (!worldState.industrialGateUnlocked || !worldState.industrialSurveyActive) {
        gameState.lastEvent = "Inner spur outpost needs the gate unlocked and active survey coverage first.";
        return true;
    }

    if (!worldState.industrialOutpostActive) {

```

```

if (InventoryCount(profile, "old_plate") < 2 || InventoryCount(profile, "repair_patch") < 1 ||
InventoryCount(profile, "power_cell") < 1) {

gameState.lastEvent = "Outpost activation needs 2 old_plate, 1 repair_patch, and 1 power_cell.";

return true;

}

ConsumeInventoryItem(profile, "old_plate", 2);

ConsumeInventoryItem(profile, "repair_patch", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.industrialOutpostActive = true;

gameState.outpostTimer = 300.0f;

gameState.lastEvent = "Inner spur outpost established. Shelter 17 now has a forward foothold beyond the
industrial gate.";

} else {

worldState.industrialOutpostActive = false;

gameState.lastEvent = "Inner spur outpost returned to standby.";

}

return true;

}

if (scriptTag == "assembly_cell") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.industrialOutpostActive || !worldState.industrialSurveyActive) {

gameState.lastEvent = "Assembly cell needs a live inner spur outpost and survey coverage first.";

return true;

}

if (!worldState.assemblyCellActive) {

if (InventoryCount(profile, "steel_scrap") < 2 || InventoryCount(profile, "old_plate") < 1 ||
InventoryCount(profile, "power_cell") < 1) {

gameState.lastEvent = "Assembly cell startup needs 2 steel_scrap, 1 old_plate, and 1 power_cell.";

return true;

}

ConsumeInventoryItem(profile, "steel_scrap", 2);

```

```
ConsumeInventoryItem(profile, "old_plate", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.assemblyCellActive = true;

gameState.assemblyTimer = 340.0f;

gameState.lastEvent = "Inner spur assembly cell brought online. Local industrial recovery has started past the gate.";

} else {

worldState.assemblyCellActive = false;

gameState.lastEvent = "Inner spur assembly cell returned to standby.";

}

return true;

}

if (scriptTag == "foundry_line") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.assemblyCellActive || !worldState.industrialOutpostActive || !worldState.railFortressActive) {

gameState.lastEvent = "Foundry line needs an active assembly cell, inner spur outpost, and Rail Fortress cover first.";

return true;

}

if (!worldState.foundryLineActive) {

if (InventoryCount(profile, "steel_scrap") < 3 || InventoryCount(profile, "old_plate") < 2 || InventoryCount(profile, "power_cell") < 1) {

gameState.lastEvent = "Foundry startup needs 3 steel_scrap, 2 old_plate, and 1 power_cell.";

return true;

}

ConsumeInventoryItem(profile, "steel_scrap", 3);

ConsumeInventoryItem(profile, "old_plate", 2);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.foundryLineActive = true;

gameState.foundryTimer = 420.0f;

gameState.lastEvent = "Inner spur foundry line brought online. Heavy plate and hull-grade fabrication resumed.";
```

```
} else {  
    worldState.foundryLineActive = false;  
    gameState.lastEvent = "Inner spur foundry line returned to standby.";  
}  
    return true;  
}  
    if (scriptTag == "reactor_yard") {  
        auto& worldState = EnsureSelectedWorldFieldState(profile);  
        if (!worldState.foundryLineActive || !worldState.assemblyCellActive || !worldState.orbitalUplinkActive) {  
            gameState.lastEvent = "Reactor yard needs an active foundry line, assembly cell, and orbital uplink first.";  
            return true;  
        }  
        if (!worldState.reactorYardActive) {  
            if (InventoryCount(profile, "power_cell") < 2 || InventoryCount(profile, "ether_shard") < 2 ||  
                InventoryCount(profile, "copper_wire") < 2) {  
                gameState.lastEvent = "Reactor yard startup needs 2 power_cell, 2 ether_shard, and 2 copper_wire.";  
                return true;  
            }  
            ConsumeInventoryItem(profile, "power_cell", 2);  
            ConsumeInventoryItem(profile, "ether_shard", 2);  
            ConsumeInventoryItem(profile, "copper_wire", 2);  
            worldState.reactorYardActive = true;  
            gameState.reactorTimer = 460.0f;  
            gameState.lastEvent = "Inner spur reactor yard brought online. Heavy energy recovery resumed beyond the  
gate.";  
        } else {  
            worldState.reactorYardActive = false;  
            gameState.lastEvent = "Inner spur reactor yard returned to standby.";  
        }  
        return true;  
    }
```

```

}

if (scriptTag == "capacitor_bank") {

    auto& worldState = EnsureSelectedWorldFieldState(profile);

    if (!worldState.reactorYardActive || !worldState.foundryLineActive || !worldState.orbitalUplinkActive) {

        gameState.lastEvent = "Capacitor bank needs an active reactor yard, foundry line, and orbital uplink first.";

        return true;

    }

    if (!worldState.capacitorBankActive) {

        if (InventoryCount(profile, "power_cell") < 2 || InventoryCount(profile, "copper_wire") < 2 ||
            InventoryCount(profile, "ether_shard") < 1) {

            gameState.lastEvent = "Capacitor bank startup needs 2 power_cell, 2 copper_wire, and 1 ether_shard.";

            return true;

        }

        ConsumeInventoryItem(profile, "power_cell", 2);

        ConsumeInventoryItem(profile, "copper_wire", 2);

        ConsumeInventoryItem(profile, "ether_shard", 1);

        worldState.capacitorBankActive = true;

        gameState.capacitorTimer = 390.0f;

        gameState.lastEvent = "Inner spur capacitor bank charged and tied into the recovery backbone.";

    } else {

        worldState.capacitorBankActive = false;

        gameState.lastEvent = "Inner spur capacitor bank returned to standby.";

    }

    return true;

}

if (scriptTag == "relay_substation") {

    auto& worldState = EnsureSelectedWorldFieldState(profile);

    if (!worldState.capacitorBankActive || !worldState.reactorYardActive || !worldState.industrialOutpostActive) {

        gameState.lastEvent = "Relay substation needs an active capacitor bank, reactor yard, and inner spur outpost first.";
    }
}

```



```

return true;
}

if (!worldState.relaySubstationActive) {

if (InventoryCount(profile, "copper_wire") < 3 || InventoryCount(profile, "power_cell") < 1 ||
InventoryCount(profile, "repair_patch") < 1) {

gameState.lastEvent = "Relay substation sync needs 3 copper_wire, 1 power_cell, and 1 repair_patch.";

return true;

}

ConsumeInventoryItem(profile, "copper_wire", 3);
ConsumeInventoryItem(profile, "power_cell", 1);
ConsumeInventoryItem(profile, "repair_patch", 1);

worldState.relaySubstationActive = true;

gameState.relaySubstationTimer = 430.0f;

gameState.lastEvent = nearest->linkTarget.empty()
? "Relay substation synchronized. Inner spur power now routes back into Shelter 17."
: "Relay substation synchronized. Backbone route online -> " + nearest->linkTarget + ".";

} else {

worldState.relaySubstationActive = false;

gameState.lastEvent = "Relay substation returned to standby.";

}

return true;

}

if (scriptTag == "service_bay") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.relaySubstationActive || !worldState.foundryLineActive || !worldState.industrialOutpostActive) {

gameState.lastEvent = "Service bay needs a live relay substation, foundry line, and inner spur outpost first.";

return true;

}

if (!worldState.serviceBayActive) {

```

```

if (InventoryCount(profile, "old_plate") < 2 || InventoryCount(profile, "repair_patch") < 1 ||
InventoryCount(profile, "power_cell") < 1) {

gameState.lastEvent = "Service bay startup needs 2 old_plate, 1 repair_patch, and 1 power_cell.";

return true;

}

ConsumeInventoryItem(profile, "old_plate", 2);

ConsumeInventoryItem(profile, "repair_patch", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.serviceBayActive = true;

gameState.serviceBayTimer = 300.0f;

gameState.lastEvent = "Inner spur service bay brought online. BT-72 can now be serviced deeper into the factory
belt.";

} else {

worldState.serviceBayActive = false;

gameState.lastEvent = "Inner spur service bay returned to standby.";

}

return true;

}

if (scriptTag == "water_reclaimer") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.serviceBayActive || !worldState.relaySubstationActive || !worldState.recoveryFabricatorActive) {

gameState.lastEvent = "Water reclaimer needs a live service bay, relay substation, and recovery fabricator first.";

return true;

}

if (!worldState.waterReclaimerActive) {

if (InventoryCount(profile, "copper_wire") < 2 || InventoryCount(profile, "old_plate") < 1 ||
InventoryCount(profile, "power_cell") < 1) {

gameState.lastEvent = "Water reclaimer startup needs 2 copper_wire, 1 old_plate, and 1 power_cell.";

return true;

}

ConsumeInventoryItem(profile, "copper_wire", 2);

```

```
ConsumeInventoryItem(profile, "old_plate", 1);

ConsumeInventoryItem(profile, "power_cell", 1);

worldState.waterReclaimerActive = true;

gameState.waterReclaimerTimer = 260.0f;

gameState.lastEvent = "Inner spur water reclaimer brought online. Long-range recovery now has a stable water source.";

} else {

worldState.waterReclaimerActive = false;

gameState.lastEvent = "Inner spur water reclaimer returned to standby.";

}

return true;

}

if (scriptTag == "lanline_service_hub") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.towerSyncRecovered) {

gameState.lastEvent = "Lanline Services stay locked until the first tower is synchronized.";

return true;

}

worldState.localRelayAvailable = true;

profile.lanlineServices.serviceHubKnown = true;

gameState.lanlineServicesVisible = true;

gameState.lastSupportAction = nearest->linkTarget.empty()

? "Lanline service hub handshake complete."

: "Lanline service hub handshake complete -> " + nearest->linkTarget + ".";

gameState.lastEvent = gameState.lastSupportAction;

return true;

}

if (scriptTag == "tank_service") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.serviceBayActive) {
```

```

gameState.lastEvent = "Tank service anchor is authored, but the service bay backbone is still offline.";
return true;
}

std::string repairEvent;

if (TryConsumeBestTankServiceKit(profile, &repairEvent)) {

gameState.lastSupportAction = "BT-72 serviced through authored tank service anchor.";

gameState.lastEvent = gameState.lastSupportAction + " " + repairEvent;

} else {

gameState.lastEvent = repairEvent.empty()

? "Tank service anchor ready. Bring a compatible service kit from Lanline support or field salvage."

: "Tank service anchor ready. " + repairEvent;

}

return true;

}

if (scriptTag == "medical_support") {

auto& worldState = EnsureSelectedWorldFieldState(profile);

if (!worldState.localRelayAvailable) {

Success

Ran command

$ $lines=Get-Content tests\SmokeChecks.cpp; $lines[1210..1260]; $lines[2719..2908]

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

bunker::World world;

world.GeneratePrototypeZone();

const bunker::MapObject* workshop = world.FindObjectByRegistryId("[%workshop_0001]");

if (!Check(workshop != nullptr, "workshop handoff smoke expected workshop anchor")) {

return false;

}

bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();

profile.selectedWorld = "service_handoff_smoke.bwld";

profile.story.awakenedFromCryo = true;

```

```
profile.story.pipPadRecovered = true;

profile.story.archiveRecovered = true;

profile.firstPlayableRoute.bt72Restored = true;

profile.story.tankLinked = true;

profile.firstPlayableRoute.clearanceBlueprintRecovered = true;

profile.firstPlayableRoute.clearanceMaterialsRecovered = true;

profile.firstPlayableRoute.clearanceModuleInstalled = true;

profile.story.exitedBunker = true;

profile.firstPlayableRoute.surfaceArrivalReached = true;

profile.story.outerRoadCleared = true;

profile.firstPlayableRoute.firstTankCombatResolved = true;

profile.partnerTank.worldPositionKnown = true;

profile.partnerTank.worldX = workshop->x;

profile.partnerTank.worldY = workshop->y;

profile.partnerTank.damage.hull = 82.0f;

profile.partnerTank.energyReserve = 100.0f;

profile.partnerTank.ammoReserve = 100.0f;

profile.character.inventory.push_back({"repair_patch", 1, 0.2f});

bunker::PlayerState player;

player.insideTank = true;

bunker::StaticEraser staticEraser;

bunker::GameState gameState;

bunker::HandleInteraction(workshop, world, player, profile, staticEraser, gameState);

return Check(profile.firstPlayableRoute.firstServicePerformed, "workshop handoff smoke expected first service
flag") &&

Check(gameState.lastEvent.find("First service halt logged") != std::string::npos,

"workshop handoff smoke expected first service log copy") &&

Check(gameState.lastEvent.find("recovery node") != std::string::npos,

"workshop handoff smoke expected relay handoff cue") &&

Check(gameState.lastEvent.find("Route beat: Recovery Sync") != std::string::npos,
```

```
"workshop handoff smoke expected recovery-sync beat cue");
}

bool RunScalableContainerLootRuntimeSmoke() {
    auto inventoryCount = [](const bunker::SessionProfile& profile, std::string_view itemId) {
        const auto it = std::find_if(
            profile.character.inventory.begin(),
            profile.character.inventory.end(),
            [&](const bunker::InventoryEntry& entry) { return entry.itemId == itemId; });
        bunker::SessionProfile profile = bunker::MakeDefaultSessionProfile();
        profile.selectedWorld = "lanline_smoke_world.bwld";
        profile.fieldCheckpointKnown = true;
        profile.fieldCheckpointWorld = profile.selectedWorld;
        profile.story.awakenedFromCryo = true;
        profile.story.pipPadRecovered = true;
        profile.story.archiveRecovered = true;
        profile.firstPlayableRoute.bt72Restored = true;
        profile.story.tankLinked = true;
        profile.firstPlayableRoute.clearanceBlueprintRecovered = true;
        profile.firstPlayableRoute.clearanceMaterialsRecovered = true;
        profile.firstPlayableRoute.clearanceModuleInstalled = true;
        profile.story.exitedBunker = true;
        profile.firstPlayableRoute.surfaceArrivalReached = true;
        profile.story.outerRoadCleared = true;
        profile.firstPlayableRoute.firstTankCombatResolved = true;
        profile.firstPlayableRoute.firstServicePerformed = true;
        profile.story.relayRecovered = true;
        profile.firstPlayableRoute.firstRecoveryNodeActivated = true;
        profile.story.returnedToBase = true;
        profile.firstPlayableRoute.debriefSummaryViewed = true;
        profile.character.collectedTapes.push_back({"tower_pylon_alpha", "Pylon Alpha", true, false, true});
```

```
profile.character.collectedTapes.push_back({"tower_pylon_beta", "Pylon Beta", true, false, true});

bunker::WorldFieldState* worldState = bunker::FindWorldFieldState(profile, profile.selectedWorld, true);

if (!Check(worldState != nullptr, "lanline services smoke failed to create world field state")) {

return false;

}

worldState->towerSyncRecovered = true;

worldState->localRelayAvailable = true;

worldState->regionalGridOnline = true;

worldState->caravanRouteActive = true;

worldState->industrialSurveyActive = true;

worldState->industrialGateUnlocked = true;

worldState->industrialOutpostActive = true;

worldState->tradeNetworkActive = true;

worldState->railFreightActive = true;

worldState->railFortressActive = true;

worldState->recoveryFabricatorActive = true;

worldState->assemblyCellActive = true;

worldState->foundryLineActive = true;

worldState->reactorYardActive = true;

worldState->capacitorBankActive = true;

worldState->relaySubstationActive = true;

worldState->serviceBayActive = true;

worldState->waterReclaimerActive = true;

worldState->orbitalUplinkActive = true;

worldState->feyRingIntercityUnlocked = true;

worldState->feyRingInterserverUnlocked = true;

const auto unlockState = bunker::BuildServicesUnlockState(profile, worldState);

if (!Check(unlockState.towerSyncRecovered, "lanline services smoke expected tower sync recovery flag")) {

return false;

}
```

```
if (!Check(unlockState.relaySubstationActive, "lanline services smoke expected relay substation unlock flag")) {  
    return false;  
}  
  
if (!Check(unlockState.serviceBayActive, "lanline services smoke expected service bay unlock flag")) {  
    return false;  
}  
  
if (!Check(unlockState.waterReclaimerActive, "lanline services smoke expected water reclaimer unlock flag")) {  
    return false;  
}  
  
if (!Check(unlockState.backboneStable, "lanline services smoke expected stable backbone")) {  
    return false;  
}  
  
if (!Check(unlockState.feyRingIntercityUnlocked, "lanline services smoke expected inter-city Fey unlock")) {  
    return false;  
}  
  
if (!Check(unlockState.feyRingInterserverUnlocked, "lanline services smoke expected inter-server Fey unlock")) {  
    return false;  
}  
  
if (!Check(bunker::IsTankServiceUnlocked(unlockState), "lanline services smoke expected tank service unlock")) {  
    return false;  
}  
  
if (!Check(bunker::IsMedicalSupportUnlocked(unlockState), "lanline services smoke expected medical support  
unlock")) {  
    return false;  
}  
  
if (!Check(unlockState.backboneStage == "Backbone Stable",  
"lanline services smoke expected mirrored stable backbone stage")) {  
    return false;  
}  
  
if (!Check(unlockState.backboneStatus.find("14/14") != std::string::npos,
```



```
"lanline services smoke expected mirrored full backbone status")) {  
    return false;  
}  
  
if (!Check(unlockState.backbonePayoff.find("stable recovery backbone") != std::string::npos,  
"lanline services smoke expected mirrored backbone payoff")) {  
    return false;  
}  
  
if (!Check(unlockState.routeEventSummary.find("rare field prompt") != std::string::npos,  
"lanline services smoke expected mirrored idle route-event summary")) {  
    return false;  
}  
  
if (!Check(!unlockState.merchantWindowActive, "lanline services smoke expected merchant window to start  
inactive")) {  
    return false;  
}  
  
bunker::LanlineServicesState state = bunker::MakeDefaultLanlineServicesState(1000);  
  
state.relayCredits = 777;  
  
state.ownedCosmetics = {"skin_bt72_ashgray", "cosmetic_relay_badge"};  
  
bunker::SupportOrder order;  
  
order.orderId = "order_service_01";  
  
order.itemId = "tank_engine_kit";  
  
order.itemLabel = "BT-72 Engine Service Kit";  
  
order.destinationNode = "Shelter 17";  
  
order.state = bunker::SupportOrderState::Queued;  
  
order.paymentCurrency = bunker::StoreCurrency::InGame;  
  
order.priceCredits = 250;  
  
order.createdAtUnix = 1000;  
  
order.etaUnix = 1360;  
  
state.supportOrders.push_back(order);  
  
bunker::LanlineServicesProfile profileSnapshot{};
```

```
bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, state);

if (!Check(profileSnapshot.relayCredits == 777, "lanline services smoke expected profile relay credits sync")) {
    return false;
}

if (!Check(profileSnapshot.ownedCosmetics.size() == 2, "lanline services smoke expected cosmetic snapshot sync")) {
    return false;
}

if (!Check(profileSnapshot.pendingSupportOrders.size() == 1 &&
profileSnapshot.pendingSupportOrders[0] == "order_service_01",
"lanline services smoke expected pending support order sync")) {
    return false;
}

const bunker::LanlineServicesSave save = bunker::BuildLanlineServicesSave(state);
if (!Check(bunker::SaveLanlineServicesSave(save, bunker::DefaultLanlineServicesSavePath()),
"lanline services smoke failed to save services state")) {
    return false;
}

bunker::LanlineServicesSave loadedSave{};
if (!Check(bunker::LoadLanlineServicesSave(bunker::DefaultLanlineServicesSavePath(), loadedSave),
"lanline services smoke failed to load services state")) {
    return false;
}

bunker::LanlineServicesState loadedState = bunker::MakeLanlineServicesStateFromSave(loadedSave, 1200);
bunker::ApplyLanlineServicesProfileSnapshot(loadedState, profileSnapshot);
bunker::AdvanceLanlineSupportOrders(loadedState, 1500);
if (!Check(bunker::CountSupportOrdersInState(loadedState, bunker::SupportOrderState::Delivered) == 1,
"lanline services smoke expected one delivered support order after time advance")) {
    return false;
}
```

```

worldState->activeRouteEventType = "merchant_window";

worldState->routeEventTimeRemaining = 20.0f;

worldState->routeEventOfferTimeRemaining = 6.0f;

const auto merchantUnlockState = bunker::BuildServicesUnlockState(profile, worldState);

if (!Check(merchantUnlockState.merchantWindowActive, "lanline services smoke expected merchant window
mirror when active")) {

return false;

}

if (!Check(merchantUnlockState.routeEventSummary.find("Merchant window offered") != std::string::npos,
"lanline services smoke expected merchant route-event summary mirror")) {

return false;

}

std::string claimSummary;

if (!Check(bunker::ClaimDeliveredSupportOrders(loadedState, profile, &claimSummary) == 1,
"lanline services smoke expected delivered support order claim")) {

return false;

}

profile.lanlineServices.relayCredits = loadedState.relayCredits + 23;

bunker::SyncLanlineServicesSessionProfile(profile, loadedState);

bunker::SyncLanlineServicesProfileSnapshot(profileSnapshot, loadedState);

return Check(loadedState.relayCredits == 777, "lanline services smoke expected relay credits after roundtrip") &&
Check(loadedState.ownedCosmetics.size() == 2, "lanline services smoke expected owned cosmetics after
roundtrip") &&
Check(loadedState.supportOrders.size() == 1, "lanline services smoke expected one support order after
roundtrip") &&
Check(loadedState.supportOrders[0].destinationNode == "Shelter 17",
"lanline services smoke expected destination node after roundtrip") &&
Check(loadedState.supportOrders[0].state == bunker::SupportOrderState::Claimed,
"lanline services smoke expected claimed support order after runtime claim") &&
Check(profileSnapshot.pendingSupportOrders.empty(),
"lanline services smoke expected pending support orders to clear after claim") &&

```

```

Check(worldState->relayCreditsSpent == 23,
"lanline services smoke expected world relay credit spend mirror after session sync") &&
Check(claimSummary.find("BT-72 Engine Service Kit") != std::string::npos,
"lanline services smoke expected claim summary to mention delivered order") &&
Check(std::any_of(
profile.character.inventory.begin(),
profile.character.inventory.end(),
[](const bunker::InventoryEntry& item) {
return item.itemId == "engine_seal" && item.count >= 1;
}),
"lanline services smoke expected engine service kit delivery to grant engine_seal inventory");
}

```

Success

Ran command

```

$ $lines=Get-Content src\SkillSystem.cpp; $lines[0..130]
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
#include "../include/SkillSystem.hpp"

namespace bunker {

namespace {

PassiveSkill* FindSkill(SessionProfile& profile, const std::string& skillId) {
for (auto& skill : profile.character.passiveSkills) {
if (skill.skillId == skillId) {
return &skill;
}
}

return nullptr;
}

bool HasRecipeItem(const SessionProfile& profile, const std::string& itemId) {
for (const auto& item : profile.character.inventory) {
if (item.itemId == itemId && item.count > 0) {

```

```

return true;

}

}

return false;

}

void UnlockSkill(SessionProfile& profile, const std::string& skillId, const std::string& fallbackName, std::string*
eventText) {

    auto* skill = FindSkill(profile, skillId);

    if (skill == nullptr) {

        profile.character.passiveSkills.push_back({skillId, fallbackName, true, true});

        if (eventText != nullptr) {

            *eventText = fallbackName + " awakened and equipped.";

        }

        return;

    }

    if (!skill->unlocked) {

        skill->unlocked = true;

        skill->equipped = true;

        if (eventText != nullptr) {

            *eventText = skill->displayName + " awakened and equipped.";

        }

    }

}

} // namespace

bool HasEquippedPassiveSkill(const SessionProfile& profile, const std::string& skillId) {

    for (const auto& skill : profile.character.passiveSkills) {

        if (skill.skillId == skillId && skill.unlocked && skill.equipped) {

            return true;

        }

    }

}

```

```
return false;
}

void RegisterArchiveSync(SessionProfile& profile, std::string* eventText) {
    profile.character.awakening.archiveSyncs += 1;
    if (profile.character.awakening.archiveSyncs >= 2) {
        UnlockSkill(profile, "skill_data_miner", "Data Miner", eventText);
    }
}

void RegisterFootKill(SessionProfile& profile, std::string* eventText) {
    profile.character.awakening.footKills += 1;
    if (profile.character.awakening.footKills >= 2) {
        UnlockSkill(profile, "skill_field_reflex", "Field Reflex", eventText);
    }
}

void RegisterTankAction(SessionProfile& profile, std::string* eventText) {
    profile.character.awakening.tankActions += 1;
    if (profile.character.awakening.tankActions >= 2) {
        UnlockSkill(profile, "skill_pilot_sync", "Pilot Sync", eventText);
    }
}

void RegisterStressSurvival(SessionProfile& profile, std::string* eventText) {
    profile.character.awakening.stressSurvivals += 1;
    if (profile.character.awakening.stressSurvivals >= 1) {
        UnlockSkill(profile, "skill_second_wind", "Second Wind", eventText);
    }
}

void RegisterHeavyCarryDrill(SessionProfile& profile, std::string* eventText) {
    profile.character.awakening.heavyCarryDrills += 1;
    if (profile.character.awakening.heavyCarryDrills >= 4) {
        UnlockSkill(profile, "skill_muscle_memory", "Muscle Memory", eventText);
    }
}
```

```

}

}

void RegisterFieldServiceUse(SessionProfile& profile, std::string* eventText) {

profile.character.awakening.fieldServiceUses += 1;

if (profile.character.awakening.fieldServiceUses >= 3 && !HasRecipeItem(profile, "recipe_repair_patch")) {

profile.character.inventory.push_back({"recipe_repair_patch", 1, 0.0f});

if (eventText != nullptr) {

*eventText = "Awakened Recipe discovered: Repair Patch.";

}

}

}

} // namespace bunker

Success

Ran command

$ git grep -n "InventoryCount\\|CountInventory\\|countInventory\\|HasInventoryItem" -- tests/SmokeChecks.cpp

## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10

tests/SmokeChecks.cpp:73:int CountInventoryItem(const bunker::SessionProfile& profile, const std::string&
itemId) {

tests/SmokeChecks.cpp:1699: auto countInventory = [](const bunker::SessionProfile& profile, const std::string&
itemId) {

tests/SmokeChecks.cpp:1751: const int vouchersBefore = countInventory(profile, "trade_voucher");

tests/SmokeChecks.cpp:1767: Check(countInventory(profile, "trade_voucher") == vouchersBefore - 1,

tests/SmokeChecks.cpp:5956: if (!Check(bunker::HasInventoryItem(patchProfile, "repair_patch") &&

tests/SmokeChecks.cpp:5967: bunker::HasInventoryItem(patchProfile, "repair_patch") &&

tests/SmokeChecks.cpp:6107: bunker::HasInventoryItem(profile, "bunker_access_card") &&

tests/SmokeChecks.cpp:6109: bunker::HasInventoryItem(profile, "%it_pippad"),

tests/SmokeChecks.cpp:6145: bunker::HasInventoryItem(profile, "old_plate") &&

tests/SmokeChecks.cpp:6146: bunker::HasInventoryItem(profile, "power_cell") &&

tests/SmokeChecks.cpp:6147: bunker::HasInventoryItem(profile, "repair_patch"),

tests/SmokeChecks.cpp:6428: bunker::HasInventoryItem(profile, "power_cell") &&

```

tests/SmokeChecks.cpp:6429: bunker::HasInventoryItem(profile, "old_plate") &&
tests/SmokeChecks.cpp:6430: bunker::HasInventoryItem(profile, "repair_patch"),
tests/SmokeChecks.cpp:6499: bunker::HasInventoryItem(profile, "bunker_access_card") &&
tests/SmokeChecks.cpp:6546: bunker::HasInventoryItem(profile, "bunker_access_card") &&
tests/SmokeChecks.cpp:6571: bunker::HasInventoryItem(profile, "power_cell") &&
tests/SmokeChecks.cpp:6572: bunker::HasInventoryItem(profile, "old_plate") &&
tests/SmokeChecks.cpp:6573: bunker::HasInventoryItem(profile, "repair_patch"),
tests/SmokeChecks.cpp:6646: const int selectedDeviceCount = CountInventoryItem(profile, "%it_pippad");
tests/SmokeChecks.cpp:6658: CountInventoryItem(profile, "%it_pippad") == selectedDeviceCount &&
tests/SmokeChecks.cpp:6707: const int selectedDeviceCount = CountInventoryItem(profile,
"%it_pippad");
tests/SmokeChecks.cpp:6713: CountInventoryItem(profile, "%it_pippad") == selectedDeviceCount &&
tests/SmokeChecks.cpp:6803: const int pipPadCountBeforeRetiredInteraction = CountInventoryItem(profile,
"%it_pippad");
tests/SmokeChecks.cpp:6814: CountInventoryItem(profile, "%it_pippad") ==
pipPadCountBeforeRetiredInteraction &&
tests/SmokeChecks.cpp:7568: CountInventoryItem(profile, "%it_pippad") == 1,
tests/SmokeChecks.cpp:7573: CountInventoryItem(profile, "%it_pippad") == 1,
tests/SmokeChecks.cpp:7585: CountInventoryItem(profile, "%it_pipboy_1_0") == 1 &&
tests/SmokeChecks.cpp:7592: CountInventoryItem(profile, "%it_pipboy_3000") == 1 &&
tests/SmokeChecks.cpp:7689: CountInventoryItem(profile, "%it_pipboy_2000_classic") == 1 &&
tests/SmokeChecks.cpp:7696: CountInventoryItem(profile, "%it_pipboy_2000_mark_vi") == 1 &&
tests/SmokeChecks.cpp:7755: CountInventoryItem(reselectProfile, "%it_pipboy_2000_mark_vi") == 1,
tests/SmokeChecks.cpp:7761: const int markViCountBeforeInvalid = CountInventoryItem(reselectProfile,
"%it_pipboy_2000_mark_vi");
tests/SmokeChecks.cpp:7765: CountInventoryItem(reselectProfile, "%it_pipboy_2000_mark_vi") ==
markViCountBeforeInvalid,
tests/SmokeChecks.cpp:7770: CountInventoryItem(reselectProfile, "%it_pipboy_2000_mark_vi") ==
markViCountBeforeInvalid,
tests/SmokeChecks.cpp:8138: bunker::HasInventoryItem(profile, "power_cell") &&
tests/SmokeChecks.cpp:8139: bunker::HasInventoryItem(profile, "old_plate") &&
tests/SmokeChecks.cpp:8140: bunker::HasInventoryItem(profile, "repair_patch"),


```
tests/SmokeChecks.cpp:8226: bunker::HasInventoryItem(profile, "bunker_access_card") &&
tests/SmokeChecks.cpp:8235: bunker::HasInventoryItem(profile, "%it_pippad") &&
tests/SmokeChecks.cpp:8275: bunker::HasInventoryItem(profile, "old_plate") &&
tests/SmokeChecks.cpp:8277: bunker::HasInventoryItem(profile, "power_cell") &&
tests/SmokeChecks.cpp:8279: bunker::HasInventoryItem(profile, "repair_patch"),
tests/SmokeChecks.cpp:8394: bunker::HasInventoryItem(profile, "%it_pippad"),
Success
```

Ran command

```
$ $lines=Get-Content tests\SmokeChecks.cpp; $lines[0..130]
```

```
## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10
```

```
#include "../include/AppPaths.hpp"
```

```
#include "../include/AtomicPersistence.hpp"
```

```
#include "../include/BuildAnnouncement.hpp"
```

```
#include "../include/GameExecution.hpp"
```

```
#include "../include/GameRuntime.hpp"
```

```
#include "../include/GameplayDescriptorRegistry.hpp"
```

```
#include "../include/HangarSystem.hpp"
```

```
#include "../include/LanlineServices.hpp"
```

```
#include "../include/LaunchSession.hpp"
```

```
#include "../include/PrefabLibrary.hpp"
```

```
#include "../include/SessionProfiles.hpp"
```

```
#include "../include/StoryRoute.hpp"
```

```
#include "../include/World.hpp"
```

```
#include "../include/WorldEditorUndo.hpp"
```

```
#include "../include/WorldEvents.hpp"
```

```
#include "../include/WorldExport.hpp"
```

```
#include "../include/WorldSemanticAuthoring.hpp"
```

```
#include "../include/WorldValidation.hpp"
```

```
#include "../Editor/src/EditorSupport.hpp"
```

```
#include <algorithm>
```

```
#include <cmath>

#include <cstdint>

#include <cstdlib>

#include <filesystem>

#include <fstream>

#include <iomanip>

#include <initializer_list>

#include <iostream>

#include <string>

#include <unordered_set>

#include <vector>

namespace fs = std::filesystem;

namespace {

struct WorkingDirectoryGuard {

explicit WorkingDirectoryGuard(const fs::path& target)

: original(fs::current_path()) {

fs::current_path(target);

}

~WorkingDirectoryGuard() {

std::error_code ec;

fs::current_path(original, ec);

}

fs::path original;

};

bool Check(bool condition, const std::string& message) {

if (!condition) {

std::cerr << "[smoke] " << message << '\n';

return false;

}

return true;

}
```

```

}

bool HasDuplicateRegistryIds(const bunker::World& world, std::string* duplicateId = nullptr) {
    std::unordered_set<std::string> seen;
    for (const auto& object : world.objects) {
        if (!seen.insert(object.registryId).second) {
            if (duplicateId != nullptr) {
                *duplicateId = object.registryId;
            }
            return true;
        }
    }
    return false;
}

int CountInventoryItem(const bunker::SessionProfile& profile, const std::string& itemId) {
    int total = 0;
    for (const auto& item : profile.character.inventory) {
        if (item.itemId == itemId) {
            total += item.count;
        }
    }
    return total;
}

bool WriteTextFile(const fs::path& path, const std::string& text) {
    std::ofstream file(path);
    if (!file.is_open()) {
        return false;
    }
    file << text;
    return true;
}

```

```

std::string ReadTextFile(const fs::path& path) {
    std::ifstream file(path);

    std::string text;

    std::string line;

    while (std::getline(file, line)) {

        text += line;

        text += '\n';

    }

    return text;

}

bool ContainsText(const std::string& text, const std::string& needle) {

    return text.find(needle) != std::string::npos;

}

void WriteRawString(std::ofstream& file, const std::string& value) {

    const auto length = static_cast<std::uint32_t>(value.size());

    file.write(reinterpret_cast<const char*>(&length), sizeof(length));

    file.write(value.data(), static_cast<std::streamsize>(value.size()));

}

void UnlockAndEquipSkill(bunker::SessionProfile& profile, const std::string& skillId) {

    for (auto& skill : profile.character.passiveSkills) {

        if (skill.skillId == skillId) {

            skill.unlocked = true;

            skill.equipped = true;

            return;

        }

    }

}

bool RunWorldRoundtrip() {

    bunker::World savedWorld;

    savedWorld.GeneratePrototypeZone();

```

```
savedWorld.metadata.name = "Smoke Test World";  
  
savedWorld.metadata.objective = "Roundtrip world persistence";  
  
savedWorld.EnsureStarterInfrastructure();  
  
if (auto* archiveTerminal = savedWorld.FindObjectByRegistryId("[%archive_0001]")) {  
  
    archiveTerminal->editorLayer = "Archive";  
  
    archiveTerminal->prefabSourceId = "prefab_archive_sync";  
  
    Success  
  
    Ran command  
  
    $ git grep -n "scriptTag == \"workshop_service\"\\|scriptTag == \"workshop\"\\|workshop_service" --  
    src\\GameRuntime.cpp src\\World.cpp tests\\SmokeChecks.cpp  
  
    ## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10  
  
    fatal: unable to resolve revision: workshop_service\\|scriptTag == \\workshop\\|workshop_service  
  
    Exit code 1  
  
    Ran command  
  
    $ git grep -n "FindObjectByScriptTag(\"lanline_service_hub\"\\|lanline_service_hub" -- src\\World.cpp  
    tests\\SmokeChecks.cpp  
  
    ## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10  
  
    fatal: unable to resolve revision: lanline_service_hub\\|lanline_service_hub  
  
    Exit code 1  
  
    Ran command  
  
    $ $lines=Get-Content tests\\SmokeChecks.cpp; $lines[8448..8570]  
  
    ## feature/start-bunker-bt72-pippad-v10...origin/feature/start-bunker-bt72-pippad-v10  
  
    std::cerr << "[smoke] failed to create sandbox: " << ec.message() << '\n';  
  
    return EXIT_FAILURE;  
  
    }  
  
    WorkingDirectoryGuard guard(sandboxRoot);  
  
    bunker::EnsureProjectDirectories();  
  
    const bool ok = RunWorldRoundtrip() &&  
  
    RunEditorWorldFileHelpersSmoke() &&  
  
    RunProfileRoundtrip() &&
```

RunProfileMigrationContractSmoke() &&
RunContinuityAnchorContractSmoke() &&
RunFirstPlayableRouteStorySmoke() &&
RunRouteBeatPresentationSmoke() &&
RunFirstPlayableRouteReadoutSmoke() &&
RunBt72RestorationObjectiveReadoutSmoke() &&
RunSurfaceArrivalWorldEventSmoke() &&
RunFirstCombatWorldEventSmoke() &&
RunFirstCombatResolutionHandoffSmoke() &&
RunWorkshopServiceRouteHandoffSmoke() &&
RunScalableContainerLootRuntimeSmoke() &&
RunRuntimeProfileSaveDoesNotReplaceAuthoringWorldSmoke() &&
RunDebriefIndustrialHandoffSmoke() &&
RunRecoveryHandoffSummarySmoke() &&
RunRecoveryBackboneStatusSmoke() &&
RunRouteEventLifecycleSmoke() &&
RunMerchantRouteEventSmoke() &&
RunWeatherWeightedRouteEventSmoke() &&
RunWorldScopedRouteSummarySmoke() &&
RunHostileAwarenessSmoke() &&
RunHumanTriggerDisciplineSmoke() &&
RunHumanCoverSeekingSmoke() &&
RunBt72CombatFeedbackSmoke() &&
RunReactiveBreakableGlassSmoke() &&
RunReactiveBreakableFoliageSmoke() &&
RunMechanicalHostileDamageSmoke() &&
RunFieldReflexRpgWeightSmoke() &&
RunBt72CrewCoordinationWeightSmoke() &&
RunBt72WeakPointComboSmoke() &&
RunBt72SeatPolicySmoke() &&

RunServiceChoiceWeightSmoke() &&
RunLauncherAnnouncementSmoke() &&
RunLanlineServicesRoundtripSmoke() &&
RunTankServiceKitSmoke() &&
RunLanlineSeatRoundtripSmoke() &&
RunLaunchTicketFlow() &&
RunGameplayDescriptorValidationSmoke() &&
RunSemanticDependencyValidationSmoke() &&
RunSemanticDependencyGraphSmoke() &&
RunSemanticLayoutSmoke() &&
RunSemanticLayoutPreserveManualSmoke() &&
RunSemanticAuthoringStateRoundtripSmoke() &&
RunSemanticAutoAnchorValidationSmoke() &&
RunPrefabLibrarySemanticStateSmoke() &&
RunLegacyPrefabManualLootMigrationSmoke() &&
RunPrefabUsageAndExportReportSmoke() &&
RunSupportedFileFormatRegistrySmoke() &&
RunExportDataScanSmoke() &&
RunStrictSemanticExportPolicySmoke() &&
RunValidatedWorldExportArtifactSmoke() &&
RunWorldExportAuditTrailSmoke() &&
RunShippingBaselineDiffSmoke() &&
RunShippingBaselineObjectAwareDriftSmoke() &&
RunExportHistoryCheckpointSelectionSmoke() &&
RunLegacySemanticAutoInferenceSmoke() &&
RunLegacyManualLootMigrationSmoke() &&
RunLegacyWorldEditorLayerInferenceSmoke() &&
RunWorldEditorUndoSmoke() &&
RunWorldReferenceGraphSmoke() &&
RunSemanticAuthoringCascadeSmoke() &&

RunStarterSemanticLinkTargetSmoke() &&
RunLegacyWorldAliasMigrationSmoke() &&
RunGameExecutionResourceLookupSmoke() &&
RunGameExecutionLootTemplateSmoke() &&
RunGameExecutionScriptBridgeSmoke() &&
RunPlayerSweepCollisionSmoke() &&
RunRuntimeCameraSmoke() &&
RunPipDeviceCapabilityContractSmoke() &&
RunBlueLinkExpansionModuleContractSmoke() &&
RunBlueLinkRuntimeInteractionSmoke() &&
RunBt72CraneRestorationContractSmoke() &&
RunBt72CraneRuntimeInteractionSmoke() &&
RunBt72ServiceNotesAndPatchRuntimeSmoke() &&
RunStarterWorldFirstRouteObjectAvailabilitySmoke() &&
RunStarterWorldFirstPlayableRouteSmoke() &&
RunStarterWorldRegistryIdentitySmoke() &&
RunStarterScenarioIdentitySurvivesPipPadPickupSmoke() &&
RunStarterScenarioIdentitySurvivesPipPadPickupRoundtripSmoke() &&
RunLoadedStarterWorldContinuesRouteAfterPipPadPickupSmoke() &&
RunLoadedProfileAndWorldContinueAfterPipPadPickupSmoke() &&
RunPipDeviceSelectionStationPersistsAfterPickupSmoke() &&
RunPipDeviceSelectionStationRoundtripSmoke() &&
RunPipDeviceReselectFlowSmoke() &&
RunPipDeviceSelectionStationRetiresAfterBunkerExitSmoke() &&
RunActivePipDeviceUsableAfterStationRetirementSmoke() &&
RunActivePipDeviceUiCompatibilityWrapperSmoke() &&
RunActivePipDeviceAccessCompatibilityWrapperSmoke() &&
RunActivePipDeviceCustomizationStateSmoke() &&
RunActivePipDeviceCustomizationCommandSmoke() &&
RunActivePipDeviceCustomizationCapabilityRulesSmoke() &&


```
RunActivePipDeviceCustomizationCommandGateSmoke() &&
RunActivePipDeviceCustomizationRuntimeBindingSmoke() &&
RunPipDeviceSelectionHelperContractSmoke() &&
RunPipDeviceItemRegistryAndStationOptionsSmoke() &&
RunPipDeviceProfileNormalizationRegistrySmoke() &&
RunActivePipDeviceCapabilityRulesSmoke() &&
RunStarterWorldRouteObjectsPersistAfterSaveLoadSmoke() &&
RunLegacyStarterWorldGetsV940RouteInfrastructureSmoke() &&
RunStarterRuntimeObjectSpecDriftSmoke() &&
RunStarterWorldRegistryIdentityRoundtripSmoke() &&
RunLoadedStarterWorldFirstPlayableRouteSmoke() &&
RunFirstPlayableRouteEndToEndSmoke() &&
RunPipPadAccessGatingSmoke();
fs::remove_all(sandboxRoot, ec);
return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}

Success
```